Journal Collaboration
Machine Learning and Knowledge Extraction

Nhan Thi Ho

Introduction to Bio-Medical Data Analysis with R

Biodata with R

MDPI

Introduction to Bio-Medical Data Analysis with R

Introduction to Bio-Medical Data Analysis with R

Biodata with R

Nhan Thi Ho

MDPI

Basel • Beijing • Wuhan • Barcelona • Belgrade • Novi Sad • Cluj • Manchester

Author
Nhan Thi Ho
Research Management Department,
Vinmec International Hospital
Hanoi, Vietnam
College of Health Sciences, VinUniversity
Hanoi, Vietnam

Editorial Office
MDPI AG
Grosspeteranlage 5
4052 Basel, Switzerland

For citation purposes, cite as indicated below:

Ho, N.T. *Introduction to Bio-Medical Data Analysis with R*; MDPI: Basel, Switzerland, 2026, pp. Page Range

ISBN 978-3-7258-5338-0 (Hbk)
ISBN 978-3-7258-5337-3 (PDF)
https://doi.org/10.3390/books978-3-7258-5337-3

The publication of this work was generously supported by Vinmec International Hospital, Hanoi, Vietnam, and the MDPI Journal *Machine Learning & Knowledge Extraction.*

© 2026 by the author/s. Licensee MDPI, Basel, Switzerland. This book is Open Access and distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

Contents

List of Figures and Tables xi

Abbreviations xvii

About the Author xix

Preface xxi

Acknowledgements xxiii

1 Introduction 1
- 1.1 About This Book 1
- 1.2 Target Audience 1
- 1.3 Main Contents of This Book 1

2 Introduction to R 3
- 2.1 What Is R? 3
- 2.2 RStudio 3
- 2.3 Rmarkdown 3
- 2.4 Base R Example 3
- 2.5 Install, Load, Update R Packages 4
- 2.6 Working with R Objects 5
  - 2.6.1 Vector 5
  - 2.6.2 Matrix 6
  - 2.6.3 Array 6
  - 2.6.4 List 7
  - 2.6.5 Data Frame 7
  - 2.6.6 Assessing the Structure of Data Objects 8
  - 2.6.7 Useful Functions to Assess R Objects 8
  - 2.6.8 Understanding Vectors 9
- 2.7 Managing Bio-Medical Data in R 11
  - 2.7.1 Entering Data 11
  - 2.7.2 Importing Data from a File 12
  - 2.7.3 Some Other Data Import Functions 13
  - 2.7.4 Exporting Data 14
  - 2.7.5 Basic Data Manipulation with “tidyverse” 15
- 2.8 Basic Data Visualization 18
  - 2.8.1 Barplot 18
  - 2.8.2 Histogram 18
  - 2.8.3 Density 19
  - 2.8.4 Histogram with Density 20

2.8.5 Scatter Plot . . . 22
2.8.6 Boxplot . . . 24
2.8.7 Pie Charts . . . 25
2.9 Summary Statistics . . . 26
2.9.1 Basic Descriptive Summary Functions . . . 26
2.9.2 Basic Statistical Test . . . 27
2.9.3 Summary Statistic Table . . . 30
2.10 Practice . . . 33
2.11 Practice Review . . . 33
2.11.1 The Requirements . . . 33
2.11.2 Further Exploration . . . 39
2.12 Useful Resources . . . 45
2.12.1 R Books . . . 45
2.12.2 Datasets . . . 45
2.13 Chapter Summary . . . 45

3 Basic Statistics with R . . . 46
3.1 Data Distribution . . . 46
3.1.1 The Normal Distribution . . . 46
3.1.2 The Binomial Distribution . . . 49
3.2 Basic Statistical Tests . . . 50
3.2.1 Tests for Comparing Continuous Variables . . . 50
3.2.2 Tests for Comparing Categorical Variables . . . 54
3.3 Data Summary and Basic Statistical Test in a Table . . . 55
3.4 Sample Size—Power Estimation . . . 58
3.4.1 Power, Sample Size for Comparing Continuous Variables . . . 58
3.4.2 Power, Sample Size for Comparing Categorical Variables . . . 60
3.4.3 Sample Size for Estimating Parameter with Desired Precision 61
3.5 Chapter Summary . . . 63

4 Linear Regression with R . . . 64
4.1 Introduction to Linear Regression . . . 64
4.2 Simple Linear Regression . . . 64
4.2.1 Simple Linear Model with R . . . 64
4.2.2 Linear Model Assumptions . . . 68
4.2.3 Checking Linear Model Assumptions . . . 68
4.2.4 Checking Influential Observations . . . 73
4.2.5 Categorical Predictor Variables . . . 73
4.3 Multiple Linear Regression . . . 85
4.4 Model Selection . . . 90

4.5 Model Validation . . . 93
4.5.1 Performance Metrics . . . 94
4.5.2 Calibration . . . 96
4.6 Useful Resources . . . 97
4.7 Chapter Summary . . . 97

5 Logistic Regression with R . . . 99
5.1 Why Logistic Regression Is Needed? . . . 99
5.2 Logistic Regression Overview . . . 100
5.2.1 Statistical Concepts . . . 100
5.2.2 Univariable and Multivariable Models . . . 108
5.3 Logistic Regression Model Building . . . 108
5.3.1 Main Steps . . . 108
5.3.2 Building Model in R . . . 109
5.4 Model Evaluation . . . 113
5.4.1 Classification Metrics . . . 113
5.4.2 Calculate, Visualize Classification Measures . . . 114
5.4.3 Discrimination Power . . . 117
5.4.4 Model Fit Statistics . . . 119
5.5 Model Validation . . . 121
5.5.1 Create Training and Test Samples . . . 121
5.5.2 Build Logitic Models on Training Data and Predict on Test Data 121
5.6 Useful Resources . . . 125
5.7 Chapter Summary . . . 125

6 Survival Data Analysis with R . . . 126
6.1 Introduction to Survival Analysis . . . 126
6.1.1 Survival Data . . . 126
6.1.2 Key Statistical Concepts . . . 127
6.1.3 Clinical Case Study: Factors Affecting Lung Cancer Survival 127
6.2 Essential R Packages for Survival Analysis . . . 127
6.2.1 Working with Dates . . . 127
6.2.2 Package “Survival” Main Functions . . . 128
6.3 Survival Curves and Models . . . 129
6.3.1 Survival Model and Curve for Overall Data . . . 129
6.3.2 Comparing Survival Times Between Groups . . . 133
6.3.3 Hazard Ratios . . . 138
6.3.4 Multivariable Cox Model . . . 138
6.3.5 Stratified Cox Model . . . 139
6.4 Model Evaluation . . . 140

6.4.1 Checking Assumptions for Cox Model . . . 140
6.4.2 Evaluating Model Accuracy . . . 143
6.4.3 Residual Analysis for Goodness-of-Fit . . . 144
6.5 Model Selection . . . 146
6.5.1 Partial Likelihood Hypothesis Tests . . . 146
6.5.2 Information Criteria . . . 147
6.5.3 Model Selection Approaches . . . 147
6.6 Transform Survival Curves . . . 149
6.6.1 Cumulative Incidence Function . . . 150
6.6.2 Cumulative Hazard Function . . . 150
6.7 More Advanced Aspects in Survival Analysis . . . 151
6.7.1 Competing Risk Analysis . . . 151
6.7.2 Time Dependent Covariates . . . 154
6.8 Useful Resources . . . 156
6.9 Chapter Summary . . . 156
7 Longitudinal Data Analysis with R . . . 157
7.1 Longitudinal Data . . . 157
7.2 Example: The Bangladesh Data . . . 157
7.2.1 About Example Data . . . 157
7.2.2 Exploratory Data Analysis . . . 158
7.3 Options for Analysis of Change . . . 160
7.3.1 Pre-Post Analysis . . . 160
7.3.2 Linear Model for Time (the "naïve" Approach) . . . 160
7.3.3 Issues of Linear Model . . . 163
7.4 Generalized Estimating Equations (GEE) . . . 163
7.4.1 Introduction to GEE . . . 163
7.4.2 GEE Example: The Bangladesh Data . . . 164
7.4.3 GEE Example: Other Types of Outcomes . . . 167
7.4.4 GEE Summary Main Characteristics . . . 170
7.5 Generalized Linear Mixed-Effects Models (GLMM) . . . 170
7.5.1 GLMM Overview . . . 170
7.5.2 Linear Mixed Effect (LME) Model Example . . . 171
7.5.3 GLMM Examples . . . 183
7.6 GEE or Mixed-Effects Model . . . 187
7.7 Useful Resources . . . 187
7.7.1 Courses . . . 187
7.7.2 Online Tutorials . . . 187
7.7.3 Books, Other Resources . . . 187
7.8 Chapter Summary . . . 187

8 Analysis of Large Consortium Data with R . . . 189
8.1 About Data . . . 189
8.1.1 Load Required Packages and Example Data . . . 190
8.1.2 Summary of Data . . . 190
8.2 Analysis Approaches . . . 195
8.2.1 Estimate Incidence of Cancer by Person Years . . . 195
8.2.2 Kaplan–Meier Plot of Time to Cancer . . . 197
8.2.3 Cox Models . . . 199
8.2.4 Subgroup Analysis . . . 201
8.2.5 Analysis of Each Study in the Consortium . . . 208
8.2.6 Meta-Analysis Across Studies Based on the Results of Each Study . . . 212
8.2.7 Sensitivity Analysis (Leave One Study Out Validation) . . . 214
8.3 Useful Resources . . . 219
8.4 Chapter Summary . . . 219

9 Analysis of Child Growth Data with R . . . 221
9.1 Data Summary . . . 221
9.1.1 Load Required Packages and Example Data . . . 221
9.1.2 Data Summary by Study Sites . . . 222
9.2 Analysis Approaches . . . 223
9.2.1 Compare Median of Anthropometric Measure for Age by Gender . . . 223
9.2.2 Temporal Trend . . . 228
9.2.3 Comparison with WHO 2007 Growth References . . . 233
9.2.4 Calculate Z-Scores and Prevalence with “Anthro” Package . . . 239
9.2.5 Calculate Prevalence of Z-Score Categories with “anthro” Package . . . 241
9.2.6 Calculate Prevalence of Nutritional Status Manually . . . 242
9.2.7 Calculate Z-Scores Manually . . . 243
9.2.8 Calculate Prevalence from z-Scores Manually . . . 250
9.2.9 WHO LMS Method for Constructing Normalized Growth Curves . . . 253
9.3 Useful Resources . . . 256
9.4 Chapter Summary . . . 257

10 Analysis of Clinical Trial Data with R . . . 258
10.1 Why R for Clinical Trial Data Analysis? . . . 258
10.2 Sample Size—Power Estimation for Clinical Trial . . . 258
10.2.1 Superiority Trials . . . 258

10.2.2 Non-Inferiority Trial . . . 262
10.3 Randomization . . . 265
10.3.1 Simple Randomization . . . 266
10.3.2 Blocked Randomization (Permuted Block Randomization) . . . 266
10.3.3 Stratified Randomization . . . 268
10.4 Examples of Published Clinical Trial Data Analysis Using R . . . 270
10.5 Case Study: Example Analysis of a Vaccine Clinical Trial Data . . . 270
10.5.1 Load Required Packages and Example Data . . . 271
10.5.2 Baseline Summary . . . 271
10.5.3 Time-to-Event Outcome Summary . . . 273
10.5.4 Vaccine Efficacy Cox Model . . . 275
10.5.5 Safety Data Summary . . . 280
10.6 Useful Resources . . . 286
10.6.1 Books, Courses . . . 286
10.6.2 Journal Articles . . . 286
10.7 Chapter Summary . . . 286

11 Analysis of Microbiome Data with "metamicrobiomeR" . . . 287
11.1 Statistical Approaches . . . 287
11.2 Example Data . . . 288
11.3 "metamicrobiomeR" Package Summary . . . 289
11.4 Package Usage . . . 289
11.4.1 Example Differential Analysis: Comparison Between Breastfeeding Statuses . . . 290
11.4.2 Illustration of Meta-Analysis . . . 297
11.5 Useful Resources . . . 304
11.6 Chapter Summary . . . 305

References . . . 306

Index . . . 314

List of Figures and Tables

Figure 1 Barplot example. 18
Figure 2 Histogram example. 19
Figure 3 Plot by group example. 19
Figure 4 Density example. 20
Figure 5 Density by group example. 20
Figure 6 Histogram with density example. 21
Figure 7 Histogram with density example fixed. 21
Figure 8 Histogram with density by group. 22
Figure 9 Scatter plot example. 22
Figure 10 Scatter plot with regression line. 23
Figure 11 Scatter plot by group. 24
Figure 12 Boxplot example. 24
Figure 13 Box plot with dots. 25
Figure 14 Pie chart. 26
Figure 15 Histogram with base R plot. 28
Figure 16 Histogram with base R plot 2. 29
Figure 17 Boxplot number of fecal samples. 35
Figure 18 Violin plot number of fecal samples. 36
Figure 19 Combined violin and box plots. 37
Figure 20 Mean plot with error bar number of fecal samples. 38
Figure 21 Mean plot with 95% CI. 38
Figure 22 Histogram by cohort. 39
Figure 23 whz spaghetti plot. 41
Figure 24 waz spaghetti plot. 41
Figure 25 waz spaghetti plot with fitted lines (linear trends). 42
Figure 26 haz spaghetti plot with fitted lines (linear trends). 43
Figure 27 All-in-one spaghetti plot. 44
Figure 28 All-in-one spaghetti plot for each index. 45
Figure 29 Example normal distribution. 47
Figure 30 Example normal distribution using "*ggplot2*". 48
Figure 31 Example binomial distribution (n = 20, p = 0.4). 50
Figure 32 Q-Q plot. 51
Figure 33 Q-Q plot using "*qqplotr*". 52
Figure 34 Intercept, slope, and residuals plot. 67
Figure 35 Residuals plot. 67
Figure 36 Residuals vs. fitted values plot. 68
Figure 37 Example trend of residuals. 69

Figure 38 Residual plot: equal variance. 69
Figure 39 Residual plot: unequal variance. 70
Figure 40 Histogram normal distribution. 71
Figure 41 Q-Q plot normal distribution. 71
Figure 42 Linear model diagnostic plot using the "performance" package. . 72
Figure 43 Histogram of continuous variable: age. 76
Figure 44 Histogram of continuous variable: bmi. 76
Figure 45 Histogram of continuous variable: children. 76
Figure 46 Histogram of continuous variable: charges. 77
Figure 47 Predicted values by smoker group. 78
Figure 48 Predicted values by region. 81
Figure 49 Residual vs. fitted plot 2. 82
Figure 50 Q-Q plot 2. 82
Figure 51 Residual vs. fitted plot: log transformation. 84
Figure 52 Q-Q plot: log transformation. 84
Figure 53 Observed and predicted values with 95% CI. 87
Figure 54 Observed and predicted values with interaction. 89
Figure 55 Model selection with LASSO. 92
Figure 56 Calibration plot. 97
Figure 57 Model outcome vs. BMI by linear regression. 100
Figure 58 Logistic Regression sigmoid curve. 101
Figure 59 Predicted Probability of outcome vs. BMI. 103
Figure 60 Sigmoid curves of univariable vs. multivariable models. 106
Figure 61 Model comparison using the "*performance*" package. 106
Figure 62 Confusion matrix. 113
Figure 63 Sensitivity by cutoff. 114
Figure 64 Specificity by cutoff. 115
Figure 65 Sensitivity and specificity. 115
Figure 66 Accuracy by cutoff. 116
Figure 67 Receiver Operating Characteristics (ROC) Curve. 118
Figure 68 Radar plot from *compare_performance()*. Source: figure by author. . 120
Figure 69 ROC test data. 122
Figure 70 Sensitivity by cutoff test data. 123
Figure 71 Specificity by cutoff test data. 123
Figure 72 Sensitivity, specificity test data. 124
Figure 73 Accuracy by cutoff test data. 124
Figure 74 Kaplan–Meier plot using base R. 132
Figure 75 Kaplan–Meier plot using "*survminer*" package. 132
Figure 76 Kaplan–Meier curve between groups with "*survminer*". 135

Figure 77 Check Proportional Hazards. 141
Figure 78 Check linearity. 143
Figure 79 Martingale Residuals vs. age. 145
Figure 80 Deviance Residuals. 146
Figure 81 Cumulative Incidence Function. 150
Figure 82 Cumulative hazard function. 151
Figure 83 Cumulative incidence of cancer progression and death by sex. . . 153
Figure 84 waz spaghetti plot. 159
Figure 85 waz mean plot. 160
Figure 86 waz linear regression plot. 162
Figure 87 Coefficient estimates for waz. 177
Figure 88 Random effects. 178
Figure 89 Predicted values “waz” vs. “age.m”. 178
Figure 90 Predicted values of waz vs. cohort. 179
Figure 91 Interaction. 179
Figure 92 Slopes of coefficients. 180
Figure 93 Residuals. 181
Figure 94 Diagnostic plot for mixed model using the “*performance*” package. 182
Figure 95 Kaplan–Meier curve overall pooled data. 199
Figure 96 Kaplan–Meier curve for each of the studies. 209
Figure 97 Meta-analysis univariable Cox model. 213
Figure 98 Meta-analysis multivariable Cox model. 214
Figure 99 Study left out: study5. 215
Figure 100 Study left out: study7. 216
Figure 101 Study left out: study8. 216
Figure 102 Study left out: study2. 216
Figure 103 Study left out: study6. 217
Figure 104 Study left out: study1. 217
Figure 105 Study left out: study3. 217
Figure 106 Study left out: study4. 218
Figure 107 Study left out: study9. 218
Figure 108 Sensitivity analysis summary: univariable. 219
Figure 109 BMI median, 95% CI between study sites. 225
Figure 110 GEE slopes and 95% CI. 231
Figure 111 Trend slopes, 95% CI for males. 232
Figure 112 Trend slopes, 95% CI combined for males. 233
Figure 113 Study data height of males $\leq$ 5 years old vs. WHO references. . . 236
Figure 114 Individual height vs. WHO references. 237
Figure 115 Centile curves for height for age. 255

Figure 116 Fan-chart height for age. 256
Figure 117 Cumulative incidence plot. 278
Figure 118 CIF stratified by risk category. 279
Figure 119 Taxa mean plot phylum. 291
Figure 120 Taxa mean plot family. 292
Figure 121 Meta-analysis nice plot. 300
Figure 122 Meta-analysis forest plot. 303

Table 1 Functions to assess R objects. 9
Table 2 Operations on single vectors. 11
Table 3 Summary table. 30
Table 4 More comprehensive summary table. 32
Table 5 Summary, statistical tests for all variables. 56
Table 6 Summary, statistical tests, adjusted options. 57
Table 7 Data summary table. 74
Table 8 Data summary. 110
Table 9 Cox model with interaction term. 156
Table 10 Mixed model summary table. 176
Table 11 Summary of data by study and overall. 192
Table 12 Summary of data by main exposure. 194
Table 13 Incidence by study. 196
Table 14 Incidence by main exposure status by study. 197
Table 15 Summary of unadjusted Cox model. 200
Table 16 Summary of adjusted Cox model. 201
Table 17 Summary of subgroup analysis results. 206
Table 18 Univariable Cox model results of each study. 210
Table 19 Multivariable Cox model results of each study. 211
Table 20 Data summary by study site. 223
Table 21 Height median, 95% CI of males. 226
Table 22 Height median difference, 95% CI of males. 227
Table 23 Slopes raw table. 230
Table 24 GEE slopes and 95% CI. 230
Table 25 Height classification prevalence and 95% CI. 238
Table 26 Prevalence, 95% CI of stunting. 239
Table 27 Prevalence of wasting and obesity based on z-scores. 243
Table 28 Prevalence of stunting based on z-scores. 243
Table 29 Prevalence, 95% CI obesity, wasting calculated manually. 251
Table 30 Prevalence, 95% CI stunting under five calculated manually. 252
Table 31 Prevalence, 95% CI obesity, thinness calculated manually. 253

Table 32 Prevalence, 95% CI of stunting calculated manually. 253
Table 33 Summary of baseline data. 272
Table 34 Event overall. 273
Table 35 Event by risk. 274
Table 36 Summary of vaccine efficacy. 277
Table 37 Summary of serious adverse events. 283
Table 38 Summary of adverse events by SOC and PT. 285
Table 39 Functions in the “metamicrobiomeR” package. 289

Abbreviations

ACC	Accuracy
AE	Adverse Event
AIC	Akaike Information Criterion
API	Application Programming Interface
AUC	Area Under the Curve
BCa	Bias-corrected and accelerated
BEZI	zero-inflated beta
BIC	Bayesian Information Criterion
BMI	Body Mass Index
CI	Confidence Interval
CHF	Cumulative Hazard Function
CIF	Cumulative Incidence Function
COVID-19	Coronavirus disease 2019
FDR	False Discovery Rate
FN	False Negative
FP	False Positive
GAMLSS	Generalized Additive Model for Location, Scale, and Shape
GEE	Generalized Estimating Equation
GLM	Generalized Linear Models
GLMM	Generalized linear mixed-effects model
HR	Hazard Ratio
ITT	Intention to treat
IQR	Interquartile range
IRR	Incident Rate Ratios
KEGG	Kyoto Encyclopedia of Genes and Genomes
KM	Kaplan–Meier
LASSO	Least Absolute Shrinkage and Selection Operator
LME	Linear mixed-effects models
MAE	Mean Absolute Error
MD	Medical Doctor
mITT	modified intention to treat
MLE	Maximum Likelihood Estimation
MSE	Mean Squared Error
NPV	Negative Predictive Value
OR	Odds Ratio
RD	Risk Difference
RMSE	Root Mean Squared Error
ROC	Receiver Operating Characteristics
RR	Rate (or Risk) Ratio
PhD	Doctor of Philosophy
PFS	Progression-Free Survival
PPV	Positive predictive value
PT	Preferred Term
QIC	Quasi Information Criterion
SAE	Serious Adverse Event

SBP	Systolic blood pressure
SD	Standard Deviation
SE	Standard Error
SHR	Sub-distribution Hazard Ratio
SOC	System Organ Class
TN	True Negative
TP	True Positive
URL	Uniform Resource Locator
USA	United States of America
WHO	World Health Organization

About the Author

Nhan Thi Ho, MD, PhD, got her PhD in (quantitative) epidemiology at Michigan State University, US. Then, she was a researcher and biostatistician at the Oxford University Clinical Research Unit, a visiting scholar in mathematical modeling at Princeton University, US, and a fellow in epidemiology at Columbia University Medical Center (CUMC), US, before joining Vinmec International Hospital in Vietnam as head of data science and bio-medical research methodology. She is experienced in biostatistics for various types of data and is experienced with various statistical software such as R, SAS, and Stata. She is the author and creator of the R package "metamicrobiomeR" for analysis and meta-analysis of microbiome data which feeds the chapter "Analysis of microbiome data with "metamicrobiomeR"" of this book. She is the founder and creator of the WeGrow-Health^{TM} AI-powered application (wegrow-health.com) for child growth tracking, prediction and analysis, which shares some methodology with the chapter "Analysis of Child Growth Data with R" of this book. She is also the primary author of a published book entitled "Clinical Research Methodology—Basic". Alongside her research position, she is also a pediatrician with working experience in major children's hospitals in Vietnam and France. In parallel, she is also an affiliated faculty member in the College of Health Sciences of VinUniversity in Vietnam.

Preface

This book includes a series of hands-on practical introductory guides for the analysis of bio-medical data using R. It is intended mainly for researchers and students in the fields of bio-medical sciences, public health, and clinical research. It will provide readers with basic statistical programming skills in R, together with practical data analysis skills and applied statistical knowledge, not only of common biostatistical topics but also of some specialized bio-medical–clinical research topics and data types. The knowledge and skills contained in this series are also applicable across other statistical software.

The author's path, moving between clinical practice and quantitative research, between high-resource academic settings and the realities of health research in lower–middle-income countries, made it clear that the gap between have statistical knowledge and being able to apply it fluently was a genuine obstacle to scientific progress. This book aims to close that gap, at least in part, for bio-medical researchers. As R is a free software that keeps growing with contribution from the scientific community, its latest version is accessible by anyone, bridging the gap between low- and high-resource settings.

The central argument of this book is that statistical programming competence and applied analytical judgment are best acquired together, through worked examples drawn from real bio-medical problems, rather than through formal statistical theory studied in isolation from practice. Each chapter is structured around a class of bio-medical research questions—What is the relationship between two continuous variables? How do we compare survival across treatment arms? How do we account for repeated measurements within the same patient?—and develops both the statistical reasoning and R implementation skills needed to answer them. The reader is not asked to run code whose purpose has not been explained, and the explanation is not purely syntactic.

Several chapters of this book draw on previously published research conducted by the author and their colleagues. The clinical trial case study in Chapter 10 uses data generated to reproduce key features of a multi-center COVID-19 vaccine trial in which the author served as a core researcher and data scientist; the original study was published in Nature Communications, and the use of mock-up example data for educational purposes here requires no additional permissions. The microbiome analysis workflows in Chapter 11 are based on methods developed and published by the author, and the "metamicrobiomeR" package on which they rely is freely available under an open-source license on R CRAN. The child growth analysis framework in Chapter 9 draws on the author's published work using World Health Organization 2006 and 2007 growth references.

Throughout, the book's philosophy is one of learning by doing. All code is fully reproducible, and the output of each code block is shown immediately beneath it, allowing readers to follow the analysis step by step before adapting it to their own data. Where the statistical concepts underlying a procedure are non-trivial, e.g., mixed-effects model specification, competing-risks survival analysis, etc., the necessary conceptual foundations are laid before R implementation is introduced. The book assumes a basic familiarity with core statistical ideas (means, variances, hypothesis testing) but requires no prior experience with R or with any other programming language.

It is the author's sincere hope that this book lowers the barrier between a research question and a rigorous, reproducible answer that is accessible to anyone willing to open R and work through an example. Bio-medical science advances most reliably when the researchers conducting it have full command of their data.

Nhan Thi Ho
Author

Acknowledgments

The author would like to thank Vinmec International Hospital for their support during the organization of the course “Bio-Medical Data Analysis with R: Intro Series” and during the preparation and publication of this book.

1. Introduction

1.1. About This Book

Data analysis is an essential part of bio-medical research. To turn research data into meaningful scientific insight and a report, researchers need a powerful and flexible analytical tool. R is a free, open-source statistical analysis software that has been extensively used across many fields of bio-medical research. It is well-known not only because it is free but also because of its power, its graphics capabilities, and its vast ecosystem of community-built packages that provide specialized functions for many statistical challenges. For a bio-medical researcher, being able to use R is a "handy skill" to take control of one's data, from cleaning and visualization to complex modeling and reporting. This book is designed to be a hands-on practical guide to using R for the specific types of data and analytical questions that bio-medical researchers face every day.

This book's practical, "learning-by-doing" approach stems from its origin. During the COVID-19 restrictions in 2020, Nhan Thi Ho, MD, PhD, developed and delivered an online course organized by Vinmec International Hospital: "Bio-Medical Data Analysis with R: Intro Series". The course's hands-on, practical focus attracted hundreds of participants from Vietnam and other countries, and it was highly evaluated by its participants. This book is a substantive editing of the lectures of the aforementioned course. This book aims to sustain and deliver the course values to a much larger global audience. Furthermore, the novel contribution of this book, which differentiates it from other books about R, is the four chapters on four special topics regarding the analysis of various types of bio-medical and clinical data and study designs, which the author Nhan Thi Ho has done and published. The philosophy of this book is not to teach theoretical statistics but to provide readers with the basic statistical programming skills in R alongside the practical data analysis skills they need to conduct and report their own research.

1.2. Target Audience

This book is designated primarily for bio-medical and clinical students and researchers. This may include biologists, physicians, and clinicians involved in research; epidemiologists and public health professionals; and undergraduate and graduate students in health sciences, biostatistics, or epidemiology. It is also aimed at students and researchers who may be using other statistical software (like SPSS, Stata, or SAS) and wish to transition to R, or who are new to programming and need a practical, applied introduction.

The book is designed as an "Intro Series". It assumes the audience has a basic understanding of core statistical concepts but requires no prior R programming experience. It builds R skills from the ground up, starting with R basics and moving progressively into more advanced, real-world bio-medical and clinical data topics. The knowledge and skills provided are also useful and applicable for those who may use other statistical software.

1.3. Main Contents of This Book

The book is structured to build skills progressively, starting with the fundamentals of R before moving into using R for core statistical methods and more advanced, specialized topics that bio-medical researchers encounter.

Part 1: The Foundation for R: The book begins with an introduction to the R environment, RStudio, and R Markdown, followed by the basics of R programming, data manipulation, data description, and visualization (Chapter 2).

Part 2: Using R for Core Statistical Methods: This section covers using R for fundamental statistical concepts, distributions, basic statistical tests, sample size—power estimation (Chapter 3), and the most common and essential statistical models in bio-medical research: Linear Regression (Chapter 4), Logistic Regression (Chapter 5), Survival (Time-to-Event) Analysis (Chapter 6), and Longitudinal Data Analysis (Chapter 7).

Part 3: More Advanced Bio-Medical Research Topics: This section covers using R for Analysis and Meta-analysis of Large Consortium Data (Chapter 8), Analysis of Child Growth Data (Chapter 9), Analysis of Clinical Trial Data (Chapter 10), and Analysis and Meta-analysis of Microbiome Data with the "metamicrobiomeR" package (Chapter 11).

After reading and following the examples in this book, readers will gain the applied statistical knowledge and practical skills necessary to analyze and report their own bio-medical and clinical research data using R.

2. Introduction to R

This chapter introduces bio-medical researchers to R and R packages. It covers basic data handling, description, summary, and visualization. It also includes a section for practice exercises and practice review to help readers acquire the skills smoothly. Readers do not need prior programming skills in R or other software to start learning.

2.1. What Is R?

R is a free software environment for statistical computing and graphics. R is open source and thus is continuously being improved by a large community of developers. It has a huge ecosystem of community-built packages that help tackle various statistical matters. To download, install, and learn more, go to R cran [1].

2.2. RStudio

RStudio is an integrated development environment (IDE) for R. It includes a console, a syntax-highlighting editor that supports direct code execution, as well as tools for plotting, history, debugging, and workspace management. To download, install, and learn more, go to the RStudio page [2].

2.3. Rmarkdown

R Markdown is a file format for making dynamic documents with R.

It turns your analyses into high-quality reproducible documents, reports, presentations, and dashboards. Learn more at the Rmarkdown page [3].

2.4. Base R Example

R comes with a set of base packages that provide fundamental functions. Here are some examples of how to use R for basic calculations, data generation, and summary statistics.

You can use R as a calculator to perform simple arithmetic operations directly in the R console.

```
# Use R as a calculator:
((2+2)*3)/2
```

```
## [1]  6
```

R has a vast number of built-in functions. For example, the *sqrt()* function calculates the square root of a number.

```
sqrt(9)
```

```
## [1]  3
```

```
mean(c(2,4,5,9))
```

```
## [1]  5
```

We can check how to use a function by adding "?" in front of a function. For example, use the function *rnorm()* to generate data, *summary()* to summarize the data, and *t.test()* to perform a *t*-test on the data.

```
#Check how to use a function
?rnorm
#use the function
data1<- rnorm(n = 30,mean = 1.5, sd = 0.5)
summary(data1)
```

```
##    Min.    1st Qu.   Median    Mean    3rd Qu.   Max.
##    0.6094  1.1069    1.3362    1.4283  1.7080    2.3906
```

```
data2<-rnorm(n = 35, mean = 1.8, sd = 1)
summary(data2)
```

```
##    Min.     1st Qu.   Median    Mean    3rd Qu.   Max.
##    -1.335   1.060     1.742     1.700   2.432     3.383
```

Basic statistical test may be done easily in R, for example we can do a *t*-test:

```
?t.test
t.test(x = data1, y = data2, alternative = "two.sided", var.equal = FALSE, conf.level
       = 0.95)
```

```
##
## Welch Two Sample t-test
##
## data: data1 and data2
## t = -1.3973, df = 48.737, p-value = 0.1687
## alternative hypothesis: true difference in means is not equal to 0
## 95 percent confidence interval:
##      -0.6633834    0.1192729
## sample estimates:
## mean of x mean of y
##      1.428314      1.700369
```

2.5. Install, Load, Update R Packages

R's functionality can be extended by installing R packages.

- In RStudio: You can install packages by navigating to Packages > Install and then typing the package name.
- From R console: You can use the *install.packages()* function by typing: *install.packages("packagename")*.

The example below shows how to install and load a single package, which is my R package "*metamicrobiomeR*" [4]:

```
#install and load package for use
install.packages("metamicrobiomeR")
library("metamicrobiomeR")
```

The example below shows how to install and load multiple packages. This code creates a vector of package names ("*knitr*" [5], "*rmarkdown*" [6], "*installr*" [7], "*haven*" [8], "*readr*" [9], "*yaml*" [10], "*tidyverse*" [11], "*kableExtra*" [12]) and then uses *lapply()* to load each of them:

```
#install multiple packages
Packages <- c("knitr","rmarkdown","installr","haven", "readr","yaml","tidyverse", "kableExtra")
install.packages(Packages)
#load multiple packages
lapply(Packages, library, character.only = TRUE)
```

The commands below are useful for updating packages:

```
# download and install newest version of the package
update.packages()
# shows which packages have updates
old.packages()
# looks for new packages that are not already installed
new.packages()
```

The commands below may be used to get help for package, function, or data:

```
#help for package
help(package = "metamicrobiomeR")
#help for function
help("taxa.compare", package = "metamicrobiomeR")
#or load package and then get help for a function
library("metamicrobiomeR")
?taxa.compare
#or get help for a function of a package by
?metamicrobiomeR::taxa.compare
#help for data and display data help page
help(tabsex4, package = "metamicrobiomeR")
```

2.6. Working with R Objects

R has several fundamental data structures, also known as objects.

2.6.1. Vector

A vector is a collection (one-dimensional array) of elements of the same data type:

```
# A vector (denoted by "c") of numeric elements
x<-c(1,3,4,7,8,10)
x
```

```
## [1]  1 3 4 7 8 10
```

```
# A vector of character elements
a<-c("red","blue","white")
a
```

```
## [1]  "red" "blue" "white"
```

```
#creating  vector
seq(1,10, by = 2)
```

```
## [1]  1 3 5 7 9
```

```
rep("boring",3)
```

```
## [1]  "boring" "boring" "boring"
```

2.6.2. Matrix

A matrix is a two-dimensional representation of a vector (two-dimensional rectangular data set) where all elements are of the same data type.

The example code below reshapes the vector x above into a matrix with two rows:

```
y<-matrix(x, y, nrow = 2)
```

```
##      [,1] [,2] [,3]
## [1,]    1    4    8
## [2,]    3    7   10
```

2.6.3. Array

An array, similar to a matrix, is an n-dimensional representation of a vector. This code creates a three-dimensional array from the vector x:

```
z <- array(x, dim = c(1, 3, 2)); z
```

```
## , , 1
##
##      [,1] [,2] [,3]
## [1,]    1    3    4
##
## , , 2
##
##      [,1] [,2] [,3]
## [1,]    7    8   10
```

2.6.4. List

A list is a collection of "bins", a versatile object that can contain any kind of R object of any data types and structures. This code creates a list containing a vector, a matrix, an array, and another vector:

```
mylist <- list(x, y, z, a); mylist
```

```
##    [[1]]
##    [1] 1 3 4 7 8 10
##
##    [[2]]
##          [,1]  [,2]  [,3]
##    [1,]  1     4     8
##    [2,]  3     7     10
##
##    [[3]]
##    , , 1
##
##          [,1]  [,2]  [,3]
##    [1,]  1     3     4
##
##    , , 2
##
##          [,1]  [,2]  [,3]
##    [1,]  7     8     10
##
##
##    [[4]]
##    [1]   "red" "blue""white"
```

2.6.5. Data Frame

A data frame is a list in tabular form where each "bin" contains a data vector of the same length. A data frame is the usual tabular data set familiar to researchers and students. Each row is a record, and each column ("bin") is a field. Both matrix and data frame are tabular data types.

- All columns in a matrix must have the same data type (numeric, character, etc.) and the same length.
- A data frame is more general than a matrix in that different columns can have different modes (numeric, character, or factor, etc.). Just like a table in a database or an Excel sheet.

This code creates a data frame with three columns: smoker, cancer, and quant, and then displays the first four rows.

```
smoker <- c("Y", "Y", "N", "N", "Y", "Y", "Y", "N", "Y")
cancer <- c("Y", "N", "N", "Y", "Y", "Y", "N", "N", "Y")
quant<-c(1,2,3,4,5,6,7,8,9)
mydf <- data.frame(smoker, cancer, quant)
mydf[1:4,] # display rows 1 to 4
```

```
##   smoker cancer quant
## 1      Y      Y     1
## 2      Y      N     2
## 3      N      N     3
## 4      N      Y     4
```

2.6.6. Assessing the Structure of Data Objects

Understanding the structure of an R object is crucial for effective data manipulation and analysis. The *str()* function provides a concise summary of the internal structure of any R object, including its class, dimensions, and the data type of its contents. The output helps quickly identify the variables and their types, which is essential before performing any operation (e.g., ensuring numeric variables are actually stored as num).

```
data("tabsex4", package = "metamicrobiomeR")
str(tabsex4)
```

```
## 'data.frame': 701 obs. of 23 variables:
## $ id: chr "k__bacteria.p__actinobacteria" "k__bacteria.p__bacteroidetes" "k__bacteria.p__firmicutes" "k__
bacteria.p__fusobacteria" ...
## $ Estimate.genderMale                : num 0.2269 -0.1549 -0.056 -0.0648 -0.0887 ...
## $ Std. Error.genderMale              : num 0.121 0.119 0.104 0.282 0.118 ...
## $ t value.genderMale                 : num 1.874 -1.305 -0.537 -0.23 -0.751 ...
## $ Pr(>|t|).genderMale                : num 0.0619 0.1931 0.5919 0.8186 0.4532 ...
## $ Estimate.bfNon_exclusiveBF         : num -0.3688 0.2557 0.2362 -0.0605 0.3699 ...
## $ Std. Error.bfNon_exclusiveBF       : num 0.139 0.136 0.116 0.364 0.134 ...
## $ t value.bfNon_exclusiveBF          : num -2.647 1.887 2.035 -0.166 2.764 ...
## $ Pr(>|t|).bfNon_exclusiveBF         : num 0.00857 0.0602 0.04276 0.86812 0.00606 ...
## $ Estimate.bfNo_BF                   : num -1.329 0.135 0.356 -0.403 1.153 ...
## $ Std. Error.bfNo_BF                 : num 0.412 0.499 0.392 1.083 0.407 ...
## $ t value.bfNo_BF                    : num -3.223 0.27 0.908 -0.372 2.834 ...
## $ Pr(>|t|).bfNo_BF                   : num 0.00141 0.7873 0.36478 0.7103 0.0049 ...
## $ Estimate.age.sample                : num 0.1251 0.0242 0.0115 0.0285 -0.1348 ...
## $ Std. Error.age.sample              : num 0.0377 0.0367 0.0319 0.0857 0.038 ...
## $ t value.age.sample                 : num 3.318 0.66 0.359 0.333 -3.549 ...
## $ Pr(>|t|).age.sample                : num 0.001022 0.510068 0.719525 0.73934 0.000448 ...
## $ pval.adjust.genderMale             : num 0.48 0.899 0.961 0.961 0.961 ...
## $ pval.adjust.bfNon_exclusiveBF      : num 0.059 0.2333 0.1767 0.9976 0.0448 ...
## $ pval.adjust.bfNo_BF                : num 0.0219 1 1 1 0.0468 ...
## $ pval.adjust.sample                 : num 0.00786 0.82595 0.85375 0.85375 0.00505 ...
## $ study                              : chr "Subramanian et al. 2014 (Bangladesh)" "Subramanian
et al. 2014 (Bangladesh)" "Subramanian et al. 2014
(Bangladesh)" "Subramanian et al. 2014 (Bangladesh)" ...
## $ pop                                : chr "Bangladesh" "Bangladesh" "Bangladesh"
"Bangladesh" ...
```

2.6.7. Useful Functions to Assess R Objects

R provides a variety of functions to inspect the characteristics of data objects (Table 1).

Table 1. Functions to assess R objects.

Function	Returns
str	summary of data object structure
attributes	list with data object attributes
mode	mode of object
typeof	type of object; similar to mode but includes double and integer, if applicable
length	length of object
class	class of object, if it exists
dim	vector with object dimensions, if applicable
nrow	number of rows, if applicable
ncol	number of columns, if applicable
dimnames	list containing vectors of names for each dimension, if applicable
rownames/colnames	vector of row names/column names of a matrix-like object
head/tail	view the first (head) or last (tail) components of R objects
names	vector of names for the list (for a data frame returns field names)
row.names	vector of row names for a data frame

Source: Author's compilation based on data from the R Manuals [1].

2.6.8. Understanding Vectors

Operation of vectors: When you perform an operation on two vectors of the same length, the operation is applied element-wise.

Of note, in R, the semicolon (;) serves as a command separator. It allows you to write multiple independent lines of code on a single horizontal line.

```
#Generate vector
x1 <- 5; y1 <- c(10, 20, 30)
x2 <- c(5, 5, 5); y2 <- c(10, 20, 30)
x1 * y1
```

```
## [1]  50 100 150
```

```
x2 * y2
```

```
## [1]  50 100 150
```

Boolean Queries and Subsetting of Vectors: You can use logical operators (<, >, <=, >=, == for equals, ! = for not equals, & for AND, | for OR) to create a logical vector and then use that vector to select a subset of elements from another vector. You can assign names to the elements of a vector, which can make the output more informative.

```
lecturer <- c('Nhan', 'Hien', 'Phuong', 'Tien', 'Duy')
lecturer
```

```
## [1]  "Nhan" "Hien" "Phuong" "Tien" "Duy"
```

```
ms.ages <- c(41, 39, 33, 26, 25)
ms.ages
```

```
## [1]  41 39 33 26 25
```

```
ms.ages >= 30 # logical vector for lecturers with ages >= 30
```

```
## [1]  TRUE TRUE TRUE FALSE FALSE
```

```
thirtysomething <- (ms.ages >= 30) & (ms.ages < 40) lecturer[thirtysomething]  #
indexing  using  logical
```

```
## [1]                          "Hien" "Phuong"
```

```
#naming  vector
names(ms.ages)<-lecturer
ms.ages
```

```
##  Nhan  Hien  Phuong  Tien  Duy
##  41    39    33      26    25
```

Note that it is important to distinguish between the logical operators & and &&, as well as | and ||, because they behave differently in terms of vectorization and evaluation. The operators && and || are not vectorized; they are mainly used in control-flow statements, and they also implement short-circuit evaluation, improving efficiency and preventing unnecessary computations. They evaluate only the first element of each logical vector and return a single TRUE or FALSE value. For &&, if the first condition is FALSE, the second condition is not evaluated. For ||, if the first condition is TRUE, the second condition is not evaluated. Below are some examples.

```
x <- c(TRUE, FALSE, TRUE)
y <- c(TRUE, TRUE, FALSE)
x && y
```

```
## [1]  TRUE
```

Only the first elements are evaluated as TRUE && TRUE $\rightarrow$ TRUE. Similarly for || as TRUE || TRUE $\rightarrow$ TRUE.

```
x || y
```

```
## [1]  TRUE
```

In the example below, the function *stop()* is never executed, because the first condition already determines the result.

```
FALSE && stop("Error")
```

```
## [1] FALSE
```

Operations on Single Vectors: Below are some operations on single vectors (Table 2).

Table 2. Operations on single vectors.

Function	Description	Function	Description
sum	summation	rev	reverse order
cumsum	cumulative sum	order	order
diff	x[i + 1] − x[i]	sort	sort
prod	product	rank	rank
cumprod	cumulative product	sample	random sample
mean or *median* or *mad*	mean or median or median absolute deviation	quantile	percentile
min or *max*	minimum or maximum (What is the lowest or highest value in this whole group?)	var	variance, covariance
pmin or *pmax*	parallel minimum or parallel maximum of the input values (For each row/index, which value is lower or higher?)	sd	standard deviation
IQR	interquartile range	table	tabulate character vector
range	range	xtabs	tabulate factor vector

Source: Author's compilation based on data from the R Manuals [1].

2.7. Managing Bio-Medical Data in R

This section focuses on methods for inputting your data into R and preparing it for use.

2.7.1. Entering Data

For small, simple datasets, you can enter data directly into R using functions like *c()* for vectors and *data.frame()* to combine them into a data frame.

```
subjname <- c('Lop', 'Chung', 'Minh')
subjno  <-  1:length(subjname)
age <- c(34, 56, 56)
sex <- c('Male', 'Male', 'Female')
dat <- data.frame(subjno, subjname, age, sex); dat
```

```
##      subjno  subjname  age  sex
## 1    1       Lop       34   Male
## 2    2       Chung     56   Male
## 3    3       Minh      56   Female
```

2.7.2. Importing Data from a File

The example below imports the dataset of Bangladeshi children [13] from the local directory:

```
#your path to dataset
fdir1_local<-"./metamicrobiomeR-master/inst/extdata/QIIME_outputs/Bangladesh/"

#Read in text file
dat1_local<-read.delim(paste0(fdir1_local,
                              "Subramanian_et_al_mapping_file.txt"),
                       header = TRUE, sep = "\t")
```

fdir1_local stores the file path to the directory. *paste0()* concatenates the directory path and the file name to create the full file path string. *read.delim()* is a base R function used to read delimited text files (*header* = *TRUE* specifies that the first row contains column names; *sep* = "\t" specifies that the data fields are separated by a tab character).

Note that *paste0()* is a simplified version of *paste()* that automatically sets the separator to an empty string. *paste0(...)* is equivalent to *paste(..., sep = "")*.

The example below imports the same dataset from GitHub:

```
# your path to dataset
# break down the url for readability
base_url <- "https://raw.githubusercontent.com/nhanhocu/metamicrobiomeR/master/" data_path <-
"inst/extdata/QIIME_outputs/Bangladesh/"
fdir1 <- paste0(base_url, data_path)
# Read in text file
dat1<-read.delim(paste0(fdir1,"Subramanian_et_al_mapping_file.txt"),
                 header = TRUE, sep = "\t")
```

The example below imports another dataset from URL:

```
base_url <- "https://stats.idre.ucla.edu/wp-content/uploads/2016/02/"
data_use <- "test-1.csv"
#Read in csv file
test.csv <- read.csv(paste0(base_url, data_use), header = TRUE)
```

One may use a more general function, *read.table()*, which reads data from a wide range of text formats, including space-separated, tab-delimited, or comma-separated files with syntax as below.

```
fileread <- read.table(file, header = FALSE, sep = "", quote = "\"",
na.strings = "NA", stringsAsFactors = FALSE, ...)
```

In which, the *file* argument specifies the location of the data file, the *header* argument indicates whether the first row of the file contains column names, the *sep* argument defines the separator used between values (" " for space, "\t" for tab, "," for comma), the quote argument specifies characters used to quote strings (e.g., quote = "\"' " meaning that both double quotes (") and single quotes (') may surround character strings), the *na.strings* argument defines which values should be interpreted as missing values (e.g., *na.strings* = *c*("*NA*", ".", "")), and the *stringsAsFactors* argument specifies whether character variables are converted into factors automatically.

2.7.3. Some Other Data Import Functions

The "*rio*" [14] package provides a convenient, single-function *import()* that can handle various file formats (CSV, Excel, SAS, or Stata, etc.), making data import much easier! All you need is *import()*!

```
# May install package "rio" if not yet available by: install.packages("rio") and
# support packages: install.packages(c('csvy', 'feather', 'fst', 'hexView', 'readODS', 'rmatio'))
library(rio)
test1<-import(paste0(fdir1,"Subramanian_et_al_mapping_file.txt"))
```

We can import one specified excel sheet using *import()* and import the whole excel files with multiple sheets using *import_list()*:

```
#import always get only 1 dataframe
test<-import(paste0(fdir1,"nature13421-s2.xlsx"), sheet = 1, skip = 1)

#Import multiple sheets of an excel file and create a list of dataframes
list1<-import_list(paste0(fdir1,"nature13421-s2.xlsx"))
# can check the structure of list1 by: str(list1)
#or view the first component list1 by: View(list1[[1]])
```

We can also import multiple files from a local directory by using *Sys.glob()* to find specified files in a directory and *import_list()* to import all of them:

```
# Get folder path
fdir1_local<-"./metamicrobiomeR-master/inst/extdata/QIIME_outputs/Bangladesh/"

#Sys.glob() allows you to find files using a regular expression
dir1_local<-Sys.glob(paste0(fdir1_local,"alpha_div_collated/*.txt))
list2<-import_list(dir1_local)
str(list2)
```

The codes below will import those same files from GitHub using the GitHub API. First, we construct the GitHub API URL for the directory contents. Then we make a GET request to the API and check if the request was successful. Then we parse the JSON response into a data frame and filter the data frame to find files matching the pattern. Then we extract the direct download URLs, '*final_urls*', for these files and use this '*final_urls*' vector to import these files using *import_list()*. We need to use three R packages for this task, along with the "*rio*" package mentioned above: "*httr*" [15], "*jsonlite*" [16], and "*dplyr*" [17].

```
# Install necessary packages if you don't have them
install.packages ("httr")
install.packages ("jsonlite")
install.packages ("dplyr")
library(httr)
library(jsonlite)
library(dplyr)

# Construct the GitHub API URL for the directory contents
# The format is:
# https://api.github.com/repos/OWNER/REPO_NAME/contents/PATH_TO_FOLDER
owner <- "nhanhocu"
repo <- "metamicrobiomeR"
path <- "inst/extdata/QIIME_outputs/Bangladesh/alpha_div_collated"

api_url <- paste0("https://api.github.com/repos/",
                                owner, "/", repo, "/contents/", path)

# Make a GET request to the API
response <- GET(api_url)

# Check if the request was successful
if (http_status(response)$category ! = "Success") { stop("Failed to retrieve
    directory contents from GitHub API.")
}

# Parse the JSON response into a data frame
# The content is in the body of the response, which we extract and parse
file_list_df <- fromJSON(content(response, "text"), flatten = TRUE)

# Filter the data frame to find files matching the pattern "*.txt"
# We also want to ensure we only get files, not subdirectories
# Note that we use the pipe operator %>% (from the magrittr package,
# widely used in the tidyverse) is a mechanism for function chaining.
# It passes the result of the expression on the left-hand side (LHS) as
# the first argument of the function on the right-hand side (RHS)
# See Section 2.7.5.1 Pipes
txt_files_df <- file_list_df %>%
    filter(type == "file", endsWith(name, ".txt"))

# Extract the direct download URLs for these files
# The 'download_url' column contains the raw file links
final_urls <- txt_files_df$download_url

# Check the resulting list of URLs
final_urls

# You can now use this 'final_urls' vector to import the files, for example:
library(rio)
list2_github <- import_list(final_urls)
```

2.7.4. Exporting Data

Once your data is clean and prepared, you will often need to save it for later use or sharing.

```
#save one dataframe as an csv file
write.csv(dat1,file = paste(fdir1,"dat1.csv",sep = ""), row.names=FALSE)

#save multiple data objects
save(dat1,list1,list2, file = paste(fdir1,"manyfile.rda", sep = ""))

#load saved files
print(load(paste(fdir1,"manyfile.rda", sep = "")))
```

2.7.5. Basic Data Manipulation with "tidyverse"

"*tidyverse*" includes a collection of open-source packages for the R programming language that share a similar design philosophy, grammar, and data structures for tidy data [11]. The key packages include "*dplyr*" [17] for data manipulation and "*ggplot2*" [18] for visualization.

Pipes

The pipe operator %>% (from the "*magrittr*" package, widely used in the "*tidyverse*") is a mechanism for function chaining. It passes the result of the expression on the left-hand side (LHS) as the first argument of the function on the right-hand side (RHS). The pipe operator (%>%) takes the output of a command and feeds it as the input of another command. This allows you to chain multiple data manipulation steps together in a clear, linear, and readable sequence.

```
library(tidyverse)
#pipe from library(magrittr), loaded along with other packages
9%>% sqrt %>% '+'(8)
```

```
## [1]  11
```

Of note is the pipe operator |>, which was introduced in base R (version 4.1.0). It serves as a way to chain multiple function calls together in a sequence that reads from left to right, rather than "inside out". The pipe operator |> takes the result of the expression on its left and passes it as the first argument to the function on its right.

```
#Harder to read: you have to read from the inside to the outside
round(sqrt(mean(c(1, 5, 10))), 2)
#Easier to read: "Take the vector, THEN mean, THEN sqrt, THEN round"
c(1, 5, 10) |> mean() |> sqrt() |> round(2)
```

Subset Data

Subset data by column and row: The "*dplyr*" package provides intuitive functions for filtering rows with *filter()* and selecting columns with *select()*:

```
dat2a<-test %>%
slice(1:50) %>%
  select(-starts_with("..."))
```

We can change column names with *rename_at()*:

```
#change column name
colnames(dat2a)<-tolower(colnames(dat2a))
oldname<-colnames(dat2a)
newname<-c("child.id",
                    "family.id",
                    "cohort",
                    "gender", "zygosity",
                    "whz","waz","haz",
                    "age.first.fe",
                    "age.last.fe",
                    "no.fe.sam",
                    "sam.int.mean",
                    "month.ebf", "age.first.solid",
                    "no.diarhea.yr",
                    "pc.days.diarrhea",
                    "frac.sam.atb.7day", "set")
dat2a<-dat2a %>% rename_at(vars(oldname), ~newname)
colnames(dat2a)
```

We can also subset data by row:

```
dat.tw<-dat2a %>% filter(cohort == "Healthy Twins & Triplets")
#more filter
dat.tw<-dat2a %>%
  filter(cohort == "Healthy Twins & Triplets")%>% filter(!is.na(zygosity))%>%
  filter(between(whz,-1,1))
#or
dat.tw<-dat2a %>%
  filter(cohort == "Healthy Twins & Triplets" &
                    !is.na(zygosity) &
                    between(whz,-1,1))
```

We can also select columns:

```
#Base R
#Select by position (cautious!)
subdat<-dat2a[,c(1,4:6)]
#or select specific column names
subdat<-dat2a[,c("child.id","gender","zygosity","whz")]

#pipes and select()
subdat<-dat2a %>% select(c(1,4:6))
subdat<-dat2a  %>%  select(child.id,gender,zygosity,whz)
```

Or discard columns:

```
#Base R
subdat<-dat2a[,-c(1,4:6)]
subdat<-dat2a[,colnames(dat2a)[!colnames(dat2a) %in%
                                  c("child.id", "gender",
                                    "zygosity", "whz")]]
#or with tidyverse
subdat<-dat2a %>% select(-c(1,4:6))
subdat<-dat2a  %>%  select(-c("child.id", "gender",
                              "zygosity", "whz"))
```

Or rename columns with *rename()*:

```
#Base R
colnames(dat2a)[colnames(dat2a) == "child.id"]<-"childid"
#with tidyverse
dat2a<-dat2a %>%
        select(childid = child.id, everything()) dat2a<-dat2a %>%
        rename(child.id = childid)
```

Editing Data

The *mutate()* function from the "*dplyr*" package is used to add new variables or modify existing ones.

```
#initial look at data
summary(dat2a)
```

```
##child.id          family.id          cohort
##Length: 50        Length: 50         Length: 50
##Class: character  Class: character   Class: character
##Mode: character   Mode: character    Mode: character
##
##
##
##gender            zygosity           whz
##Length: 50        Length: 50         Min.: −1.73842
##Class: character  Class: character   1st Qu.: −0.88598
##Mode: character   Mode: character    Median: −0.47063
##                                     Mean: −0.42223
##                                     3rd Qu.: −0.03458
##                                     Max.: 1.20133
##waz               haz                age.first.fe
##Min.: −3.7555     Min.: −4.6586      Min.: 1.00
##1st Qu.: −2.6037  1st Qu.: −3.4314   1st Qu.: 5.00
##Median: −1.5116   Median: −1.5749    Median: 6.00
##Mean: −1.5148     Mean: −1.8570      Mean: 9.78
##3rd Qu.: −0.71893rd Qu.: −0.7189     3rd Qu.: 9.75
##Max.: 1.6698      Max.: 1.6698       Max.: 37.00
##age.last.fe       no.fe.sam          sam.int.mean
##Min.: 286.0       Min.: 9.00         Min.: 22.56
##1st Qu.: 455.0    1st Qu.: 18.00     1st Qu.: 28.29
##Median: 700.5     Median: 21.00      Median: 30.48
##Mean: 600.6       Mean: 19.92        Mean: 31.34
##3rd Qu.: 706.0    3rd Qu.: 23.00     3rd Qu.: 33.27
##Max.: 738.0       Max.: 26.00        Max.: 42.29
##month.ebf         age.first.solid    no.diarhea.yr
##Min.: 0.0000      Min.: 1.216        Min.: 0.000
##1st Qu.: 0.1971   1st Qu.: 4.920     1st Qu.: 1.271
##Median: 1.0348    Median: 6.932      Median: 2.916
##Mean: 2.0716      Mean: 6.690        Mean: 2.971
##3rd Qu.: 3.2523   3rd Qu.: 8.361     3rd Qu.: 4.054
##Max.: 7.3260      Max.: 9.790        Max.: 8.814
##pc.days.diarrhea  frac.sam.atb.7dayset
##Min.: 0.000       Min.: 0.00000      Length: 50
##1st Qu.: 1.163    1st Qu.: 0.08083   Class: character
##Median: 2.742     Median: 0.11882    Mode: character
##Mean: 3.125       Mean: 0.18328
##3rd Qu.: 4.314    3rd Qu.: 0.28373
##Max.: 10.826      Max.: 0.50000
```

```
dat2a<-dat2a %>%
  mutate(zygosity = replace(zygosity,zygosity == "NA","singleton")) %>%
  mutate(zygosity = replace(zygosity,zygosity == "not tested",NA))
```

2.8. Basic Data Visualization

Data visualization is an essential step in bio-medical data analysis. The "*ggplot2*" package (part of "*tidyverse*") is the most popular and powerful tool for creating publication-quality graphics in R. "*ggplot2*" uses a grammar of graphics that builds plots layer by layer [18]. While "*ggplot2*" is the key package of the "*tidyverse*", it is a standalone package with its own logic based on the grammar of graphics. "*ggplot2*" can be installed and loaded on its own to create high-quality visualizations without the other 20+ "*tidyverse*" packages. The reason they are so often paired together is that "*ggplot2*" is designed to work best with tidy data, which are supported by other packages (e.g., "*dplyr*" or "*tidyr*") in "*tidyverse*". Below are some examples of basic data visualization with "*ggplot2*".

2.8.1. Barplot

We can create a barplot with *geom_bar()* (Figure 1):

```
#One may load "ggthemes" package for some extra themes, geoms, and scales for 'ggplot2'. #It is
not necessary for basic plots: library(ggthemes)
ggplot(dat2a, aes(gender))+
  geom_bar()
```

Figure 1. Barplot example. Source: Figure by the author.

2.8.2. Histogram

We can create a histogram with *geom_histogram()* (Figure 2):

```
ggplot(dat2a, aes(whz))+
  geom_histogram(bins = 10)
#May use breaks instead of bins to fully control bin edges
ggplot(dat2a, aes(whz))+
  geom_histogram(breaks = seq(min(dat2a$whz, na.rm = TRUE),
                              max(dat2a$whz, na.rm = TRUE), length.out = 11)) #bins + 1
```

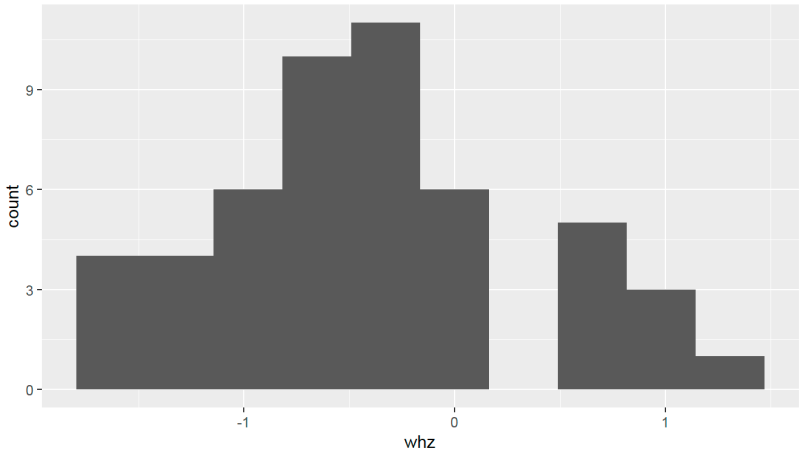


Figure 2. Histogram example. Source: Figure by author.

We can create a plot by group to compare the distribution across a categorical group (e.g., "gender") by using the *fill* aesthetic (Figure 3).

```
ggplot(dat2a,aes(whz,fill = gender))+
    geom_histogram(bins = 10, alpha = 0.5, position = "identity")
#bins are used in geom_histogram() for simplicity
```

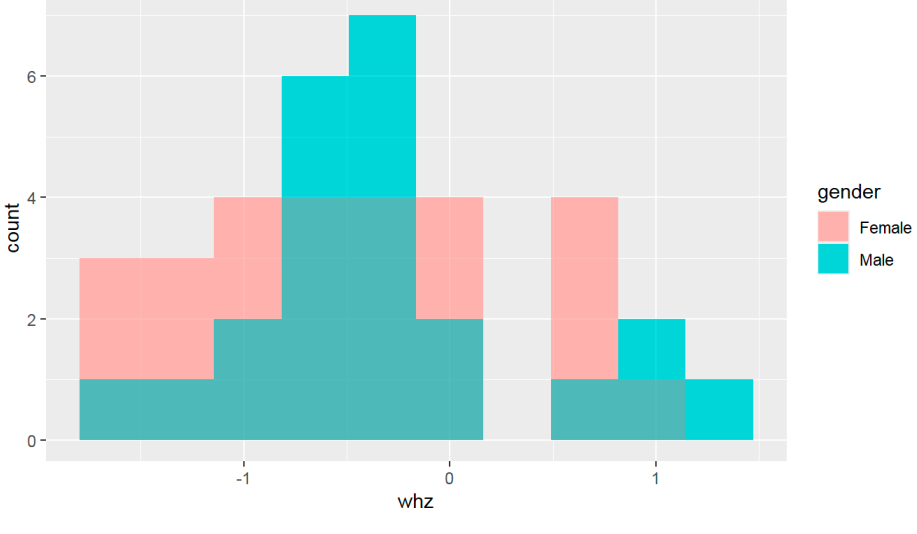


Figure 3. Plot by group example. Source: Figure by author.

2.8.3. Density

We can create a density plot with *geom_density()* (Figure 4):

```
ggplot(dat2a, aes(whz)) +
    geom_density()
```

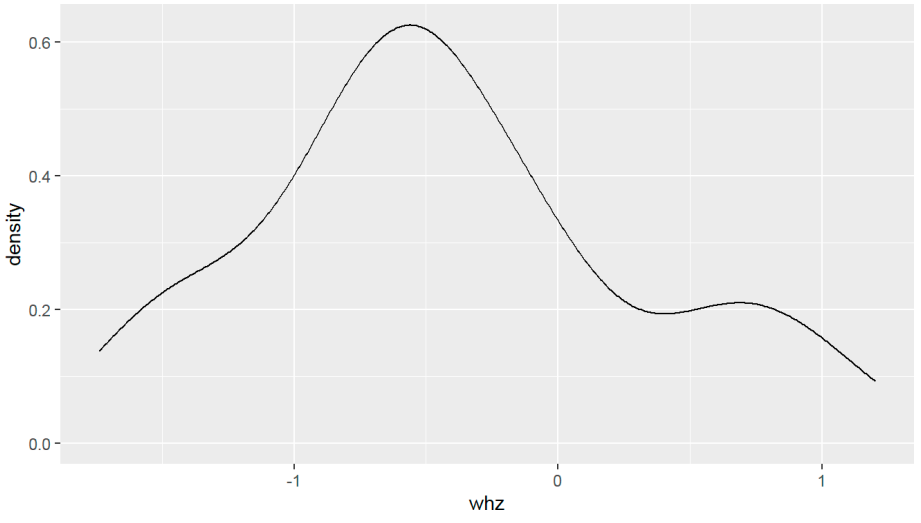


Figure 4. Density example. Source: Figure by author.

Similar to the above, we can create a plot by group using *fill*. Note that we use the parameter *alpha* to control transparency (opacity) of graphical elements: *alpha* = 1 = Fully opaque (solid color); *alpha* = 0 = Fully transparent (invisible); *alpha* = 0.5 = 50% transparent (Figure 5).

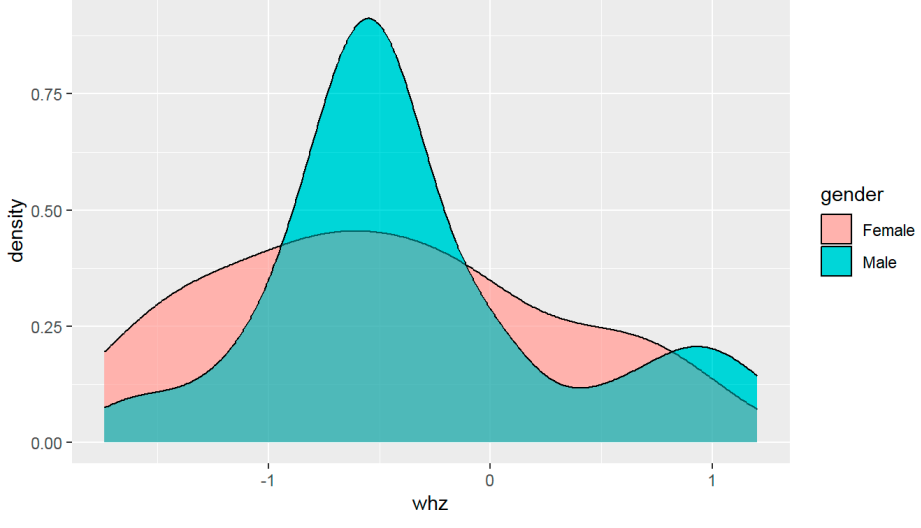


Figure 5. Density by group example. Source: Figure by author.

```
ggplot(dat2a, aes(whz, fill = gender)) +
  geom_density(alpha = 0.5)
```

2.8.4. Histogram with Density

We can combine a histogram and density into one plot (Figure 6):

```
ggplot(dat2a,aes(whz))+ geom_histogram(bins =
    10)+ geom_density()
```

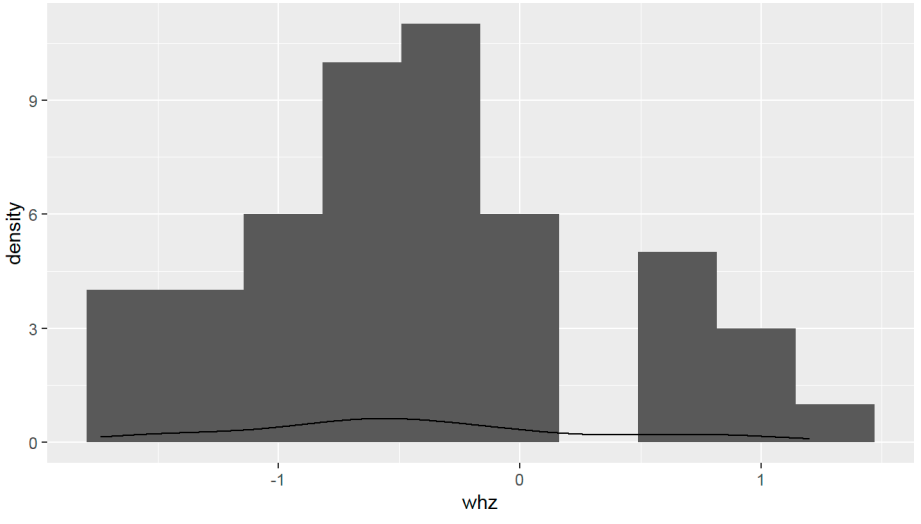


Figure 6. Histogram with density example. Source: Figure by author.

Something goes wrong! The density curve is on a different scale than the count-based histogram. To fix this, map the y-axis of the histogram to density using *after_stat(density)* (Figure 7):

```
ggplot(dat2a)+
  geom_histogram(aes(x = whz, y = after_stat(density)), bins = 10)+
  geom_density(aes(whz))
```

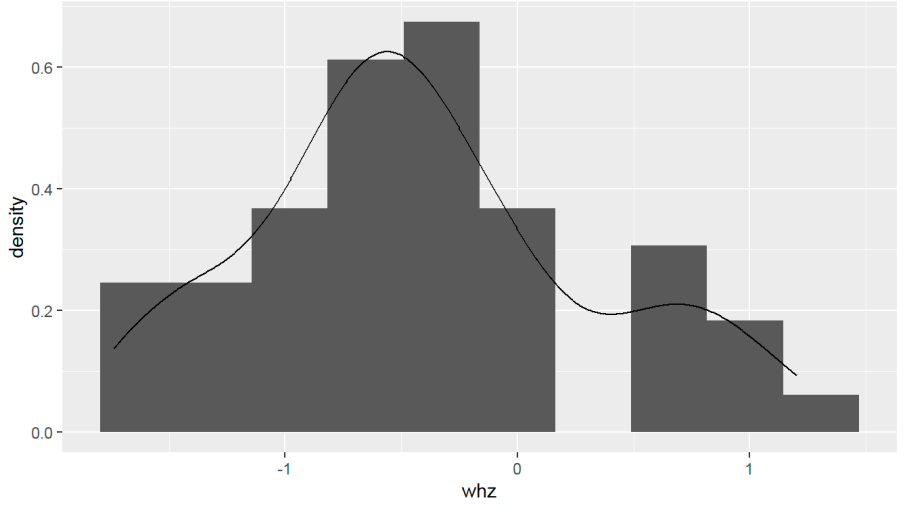


Figure 7. Histogram with density example fixed. Source: Figure by author.

Again, we can create a plot by group using *fill* (Figure 8):

```
ggplot(dat2a)+
  geom_histogram(aes(x = whz, y = after_stat(density), fill = gender), bins = 10, position =
                   "identity", alpha = 0.5)+
  geom_density(aes(whz, color = gender))
```

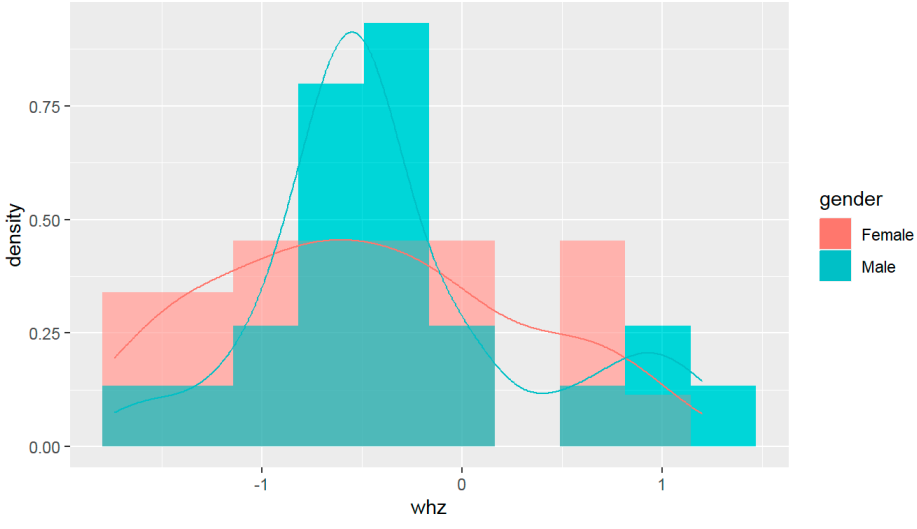


Figure 8. Histogram with density by group. Source: Figure by author.

2.8.5. Scatter Plot

We can create scatter plots with *geom_point()* (Figure 9):

```
ggplot(dat2a,aes(x = month.ebf, y = whz)) +
  geom_point()
```

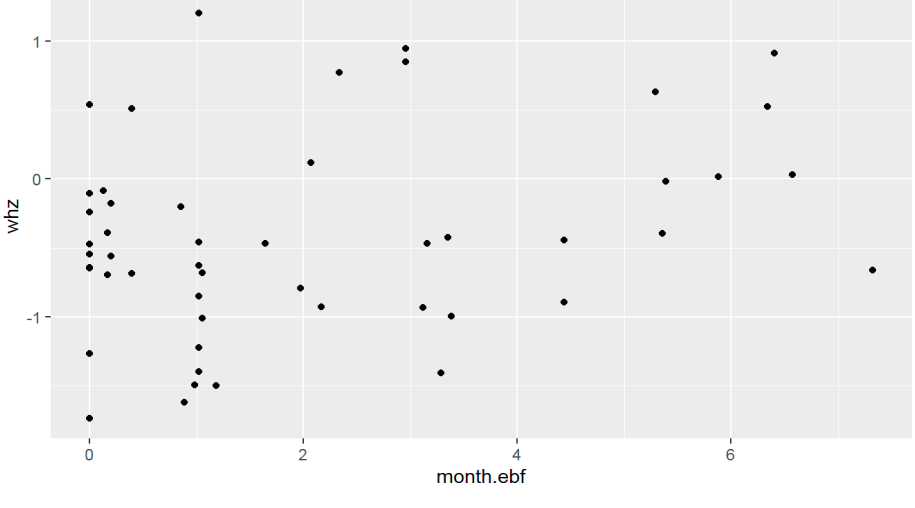


Figure 9. Scatter plot example. Source: Figure by author.

With Regression Line

A smoothed line (e.g., a regression line) can be added to summarize the trend. *stat_smooth(method = 'lm')* adds a straight line (a linear model, Linear Model, or *lm* is used here) representing the fitted relationship, along with its standard error (the shaded area) (Figure 10).

```
ggplot(dat2a,aes(x = month.ebf, y = whz)) +
  geom_point()+
  stat_smooth(method = 'lm')
#May use geom_smooth() instead of stat_smooth()
# stat_smooth() = statistical transformation + default geom
# geom_smooth() = geom wrapper around stat_smooth()
ggplot(dat2a, aes(x = month.ebf, y = whz)) +
  geom_point()+
  geom_smooth(method = 'lm')
```

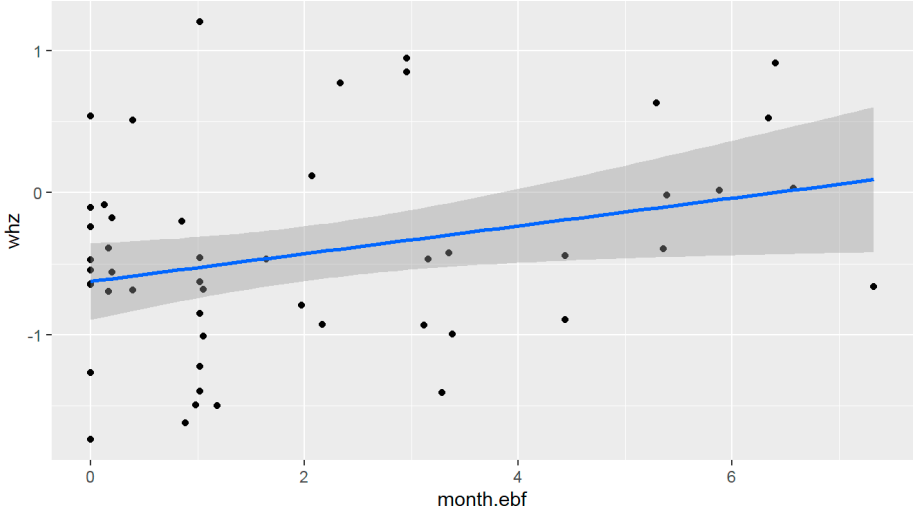


Figure 10. Scatter plot with regression line. Source: Figure by author.

By group: in the example below, the color aesthetic is mapped to "gender", creating separate colors and separate regression lines for each group (Figure 11).

```
ggplot(dat2a,aes(x = month.ebf, y = whz, color = gender)) +
  geom_point()+
  geom_smooth(method = 'lm')
```

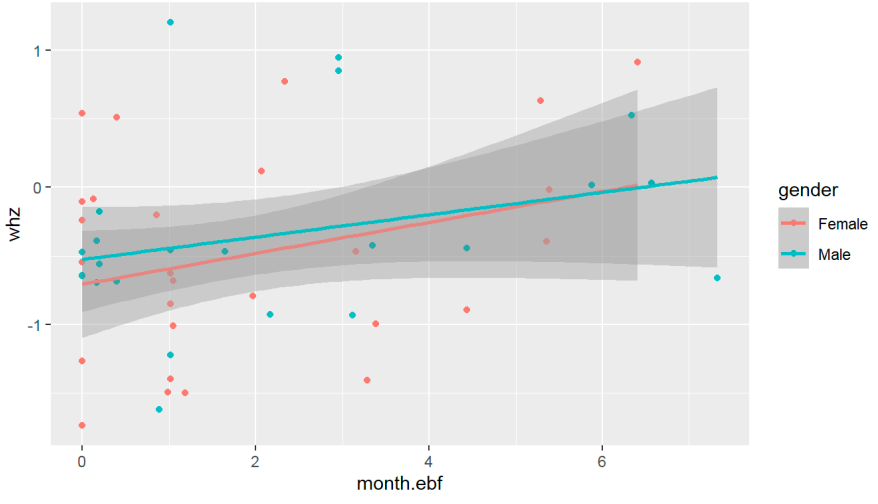


Figure 11. Scatter plot by group. Source: Figure by author.

2.8.6. Boxplot

We can create boxplots with *geom_boxplot()* (Figure 12):

```
ggplot(dat2a,aes(x = gender, y = month.ebf, color = gender)) +
  geom_boxplot()
```

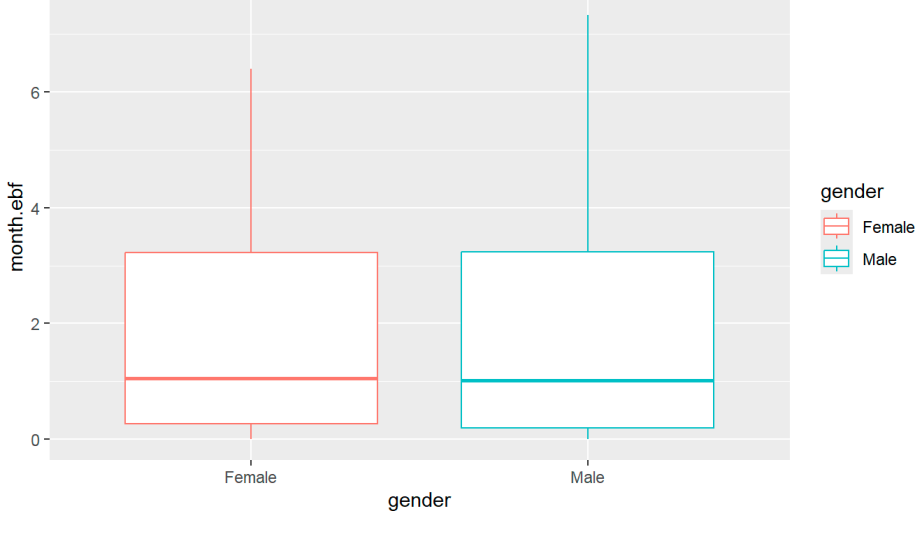


Figure 12. Boxplot example. Source: Figure by author.

Box Plot with Dots: *geom_dotplot()* plots the individual data points, stacking them (*stackdir* = '*center*') along the y-axis (*binaxis* = '*y*') (Figure 13).

```
ggplot(dat2a,aes(x = gender, y = month.ebf, color = gender)) +
  geom_boxplot()+
  geom_dotplot(binaxis = 'y', stackdir = 'center', dotsize = 1)
```

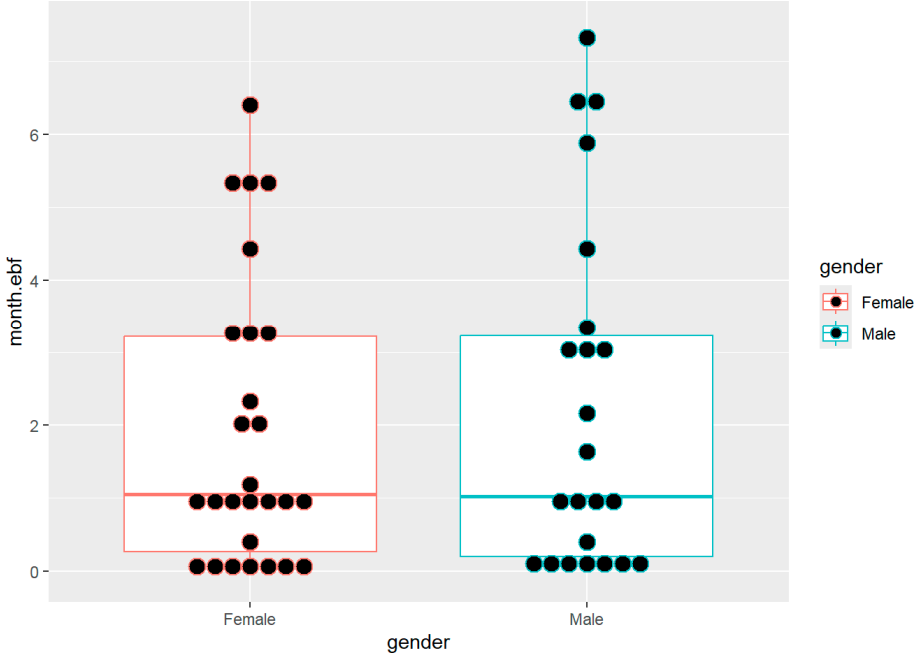


Figure 13. Box plot with dots. Source: Figure by author.

2.8.7. Pie Charts

In the example below, the data is grouped by gender, and the count of each group is calculated (*summarise(count = n())*). Then, *ggplot()* creates the plot using the previously generated data from *summarise()* and *geom_bar(stat = "identity") + coord_polar("y", start = 0)* to generate a pie plot. The *summarise()* function from "*dplyr*" is the primary tool for computing descriptive statistics, often used with *group_by()* for group-level summaries (Figure 14).

```
dat2a %>%
  group_by(gender) %>%
  summarise(count = n()) %>%
  ggplot(aes(x = "", y = count, fill = gender)) +
  geom_bar(stat = "identity") +
  coord_polar("y", start = 0)
```

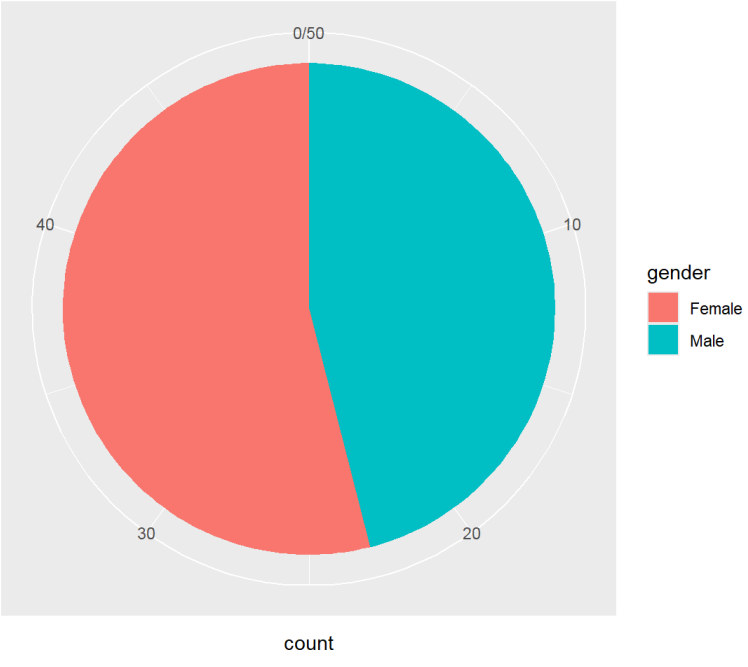


Figure 14. Pie chart. Source: Figure by author.

For more data visualization, below are some useful resources:

- "*ggplot2*" tutorial (http://r-statistics.co/Complete-Ggplot2-Tutorial-Part1-With-R-Code.html, accessed on 1 January 2026);
- Statistical tools for high-throughput data analysis: "*ggplot2*" essentials (http://www.sthda.com/english/wiki/ggplot2-essentials, accessed on 1 January 2026);
- "*ggplot2*" cheat sheet (https://rstudio.github.io/cheatsheets/html/data-visualization.html, accessed on 1 January 2026).

2.9. Summary Statistics

2.9.1. Basic Descriptive Summary Functions

The *summarise()* function from "*dplyr*" is the primary tool for computing descriptive statistics, often used with *group_by()* for group-level summaries [17].

```
# Summary of variable month.ebf
dat2a %>% summarise(
      count = n(),
      mean = mean(month.ebf, na.rm = TRUE),
      sd = sd(month.ebf, na.rm = TRUE),
      median = median(month.ebf, na.rm = TRUE),
      IQR = IQR(month.ebf, na.rm = TRUE))
```

```
##   count     mean       sd   median      IQR
## 1    50 2.071644 2.154823 1.034836 3.055231
```

We can do summary by group with *group_by()*:

```r
# Summary of month.ebf by cohort
dat2a %>% group_by(cohort) %>%
summarise(
      count = n(),
      mean = mean(month.ebf, na.rm = TRUE),
      sd = sd(month.ebf, na.rm = TRUE),
      median = median(month.ebf, na.rm = TRUE),
      IQR = IQR(month.ebf, na.rm = TRUE))
```

```
## # A tibble: 2 × 6
## cohort                          count  mean   sd     median  IQR
## <chr>                           <int>  <dbl>  <dbl>  <dbl>   <dbl>
## 1 Healthy Singleton Birth Cohort 25    3.45   2.24   3.29    3.71
## 2 Healthy Twins & Triplets       25    0.694  0.706  0.854   1.02
```

We can also do summary by subgroup of another group (e.g., by "gender" within "Healthy Twins & Triplets"):

```r
dat2a %>% filter(cohort == "Healthy Twins & Triplets") %>%
   group_by(gender) %>%
   summarise(
      count = n(),
      mean = mean(month.ebf, na.rm = TRUE),
      sd = sd(month.ebf, na.rm = TRUE),
      median = median(month.ebf, na.rm = TRUE),
      IQR = IQR(month.ebf, na.rm = TRUE))
```

```
## # A tibble: 2 × 6
## gender   count  mean   sd     median  IQR
## <chr>    <int>  <dbl>  <dbl>  <dbl>   <dbl>
## 1 Female 18     0.772  0.770  0.920   1.04
## 2 Male   7      0.493  0.498  0.197   0.920
```

We can add *filter()* to select specified rows:

```r
dat2a %>% filter(cohort == "Healthy Twins & Triplets") %>%
filter(!is.na(zygosity)) %>%
   group_by(gender) %>%
   summarise(
      count = n(),
      mean = mean(month.ebf, na.rm = TRUE),
      sd = sd(month.ebf, na.rm = TRUE),
      median = median(month.ebf, na.rm = TRUE),
      IQR = IQR(month.ebf, na.rm = TRUE))
```

```
## # A tibble: 2 × 6
## gender   count  mean   sd     median  IQR
## <chr>    <int>  <dbl>  <dbl>  <dbl>   <dbl>
## 1 Female 16     0.749  0.816  0.690   1.03
## 2 Male   7      0.493  0.498  0.197   0.920
```

2.9.2. Basic Statistical Test

The example below uses Welch's *t*-test to compare continuous variables between two groups (Figure 15):

```
#t-test
summary(dat2a$whz)
```

```
##    Min.      1st Qu.    Median     Mean      3rd Qu.    Max.
##    -1.73842  -0.88598   -0.47063   -0.42223  -0.03458   1.20133
```

```
hist(dat2a$whz)
```

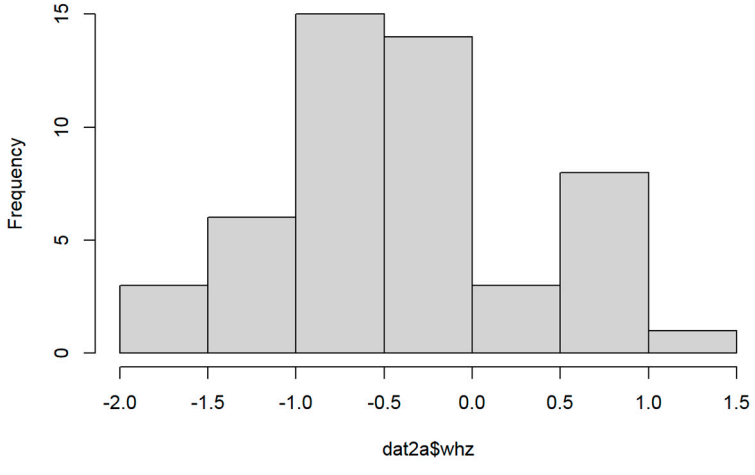


Figure 15. Histogram with base R plot. Source: Figure by author.

```
t.test(whz~gender, data = dat2a)
```

```
##
## Welch Two Sample t-test
##
## data: whz by gender
## t = -0.7285, df = 47.802, p-value = 0.4699
## alternative hypothesis: true difference in means between group Female and group Male is not
equal to 0
## 95 percent confidence interval:
##                              -0.5572199                  0.2608470
## sample estimates:
##                              mean in group Female        mean in group Male
##                              -0.4903957                  -0.3422092
```

The example below uses the Non-Parametric (Wilcoxon or Mann–Whitney) test to compare continuous variables (Figure 16):

```
summary(dat2a$month.ebf)
```

```
##    Min.      1st Qu.    Median     Mean      3rd Qu.    Max.
##    0.0000    0.1971     1.0348     2.0716    3.2523     7.3260
```

```
hist(dat2a$month.ebf)
```

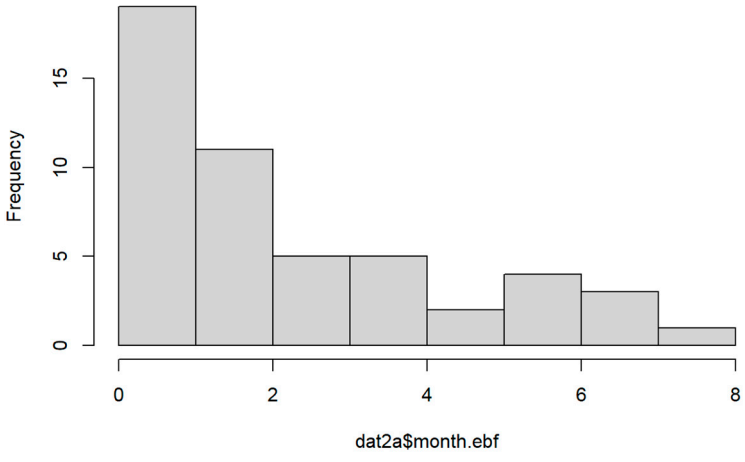


Figure 16. Histogram with base R plot 2. Source: Figure by author.

```
#wilcox.test
wilcox.test(month.ebf ~ gender, data = dat2a, conf.int = TRUE)
```

```
## Wilcoxon rank sum test with continuity correction
##
## data: month.ebf by gender
## W = 292.5, p-value = 0.7324
## alternative hypothesis: true location shift is not equal to 0
## 95 percent confidence interval:
##    −1.0512976       0.8540954
## sample estimates:
## difference in location
##    −0.03287068
```

The examples below use chi-square and Fisher's exact tests to compare categorical variables:

```
#Chi-square
table(dat2a$cohort, dat2a$gender)
```

```
##
##                                 Female  Male
## Healthy Singleton Birth Cohort  9       16
## Healthy Twins & Triplets        18      7
```

```
chisq.test(dat2a$cohort,dat2a$gender)
```

```
##
## Pearson's Chi-squared test with Yates' continuity
## correction
##
## data: dat2a$cohort and dat2a$gender
## X-squared = 5.153, df = 1, p-value = 0.02321
```

```
#Fisher
fisher.test(dat2a$cohort,dat2a$gender)
```

```
##
## Fisher's Exact Test for Count Data
##
## data: dat2a$cohort and dat2a$gender
## p-value = 0.02224
## alternative hypothesis: true odds ratio is not equal to 1
## 95 percent confidence interval:
## 0.05544835 0.83324786
## sample estimates:
## odds ratio
## 0.2261644
```

2.9.3. Summary Statistic Table

Packages for Quick, Nice Summary Table

The "*arsenal*" package provides handy R functions for large-scale statistical summaries. The *tableby()* function of the arsenal package is used to create a summary table that includes basic statistical tests for comparing many variables between groups [19].

In the example below, the *tableby()* function is applied to the Bangladesh children dataset above using a formula like "*group variable ~ variables to be summarized*" to summarize those variables stratified by the cohort group and perform statistical tests for difference (Table 3). The packages "*tidyverse*" [11] for data handling and visualization, "*knitr*" [5], and "*kableExtra*" [12] for making tables will be used throughout the chapters of this book.

```
library(arsenal)
ca.vars<-c("cohort","gender","zygosity")
co.vars<-c("whz", "waz", "haz","age.first.fe","age.last.fe", "no.fe.sam","sam.int.mean",
           "month.ebf","age.first.solid","no.diarhea.yr",
           "pc.days.diarrhea","frac.sam.atb.7day")
varuse<-c(co.vars,ca.vars[!ca.vars %in% "cohort"]) mylabels
<-as.list(varuse)
names(mylabels)<-varuse
tab1 <- tableby(as.formula(paste("cohort",paste(varuse,collapse = "+"),sep = "~")), data
                = dat2a)
kable(summary(tab1,
        labelTranslations = mylabels, text
        = TRUE),
      caption = "Summary table")%>%
  kable_styling(latex_options = c("scale_down", "hold_position"))
```

Table 3. Summary table.

	Healthy Singleton Birth Cohort (N = 25)	**Healthy Twins & Triplets (N = 25)**	**Total (N = 50)**	***p*-Value**
whz				0.683
- Mean (SD)	−0.380 (0.727)	−0.464 (0.724)	−0.422 (0.719)	
- Range	−1.623–0.949	−1.738–1.201	−1.738–1.201	
waz				<0.001
- Mean (SD)	−0.483 (0.806)	−2.546 (0.561)	−1.515 (1.248)	
- Range	−1.741–1.670	−3.755–1.392	−3.755–1.670	
haz				<0.001
- Mean (SD)	−0.483 (0.806)	−3.231 (0.956)	−1.857 (1.641)	
- Range	−1.741–1.670	−4.659–1.412	−4.659–1.670	

Table 3. *Cont.*

	Healthy Singleton Birth Cohort (N = 25)	**Healthy Twins & Triplets (N = 25)**	**Total (N = 50)**	***p*-Value**
age.first.fe				0.008
- Mean (SD)	6.360 (2.782)	13.200 (12.124)	9.780 (9.366)	
- Range	3.000–15.000	1.000–37.000	1.000–37.000	
age.last.fe				<0.001
- Mean (SD)	710.400 (12.430)	490.880 (152.553)	600.640 (154.167)	
- Range	700.000–738.000	286.000–730.000	286.000–738.000	
no.fe.sam				0.001
- Mean (SD)	21.920 (2.019)	17.920 (5.338)	19.920 (4.476)	
- Range	18.000–24.000	9.000–26.000	9.000–26.000	
sam.int.mean				<0.001
- Mean (SD)	34.003 (3.813)	28.677 (3.090)	31.340 (4.363)	
- Range	30.087–42.294	22.562–35.000	22.562–42.294	
month.ebf				<0.001
- Mean (SD)	3.449 (2.242)	0.694 (0.706)	2.072 (2.155)	
- Range	0.000–7.326	0.000–2.332	0.000–7.326	
age.first.solid				0.254
- Mean (SD)	6.352 (2.236)	7.028 (1.884)	6.690 (2.074)	
- Range	1.216–9.790	3.088–9.396	1.216–9.790	
no.diarhea.yr				<0.001
- Mean (SD)	4.278 (2.135)	1.664 (1.345)	2.971 (2.205)	
- Range	0.520–8.814	0.000–4.986	0.000–8.814	
pc.days.diarrhea				0.007
- Mean (SD)	4.090 (2.223)	2.159 (2.601)	3.125 (2.585)	
- Range	0.142–10.826	0.000–9.563	0.000–10.826	
frac.sam.atb.7day				0.070
- Mean (SD)	0.219 (0.145)	0.148 (0.125)	0.183 (0.139)	
- Range	0.000–0.500	0.000–0.462	0.000–0.500	
gender				0.011
- Female	9 (36.0%)	18 (72.0%)	27 (54.0%)	
- Male	16 (64.0%)	7 (28.0%)	23 (46.0%)	
zygosity				<0.001
- N-Miss	0	2	2	
- DZ	0 (0.0%)	12 (52.2%)	12 (25.0%)	
- Fraternal co-twin in set of triplets	0 (0.0%)	1 (4.3%)	1 (2.1%)	
- MZ	0 (0.0%)	8 (34.8%)	8 (16.7%)	
- MZ co-twin in set of triplets	0 (0.0%)	2 (8.7%)	2 (4.2%)	
- singleton	25 (100.0%)	0 (0.0%)	25 (52.1%)	

Source: Table by author.

Readers feel free to try other R packages for nice data summary tables.

More Comprehensive Table (Control)

The *tableby.control()* function allows fine-tuning of the output, such as changing the statistical tests used (e.g., using a Wilcoxon test (*wilcox*) instead of a *t*-test) and adjusting the summary statistics displayed (e.g., adding Mean (95% CI)) (Table 4).

```
my_controls <- tableby.control( test
  = TRUE,
  total = TRUE,
  numeric.test = "wilcox", cat.test = "fe",
  numeric.stats = c("meansd", "medianq1q3", "range", "Nmiss2"),
  cat.stats = c("countpct", "Nmiss2"),
  stats.labels = list( meansd
    = "Mean (SD)",
    medianq1q3 = "Median (Q1, Q3)", range
    = "Min - Max",
    Nmiss2 = "Missing"
  )
)
tab2 <- tableby(as.formula(paste("cohort",paste(varuse,collapse = "+"), sep = "~")), data
                = dat2a,
                control = my_controls)
kable(summary(tab2, labelTranslations = mylabels, text = TRUE),
      caption = "More comprehensive summary table")%>%
  kable_styling(latex_options = c("scale_down", "hold_position"))
```

Table 4. More comprehensive summary table.

	Healthy Singleton Birth Cohort (N = 25)	Healthy Twins & Triplets (N = 25)	Total (N = 50)	*p*-Value
whz				0.705
- Mean (SD)	−0.380 (0.727)	−0.464 (0.724)	−0.422 (0.719)	
- Median (Q1, Q3)	−0.468 (−0.897, 0.015)	−0.560 (−0.853, −0.104)	−0.471 (−0.886, −0.035)	
- Min–Max	−1.623–0.949	−1.738–1.201	−1.738–1.201	
- Missing	0	0	0	
waz				<0.001
- Mean (SD)	−0.483 (0.806)	−2.546 (0.561)	−1.515 (1.248)	
- Median (Q1, Q3)	−0.706 (−1.031, −0.030)	−2.606 (−2.879, −2.295)	−1.512 (−2.604, −0.719)	
- Min–Max	−1.741–1.670	−3.755–−1.392	−3.755–1.670	
- Missing	0	0	0	
haz				<0.001
- Mean (SD)	−0.483 (0.806)	−3.231 (0.956)	−1.857 (1.641)	
- Median (Q1, Q3)	−0.706 (−1.031, −0.030)	−3.437 (−4.032, −2.551)	−1.575 (−3.431, −0.719)	
- Min–Max	−1.741–1.670	−4.659–−1.412	−4.659–1.670	
- Missing	0	0	0	
age.first.fe				0.077
- Mean (SD)	6.360 (2.782)	13.200 (12.124)	9.780 (9.366)	
- Median (Q1, Q3)	5.000 (5.000, 8.000)	8.000 (5.000, 15.000)	6.000 (5.000, 9.750)	
- Min–Max	3.000–15.000	1.000–37.000	1.000–37.000	
- Missing	0	0	0	
age.last.fe				<0.001
- Mean (SD)	710.400 (12.430)	490.880 (152.553)	600.640 (154.167)	
- Median (Q1, Q3)	704.000 (701.000, 715.000)	455.000 (366.000, 638.000)	700.500 (455.000, 706.000)	
- Min–Max	700.000–738.000	286.000–730.000	286.000–738.000	
- Missing	0	0	0	
no.fe.sam				0.011
- Mean (SD)	21.920 (2.019)	17.920 (5.338)	19.920 (4.476)	
- Median (Q1, Q3)	23.000 (21.000, 23.000)	18.000 (13.000, 22.000)	21.000 (18.000, 23.000)	
- Min–Max	18.000–24.000	9.000–26.000	9.000–26.000	
- Missing	0	0	0	
sam.int.mean				<0.001
- Mean (SD)	34.003 (3.813)	28.677 (3.090)	31.340 (4.363)	
- Median (Q1, Q3)	33.143 (31.409, 35.350)	28.154 (27.048, 30.350)	30.477 (28.290, 33.271)	
- Min–Max	30.087–42.294	22.562–35.000	22.562–42.294	
- Missing	0	0	0	
month.ebf				<0.001
- Mean (SD)	3.449 (2.242)	0.694 (0.706)	2.072 (2.155)	
- Median (Q1, Q3)	3.285 (1.643, 5.355)	0.854 (0.000, 1.018)	1.035 (0.197, 3.252)	
- Min–Max	0.000–7.326	0.000–2.332	0.000–7.326	
- Missing	0	0	0	
age.first.solid				0.388
- Mean (SD)	6.352 (2.236)	7.028 (1.884)	6.690 (2.074)	
- Median (Q1, Q3)	6.209 (4.599, 8.213)	7.063 (6.078, 8.476)	6.932 (4.920, 8.361)	
- Min–Max	1.216–9.790	3.088–9.396	1.216–9.790	
- Missing	0	0	0	

Table 4. *Cont.*

	Healthy Singleton Birth Cohort (N = 25)	Healthy Twins & Triplets (N = 25)	Total (N = 50)	*p*-Value
no.diarhea.yr				<0.001
- Mean (SD)	4.278 (2.135)	1.664 (1.345)	2.971 (2.205)	
- Median (Q1, Q3)	3.957 (3.008, 5.170)	1.276 (0.802, 2.003)	2.916 (1.271, 4.054)	
- Min–Max	0.520–8.814	0.000–4.986	0.000–8.814	
- Missing	0	0	0	
pc.days.diarrhea				<0.001
- Mean (SD)	4.090 (2.223)	2.159 (2.601)	3.125 (2.585)	
- Median (Q1, Q3)	3.930 (2.996, 4.755)	1.359 (0.440, 2.740)	2.742 (1.163, 4.314)	
- Min–Max	0.142–10.826	0.000–9.563	0.000–10.826	
- Missing	0	0	0	
frac.sam.atb.7day				0.109
- Mean (SD)	0.219 (0.145)	0.148 (0.125)	0.183 (0.139)	
- Median (Q1, Q3)	0.217 (0.087, 0.318)	0.111 (0.077, 0.167)	0.119 (0.081, 0.284)	
- Min–Max	0.000–0.500	0.000–0.462	0.000–0.500	
- Missing	0	0	0	
gender				0.022
- Female	9 (36.0%)	18 (72.0%)	27 (54.0%)	
- Male	16 (64.0%)	7 (28.0%)	23 (46.0%)	
- Missing	0	0	0	
zygosity				<0.001
- DZ	0 (0.0%)	12 (52.2%)	12 (25.0%)	
- Fraternal co-twin in set of triplets	0 (0.0%)	1 (4.3%)	1 (2.1%)	
- MZ	0 (0.0%)	8 (34.8%)	8 (16.7%)	
- MZ co-twin in set of triplets	0 (0.0%)	2 (8.7%)	2 (4.2%)	
- singleton	25 (100.0%)	0 (0.0%)	25 (52.1%)	
- Missing	0	2	2	

Source: Table by author.

2.10. Practice

Write R codes to:

- Import sheet 2 of excel data file "nature13421-s2.xlsx" in the path for datasets *fdir1* introduced above;
- Rename column names to sensible names, remove redundant rows;
- Save your cleaned data to a local disk and load the saved data for use;
- Make a table showing the number of fecal samples for each child ID;
- Summarize the number of fecal samples in all child IDs and by cohort of singletons vs. twins/triplets;
- Plot the number of fecal samples by cohort of singletons vs. twins/triplets.

2.11. Practice Review

2.11.1. The Requirements

We now practice writing R code for the requirements above. First, we import sheet 2 of the Excel data file using the "*rio*" package [14]:

```
library(rio)
#import all sheets
list1<-import_list(paste(fdir1,"nature13421-s2.xlsx", sep = ""))
#get sheet 2
sh2<-list1[[2]]

#import sheet 2 only
sh2<-import(paste(fdir1,"nature13421-s2.xlsx", sep = ""), sheet = 2, skip = 1)
```

We rename column names to sensible names and remove redundant rows:

```
library(tidyverse)
colnames(sh2)
colnames(sh2)[!is.na(sh2[1,])]<-sh2[1,][!is.na(sh2[1,])]
#or just do all at a time oldname<-
colnames(sh2)
newname<-c("cohort",
                    "family.id",
                    "child.id",
                    "sample.id",
                    "age.d",
                    "age.m", "whz","waz","haz",
                    "breast.milk", "formula",
                    "solid",  "diarrhea",
                    "antibiotic.7d", "medication",
                    "no.sequence",
                    "serun.id", "barcode")
sh2<-sh2%>% rename_at(vars(oldname), ~newname)
colnames(sh2)
#remove unused rows
sh2<-sh2%>%
  filter(!is.na(family.id))
```

You may want to save your cleaned data to the local disk and load the saved data for use:

```
#save data as csv
write.csv (sh2, file = paste(fdir1, "sh2.csv", sep = ""), row.names = FALSE)
#read the data back to R
sh2<- read.csv(paste(fdir1,"sh2.csv", sep  =  ""))
```

Now, we make a table showing the number of fecal samples for each child ID and summarize the number of fecal samples for all child IDs:

```
nsam<-sh2%>% group_by(child.id)  %>%
  summarise(no.sam = n())
View(nsam)
nsam  %>%
    summarise(
      mean = mean(no.sam,  na.rm = TRUE),
      sd = sd(no.sam,  na.rm = TRUE),
      median = median(no.sam,  na.rm = TRUE),
      IQR = IQR(no.sam,  na.rm = TRUE))
```

```
## # A tibble: 1 × 4
##           mean    sd        median    IQR
## <dbl>    <dbl>   <dbl>     <dbl>
## 1         19.9    4.48      21        5
```

We summarize the number of fecal samples per "child.id" by cohort of singletons vs. twins/triplets:

```
nsam<-sh2%>%
  group_by(cohort,child.id)  %>%
  summarise(no.sam = n()) %>%
  summarise(
    mean = mean(no.sam,  na.rm = TRUE),
    sd = sd(no.sam,  na.rm = TRUE),
    median = median(no.sam,  na.rm = TRUE),
    IQR = IQR(no.sam,  na.rm = TRUE))
nsam
```

```
## # A tibble: 2 × 5
## cohort                          mean   sd     median  IQR
## <chr>                           <dbl>  <dbl>  <int>   <dbl>
## 1 Healthy Singleton Birth Cohort 21.9  2.02   23      2
## 2 Healthy Twins & Triplets       17.9  5.34   18      9
```

We create a boxplot for the number of fecal samples by cohort of singletons vs. twins/triplets (Figure 17):

```
sh2%>%
  group_by(cohort,child.id)  %>%
  summarise(no.sam = n()) %>%
  ggplot(aes(x  =  cohort,y  =  no.sam,fill  =  cohort))  +
  geom_boxplot()+
  ylab("Number  of  fecal  samples")+
  geom_dotplot(binaxis  =  'y',  stackdir  =  'center',  dotsize  =  1)+
  theme(axis.text.x  =  element_blank())
```

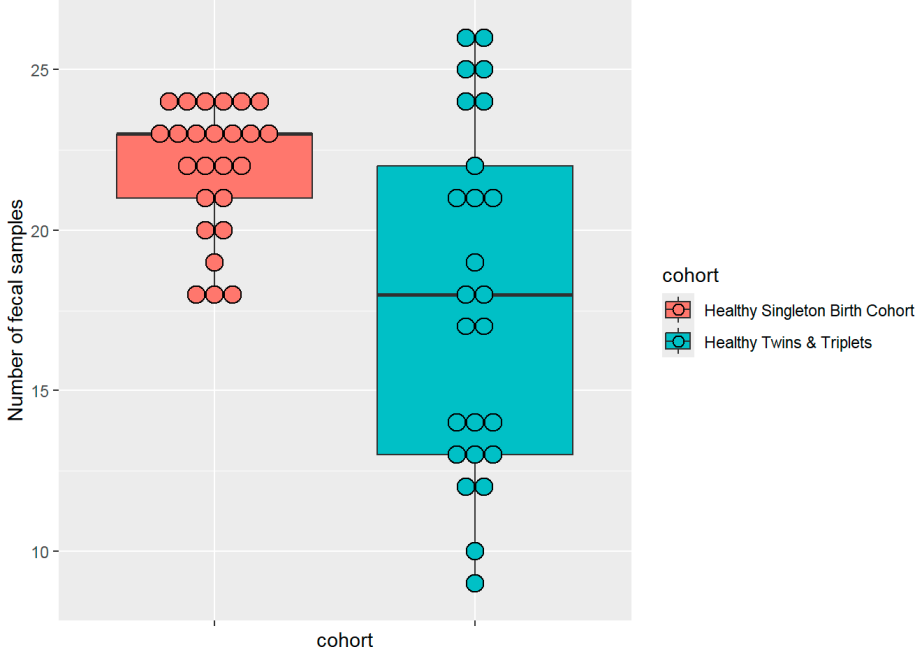


Figure 17. Boxplot number of fecal samples. Source: Figure by author.

We can create a violin plot, which is similar to a box plot but shows the density within groups (not as much information provided as in the box plot) (Figure 18):

```
sh2%>% group_by(cohort,child.id)  %>%
   summarise(no.sam = n()) %>%
   ggplot(aes(x  =  cohort,y  =  no.sam,fill  =  cohort))  +
   geom_violin()+
   ylab("Number of fecal samples")+
   theme(axis.text.x = element_blank())
```

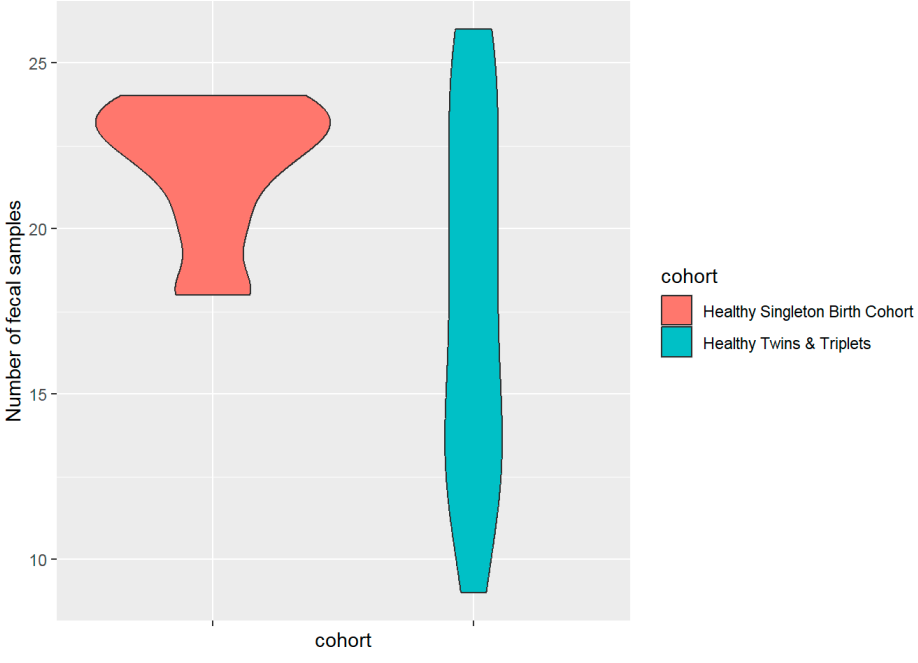


Figure 18. Violin plot number of fecal samples. Source: Figure by author.

Of note, one may combine the boxplot and the violin plot as follows (Figure 19):

```
sh2%>%
group_by(cohort, child.id) %>%
summarise(no.sam = n(), .groups = "drop") %>%
ggplot(aes(x = cohort, y = no.sam, fill = cohort)) +
# The Violin Plot (The "Background" shape)
geom_violin(alpha = 0.5) +

# The Box Plot (Narrowed to fit inside the violin)
geom_boxplot(width = 0.1, fill = "white", outlier.shape = NA) +

# Optional: Dotplot (if you want the raw points)
geom_dotplot(binaxis = 'y', stackdir = 'center', dotsize = 0.5, alpha = 0.3) +

ylab("Number of fecal samples") +
theme_minimal() +
theme(axis.text.x = element_blank())
```

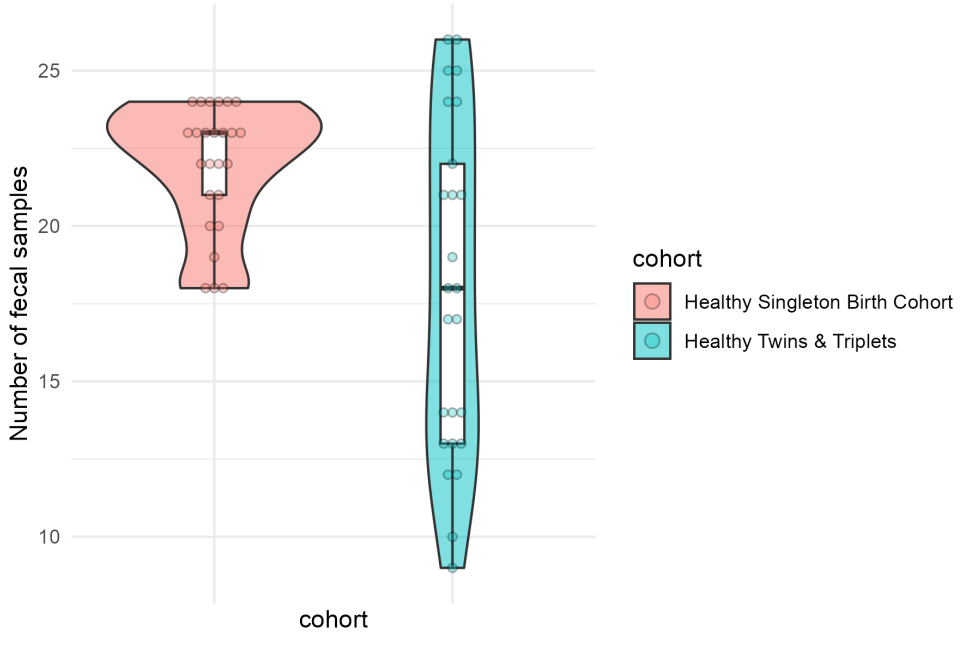


Figure 19. Combined violin and box plots. Source: Figure by author.

We can create a bar plot for the number of fecal samples by cohort of singletons vs. twins/triplets, and a summary of the number of fecal samples:

```
nsam<-sh2%>%
   group_by(cohort,child.id)  %>%
   summarise(no.sam = n()) %>%
   summarise(
      no.child = n(),
      mean = mean(no.sam,  na.rm = TRUE),
      sd = sd(no.sam,  na.rm = TRUE),
      se = sd/sqrt(n()),
      ci = 1.96*se,
      median = median(no.sam,  na.rm = TRUE),
      IQR = IQR(no.sam,  na.rm = TRUE))
nsam
```

```
## # A tibble: 2 × 8
## cohort                no.child  mean  sd    se    ci    median  IQR
## <chr>                 <int>     <dbl> <dbl> <dbl> <dbl> <int>   <dbl>
## 1 Healthy Singleto~   25        21.9  2.02  0.404 0.791 23      2
## 2 Healthy Twins & ~   25        17.9  5.34  1.07  2.09  18      9
```

We can create a mean plot with an error bar. The error bars represent standard errors (SE) (Figure 20):

```
ggplot(nsam, aes(x = cohort, y =  mean, color=cohort)) +
      geom_point(size = 3 ) +
      geom_errorbar(aes(ymin = mean-se, ymax = mean+se),
                    width = .2,              # Width of the error bars
                    position = position_dodge(.9))+
                    ylab("Number of fecal samples (mean +/- SE)")+
   theme(axis.text.x =  element_blank())
```

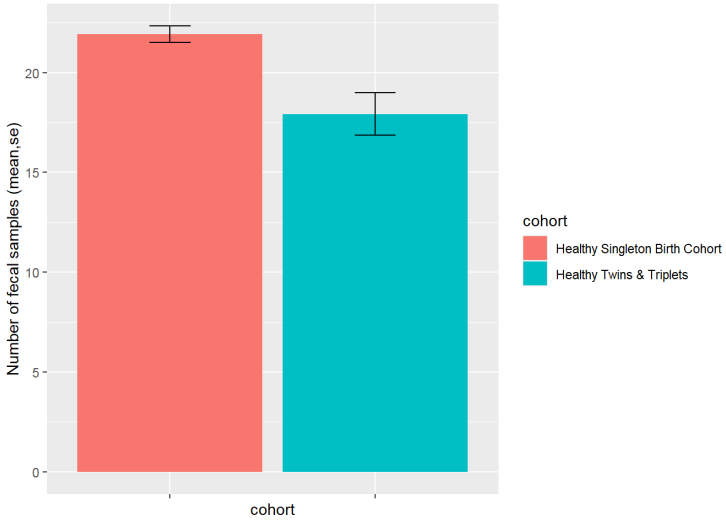


Figure 20. Mean plot with error bar number of fecal samples. Source: Figure by author.

We can replace the error bar with 95% confidence interval (CI) for the mean plot (Figure 21):

```
ggplot(nsam, aes(x = cohort, y = mean, color = cohort)) +
  geom_point(size = 3) +
  geom_errorbar(aes(ymin = mean-ci, ymax = mean+ci),
                width = .2,              # Width of the error bars
                position = position_dodge(.9))+
  ylab("Number of fecal samples (mean, 95%CI)")+
 theme(axis.text.x = element_blank())
```

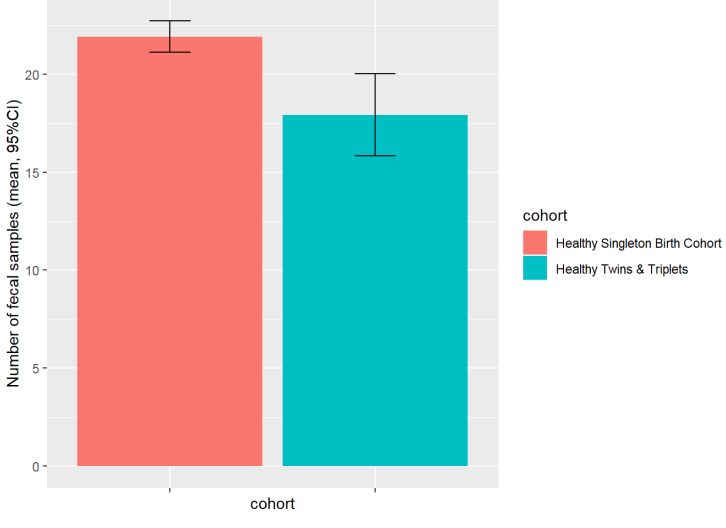


Figure 21. Mean plot with 95% CI. Source: Figure by author.

We can create a histogram with density for the number of fecal samples by cohort of singletons vs. twins/triplets (Figure 22):

```
sh2%>% group_by(cohort,child.id) %>%
  summarise(no.sam = n()) %>%
  ggplot(aes(no.sam,y = after_stat(density), fill = cohort))+
  geom_histogram(bins = 10, alpha = 0.5, position = "identity")+
  geom_density(aes(no.sam, color = cohort), alpha = 0.5)
```

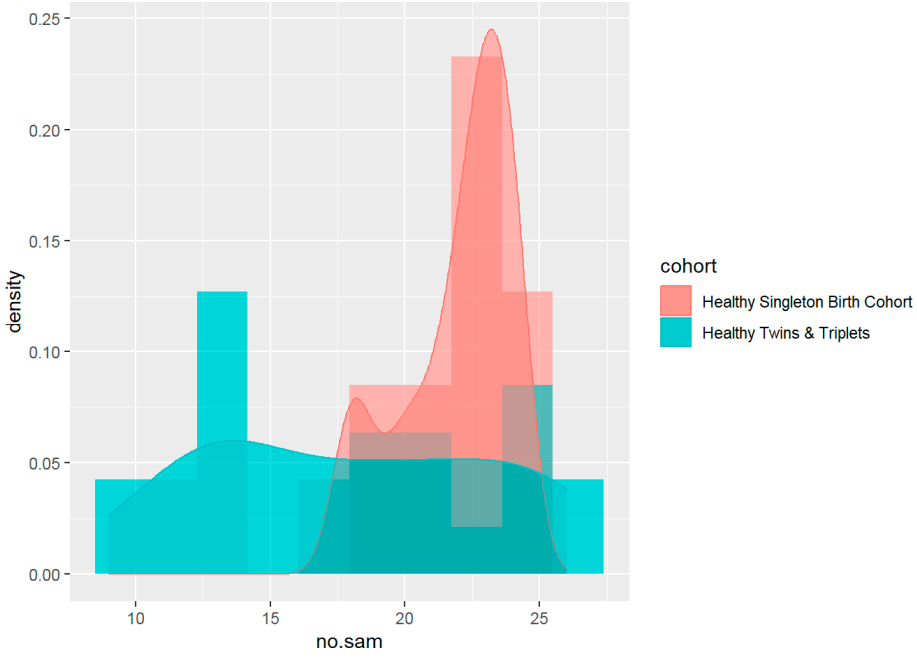


Figure 22. Histogram by cohort. Source: Figure by author.

2.11.2. Further Exploration

We can do further exploration beyond the practice requirements above. For example, we can do some basic statistical tests.

```
nsam<-sh2%>%
  group_by(cohort,child.id) %>%
  summarise(no.sam = n())
head(nsam)
```

```
##   # A tibble: 6 × 3
##   # Groups: cohort      [1]
##     cohort                                child.id      no.sam
##     <chr>                                 <chr>         <int>
##   1 Healthy Singleton   Birth   Cohort    Bgsng7018     21
##   2 Healthy Singleton   Birth   Cohort    Bgsng7035     23
##   3 Healthy Singleton   Birth   Cohort    Bgsng7052     20
##   4 Healthy Singleton   Birth   Cohort    Bgsng7063     20
##   5 Healthy Singleton   Birth   Cohort    Bgsng7071     18
##   6 Healthy Singleton   Birth   Cohort    Bgsng7082     18
```

```
wilcox.test(no.sam~cohort, nsam, conf.int = TRUE)
```

```
## Wilcoxon rank sum test with continuity correction
##
## data: no.sam by cohort
## W = 443.5, p-value = 0.01092
## alternative hypothesis: true location shift is not equal to 0
## 95 percent confidence interval:
##       1.000066     7.000058
## sample estimates:
## difference in location
##       4.000054
```

```
t.test(no.sam~cohort, nsam)
```

```
##
## Welch Two Sample t-test
##
## data: no.sam by cohort
## t = 3.5045, df = 30.73, p-value = 0.001427
## alternative hypothesis: true difference in means between group Healthy Singleton
## Birth Cohort and group Healthy Twins & Triplets is not equal to 0
## 95 percent confidence interval:
##     1.671264     6.328736
## sample estimates:
## mean in group Healthy Singleton Birth Cohort
##     21.92
## mean in group Healthy Twins & Triplets
##     17.92
```

To decide which test is appropriate, the readers may see Chapter "Basic Statistics" of this book.

Note that there is a difference in data type between sheet 1 and sheet 2:

- Sheet 1 is cross-sectional data.
- Sheet 2 is longitudinal data: many observations per "child.id".

Therefore, different ways of data description and analysis may be used.

For example, for longitudinal data, we can do visualization with a spaghetti plot for "whz" (Figure 23):

```
sh2%>% filter(!is.na(whz)) %>%
  ggplot()+
  geom_point(aes(x = age.m, y = whz, group = child.id, colour = cohort))+
  geom_line(aes(x = age.m, y = whz, group = child.id, colour = cohort), size = 0.3)
```

Figure 23. whz spaghetti plot. Source: Figure by author.

We can do more exploration with "waz" (Figure 24):

```
sh2%>% filter(!is.na(waz)) %>%
  ggplot()+
  geom_point(aes(x = age.m,  y = waz,  group = child.id, colour  =  cohort))+
  geom_line(aes(x = age.m,  y = waz,  group = child.id, colour  =  cohort), size  =  0.3)
```

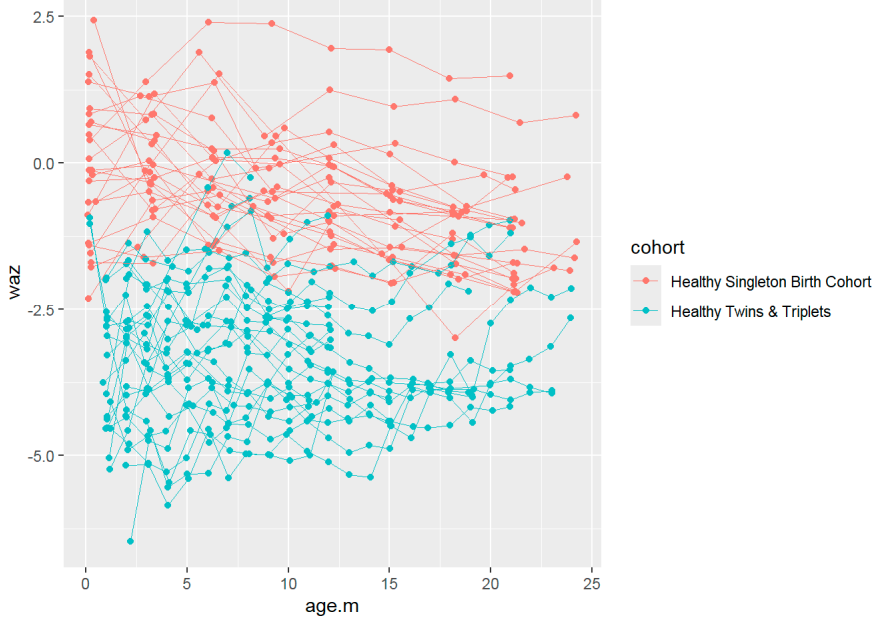


Figure 24. waz spaghetti plot. Source: Figure by author.

We can add fitted lines (linear trends) to the plot (Figure 25):

```
sh2%>% filter(!is.na(waz)) %>%
  ggplot()+
  geom_point(aes(x = age.m, y = waz, group = child.id,
                 colour = cohort))+
  geom_line(aes(x = age.m, y = waz, group = child.id, colour = cohort), size = 0.3)+
  stat_smooth(aes(x = age.m, y = waz, colour = cohort), method = 'lm', size = 2)
```

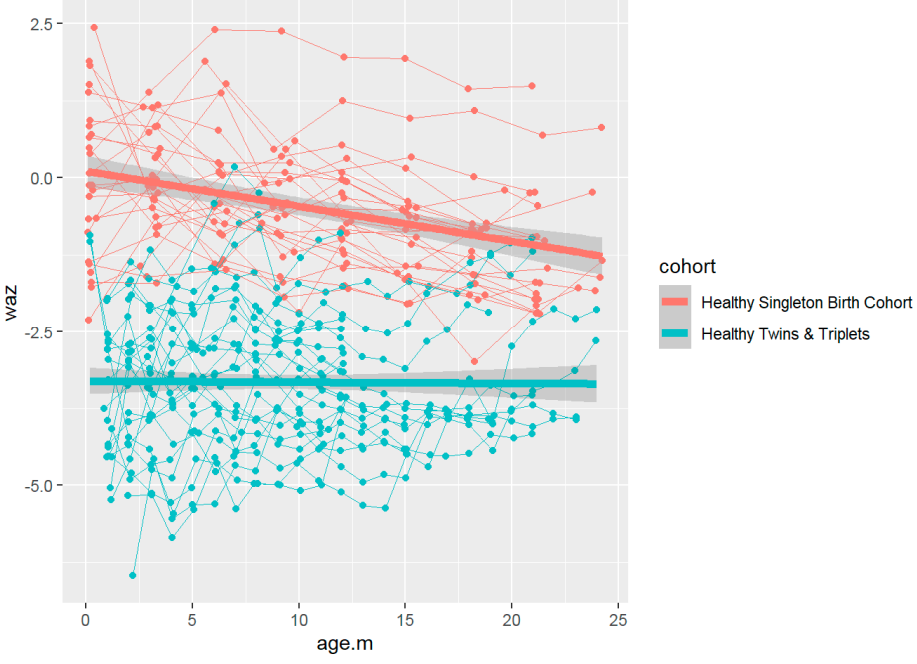


Figure 25. waz spaghetti plot with fitted lines (linear trends). Source: Figure by author.

We can do more similar exploration with "haz" (Figure 26):

```
sh2%>% filter(!is.na(haz)) %>%
  ggplot()+
  geom_point(aes(x = age.m, y = haz, group = child.id, colour = cohort))+
  geom_line(aes(x = age.m, y = haz, group = child.id, colour = cohort), size = 0.3)+
  stat_smooth(aes(x = age.m, y = haz, colour = cohort), method = 'lm', size = 2)
```

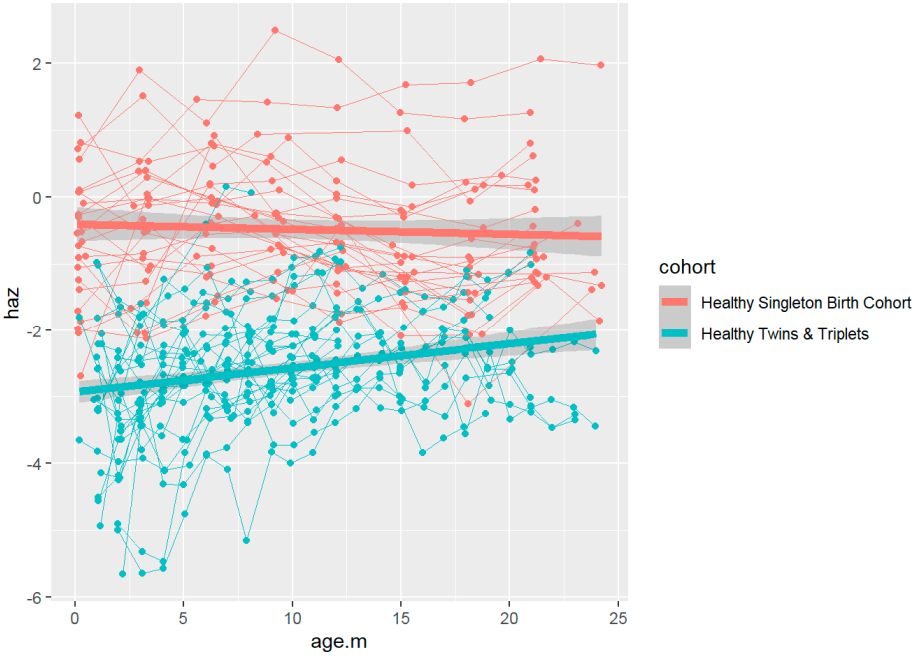


Figure 26. haz spaghetti plot with fitted lines (linear trends). Source: Figure by author.

To test for the difference between two groups, please check out the chapter "Longitudinal data analysis with R" in this book.

We can combine all the above plots into an all-in-one spaghetti plot of three indices by cohort, adding each item to the plot and using *facet_grid()* (Figure 27):

```
sh2%>%
    filter(!is.na(haz) & !is.na(waz) & !is.na(whz)) %>% ggplot()+
    #add each item to the plot
    geom_point(aes(x = age.m, y = haz,
                   group = child.id,color = "haz"))+
    geom_line(aes(x = age.m, y = haz,
                  group = child.id, color = "haz"), size = 0.3)+
    stat_smooth(aes(x = age.m, y = haz,
                    color = "haz"), method = 'lm',size = 2)+
                    geom_point(aes(x = age.m, y = waz,
                   group = child.id, color = "waz"))+
    geom_line(aes(x = age.m, y = waz,
                  group = child.id, color = "waz"), size = 0.3)+
    stat_smooth(aes(x = age.m, y = waz,
                    color = "waz"), method = 'lm',size = 2)+
    geom_point(aes(x = age.m, y = whz,
                   group = child.id, color = "whz"))+
    geom_line(aes(x = age.m, y = whz,
                  group = child.id, color = "whz"), size = 0.3)+
    stat_smooth(aes(x = age.m, y = whz,
                    color = "whz"), method = 'lm', size = 2)+
                    #divide the plot into multiple panels (cohorts)
    facet_grid(.~ cohort)+
theme(legend.title = element_blank())
```

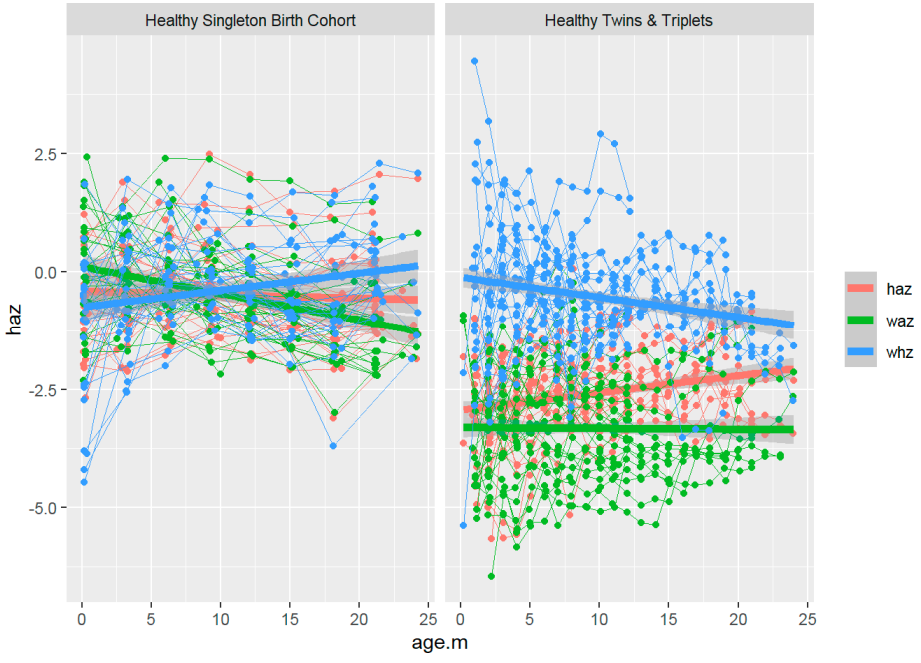


Figure 27. All-in-one spaghetti plot. Source: Figure by author.

We can create each plot for each index by cohort and then combine all plots into an all-in-one spaghetti plot using *ggarrange()* of the "*ggpubr*" package (Figure 28) [20]:

```
p1<-sh2%>%
    filter(!is.na(haz) & !is.na(waz) & !is.na(whz)) %>% ggplot()+
    geom_point(aes(x = age.m, y = haz, group = child.id, colour = cohort))+
    geom_line(aes(x = age.m, y = haz, group = child.id, colour = cohort), size =
                          0.3)+
    stat_smooth(aes(x = age.m, y =haz, colour = cohort), method = 'lm', size = 2)+
    theme(legend.position = "bottom")
p2<-sh2%>%
    filter(!is.na(haz) & !is.na(waz) & !is.na(whz)) %>%
    ggplot()+
    geom_point(aes(x = age.m, y = waz, group = child.id, colour = cohort))+
    geom_line(aes(x = age.m, y = waz, group = child.id, colour = cohort), size =
                          0.3)+
    stat_smooth(aes(x= age.m, y  =waz, colour = cohort), method = 'lm', size = 2)+
    theme(legend.position = "bottom")
p3<-sh2%>%
    filter(!is.na(haz) & !is.na(waz) & !is.na(whz)) %>% ggplot()+
    geom_point(aes(x = age.m, y =    whz, group =    child.id, colour = cohort))+
  geom_line(aes(x = age.m, y =    whz, group =    child.id, colour = cohort), size = 0.3)+
  stat_smooth(aes(x= age.m, y  =whz, colour = cohort), method = 'lm',size = 2)+
  theme(legend.position = "bottom")

  # Combine 3 plots into 1 plot
  install.packages("ggpubr")
  library(ggpubr)
  ggarrange(p1, p2, p3,nrow = 1, common.legend  = TRUE, legend= "bottom")
```

Figure 28. All-in-one spaghetti plot for each index. Source: Figure by author.

2.12. Useful Resources

2.12.1. R Books

- Population Health Data Science with R [21].
- Modern R with the tidyverse [22].

2.12.2. Datasets

- The datasets used in some examples of this book are of Bangladeshi infants published by Subramanian et al. [13]. These datasets may be downloaded from Gordon's lab website, or from my GitHub repo (https://github.com/nhanhocu/metamicrobiomeR/tree/master/inst/extdata/QIIME_outputs/Bangladesh, accessed on 1 January 2026).

2.13. Chapter Summary

- Data Import: the "*rio*" package, particularly the *import()* and *import_list()* functions, is highly useful.
- Data manipulation: the "*tidyverse*" collection of packages, especially the key functions from the "*dplyr*" package, like *filter()*, *select()*, *rename()*, and *mutate()*, are very handy.
- Data Visualization: the "*ggplot2*" package is the primary tool for creating a wide range of graphics.
- Descriptive statistics: the *summarise()* function from "*dplyr*", often in combination with *group_by()* for group-level summaries is handy.
- Creating summary tables: the "*arsenal*" package (with function *tableby()*) is among the useful packages.

3. Basic Statistics with R

This chapter introduces foundational statistical concepts and their practical implementation using R. It is essential to first understand the nature of bio-medical data, particularly its distribution, as this determines the choice of appropriate statistical tests. This chapter covers the two most critical data distributions (the Normal Distribution for continuous data and the Binomial Distribution for binary outcomes) and provides the essential R functions for working with them. Furthermore, this chapter introduces the readers to the most commonly used statistical tests (the *t*-test and its non-parametric alternatives (Wilcoxon and Kruskal–Wallis tests) for comparison of continuous variables and the chi-square/Fisher's Exact tests for comparisons of categorical variables). This chapter also includes a section about power-sample size estimation for bio-medical studies.

3.1. Data Distribution

Understanding the distribution of the data would help with choosing proper statistical tests and interpreting the findings. The two most common and important distributions (normal and binomial distributions) will be discussed.

First, we load the main R packages used in this chapter ("*knitr*" [5], "*rmarkdown*" [6], "*kableExtra*" [12], "*arsenal*" [19], "*car*" [23], "*pwr*" [24], "*samplesizeestimator*" [25], "*presize*" [26]):

```
#load multiple packages
Packages <- c("knitr", "rmarkdown", "kableExtra", "arsenal", "car", "pwr", "samplesizeestimator", "presize")
lapply(Packages, library, character.only = TRUE)
```

3.1.1. The Normal Distribution

According to the Central Limit Theorem, the distribution of sample means from any population will tend to be normal as the sample size increases. The normal distribution is defined by two key parameters: the mean (μ), which marks its center, and the standard deviation (σ), which describes its spread [27]. Below are key R functions related to the normal distribution:

- *dnorm(x, mean, sd)*: Density. Calculates the height of the curve at a point x.
- *pnorm(q, mean, sd)*: Probability. Calculates the cumulative probability (area under the curve) up to a point q.
- *qnorm(p, mean, sd)*: Quantile. The inverse of *pnorm*. Finds the value x associated with a cumulative probability p.
- *rnorm(n, mean, sd)*: Random. Generates n random numbers from a normal distribution.

Below is an example to visualize the normal distribution of adult male height in a population with a mean of 178 cm and a standard deviation of 7 cm (Figure 29).

```r
# First, generate a large sample of random data to simulate the population
set.seed(123) # for reproducibility
simulated_heights <- rnorm(1000, mean = 178, sd = 7) #Create a histogram to
see the sample's distribution hist(simulated_heights,
        breaks = 30, # Suggest more bins for a smoother look
        freq = FALSE, # Plot density, not frequency, to allow overlay
        main = "Distribution of Adult Male Heights", xlab = "Height
        (cm)",
        col = "lightblue")
#Overlay the theoretical normal curve
curve(dnorm(x, mean = 178, sd = 7), from = 150, to =
         210,
         add = TRUE, # Add to the existing plot
         col = "darkred",
         lwd = 2) # Line width #Add a
legend
legend("topright", legend = c("Simulated Data",
                                        "Theoretical Normal Curve"),
          col = c("lightblue", "darkred"), lty = 1, lwd = 2,
          bty = "n")
```

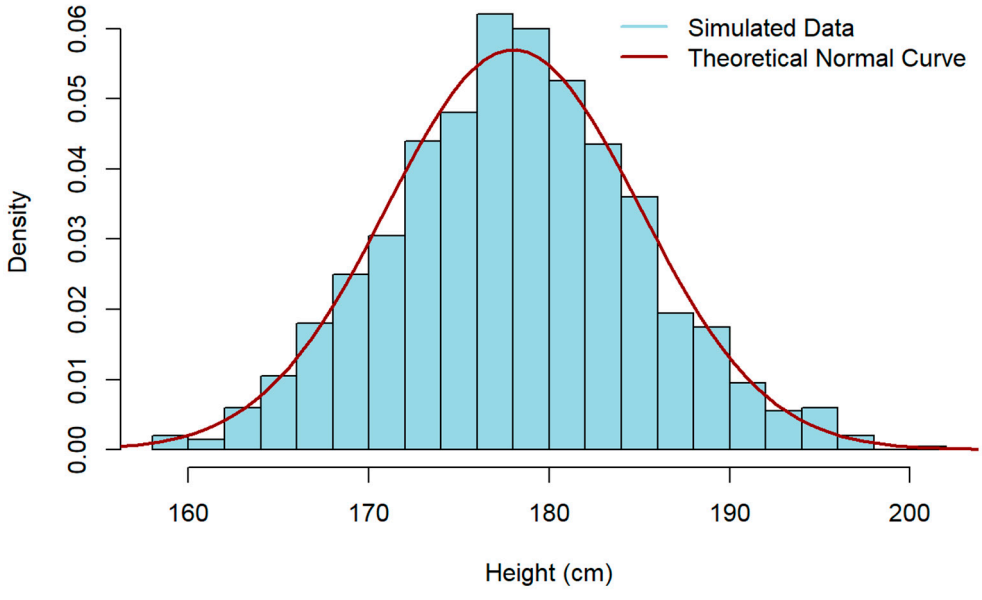


Figure 29. Example normal distribution. Source: Figure by author.

The example code provided above uses several fundamental Base R functions to construct a multi-layered visualization:

- *hist()*: This is the primary function used to create a histogram.
    - *breaks = 30*: This argument specifies the number of bins. Increasing this number provides a smoother, more detailed look at the sample's distribution.
    - *freq = FALSE*: Crucially, this setting tells R to plot the density on the y-axis rather than the raw frequency (counts). This is necessary to ensure the

histogram and the theoretical curve share the same vertical scale for a proper overlay.

- *curve()*: This function is used to draw the mathematical representation of a function over a specified range.
  - *dnorm(x, mean = 178, sd = 7)*: This is the density function for the normal distribution, defining the shape of the curve.
  - *add = TRUE*: This is a vital parameter that instructs R to draw the curve on top of the existing histogram rather than creating a completely new plot.
- *legend()*: This function adds a descriptive key to the plot to help the reader distinguish between different layers (in this case, the Simulated Data and the Theoretical Normal Curve).
  - *topright*: Specifies the location of the legend on the plot canvas.
  - *bty = "n"*: This removes the box border around the legend for a cleaner appearance.

Note that we illustrate the use of standard R plots in this chapter. One may also use "*ggplot2*" [18] for plotting as introduced in the previous chapter. For example, the above plot can be done with "*ggplot2*" as below (Figure 30).

```
ggplot(data.frame(simulated_heights), aes(x = simulated_heights)) +
 geom_histogram(aes(y = after_stat(density), fill = "Simulated Data"), bins = 30, color = "black") +
 stat_function(aes(color = "Theoretical Normal Curve"), fun = dnorm, args = list(mean = 178, sd = 7), size = 1) +
 labs(title = "Distribution of Adult Male Heights", x = "Height (cm)", y = "Density", fill = "", color = "") +
 theme_minimal() + theme(legend.position = "top") +
 scale_fill_manual(values = c("Simulated Data" = "lightblue")) +
 scale_color_manual(values = c("Theoretical Normal Curve" = "darkred"))
```

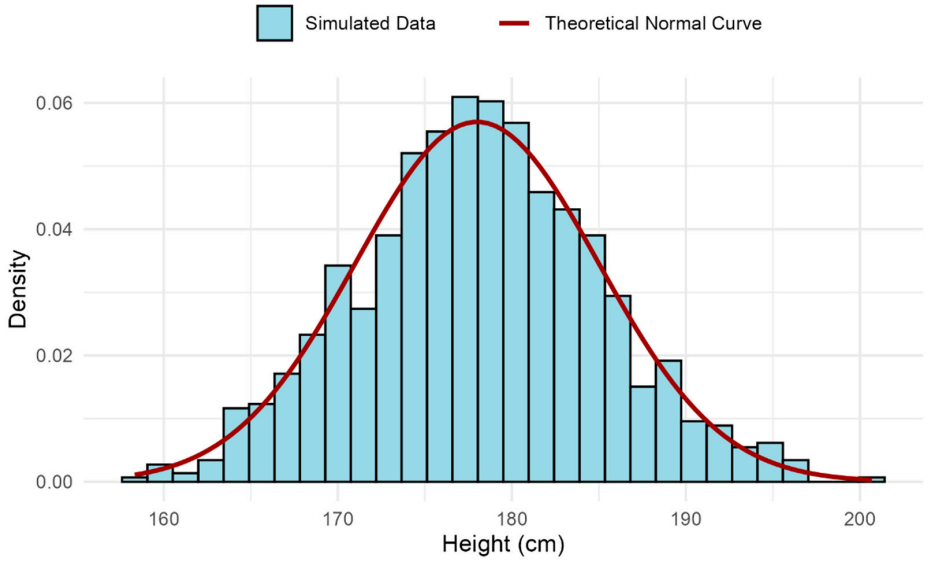


Figure 30. Example normal distribution using "*ggplot2*". Source: Figure by author.

3.1.2. The Binomial Distribution

The binomial distribution is used for discrete data. It models the number of "successes" in a fixed number (n or size) of independent trials, where the probability of success (p) is the same for each trial [28]. For example, in a trial of 20 patients, a new drug has a known response probability of 40%. What is the probability that exactly eight patients will respond? We use *dbinom()* (the density or, for discrete data, the probability mass function).

```
dbinom(x = 8, size = 20, prob = 0.40)
```

```
## [1]  0.1797058
```

The output shows that there is approximately an 18.0% chance that exactly 8 out of 20 patients will respond. Another question is: what is the probability that "8 or fewer" patients will respond? We use *pbinom()* for the cumulative probability.

```
pbinom(q = 8, size                                    = 20, prob = 0.40)
```

```
## [1]  0.5955987
```

The output shows that there is a 59.6% chance that 8 or fewer patients will respond to the treatment.

We can visualize the entire probability distribution for this example (Figure 31).

```
# Possible outcomes: 0 to 20 successes
outcomes <- 0:20
# Calculate the probability for each outcome
probabilities <- dbinom(x = outcomes, size      = 20, prob = 0.40)

# Create a bar plot
barplot(probabilities,
                names.arg = outcomes,
                main = "Binomial Distribution (n = 20, p = 0.4)",
                xlab = "Number of Responding Patients",
                ylab =  "Probability",
                col = "skyblue")
```

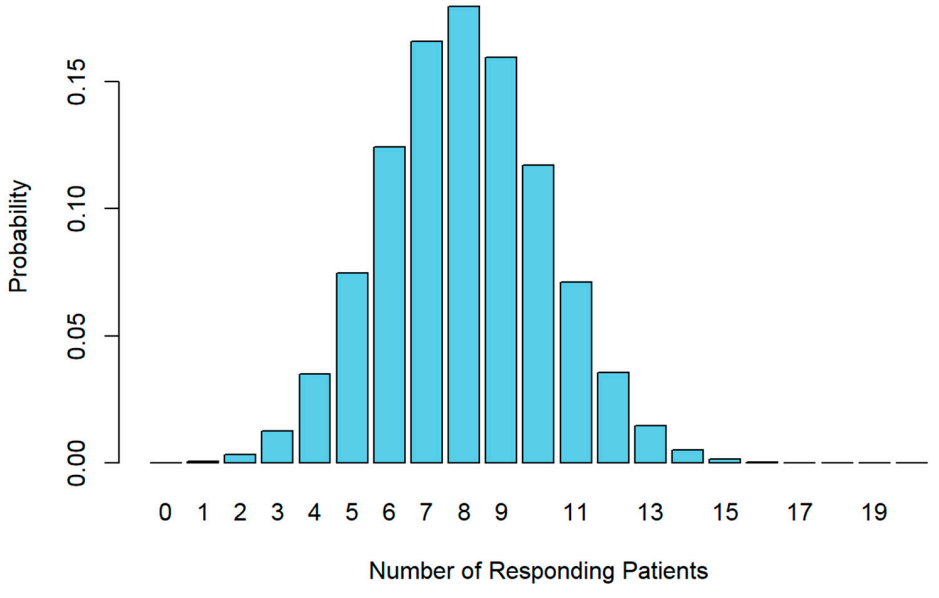


Figure 31. Example binomial distribution (n = 20, p = 0.4). Source: Figure by author.

The bar plot shows the probability of each possible outcome, from 0 successes to 20. We can see the most likely outcome is eight successes, and the probabilities tail off as we move away from the mean (which is np = 20*0.4 = 8).

The relationship between the binomial distribution and the normal distribution:

For large n, the binomial distribution approximates the normal distribution with mean = np and variance = np(1 − p). This approximation explains why proportions can often be analyzed using normal-based methods when the sample size is large.

3.2. Basic Statistical Tests

3.2.1. Tests for Comparing Continuous Variables

Continuous variables (e.g., BMI, cholesterol, and blood pressure) are compared between groups using parametric tests when the data are approximately normal and non-parametric tests otherwise. Another option for data with a non-normal distribution is to find a transformation that makes the data (approximately) normal, then use parametric tests for the transformed data.

Compare Mean to a Constant

To test if the mean of a single sample significantly differs from a known or hypothesized population mean. The *t*-test may be used in this case with the assumption:

- Independence of observations.
- Data are approximately normally distributed.

Example: Using the built-in iris dataset, we test if the mean sepal length ("Sepal.Length") of the virginica species is significantly different from a hypothesized value of 6.1 cm (this is a hypothesized value for example only, not a real value). First,

we check for normal distribution with a Q-Q plot using *qqnorm()* and *qqline()*, then perform a *t*-test (Figure 32).

```
# get the data
virginica_sepal_length <- iris$Sepal.Length[iris$Species == "virginica"]
# Check the normality assumption with a Q-Q Plot
qqnorm(virginica_sepal_length,
       main = "Q-Q Plot for Virginica Sepal Length")
qqline(virginica_sepal_length, col = "red", lwd = 2)
```

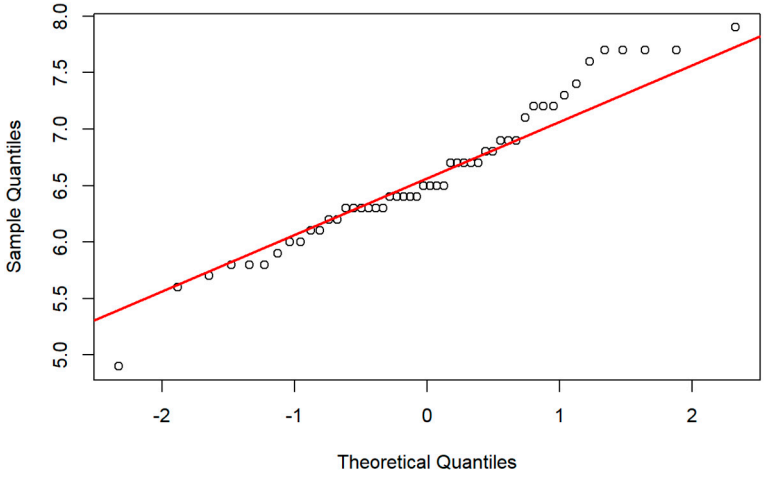


Figure 32. Q-Q plot. Source: Figure by author.

```
# 3. Perform the t-test
t.test(virginica_sepal_length, mu = 6.1)
```

```
##
## One Sample t-test
##
## data: virginica_sepal_length
## t = 5.4266, df = 49, p-value = 0.000001765
## alternative hypothesis: true mean is not equal to 6.1
## 95 percent confidence interval:
##          6.407285                  6.768715
## sample estimates:
## mean of x
##          6.588
```

The data fall roughly along the straight line (approximates normal distribution), so a *t*-test may be used. The results show that the mean is significantly different from 6.1 (p-value < 0.05 and 95% CI confidence interval does not include 6.1).

Note that one may try functions *stat_qq_band()* and *stat_qq_line()* of the package "*qqplotr*" [29], which is used together with the package "*ggplot2*" for the above Q-Q plot (Figure 33).

```
library(qqplotr)
library(ggplot2)
# Q-Q Plot with 95%CI
ggplot(data.frame(virginica_sepal_length),
aes(sample = virginica_sepal_length)) +
stat_qq_band() +  stat_qq_line() +  stat_qq_point() +
 labs(x = "Theoretical Quantiles",
y = "Sample Quantiles", title = "Q-Q Plot with Confidence Band"))
```

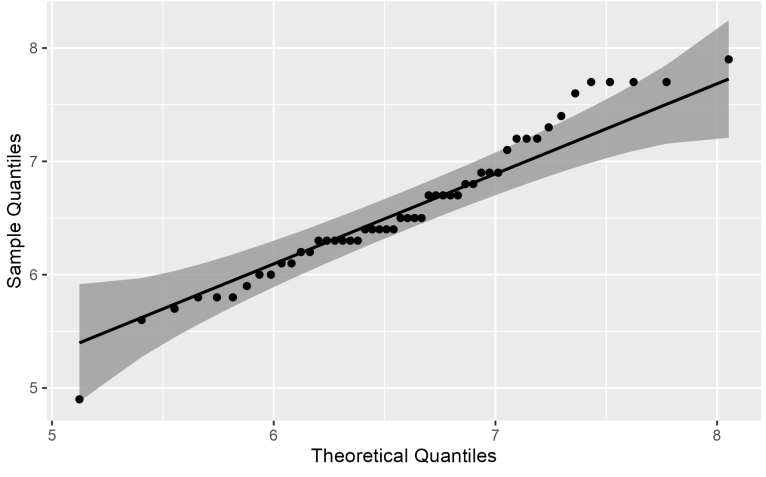


Figure 33. Q-Q plot using "*qqplotr*". Source: Figure by author.

Compare Means of Two Independent Groups

The assumptions for comparing the means of two independent groups using a two-sample *t*-test are:

- Independence of the two groups.
- Approximate normality within each group.
- Homogeneity of variances (the variances in the two groups are roughly equal). R's default *t.test()* uses the Welch *t*-test, which does not require this assumption and is therefore a robust choice. As such, this test is typically used directly without first performing formal tests of variance equality.

In practice, strict pre-testing of assumptions to decide which statistical test to apply is not generally recommended, because preliminary hypothesis testing (e.g., testing for equal variances before choosing the *t*-test version) can lead to incorrect inference and inflated Type I error rates. Instead, a more robust strategy is to use statistical procedures that remain valid when assumptions are not perfectly satisfied. Diagnostic tools such as Levene's test (available in the car package) may still be used to explore the degree of variance heterogeneity, but the result should be interpreted descriptively rather than used as a strict decision rule for selecting the statistical test.

The example below will test if there is a significant difference in the miles per gallon (mpg) between cars with automatic (am = 0) and manual (am = 1) transmissions in the *mtcars* dataset of the "*car*" package [23]. Check for homogeneity of variances using Levene's test (optional):

```r
library(car)
leveneTest(mpg ~ as.factor(am), data = mtcars)
```

```
## Levene's Test for Homogeneity of Variance (center = median)
##       Df  F value  Pr(>F)
## group  1  4.1876  0.04957 *
##       30
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The p-value is >0.05, so we do not reject the null of equal variances. Even so, the Welch t-test is a safe default.

Now we perform the t-test:

```r
t.test(mpg ~ am, data = mtcars)
```

```
##
## Welch Two Sample t-test
##
## data: mpg by am
## t = -3.7671, df = 18.332, p-value = 0.001374
## alternative hypothesis: true difference in means between group 0 and group 1
## is not equal to 0
## 95 percent confidence interval:
##          -11.280194                -3.209684
## sample estimates:
## mean in group 0 mean in group 1
##        17.14737        24.39231
```

The results show that the two sample means are significantly different (p-value < 0.05, 95% CI of the difference does not include 0).

Non-Parametric Alternative to t-Test

If the normal distribution assumption does not hold, Wilcoxon's rank sum test (*wilcox.test()*) may be used instead of the t-test to compare continuous variables of two groups.

```r
wilcox.test(mpg ~ am, data = mtcars, conf.int = TRUE)
```

```
## Wilcoxon rank sum test with continuity correction
##
## data: mpg by am
## W = 42, p-value = 0.001871
## alternative hypothesis: true location shift is not equal to 0
## 95 percent confidence interval:
##          -11.699942                -2.900042
## sample estimates:
## difference in location
##          -6.799963
```

The Wilcoxon rank-sum test is a nonparametric method that does not assume normality. Rather than comparing group means, it evaluates whether observations from one group tend to be systematically larger or smaller than those from the other group. A statistically significant result indicates a difference between the group distributions, typically interpreted as a difference in location when the distributions have similar shapes.

For descriptive reporting, results are appropriately summarized using the median and interquartile range (IQR) for each group instead of the mean and standard deviation (SD).

3.2.2. Tests for Comparing Categorical Variables

Chi-square test (*chisq.test()*) may be used to test for a statistically significant association between two categorical variables, with the assumption:

- Data are counts;
- Independence of observations;
- Adequate expected cell counts (expected counts > 5).

The following example will test if there is an association between the number of gears ("gear") and the number of carburetors ("carb") using the *mtcars* dataset.

```
# Create a contingency table
contingency_table <- table(mtcars$gear, mtcars$carb)
contingency_table
```

```
##
##    1  2  3  4  6  8
##3  3  4  3  5  0  0
##4  4  4  0  4  0  0
##5  0  2  0  1  1  1
```

```
# Perform the test
chisq_result <- chisq.test(contingency_table)
chisq_result
```

```
##
## Pearson's Chi-squared test
##
## data: contingency_table
## X-squared = 16.518, df = 10, p-value = 0.08573
```

```
# Check the expected counts assumption
chisq_result$expected
```

```
##
##        1          2       3         4         6         8
##   3 3.28125  4.6875 1.40625 4.6875  0.46875   0.46875
##   4 2.62500  3.7500 1.12500 3.7500  0.37500   0.37500
##   5 1.09375  1.5625 0.46875 1.5625  0.15625   0.15625
```

For small samples or when expected cell counts are < 5, the asymptotic approximation underlying the chi-square test may not be reliable. In such situations, exact tests are often recommended. A commonly used approach is Fisher's Exact Test (*fisher.test()*), which calculates the exact probability of the observed table (and more extreme tables) under the null hypothesis of independence. Fisher's test is particularly useful for small sample sizes and sparse contingency tables. Fisher's test computes the exact probability of the observed configuration (and more extreme ones) under the null hypothesis of independence. It is reliable even for very small datasets, which are commonly used in pilot studies or rare-event analyses. In the example above, many expected counts are <5, so Fisher's exact test may be used.

```r
fisher.test(contingency_table)
```

```
##
## Fisher's Exact Test for Count Data
##
## data: contingency_table
## p-value = 0.2434
## alternative hypothesis: two.sided
```

Fisher's exact test is a conditional test, meaning that it conditions on the observed marginal totals of the contingency table. While this approach guarantees exact control of the Type I error rate, it may sometimes be conservative, resulting in reduced statistical power.

For this reason, unconditional exact tests have been proposed as alternatives. These tests do not condition on fixed margins and can provide greater statistical power while still maintaining appropriate Type I error control. Two commonly discussed unconditional tests are:

- Barnard's exact test;
- Boschloo's exact test.

These tests are particularly useful for 2×2 contingency tables, where they can outperform Fisher's test in terms of statistical power. In R, implementations of these methods are available in specialized packages such as "*Exact*" [30] and "*exact2x2*" [31].

3.3. Data Summary and Basic Statistical Test in a Table

As introduced in the previous chapter, the "*arsenal*" package may be used to summarize and perform corresponding statistical tests for many variables in a dataset [19]. The example below summarizes and compares all the variables in the dataset *mtcars* between *am* groups and presents the results in a publication-ready table. Note that, as the default in *tableby()*, the chi-square test and one-way ANOVA test are used for categorical variables and continuous variables, respectively, for the comparison between three or more groups (Table 5).

```r
library(arsenal)
varsum<-colnames(mtcars)[!colnames(mtcars) %in% "am"] mylabels
<-as.list(varsum)
names(mylabels)<-varsum
tab1 <- tableby(as.formula(paste("am", paste(varsum, collapse = "+"),
                                 sep = "~")),
                                 data = mtcars)
kable(summary(tab1,
          labelTranslations = mylabels, text =
          TRUE),
       longtable = TRUE,
       caption = "Summary, statistical tests for all variables")%>%
  kable_styling(latex_options = c("scale_down",
                                  "hold_position",
                                  "repeat_header"))
```

Table 5. Summary, statistical tests for all variables.

	0 (N = 19)	**1 (N = 13)**	**Total (N = 32)**	***p*-Value**
mpg				<0.001
- Mean (SD)	17.147 (3.834)	24.392 (6.167)	20.091 (6.027)	
- Range	10.400–24.400	15.000–33.900	10.400–33.900	
cyl				0.002
- Mean (SD)	6.947 (1.545)	5.077 (1.553)	6.188 (1.786)	
- Range	4.000–8.000	4.000–8.000	4.000–8.000	
disp				<0.001
- Mean (SD)	290.379 (110.172)	143.531 (87.204)	230.722 (123.939)	
- Range	120.100–472.000	71.100–351.000	71.100–472.000	
hp				0.180
- Mean (SD)	160.263 (53.908)	126.846 (84.062)	146.688 (68.563)	
- Range	62.000–245.000	52.000–335.000	52.000–335.000	
drat				<0.001
- Mean (SD)	3.286 (0.392)	4.050 (0.364)	3.597 (0.535)	
- Range	2.760–3.920	3.540–4.930	2.760–4.930	
wt				<0.001
- Mean (SD)	3.769 (0.777)	2.411 (0.617)	3.217 (0.978)	
- Range	2.465–5.424	1.513–3.570	1.513–5.424	
qsec				0.206
- Mean (SD)	18.183 (1.751)	17.360 (1.792)	17.849 (1.787)	
- Range	15.410–22.900	14.500–19.900	14.500–22.900	
vs				0.357
- Mean (SD)	0.368 (0.496)	0.538 (0.519)	0.438 (0.504)	
- Range	0.000–1.000	0.000–1.000	0.000–1.000	
gear				<0.001
- Mean (SD)	3.211 (0.419)	4.385 (0.506)	3.688 (0.738)	
- Range	3.000–4.000	4.000–5.000	3.000–5.000	
carb				0.754
- Mean (SD)	2.737 (1.147)	2.923 (2.178)	2.812 (1.615)	
- Range	1.000–4.000	1.000–8.000	1.000–8.000	

Source: Table by author.

The summary options and statistical tests by the *tableby()* may be adjusted. Below is an example for setting the options with *tableby.control()* then re-applying the *tableby()* function. In this example, we use Kruskal–Wallis' test and add some other data summary options (Table 6):

```r
# set summary options and statistical tests of choice
my_controls <- tableby.control(
    test = TRUE,
    total = TRUE,
    numeric.test = "kwt",
    cat.test = "chisq",
    numeric.stats =  c("meansd", "medianq1q3","meanCI","range","Nmiss2"),
    cat.stats = c("countpct", "Nmiss2"),
    stats.labels = list(
        meansd = "Mean (SD)",
        medianq1q3    = "Median (Q1, Q3)",
        range = "Min - Max",
        Nmiss2 = "Missing",
        meanCI = "Mean (95%CI)"
    )
)
# Apply the tableby function again:
varsum<-colnames(mtcars)[!colnames(mtcars) %in% "am"]
mylabels <-as.list(varsum)
names(mylabels)<-varsum
tab1 <- tableby(as.formula(paste("am", paste(varsum,collapse = "+"),
                                 sep = "~")),
                                 data = mtcars,
                                 control = my_controls)
kable(summary(tab1,
              labelTranslations = mylabels, text =
              TRUE),
          caption = "Summary, statistical tests, adjusted options") %>%
          kable_styling(latex_options = c("scale_down", "hold_position"))
```

Table 6. Summary, statistical tests, adjusted options.

	0 (N = 19)	**1 (N = 13)**	**Total (N = 32)**	***p*-Value**
mpg				0.002
- Mean (SD)	17.147 (3.834)	24.392 (6.167)	20.091 (6.027)	
- Median (Q1, Q3)	17.300 (14.950, 19.200)	22.800 (21.000, 30.400)	19.200 (15.425, 22.800)	
- Mean (95% CI)	17.147 (15.299, 18.995)	24.392 (20.666, 28.119)	20.091 (17.918, 22.264)	
- Min–Max	10.400–24.400	15.000–33.900	10.400–33.900	
- Missing	0	0	0	
cyl				0.004
- Mean (SD)	6.947 (1.545)	5.077 (1.553)	6.188 (1.786)	
- Median (Q1, Q3)	8.000 (6.000, 8.000)	4.000 (4.000, 6.000)	6.000 (4.000, 8.000)	
- Mean (95% CI)	6.947 (6.203, 7.692)	5.077 (4.139, 6.015)	6.188 (5.544, 6.831)	
- Min–Max	4.000–8.000	4.000–8.000	4.000–8.000	
- Missing	0	0	0	
disp				<0.001
- Mean (SD)	290.379 (110.172)	143.531 (87.204)	230.722 (123.939)	
- Median (Q1, Q3)	275.800 (196.300, 360.000)	120.300 (79.000, 160.000)	196.300 (120.825, 326.000)	
- Mean (95% CI)	290.379 (237.278, 343.480)	143.531 (90.834, 196.228)	230.722 (186.037, 275.407)	
- Min–Max	120.100–472.000	71.100–351.000	71.100–472.000	
- Missing	0	0	0	
hp				0.044
- Mean (SD)	160.263 (53.908)	126.846 (84.062)	146.688 (68.563)	
- Median (Q1, Q3)	175.000 (116.500, 192.500)	109.000 (66.000, 113.000)	123.000 (96.500, 180.000)	
- Mean (95% CI)	160.263 (134.280, 186.246)	126.846 (76.048, 177.645)	146.688 (121.968, 171.407)	
- Min–Max	62.000–245.000	52.000–335.000	52.000–335.000	
- Missing	0	0	0	
drat				<0.001
- Mean (SD)	3.286 (0.392)	4.050 (0.364)	3.597 (0.535)	
- Median (Q1, Q3)	3.150 (3.070, 3.695)	4.080 (3.850, 4.220)	3.695 (3.080, 3.920)	
- Mean (95% CI)	3.286 (3.097, 3.475)	4.050 (3.830, 4.270)	3.597 (3.404, 3.789)	
- Min–Max	2.760–3.920	3.540–4.930	2.760–4.930	
- Missing	0	0	0	

Table 6. *Cont.*

	0 (N = 19)	1 (N = 13)	Total (N = 32)	*p*-Value
wt				<0.001
- Mean (SD)	3.769 (0.777)	2.411 (0.617)	3.217 (0.978)	
- Median (Q1, Q3)	3.520 (3.438, 3.843)	2.320 (1.935, 2.780)	3.325 (2.581, 3.610)	
- Mean (95% CI)	3.769 (3.394, 4.144)	2.411 (2.038, 2.784)	3.217 (2.864, 3.570)	
- Min–Max	2.465–5.424	1.513–3.570	1.513–5.424	
- Missing	0	0	0	
qsec				0.258
- Mean (SD)	18.183 (1.751)	17.360 (1.792)	17.849 (1.787)	
- Median (Q1, Q3)	17.820 (17.175, 19.170)	17.020 (16.460, 18.610)	17.710 (16.892, 18.900)	
- Mean (95% CI)	18.183 (17.339, 19.027)	17.360 (16.277, 18.443)	17.849 (17.204, 18.493)	
- Min–Max	15.410–22.900	14.500–19.900	14.500–22.900	
- Missing	0	0	0	
vs				0.349
- Mean (SD)	0.368 (0.496)	0.538 (0.519)	0.438 (0.504)	
- Median (Q1, Q3)	0.000 (0.000, 1.000)	1.000 (0.000, 1.000)	0.000 (0.000, 1.000)	
- Mean (95% CI)	0.368 (0.130, 0.607)	0.538 (0.225, 0.852)	0.438 (0.256, 0.619)	
- Min–Max	0.000–1.000	0.000–1.000	0.000–1.000	
- Missing	0	0	0	
gear				<0.001
- Mean (SD)	3.211 (0.419)	4.385 (0.506)	3.688 (0.738)	
- Median (Q1, Q3)	3.000 (3.000, 3.000)	4.000 (4.000, 5.000)	4.000 (3.000, 4.000)	
- Mean (95% CI)	3.211 (3.009, 3.412)	4.385 (4.079, 4.691)	3.688 (3.421, 3.954)	
- Min–Max	3.000–4.000	4.000–5.000	3.000–5.000	
- Missing	0	0	0	
carb				0.720
- Mean (SD)	2.737 (1.147)	2.923 (2.178)	2.812 (1.615)	
- Median (Q1, Q3)	3.000 (2.000, 4.000)	2.000 (1.000, 4.000)	2.000 (2.000, 4.000)	
- Mean (95% CI)	2.737 (2.184, 3.290)	2.923 (1.607, 4.239)	2.812 (2.230, 3.395)	
- Min–Max	1.000–4.000	1.000–8.000	1.000–8.000	
- Missing	0	0	0	

Source: Table by author.

Readers may try other packages like "*tableone*" [32] or "*gtsummary*" [33], which may also generate publication-ready baseline tables for bio-medical papers.

3.4. Sample Size—Power Estimation

A well-designed study has a high probability of detecting a true effect. The probability of detecting a true effect of a study is its "statistical power". A good study should have a power ≥ 0.8.

Power analysis typically involves the following components [34]:

- Sample Size: The number of subjects in the study.
- Effect Size (e.g., Cohen's d): The magnitude of the effect (e.g., a large difference is easier to detect than a small one).
- Significance Level: The threshold for statistical significance (usually 0.05).
- Power: The probability of correctly rejecting a false null hypothesis.

The "*pwr*" [24], "*samplesizeestimator*" [25], and "*presize*" [26] packages may be used for power—sample size estimation in the examples below.

3.4.1. Power, Sample Size for Comparing Continuous Variables

The following example uses a *t*-test for comparing continuous variables between two groups: A researcher is testing a new drug to lower cholesterol. They hypothesize

the drug will lower LDL cholesterol by 15 mg/dL compared to a placebo. The standard deviation of LDL cholesterol is known to be 30 mg/dL.

The sample size needed for 80% power (using *pwr.t.test()* of the "*pwr*" package [24]):

```
library(pwr)
# Calculate effect size (Cohen's d)
effect_size_d <- 15 / 30 # Mean difference / standard deviation

# Calculate sample size
pwr.t.test(d = effect_size_d, sig.level = 0.05, power = 0.80,
           type = "two.sample")
```

```
##
## Two-sample t test power calculation
##
##   n           = 63.76561
##   d           = 0.5
##   sig.level   = 0.05
##   power       = 0.8
##   alternative = two.sided
##
## NOTE: n is the number in *each* group
```

Note that one may also use *power.t.test()* from the base package "stats"of base R [1].

```
# Use delta = raw mean difference, sd = standard deviation
power.t.test (delta = 15, sd = 30, sig.level = 0.05, power = 0.80,
              type = "two.sample")
```

```
## Two-sample t test power calculation
##
## n = 63.76576
## delta = 15
## sd = 30
## sig.level = 0.05
## power = 0.8
## alternative = two.sided
##
## NOTE: n is number in *each* group
```

The output shows that approximately 64 participants are needed in each group, and a total of 128 participants are needed for the study.

For some reason, the study stops after recruiting 40 subjects per group. What is the power for detecting the difference?

```
pwr.t.test(n = 40, d = 0.5, sig.level = 0.05, type= "two.sample")
```

```
##
##Two-sample t test power calculation
##
##n = 40
##d = 0.5
##sig.level = 0.05
##power = 0.5981469
##alternative = two.sided
##
## NOTE: n is number in *each* group
```

The output shows that with 40 subjects per group, the power drops to 0.6.

3.4.2. Power, Sample Size for Comparing Categorical Variables

The example below uses the chi-square test for comparing categorical variables between two groups: A city wants to test a new educational campaign to increase flu vaccination rates. The current rate (p_control) is 40%. They will consider the campaign a success if it raises the rate to 55% (p_campaign). We can calculate the sample size needed using *pwr.2p.test()* after estimating effect size using *ES.h()*:

```
# Calculate effect size h
effect_size_h <- ES.h(p1 = 0.55, p2 = 0.40)
effect_size_h
```

```
## [1]  0.3015253
```

```
# Calculate sample size per group
pwr.2p.test(h = effect_size_h,
            sig.level = 0.05,
            power = 0.80,
            alternative = "two.sided")
```

```
##
## Difference of proportion power calculation for binomial distribution
## (arcsine transformation)
##
## h = 0.3015253
## n = 172.6589
## sig.level = 0.05
## power = 0.8
## alternative = two.sided
##
## NOTE: same sample sizes
```

Note that one may also use *power.prop.test()* from the base package "stats" [1]. *power.prop.test()* internally computes the effect size from p1 and p2. One does not need to input Cohen's h.

```r
power.prop.test(p1 = 0.55, p2 = 0.40,
                sig.level = 0.05,
                power = 0.80,
                alternative = "two.sided")
```

```
## Two-sample comparison of proportions power calculation
## n = 172.7999
## p1 = 0.55
## p2 = 0.4
## sig.level = 0.05
## power = 0.8
## alternative = two.sided
##
## NOTE: n is number in *each* group
```

The output shows that approximately 173 participants are needed in each group (total N = 346) to have an 80% chance of detecting a true increase in vaccination rate from 40% to 55%.

3.4.3. Sample Size for Estimating Parameter with Desired Precision

This is a different type of sample size calculation. Here, the goal is not to test a hypothesis but to estimate a population parameter (like a mean or proportion) with a pre-specified level of precision E (e.g., a narrow confidence interval).

Estimating Mean

The example below estimates the mean cholesterol level in a specific patient population. Based on prior literature, they expect the standard deviation (σ) to be 40 mg/dL. They want their final estimate to be accurate within $\pm$ 5 mg/dL with 95% confidence.

The package "*samplesizeestimator*" may be used for estimating the mean with absolute precision [25].

```r
library(samplesizeestimator)
sample_size <- estm(sig = 40, prec = 5, alp = 0.05,
                    relative = FALSE) #alpha = 0.05 corresponding to 95%CI
print(sample_size)
```

```
## Sample_Size
## [1] 246
##
## [[2]]
## [1] " Description: Sample size is estimated with an expected"
## [2] "standard deviation of the 'outcome of interest' among the"
## [3] "'population' as 40 units at 5% level of significance with"
## [4] "an absolute precision of 5 units, the estimated sample"
## [5] "size is 246."
```

The output shows that 246 patients are needed to be 95% confident that the true population mean cholesterol is within 5 mg/dL of their sample mean.

Estimating Proportion

The example below estimates the proportion of patients who are "very satisfied" with care in a hospital. The estimate should be within $\pm$ 4% with 95% confidence. There is no prior information about the true proportion.

The package "*presize*" [35] may be used with various method options (using *prec_prop()*). Note that, if no best guess for the proportion exists, use $p = 0.5$, as this is the most conservative choice and yields the largest required sample size.

```
library(presize)
prec_prop(p = 0.5, conf.width = 0.08, conf.level = 0.95,
          method = "wald") # CI width = margin of error (0.04) *2
```

```
##
## sample size for a proportion with Wald confidence interval.
##
##      p    padj   n          conf.width   conf.level   lwr    upr
## 1    0.5  0.5    600.2279   0.08         0.95         0.46   0.54
##
## NOTE: padj is the adjusted proportion, from which the ci is calculated.
```

The output shows that 601 patients are needed to be 95% confident that the true proportion of "very satisfied" patients is within 4 percentage points of their survey result.

One may calculate the expected confidence interval width for a given sample size using *prec_prop()*.

```
prec_prop(p=0.5, n =601, method = "wald")
```

```
##
## precision for a proportion with Wald confidence interval.
##
##      p    padj   n    conf.width   conf.level   lwr         upr
## 1    0.5  0.5    601  0.0799486    0.95         0.4600257   0.5399743
##
## NOTE: padj is the adjusted proportion, from which the ci is calculated.
```

Note that the Wald confidence interval (method used above) can have poor coverage properties, especially for moderate sample sizes or proportions far from 0.5. As a result, the true coverage probability may be lower than the nominal 95%. For this reason, more reliable interval estimators may be used, such as:

- Continuity-corrected intervals, which adjust the binomial approximation to improve accuracy.
- The Clopper–Pearson exact interval, which is based on the exact binomial distribution, guarantees that the coverage probability is at least the nominal confidence level.

The Clopper–Pearson interval avoids the under-coverage problem associated with the Wald interval and is therefore often preferred when reporting confidence intervals for proportions. The Clopper–Pearson interval can be calculated using *binom.test()*. For example, suppose that in a sample of 601 patients, 300 patients ($p = 0.5$) report being very satisfied:

```
binom.test(300, 601, conf.level = 0.95)
```

```
## Exact binomial test
##
## data: 300 and 601
## number of successes = 300, number of trials = 601,
## p-value = 1
## alternative hypothesis: true probability of success is not equal to 0.5
## 95 percent confidence interval:
##           0.4584504                   0.5398940
## sample estimates:
## probability of success
##           0.4991681
```

Of note, one could also, for example, use the function *ssize.prop.ci* of package "*MKpower*" [36] with various method options for sample size calculation.

```
ssize.prop.ci(prop = 0.5, width = 0.08, method = "wald")
```

```
## Sample size calculation by method of wald
##
## n = 600.2279
## prop = 0.5
## width = 0.08
## conf.level = 0.95
##
## NOTE: Two-sided confidence interval
```

```
ssize.prop.ci(prop = 0.5, width = 0.08, method = "wald-cc")
```

```
## Sample size calculation by method of wald-cc
##
## n = 624.9779
## prop = 0.5
## width = 0.08
## conf.level = 0.95
##
## NOTE: Two-sided confidence interval
```

```
ssize.prop.ci(prop = 0.5, width = 0.08, method = "clopper-pearson")
```

```
## Sample size calculation by method of clopper-pearson
##
## n = 625
## prop = 0.5
## width = 0.08
## conf.level = 0.95
##
## NOTE: Two-sided confidence interval
```

3.5. Chapter Summary

- Examining the data distribution is important before performing statistical tests;
- T-test (*t.test()*) (normal assumption required) and its non-parametric alternatives (*wilcox.test()*, *kruskal.test()*) are the common statistical tests used for testing continuous variables;
- Chi-square tests (*chisq.test()*) and Fisher's exact tests (*fisher.test()*) are the common statistical tests used for testing categorical variables;
- Power—sample size analysis may be done with the "*pwr*", "*samplesizeestimator*", and "*presize*" packages.

4. Linear Regression with R

This chapter provides a basic guide to fitting and interpreting Simple and Multiple Linear Regression models for bio-medical data using R. The chapter begins by introducing basic concepts of the linear model, including its core formula and key assumptions such as linearity, independence, and homoscedasticity. The practical focus of the chapter centers on the *lm()* function in R for fitting linear models. Readers will also learn how to interpret the model output, perform essential model diagnostic checks, model selection, and model validation.

4.1. Introduction to Linear Regression

Linear regression evaluates the relationship between a continuous dependent variable (outcome) and one or more independent variables (predictors). From a sample of data, linear regression is used to estimate population parameters characterizing these relationships and their uncertainties.

Linear regression has two primary applications:

- Estimating Effects: It helps estimate the specific effect of a predictor on the outcome, particularly while controlling for other variables. This is a key strength of multiple linear regression, which allows you to statistically estimate the impact of one variable (e.g., smoking) while controlling the confounding influence of others (e.g., age or weight).
- Prediction: It helps predict outcomes for future observations. Once a model is built and validated, you can input the predictor values for a new individual, and the model will forecast the most likely value for their outcome variable.

First, we load the main R packages used in this chapter ("*knitr*" [5], "*rmarkdown*" [6], "*tidyverse*" [11], "*arsenal*" [19], "*kableExtra*" [12], "*yardstick*" [37]):

```
#load multiple packages
Packages <- c("knitr", "rmarkdown", "tidyverse", "arsenal",
              "kableExtra", "yardstick")
lapply(Packages, library, character.only = TRUE)
```

4.2. Simple Linear Regression

4.2.1. Simple Linear Model with R

Simple linear regression quantifies the relationship between one outcome and one predictor. Below is a generated example of data:

```
##      x         y
## 1 3.620757 0.8449674
## 2 3.035641 0.8304951
## 3 3.773154 0.9622945
## 4 4.272489 0.9096735
## 5 3.370975 0.8072324
## 6 2.837146 0.8259437
```

In which

- y is outcome or response or dependent variable (DV);
- x is the predictor or regressor or independent variable (IV).

Simple linear regression assumes a linear relationship between x and y. The formula for the simple linear regression model may be written as:

$$y = \beta_0 + \beta_1 x + \epsilon$$

In which

- β_0 is the intercept or the predicted y when $x = 0$.
- β_1 is the slope or the predicted change in y for a one-unit increase in x.
- ϵ is the error term or the deviation of the predicted y value from the observed y [38].

A simple linear model may be fitted with base R using *lm()* as follows:

```
options(scipen = 0) #adjusting the global printing option to display p-values in scientific notation
slm.fit <- lm(y ~ x, data= df)
```

The *lm()* returns complex objects which may be examined by the following:

- *coef()*: model coefficients
- *summary()*: coefficients, standard errors, *p*-values, model F-test, etc.
- *predict()*: model predictions
- *residuals()*: various kinds of residuals
- *confint()*: confidence intervals for coefficients
- *plot()*: diagnostic plots

The result output may be summarized as follows:

```
summary(slm.fit)
```

```
## Call:
## lm(formula = y ~ x, data = df)
## Residuals:
##                Min          1Q          Median          3Q          Max
##                −0.25771   −0.07126    0.00101    0.05635    0.32485
##
## Coefficients:
##                Estimate    Std. Error    t value    Pr(>|t|)
## (Intercept)    0.525215    0.019149      27.43      <2×10-16 ***
## x              0.111954    0.006056      18.48      <2×10-16 ***
## ---
## Signif. codes: 0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1
##
## Residual standard error: 0.1034 on 298 degrees of freedom
## Multiple R-squared: 0.5342, Adjusted R-squared: 0.5326
## F-statistic: 341.7 on 1 and 298 DF, p-value: <2.2×10-16
```

The *summary()* output shows:

- Model formula.
- Summary of residual distribution.

- Estimated coefficients, standard errors, t-values, and p-values (for testing a hypothesis that a coefficient equals zero).
- A quick visual shortcut to p-values by *.
- Residual standard error measures the average amount that observed responses deviate from the fitted regression line.
- Degrees of freedom (df) represent the number of independent observations (N) in the data that are available to estimate a parameter. For linear regression, p = the number of predictors +1, and $df = N - p$.
- R-square (R^2), a measure to test if the model is a good fit, shows the amount of variance of y explained by x.

$$R^2 = Explained\ variation\ of\ the\ model / Total\ variation\ of\ the\ model$$

  If the model fits the data well, R^2 is near 1, and if the model fits the data poorly, R^2 is near 0. *Adjusted* R^2 shows the same as R^2 but adjusted by the number of cases and the number of variables. When the number of variables is small and the number of outcomes is very large, then the *Adjusted* R^2 is closer to R^2.
- F-statistic: indicates whether the overall model is statistically significant (at least one predictor is related to the outcome).

In the simple linear regression model, the test of the overall model is equivalent to the test of the slope. In the summary output, the p-value for the F-statistic is identical to the p-value for the predictor's t-test, and the F-statistic is exactly the square of the t-value. Both assess the same null hypothesis: that there is no linear relationship between the predictor and the outcome.

The *summary()* output above shows that x is positively and significantly associated with y. One unit increase in x is associated with a 0.1119536 increase in y.

Confidence intervals may be computed with *confint()*:

```
ci ci <- confint(slm.fit)
```

```
##                2.5%       97.5%
## (Intercept) 0.48753000.5628993
## x           0.10003480.1238724
```

The output shows that we are 95% confident that the true population slope is between 0.100 and 0.124. Critically, this interval does not contain 0. This reinforces the p-value's conclusion that x has a statistically significant, positive effect on y.

The intercept, the slope, and the residuals are visualized as follows (Figure 34).

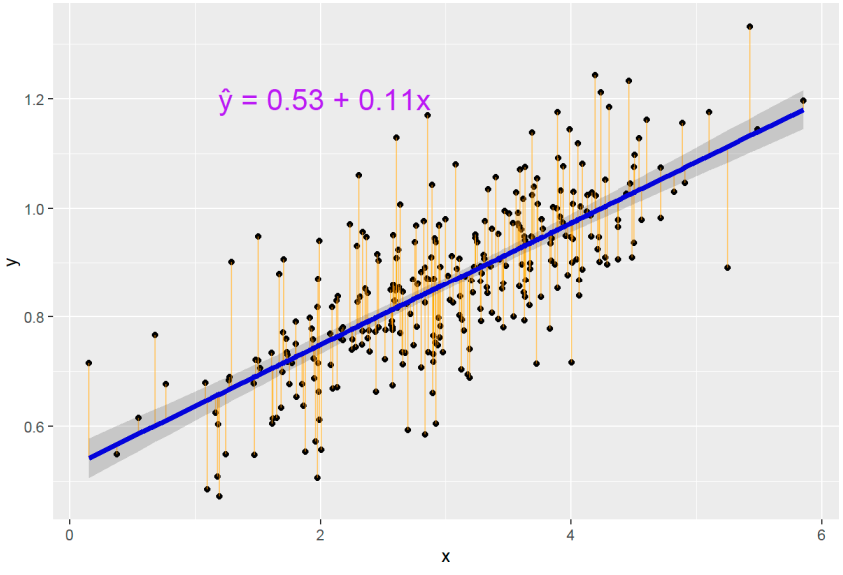


Figure 34. Intercept, slope, and residuals plot. Source: Figure by author.

The straight blue line in the image above represents the predicted values. The model fit may be assessed by plotting the predicted values to evaluate if they look reasonable against the observed data. The shaded area represents the confidence interval of predicted values.

The orange vertical line from the straight line to the observed data value is the residual. For a good model, the sum of the residuals should be approximately zero or as low as possible, and the residual plot should look random. If there is a pattern in the residual plot, the simple linear model may not fit the data well, and there may be a need to transform the variable (e.g., adding a quadratic or cubic term, . . .).

The residuals may be plotted in R using *plot()* as follows (Figure 35):

```
plot(slm.fit$residuals)
```

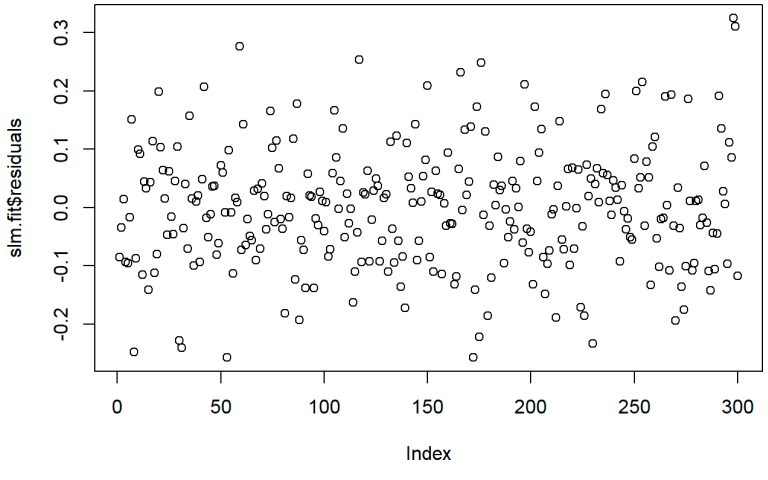


Figure 35. Residuals plot. Source: Figure by author.

The residual plot should be a random, horizontal cloud of points centered around 0. The plot above shows exactly this, with no obvious trend or pattern. This suggests our simple linear model is a reasonable fit for the data.

4.2.2. Linear Model Assumptions

Below are assumptions for inferences made from a regression model [38].

- L: Linear relationship between the mean outcome (Y) and the predictor variable (X);
- I: The errors are independent;
- N: The outcomes are normally distributed at each level of the predictor variable (X);
- E: The variance of the outcomes is equal for all levels of the predictor variable (X) (Homoscedasticity).

If the assumptions are violated, the model estimates and inferences may not be valid.

4.2.3. Checking Linear Model Assumptions

Linear Assumption

If the linear assumption is met, the residuals should be random, and there should be no systematic trend or pattern in the residual plot.

Diagnostic plots may be done by *plot()* on the *lm* object. The first plot is the residuals vs. fitted values plot (Figure 36).

```
# which = 1 plots residuals vs. fitted values plot
plot(slm.fit, which = 1)
```

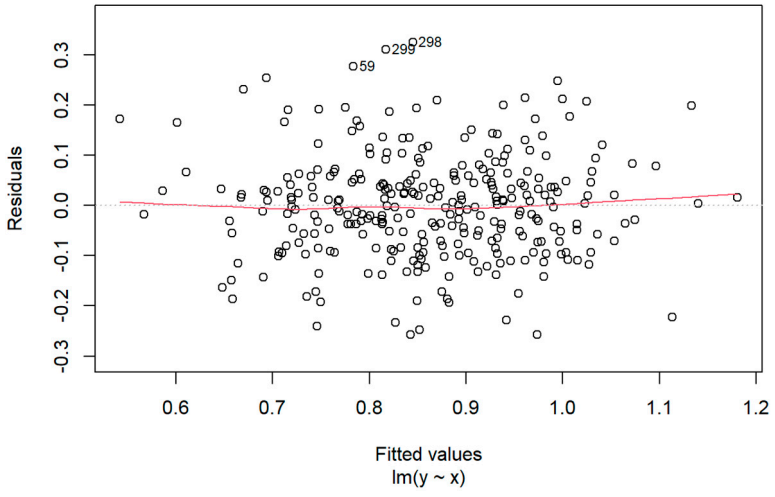


Figure 36. Residuals vs. fitted values plot. Source: Figure by author.

In our example, there is no trend in the residual plot. If there is a trend in the residuals, it suggests a nonlinear relationship between the predictor and outcome variables. In this case, one may consider adding a non-linear term to the model (e.g., cubic, quadratic, and splines, . . .) or using non-linear transformations (e.g., logarithm) to either predictor or outcome variables.

Below is an example plot showing a clear trend of residuals (Figure 37).

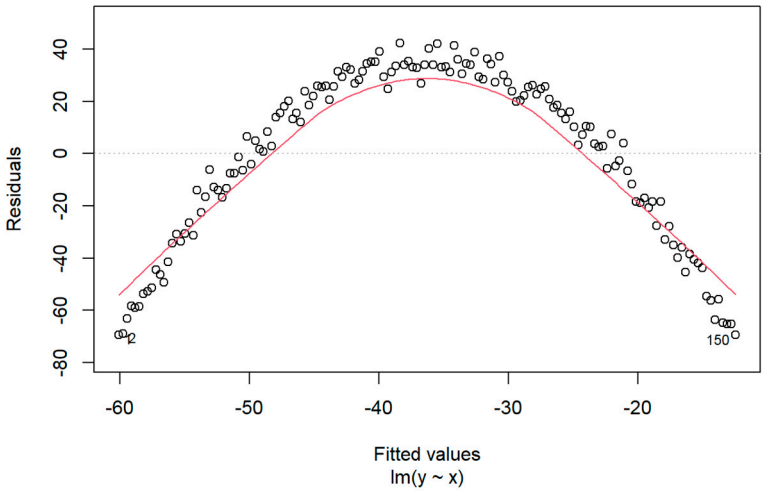


Figure 37. Example trend of residuals. Source: Figure by author.

Equal Variance Assumption (Homoscedasticity)

If this assumption is violated, standard errors will be inaccurate, resulting in inaccurate test statistics, confidence intervals, and p-values. For this assumption, the diagnostic plot of residuals vs. fitted values should show random scatter around the zero line with equal spread across the range of fitted values, as in the residual plot of our example (Figure 38).

```
# which = 1 plots residuals vs. fitted values plot
plot(slm.fit, which = 1)
```

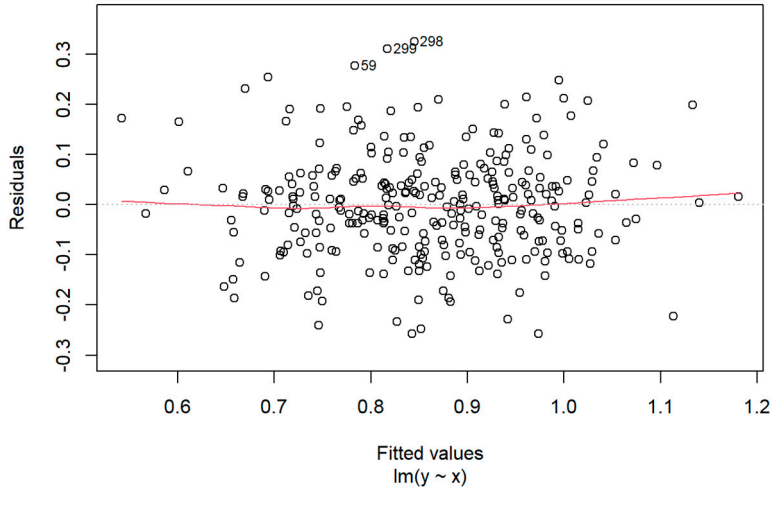


Figure 38. Residual plot: equal variance. Source: Figure by author.

Unequal variance or heteroscedasticity is often observed in the form of increasing variance of the residuals with increasing fitted values, as shown in the figure below (Figure 39).

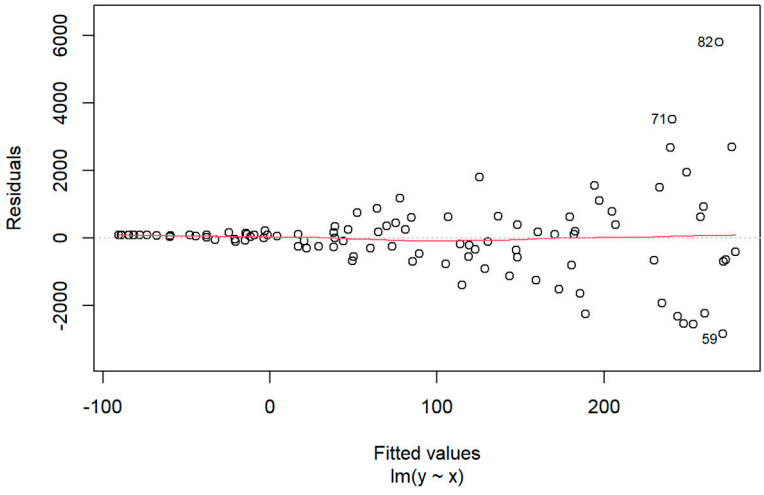


Figure 39. Residual plot: unequal variance. Source: Figure by author.

If equal variance assumption is violated, one may consider:

- Transformation of variables (e.g., log transformation of the outcome variable);
- Using robust standard errors (heteroscedastic-consistent or "sandwich") (may be done with R package "*sandwich*" [39]);
- If the data does not have a hierarchical or clustered structure (which would require a mixed-effects model), a Generalized Least Squares (GLS) model is the recommended approach. GLS allows to explicitly model the variance structure (may be done with the R package "*nlme*" [40]).

Independence Assumption

The independence assumption is not typically checked with a plot but is assessed based on the study design. If this assumption is violated, it may affect the accuracy of confidence intervals and *p*-values.

Common causes of data dependency are:

- Sampling in clusters;
- Repeated measurements of the same subjects.

The approaches to handle this assumption violation are:

- Model data dependency with mixed/multilevel models, fixed effect models, ... (may be done with R package "*lme4*" [41]);
- Using cluster robust standard errors.

Normal Distribution Assumption

If the errors are not normally distributed, the accuracy of confidence intervals and *p*-values may be affected, especially in the case of a small sample size (the impact is less with a large sample size).

Histogram and Q-Q plot of residuals may be used as diagnostic plots (Figure 40):

```
# A simple histogram of residuals
hist(slm.fit$residuals, breaks = 20)
```

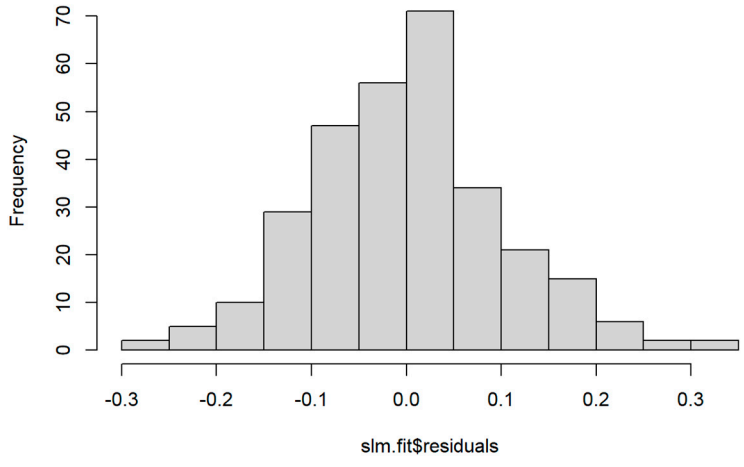


Figure 40. Histogram normal distribution. Source: Figure by author.

```
# Normal Quantile to Quantile plot
qqnorm(slm.fit$residuals)
qqline(slm.fit$residuals)
```

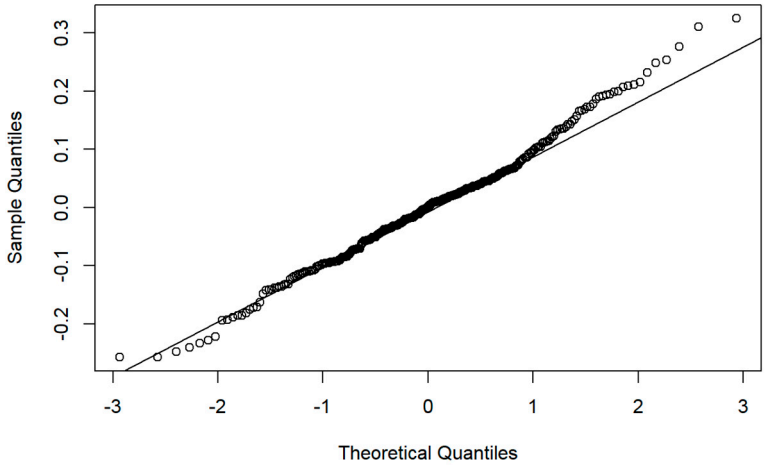


Figure 41. Q-Q plot normal distribution. Source: Figure by author.

To support the normality assumption, the histogram should show a symmetric, bell-shaped curve, and the Q-Q plot should display the points falling closely along the straight diagonal line, as seen in the figures above (Figure 41).

If the errors are not normally distributed, the Q-Q plot will show deviation from the diagonal line. The approaches to handle this issue may be:

- Transformation (e.g., logarithm, . . .) of the outcome variable;
- Using generalized linear models (may be done with *glm()* function).

Note that one may try the "*performance*" package [42], which provides a highly intuitive and comprehensive visual suite for checking model assumptions all at once. The *check_model()* function generates a multi-panel figure that visually assesses linearity, homogeneity of variance (homoscedasticity), normality of residuals,

influential observations, and collinearity, complete with subtitles guiding you on how to interpret each plot. The visualization may be done together with the package "*see*" (Figure 42) [43].

```
# Install and load the package
install.packages(c("performance","see"))
library(performance)
library(see)
# Run comprehensive diagnostic checks
pd<-check_model(slm.fit)
plot(pd)
```

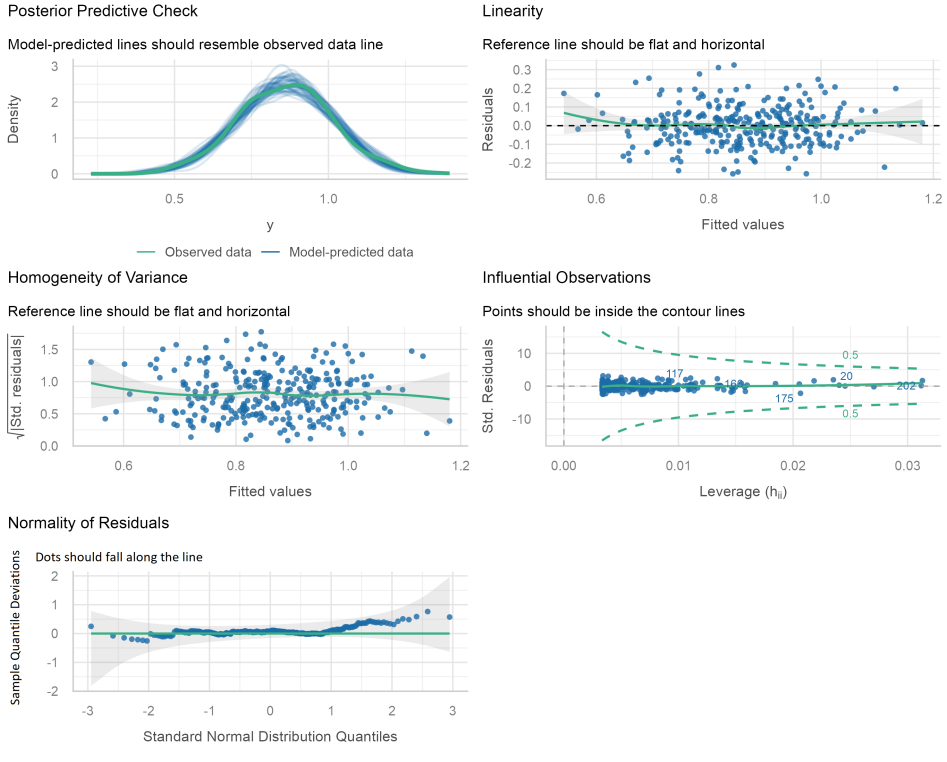


Figure 42. Linear model diagnostic plot using the "performance" package. Source: figure by the author.

The output shows that the linear regression model fits the data reasonably well (Figure 42):

- The model predictions closely resemble the observed distribution of the outcome.
- The residuals show no strong evidence of nonlinearity.
- The variance of residuals appears approximately constant.
- No highly influential observations are present.
- The residual distribution is approximately normal.

Therefore, the key assumptions of the linear regression model appear to be largely satisfied, suggesting that the model estimates and statistical inference are likely to be reliable.

4.2.4. Checking Influential Observations

Influential observations change model estimates substantially when included vs. when excluded from the data. Model coefficients change when a particular observation is excluded and may be evaluated by influence diagnostic measures:

- *dfbeta* measures how a specific coefficient changes when deleting an observation;
- Cook's distance measures how much all coefficients change when deleting an observation.

Influence diagnostic measures may be done with *influence.measures()* on an *lm* object, which results in several influence measures, including *dfbetas* for each coefficient and Cook's distance.

If there are any unusually large influence measures, they will be flagged by an * in the *infl* column.

```
influence <- influence.measures(slm.fit)
str(influence)
```

```
##   List of 3
##   $ infmat: num [1:300, 1:6] 0.01358 -0.00544 -0.00353 0.04819
##   ..- attr(*, "dimnames") = List of 2
##   .. ..$ : chr [1:300] "1" "2" "3" "4" ...
##   .. ..$ : chr [1:6] "dfb.1_" "dfb.x" "dffit" "cov.r" ...
##   $ is.inf: logi [1:300, 1:6] FALSE FALSE FALSE FALSE FALSE FALSE ...
##   ..- attr(*, "dimnames") = List of 2
##   .. ..$ : chr [1:300] "1" "2" "3" "4" ...
##   .. ..$ : chr [1:6] "dfb.1_" "dfb.x" "dffit" "cov.r" ...
##   $ call : language lm(formula = y ~ x, data = df)
##   - attr(*, "class") = chr "infl"
```

4.2.5. Categorical Predictor Variables

Categorical predictor variables with k ($\geq$2) categories will have k $-$ 1 coefficient estimates. The simplest categorical variable is a dummy variable with 2 categories, 0 and 1, denoting not belonging or belonging to a category. An example of this dummy variable may be "smoker" with categories "no" (equivalent to 0) and "yes" (equivalent to 1). Categorical variables with k categories are treated as k $-$ 1 dummy variables, with the omitted category treated as the reference category. Categorical variables in R are represented by factor variables, and the factor order and reference group may be selected. When performing *lm()*, R will automatically create dummy variables for each level of factor except for the first level (which is used as the reference group).

Example Simple Linear Regression with Categorical Predictor Variable

We will use example data regarding the association of medical costs and patient information from GitHub. We first load the data and summarize all variables with the "*arsenal*" package (Table 7) [19].

```r
# Load example linear regression data
url_base<-"https://raw.githubusercontent.com/stedy/"
data_dir<-"Machine-Learning-with-R-datasets/master/insurance.csv"
dlm<-read.csv(paste0(url_base,data_dir))
colnames(dlm)<-tolower(colnames(dlm))
str(dlm)
```

```
## 'data.frame': 1338 obs. of 7 variables: $ age : int 19 18 28 33 32 31 46 37 37 60 ...
## $ sex : chr "female" "male" "male" "male" ... $ bmi : num 27.9 33.8 33 22.7 28.9 .
## . . $ children: int 0 1 3 0 0 0 1 3 2 0 ... $ smoker : chr "yes" "no" "no" "no" ... $
## region
## : chr "southwest" "southeast" "southeast" "northwest" ... $ charges : num
## 16885 1726 4449 21984 3867 ...
```

```r
#turn character variables into factor variables
dlm[,colnames(dplyr::select_if(dlm, is.character))] <-
  lapply(dlm[,colnames(dplyr::select_if(dlm, is.character))],
         as.factor)

#summary of all variables in the dataset
my_controls <- tableby.control(
  numeric.stats = c("meansd", "medianq1q3","meanCI","range","Nmiss2"),
  cat.stats = c("countpct", "Nmiss2"),
  stats.labels = list( meansd =
    "Mean (SD)",
    medianq1q3 = "Median (Q1, Q3)",
    range = "Min - Max",
    Nmiss2 = "Missing", meanCI =
    "Mean (95%CI)"
  )
)
mylabels <-as.list(colnames(dlm))
names(mylabels)<-colnames(dlm)
tabs <- tableby(as.formula(paste("",paste(colnames(dlm),
                                          collapse = "+"),sep = "~")),
                data = dlm,
                control = my_controls) kable(summary(tabs,labelTranslations =
mylabels, text = TRUE),
      caption = "Data summary table", longtable
      = TRUE)%>%
  kable_styling(latex_options = c("scale_down", "hold_position", "repeat_header"))
```

Table 7. Data summary table.

	Overall (N = 1338)
age	
- Mean (SD)	39.207 (14.050)
- Median (Q1, Q3)	39.000 (27.000, 51.000)
- Mean (95% CI)	39.207 (38.454, 39.961)
- Min–Max	18.000–64.000
- Missing	0
sex	
- female	662 (49.5%)
- male	676 (50.5%)
- Missing	0

Table 7. *Cont.*

	Overall (N = 1338)
bmi	
- Mean (SD)	30.663 (6.098)
- Median (Q1, Q3)	30.400 (26.296, 34.694)
- Mean (95% CI)	30.663 (30.336, 30.990)
- Min–Max	15.960–53.130
- Missing	0
children	
- Mean (SD)	1.095 (1.205)
- Median (Q1, Q3)	1.000 (0.000, 2.000)
- Mean (95% CI)	1.095 (1.030, 1.160)
- Min–Max	0.000–5.000
- Missing	0
smoker	
- no	1064 (79.5%)
- yes	274 (20.5%)
- Missing	0
region	
- northeast	324 (24.2%)
- northwest	325 (24.3%)
- southeast	364 (27.2%)
- southwest	325 (24.3%)
- Missing	0
charges	
- Mean (SD)	13,270.422 (12,110.011)
- Median (Q1, Q3)	9382.033 (4740.287, 16,639.913)
- Mean (95% CI)	13,270.422 (12,620.954, 13,919.890)
- Min–Max	1121.874–63,770.428
- Missing	0

Source: Table by author.

We then examine data distribution with a histogram (Figures 43–46):

```
# Get numeric variables
numeric_vars <- colnames(dplyr::select_if(dlm, is.numeric))
# Create a loop the do histogram for each of all numeric variables
for (i in 1:length(numeric_vars)) {
      print(ggplot(data = dlm, aes(x = dlm[, numeric_vars[i]])) +
      geom_histogram(colour = "black", fill = "lightblue") +
      theme_bw() +
                  labs(x = numeric_vars[i]))
}
```

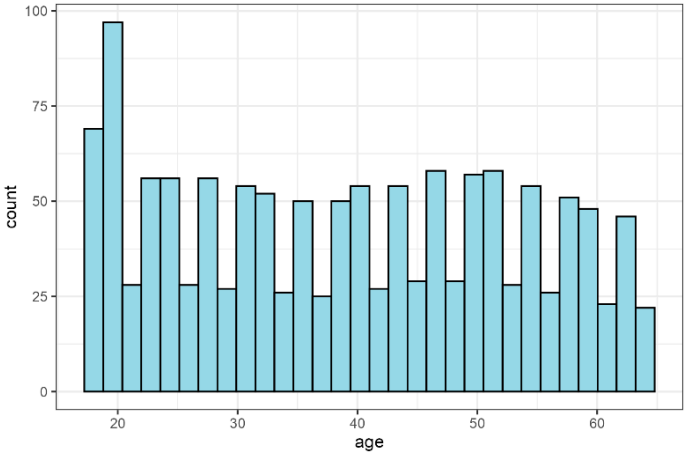


Figure 43. Histogram of continuous variable: age. Source: Figure by author.

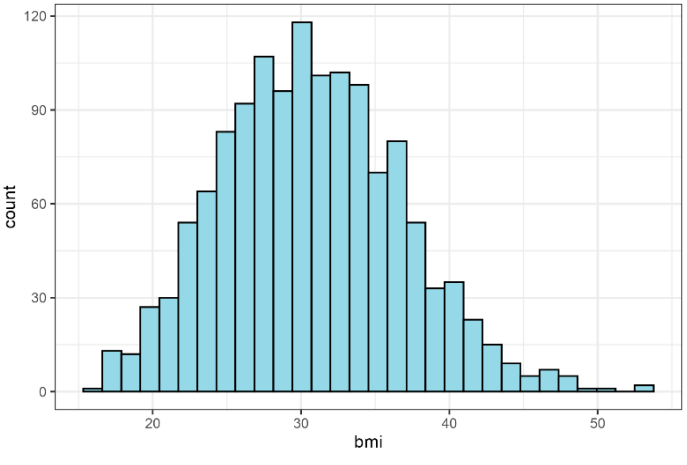


Figure 44. Histogram of continuous variable: bmi. Source: Figure by author.

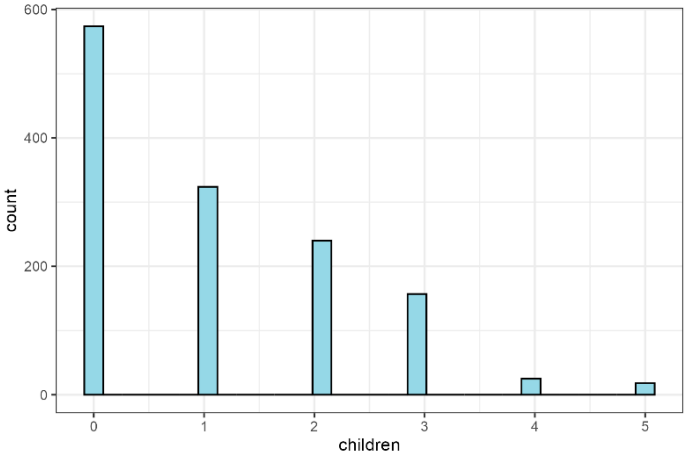


Figure 45. Histogram of continuous variable: children. Source: Figure by author.

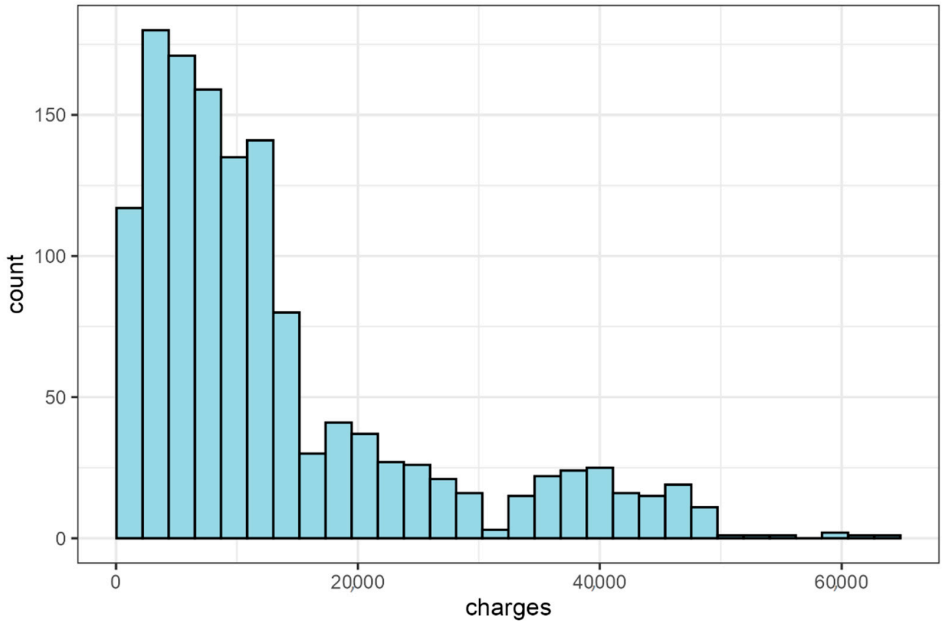


Figure 46. Histogram of continuous variable: charges. Source: Figure by author.

We fit an example simple linear model with "charges" as outcome and "smoker" as categorical predictor.

```
flm<-lm(charges ~ smoker, data = dlm)
sumflm<-summary(flm)
sumflm
```

```
## Call:
## lm(formula = charges ~ smoker, data = dlm)
## Residuals:
##                Min         1Q     Median         3Q        Max
##             −19,221      −5042       −919       3705     31,720
##
## Coefficients:
##                Estimate   Std. Error   t value   Pr(>|t|)
## (Intercept)    8434       229          36.8      <2×10-16 ***
## smokeryes      23,616     506          46.7      <2×10-16 ***
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 7470 on 1336 degrees of freedom
## Multiple R-squared: 0.62, Adjusted R-squared: 0.619
## F-statistic: 2.18×10³ on 1 and 1336 DF, p-value: <2×10-16
```

```
# Add confidence interval to summary
ciflm <- confint(flm)
sumciflm <- cbind(sumflm$coefficients, ciflm)
round(sumciflm, 4)
```

##	Estimate	Std. Error	t value	Pr(>|t|)	2.5%
## (Intercept)	8434.268	229.0142	36.8286	0	7985.002
## smokeryes	23,615.964	506.0753	46.6649	0	22,623.175
##	97.5%				
## (Intercept)	8883.535				
## smokeryes	24,608.752				

From the output, "smoker" = "no" is expected to have charges of 8434.2682979, and "smoker" = "yes" is expected to have charges of 23,615.9635337 higher than "smoker" = "no".

Predicted "charges" for smokers and non-smokers may be calculated and visualized as follows using *predict()* and *ggplot()* (Figure 47).

```
predictions <- predict(flm, newdata = dlm, interval = "confidence")
new_data <- cbind(dlm, as.data.frame(predictions))
#plot
ggplot(dlm, aes(smoker, charges)) +
  geom_point(color = "grey32", size = 0.75) +
  geom_errorbar(data = new_data, aes(x = smoker,
                                     ymin = lwr, ymax = upr, color = smoker),
                width = 0.1) +
  geom_point(data = new_data,
                                  aes(smoker, fit, color = smoker), size = 3) +
  labs(x = "smoker", y = "charges") +
  theme_bw()
```

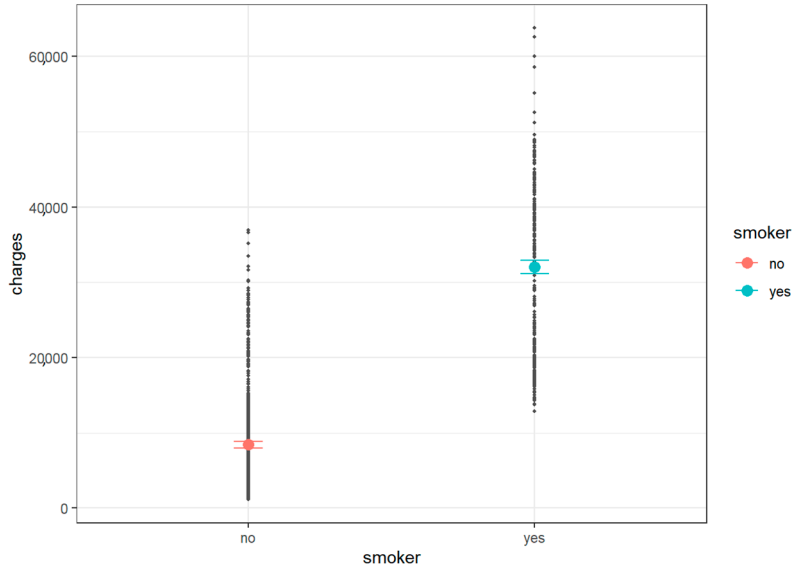


Figure 47. Predicted values by smoker group. Source: Figure by author.

The plot shows a clear difference in predicted "charges" between non-smokers and smokers, corresponding to a significant coefficient of "smoker" in the simple linear model.

Note that, for a binary categorical predictor (e.g., smoker), the linear model is mathematically equivalent to an independent Student's *t*-test (assuming equal variances).

```
#equivalent t-test
t.test(charges ~ smoker, data = dlm, var.equal = TRUE)
```

```
## Two Sample t-test
##
## data:charges by smoker
## t = −46.665, df = 1336, p-value < 2.2 × 10-16
## alternative hypothesis: true difference in means between group no and
## group yes is not equal to 0
## 95 percent confidence interval:
##        −24,608.75                    −22,623.17
## sample estimates:
##        mean in group                 no mean in group yes
##        8434.268                      32050.232
```

Note that the *mean in group yes* is the sum of *(Intercept)* and *smokeryes* estimates above (32,050.232 = 8434.268 + 23,615.964).

We fit another example, a simple linear model with "charges "as outcome and "region" as categorical predictor (region has four categories).

```
flm.r<-lm(charges ~ region, data = dlm)
sumflm.r<-summary(flm.r)
sumflm.r
```

```
## Call:
## lm(formula = charges ~ region, data = dlm)
##
## Residuals:
##                      Min       1Q       Median    3Q          Max
##                      −13,614   −8463    −3793     3385        49,035
##
## Coefficients:
##                      Estimate  Std. Error  t value  Pr(>|t|)
## (Intercept)          13,406.4  671.3       19.971   <2×10-16 ***
## regionnorthwest      −988.8    948.6       −1.042   0.297
## regionsoutheast      1329.0    922.9       1.440    0.150
## regionsouthwest      −1059.4   948.6       −1.117   0.264
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 12080 on 1334 degrees of freedom
## Multiple R-squared: 0.006634, Adjusted R-squared: 0.0044
## F-statistic: 2.97 on 3 and 1334 DF, p-value: 0.03089
```

```
# Add confidence interval to summary
ciflm.r <- confint(flm.r)
sumciflm.r <- cbind(sumflm.r$coefficients, ciflm.r)
round(sumciflm.r, 4)
```

```
##                      Estimate        Std. Error   t value   Pr(>|t|)
## (Intercept)          13,406.3845     671.2966     19.9709   0.0000
## regionnorthwest      −988.8091       948.6263     −1.0424   0.2974
## regionsoutheast      1329.0269       922.9071     1.4400    0.1501
## regionsouthwest      −1059.4471      948.6263     −1.1168   0.2643
##                      2.5%            97.5%
## (Intercept)          12,089.4724     14,723.2966
## regionnorthwest      −2849.7709      872.1526
## regionsoutheast      −481.4805       3139.5343
## regionsouthwest      −2920.4089      801.5146
```

"region" = "northeast" is used as a reference group, and other "region" categories are turned into three dummy variables and are compared with the reference group "northeast". From the output, region "northeast" is predicted to have "charges" of 13,406.3845164. Region "northwest" is predicted to have charges of 988.8091424 lower than region "northeast". Region "southeast" is predicted to have "charges" of 1329.0269212 higher than region "northeast", and so on. However, the differences between "region" categories are not significant.

Note that a linear model with a categorical predictor having more than two levels (e.g., "region") corresponds to a one-way ANOVA. Alternatively, you can use the *oneway.test()* function, which does not assume equal variances by default. Because the overall F-test for region yields a significant result (p-value = 0.03089), it indicates that at least one region differs from the others. We should introduce the *anova()* function to view the formal ANOVA table and perform post hoc tests (like Tukey's HSD) to identify exactly which regions differ.

```
#equivalent 1-way ANOVA
oneway.test(charges ~ region, data = dlm, var.equal = TRUE)
```

```
## One-way analysis of means
##
## data: charges and region
## F = 2.9696, num df = 3, denom df = 1334, p-value = 0.03089
```

```
#Display ANOVA table
anova(flm.r)
```

```
## Analysis of Variance Table
##
## Response: charges
##              Df       Sum Sq       Mean Sq    F value    Pr(>F)
## region        3    1.3008×10⁹     433586560     2.9696   0.03089 *
## Residuals  1334   1.9477×10¹¹     146007093
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
# Perform Tukey's Honest Significant Difference (HSD) post-hoc test
TukeyHSD(aov(charges ~ region, data = dlm))
```

```
## Tukey multiple comparisons of means
## 95% family-wise confidence level
##
## Fit: aov(formula = charges ~ region, data = dlm)
##
## $region
##                           diff         lwr         upr     p adj
## northwest-northeast   -988.8091 -3428.93434  1451.31605 0.7245243
## southeast-northeast   1329.0269 -1044.94167  3702.99551 0.4745046
## southwest-northeast   -1059.4471 -3499.57234  1380.67806 0.6792086
## southeast-northwest   2317.8361   -54.19944  4689.87157 0.0582938
## southwest-northwest   -70.6380 -2508.88256  2367.60656 0.9998516
## southwest-southeast   -2388.4741 -4760.50957  -16.43855 0.0476896
```

The output shows the significant difference between the southwest and the southeast.

Predicted "charges" for "region" may be calculated and visualized as follows (Figure 48).

```
predictions <- predict(flm.r, newdata = dlm, interval = "confidence")
new_data <- cbind(dlm, as.data.frame(predictions))
#plot
ggplot(new_data, aes(region, charges)) +
  geom_point(color = "grey32", size = 0.75) +
  geom_errorbar(data = new_data, aes(x = region,
                                     ymin = lwr, ymax = upr, color = region),
                width = 0.1) +
  geom_point(data = new_data,
  aes(region, fit, color = region), size = 3) +
  labs(x = "region", y = "charges") +
  theme(axis.text.x = element_blank())+
  theme_bw()
```

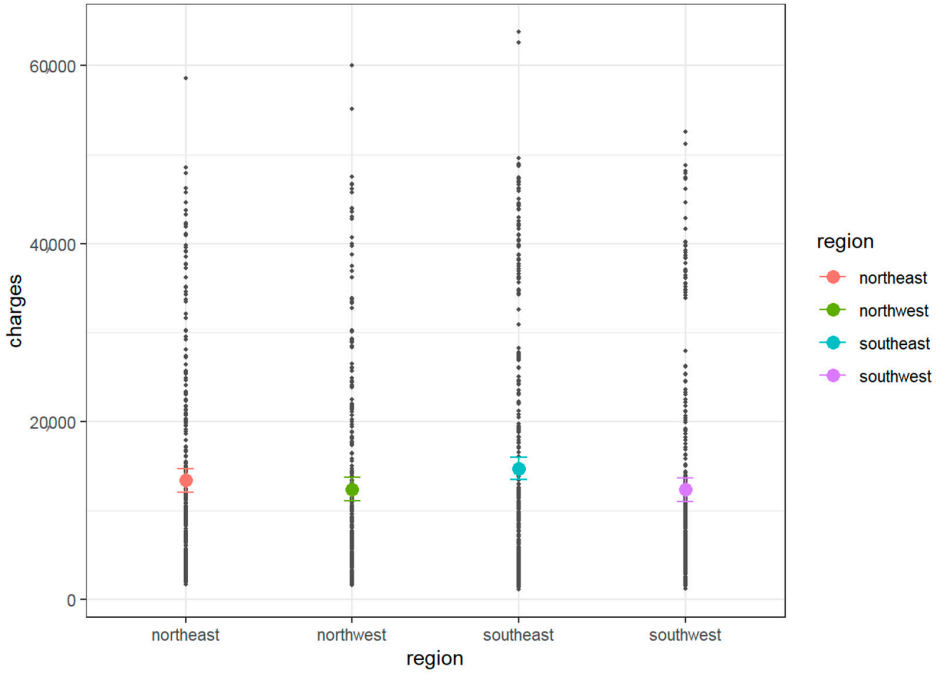


Figure 48. Predicted values by region. Source: Figure by author.

The plot shows similar predicted "charges" between "region" categories, which corresponds to non-significant coefficients of "region" categories vs. the reference category "northeast".

Diagnostic plots may be done with a residual vs. fitted plot for homoscedasticity and a Q-Q plot for normality of residuals (Figure 49):

```
plot(flm.r, which = 1)
```

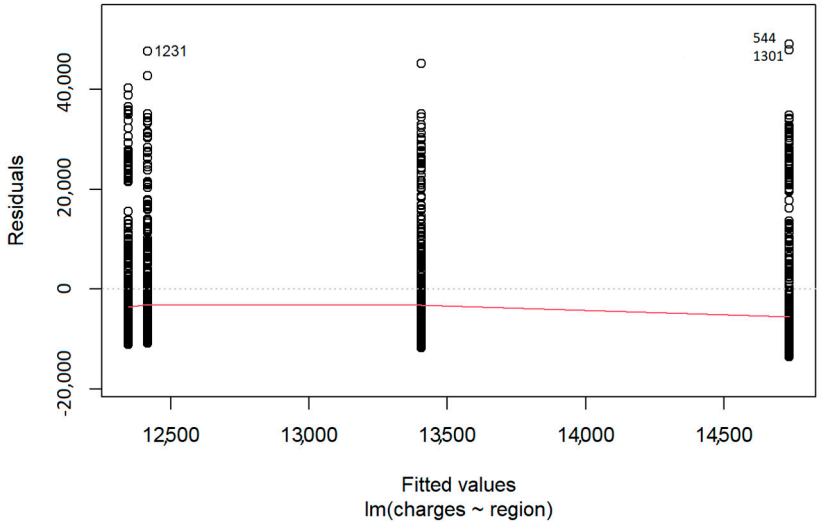


Figure 49. Residual vs. fitted plot 2. Source: Figure by author.

```
qqnorm(residuals(flm.r))
qqline(residuals(flm.r))
```

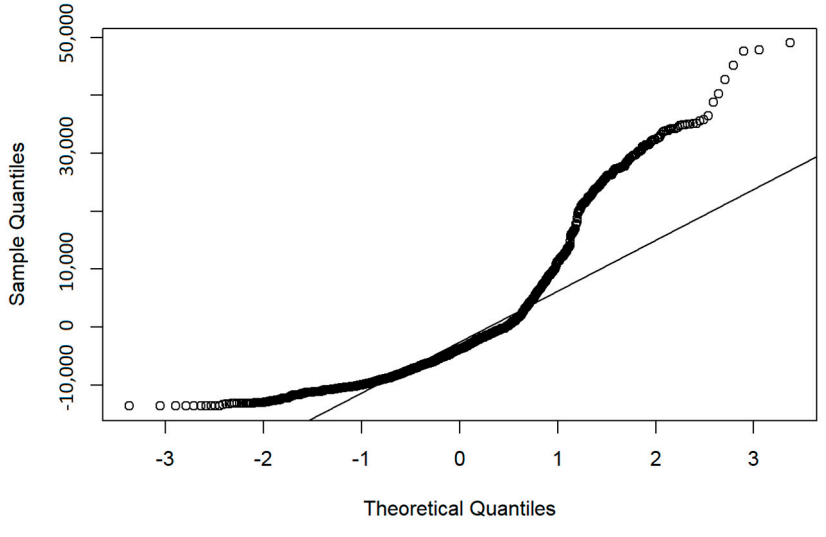


Figure 50. Q-Q plot 2. Source: Figure by author.

With the above Q-Q plot (Figure 50), the points form a strong S-curve and deviate significantly from the diagonal line; thus, the normality assumption does not seem to be met. The histogram of "charges" also shows a skewed distribution (not a symmetric bell shape). Therefore, you may try logarithm transformation of the outcome "charges", refit the linear model, and re-perform diagnostic plots.

```
dlm$logcharges<-log(dlm$charges)
flm.lr<-lm(logcharges ~ region, data = dlm)
sumflm.lr<-summary(flm.lr)
sumflm.lr
```

```
##
## Call:
## lm(formula = logcharges ~ region, data = dlm)
##
## Residuals:
##                      Min          1Q      Median          3Q         Max
##                  -2.09965    -0.64192     0.03622     0.62333     1.94064
##
## Coefficients:
##                  Estimate  Std. Error  t value  Pr(>|t|)
## (Intercept)      9.16877   0.05106     179.562  <2e-16 ***
## regionnorthwest  -0.09903  0.07216     -1.372   0.1701
## regionsoutheast  -0.04637  0.07020     -0.660   0.5091
## regionsouthwest  -0.13767  0.07216     -1.908   0.0566 .
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.9191 on 1334 degrees of freedom
## Multiple R-squared: 0.003143, Adjusted R-squared: 0.0009012
## F-statistic: 1.402 on 3 and 1334 DF, p-value: 0.2406
```

```
# Add confidence interval to summary
ciflm.lr <- confint(flm.lr)
sumciflm.lr <- cbind(sumflm.lr$coefficients, ciflm.lr)
round(sumciflm.lr, 4)
```

```
##                  Estimate Std. Error t value  Pr(>|t|)2.5%
## (Intercept)      9.1688   0.0511     179.5621 0.0000 9.0686
## regionnorthwest -0.0990   0.0722     -1.3725  0.1701 -0.2406
## regionsoutheast -0.0464   0.0702     -0.6605  0.5091 -0.1841
## regionsouthwest -0.1377   0.0722     -1.9079  0.0566 -0.2792
##                  97.5%
## (Intercept)      9.2689
## regionnorthwest 0.0425
## regionsoutheast 0.0913
## regionsouthwest 0.0039
```

```
#diagnostic plot
plot(flm.lr, which = 1)
```

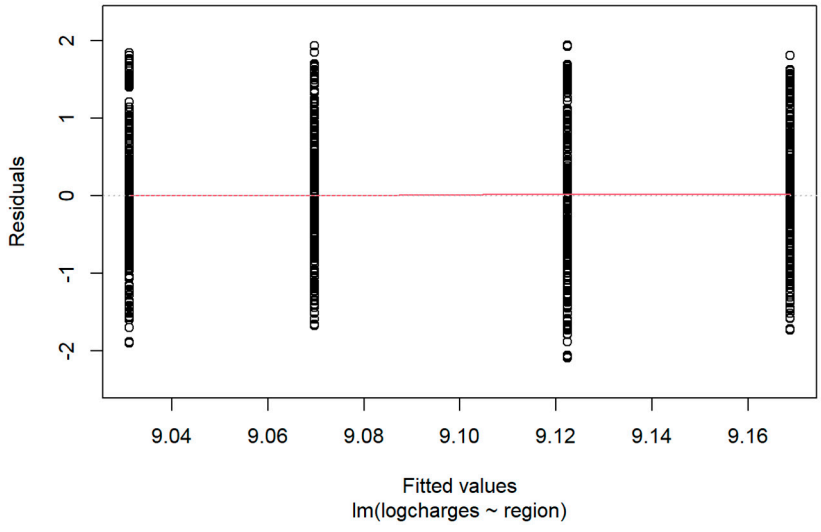


Figure 51. Residual vs. fitted plot: log transformation. Source: Figure by author.

```
qqnorm(residuals(flm.lr))
qqline(residuals(flm.lr))
```

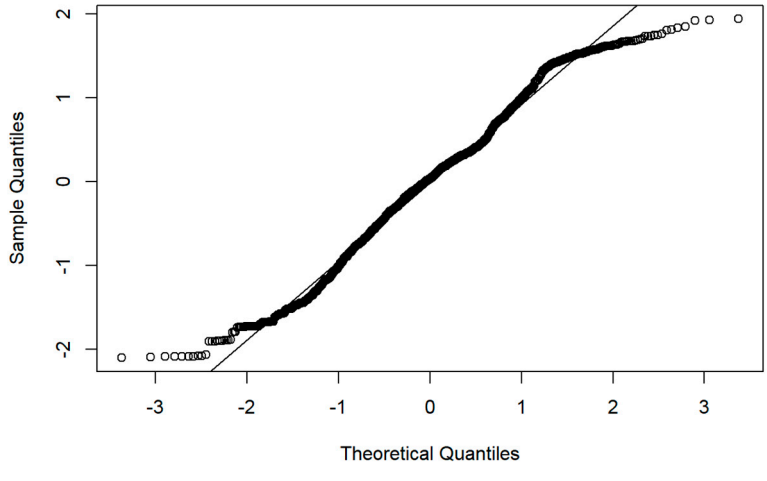


Figure 52. Q-Q plot: log transformation. Source: Figure by author.

The diagnostic plots (Figures 51 and 52) look better after a logarithmic transformation of the outcome "charges". The residual vs. fitted plot shows that the vertical spread (variance) within each of the four groups is now much more similar, indicating that the log transformation has stabilized the variance and largely fixed the heteroscedasticity. The Q-Q plot shows that the points now fall much closer to the diagonal line, indicating the residuals of this new model are approximately normally distributed. The log transformation was a necessary and successful step to meet the model's assumptions. All subsequent models will use "logcharges" as the outcome.

Note that, instead of creating a new column for log-transformed variables in your data frame, you can apply the transformation directly within the *lm()* formula:

```r
# Applying log transformation directly in the formula
flm.lr<-lm(log(charges) ~ region, data = dlm)
```

4.3. Multiple Linear Regression

Multiple linear regression allows the estimation of the effects of multiple predictor variables simultaneously on the outcome variable. It allows us to estimate the effect of a variable while controlling for other variables in the model. For the example of medical costs and patient information data above, multiple linear regression may be done by adding several variables as predictors in the linear model, for example, "bmi" and "smoker" as predictors and the logarithm of "charges" ("logcharges") as the outcome. First, we fit a simple linear regression for each of the predictors, "bmi" or "smoker", and then we fit a multiple linear regression for both "bmi" and "smoker" as predictors to see the difference between models.

```r
# Simple linear regression model with only bmi as
# predictor and logarithm charges as outcome
flm.b<-lm(logcharges ~ bmi, data = dlm)
sumflm.b<-summary(flm.b)
sumflm.b
```

```
## Call:
## lm(formula = logcharges ~ bmi, data = dlm)
##
## Residuals:
##                Min         1Q     Median         3Q        Max
##            -2.48894  -0.63536    0.03136    0.68007    1.95182
##
## Coefficients:
##                Estimate  Std. Error  t value  Pr(>|t|)
## (Intercept)    8.485243  0.127833    66.378   <2×10-16 ***
## bmi            0.020005  0.004089    4.892    1.12×10-6 ***
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.9117 on 1336 degrees of freedom
## Multiple R-squared: 0.0176, Adjusted R-squared: 0.01687
## F-statistic: 23.94 on 1 and 1336 DF, p-value: 1.117×10-6
```

```r
# Add confidence interval to summary
ciflm.b <- confint(flm.b)
sumciflm.b <- cbind(sumflm.b$coefficients, ciflm.b)
round(sumciflm.b, 4)
```

```
##                Estimate  Std. Error  t value  Pr(>|t|)  2.5%     97.5%
## (Intercept)    8.4852    0.1278      66.3778  0         8.2345   8.736
## bmi            0.0200    0.0041      4.8925   0         0.0120   0.028
```

```r
# Simple linear regression model with only smoker as predictor
# and logarithm charges as outcome
flm.s<-lm(logcharges ~ smoker, data = dlm)
sumflm.s<-summary(flm.s)
sumflm.s
```

```
## Call:
## lm(formula = logcharges ~ smoker, data = dlm)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
##  −1.7655  −0.4370   0.1237   0.4785   1.7280
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  8.78823    0.02105  417.52  <2×10-16 ***
## smokeryes    1.51588    0.04651   32.59  <2×10-16 ***
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.6866 on 1336 degrees of freedom
## Multiple R-squared: 0.4429, Adjusted R-squared: 0.4425
## F-statistic: 1062 on 1 and 1336 DF, p-value: <2.2×10-16
```

```
# Add confidence interval to summary
ciflm.s <- confint(flm.s)
sumciflm.s <- cbind(sumflm.s$coefficients, ciflm.s)
round(sumciflm.s, 4)
```

```
##             Estimate Std. Error t value Pr(>|t|) 2.5% 97.5%
## (Intercept)   8.7882     0.0210 417.5206        0 8.7469 8.8295
## smokeryes     1.5159     0.0465  32.5902        0 1.4246 1.6071
```

```
# Multiple linear regression model with smoker and
# bmi as predictors and logarithm charges as outcome
flm.sb<-lm(logcharges ~ smoker + bmi, data = dlm)
sumflm.sb<-summary(flm.sb)
sumflm.sb
```

```
##
## Call:
## lm(formula = logcharges ~ smoker + bmi, data = dlm)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## −2.17030 −0.40754  0.09871  0.44368  1.75504
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept) 8.186576   0.095254  85.945  <2×10-16 ***
## smokeryes   1.514765   0.045818  33.061  <2×10-16 ***
## bmi         0.019629   0.003033   6.472  1.36×10-10 ***
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.6763 on 1335 degrees of freedom
## Multiple R-squared: 0.4598, Adjusted R-squared: 0.459
## F-statistic: 568.3 on 2 and 1335 DF, p-value: <2.2×10-16
```

```
# Add confidence interval to summary
ciflm.sb <- confint(flm.sb)
sumciflm.sb <- cbind(sumflm.sb$coefficients, ciflm.sb)
round(sumciflm.sb, 4)
```

```
##Estimate          Std. Error t value Pr(>|t|) 2.5%   97.5%
##(Intercept) 8.18660.0953     85.9449 0        7.9997 8.3734
##smokeryes   1.51480.0458     33.0606 0        1.4249 1.6046
##bmi         0.01960.0030     6.4715  0        0.0137 0.0256
```

From the output of multiple linear regression, the coefficients change slightly for both "smoker" and "bmi" as compared to those of the simple linear regression models when one of these predictors was used alone.

The interpretation of coefficients is as follows:

- "logcharges" when "smoker" = no and "bmi" = 0 is 8.1865764.
- After controlling for "bmi", "smoker" = yes is expected to have "logcharges" of 1.514765 higher than "smoker" = no.
- After controlling for "smoker", each unit increase in "bmi" is associated with an increase of 0.0196287 in "logcharges".

Both *smokeryes* and *bmi* have very small *p*-values, so both are significant predictors even when controlling for each other. R-squared of 0.4598 indicates that together, these two variables explain 45.98% of the variance in "logcharges".

Note that, when a multiple regression model includes both a continuous predictor (like "bmi") and a categorical predictor (like "smoker"), this specific formulation is historically and statistically considered an Analysis of Covariance (ANCOVA). It allows us to compare the means of the categorical groups while statistically adjusting for the continuous covariate.

Observed and predicted values with 95% confidence intervals of the above multiple regression model may be visualized as follows (Figure 53):

```
predictions <- predict(flm.sb, newdata = dlm, interval = "confidence")
new_data <- cbind(dlm, as.data.frame(predictions))
#plot
ggplot(new_data, aes(bmi, logcharges, color = smoker)) +
  geom_point() +
    geom_ribbon(data = new_data, aes(x = bmi, ymin = lwr, ymax = upr, fill = smoker),
                                 alpha = 0.5) +
    geom_line(data = new_data, aes(bmi, fit, color = smoker),
                                 linewidth = 1) +
    labs(x = "BMI", y = "log(charges)") +
    theme_bw()
```

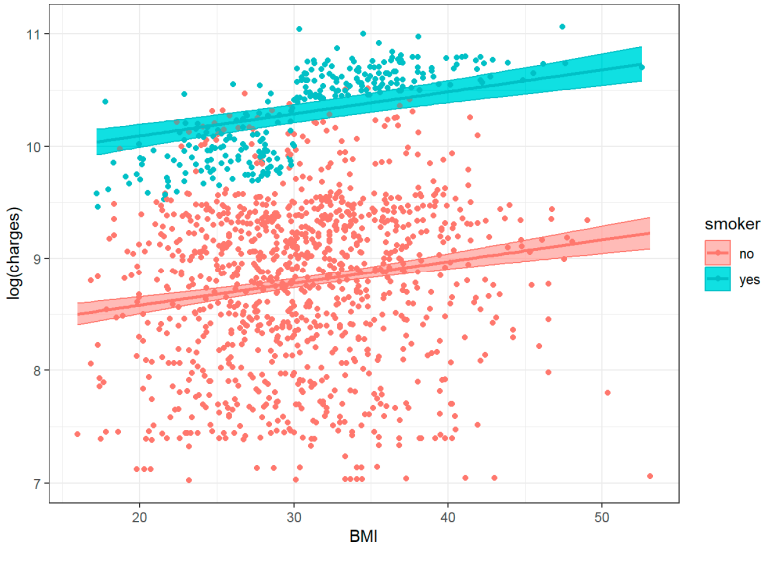


Figure 53. Observed and predicted values with 95% CI. Source: Figure by author.

In the above multiple linear regression model with "smoker" and "bmi" as predictors, the slopes for "logcharges" against "bmi" are the same for "smoker" = no and "smoker" = yes. However, the above plot shows the difference in the association of "logcharges" against "bmi" between "smoker" = no and "smoker" = yes. Thus, the multiple linear regression model above may not fit the data well, especially for "smoker" = yes.

To address this difference, an interaction term between "smoker" and "bmi" may be added to the model:

```
flm.i<-lm(logcharges ~ smoker + bmi +smoker*bmi, data = dlm)
sumflm.i<-summary(flm.i)
sumflm.i
```

```
##
## Call:
## lm(formula = logcharges ~ smoker + bmi + smoker * bmi, data = dlm)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.98216 -0.37399  0.07215  0.44881  1.72280
##
## Coefficients:
##                Estimate Std. Error t value Pr(>|t|)
## (Intercept)    8.443124   0.106184  79.514  <2e-16 ***
## smokeryes      0.342811   0.227399   1.508  0.131911
## bmi            0.011259   0.003399   3.313  0.000949 ***
## smokeryes:bmi  0.038179   0.007259   5.259  1.68e-07 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.6697 on 1334 degrees of freedom
## Multiple R-squared:  0.4708, Adjusted R-squared:  0.4696
## F-statistic: 395.6 on 3 and 1334 DF,  p-value: < 2.2e-16
```

```
# Add confidence interval to summary
ciflm.i <- confint(flm.i)
sumciflm.i <- cbind(sumflm.i$coefficients, ciflm.i) round(sumciflm.i, 4)
```

##	Estimate	Std. Error	t value	Pr(>|t|)	2.5%
## (Intercept)	8.4431	0.1062	79.5145	0.0000	8.2348
## smokeryes	0.3428	0.2274	1.5075	0.1319	-0.1033
## bmi	0.0113	0.0034	3.3126	0.0009	0.0046
## smokeryes:bmi	0.0382	0.0073	5.2595	0.0000	0.0239
##	97.5%				
## (Intercept)	8.6514				
## smokeryes	0.7889				
## bmi	0.0179				
## smokeryes:bmi	0.0524				

And we re-perform the plot of the model with interaction term (Figure 54):

```
predictions <- predict(flm.i, newdata = dlm, interval = "confidence")
new_data <- cbind(dlm, as.data.frame(predictions))
#plot
ggplot(new_data, aes(bmi, logcharges, color = smoker)) +
  geom_point() +
  geom_ribbon(data = new_data, aes(x = bmi, ymin = lwr,
                                   ymax = upr, fill = smoker),
              alpha = 0.5) +
  geom_line(data = new_data, aes(bmi, fit, color = smoker), linewidth = 1) +
  labs(x = "BMI", y = "log(charges)") + theme_bw()
```

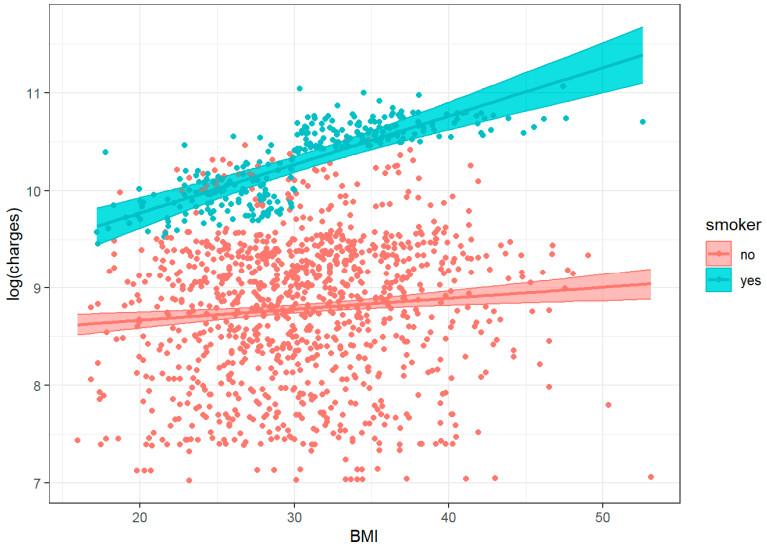


Figure 54. Observed and predicted values with interaction. Source: Figure by author.

Now the plot shows two different slopes for "smoker" = no and "smoker" = yes, and the model fits the data better, especially for "smoker" = yes.

The interpretation of the coefficients of the multiple linear regression model with an interaction term is as follows:

- "logcharges" when "smoker" = no and "bmi" = 0 is 8.443124;
- When "bmi" = 0, "logcharges" when "smoker" = yes is 0.3428113 higher when "smoker" = no;
- When "smoker" = no, for each unit increase in "bmi", "logcharges" increases by 0.011259;
- Coefficient for the interaction term expresses the change in a variable's effect when the interacting variable is increased by one unit:
  - The effect of "bmi" on "logcharges" increases by 0.0381793 ("logcharges" per one unit of "bmi") in "smoker" = yes vs. "smoker" = no.
  - For each unit increase in "bmi", the effect of "smoker" = yes vs. "smoker" = no on "logcharges" increases by 0.0381793.

The effect of "bmi" for "smoker" = yes or "smoker" = no may be calculated by adding the coefficient of "bmi" 0.011259 to the interaction coefficient, 0.0381793, multiplied by "smoker" as a dummy variable 0, 1:

- "bmi" effect ("smoker" = no) = 0.011259 + 0.0381793 *0;
- "bmi" effect ("smoker" = yes) = 0.011259 + 0.0381793 *1.

The effect of "smoker" for any "bmi" may be calculated by adding the coefficient of "smoker", 0.3428113, to the interaction coefficient, 0.0381793, and multiplying by "bmi":

- "smoker" effect ("bmi" = 15) = 0.3428113 + 0.0381793 *15.

4.4. Model Selection

Model selection is the process of systematically choosing the best subset of predictors from a larger set to include in the final model. The goal is to create a model that is both parsimonious (as simple as possible) and has high explanatory power. Several statistical criteria may be used for model selection:

- Adjusted R-squared: penalizes the model for having more predictors. It increases only if the new predictor improves the model more than would be expected by chance. When comparing models, a model with a higher adjusted R-squared is usually preferred.
- Akaike Information Criterion (AIC) and Bayesian Information Criterion (BIC): These are information-based criteria that balance model fit with model complexity. Lower values indicate a better model.

Below is an example using the *step()* function to perform both backward and forward stepwise selection based on AIC. First, a *full_model* is created containing all predictors we want to consider. The *step(full_model, . . .)* function then iteratively adds and removes predictors from this model, checking the AIC at each step (*trace = 0* simply suppresses the printing of every single step). The final result, *stepwise_model*, is the model with the lowest possible AIC that the step function could find.

```
# Fit a full model with multiple predictors
full_model <- lm(logcharges ~ age + sex + bmi + smoker +
                                smoker*bmi + children + region, data = dlm)

# Use the step() function to perform backward
# stepwise selection based on AIC
# The 'direction'can be "backward", "forward", or "both"
stepwise_model <- step(full_model, direction = "both",  trace = 0)

# View the summary of the selected model
summary(stepwise_model)
```

```
##
## Call:
## lm(formula = logcharges ~ age + sex + bmi + smoker + smoker *
## bmi + children + region, data = dlm)
##
## Residuals:
##                    Min          1Q      Median          3Q         Max
##                −0.91507    −0.17764    −0.05143     0.04641     2.22317
##
## Coefficients:
##                    Estimate   Std. Error  t value   Pr(>|t|)
## (Intercept)        7.3373697  0.0766733   95.697    <2×10-16 ***
## age                0.0347953  0.0008429   41.281    <2×10-16 ***
## sexmale            −0.0870645 0.0236073   −3.688    0.000235 ***
## bmi                0.0034060  0.0022676   1.502     0.133336
## smokeryes          0.1564189  0.1459995   1.071     0.284199
## children           0.1031486  0.0097594   10.569    <2×10-16 ***
## regionnorthwest    −0.0711306 0.0337354   −2.108    0.035176 *
## regionsoutheast    −0.1627269 0.0339029   −4.800    1.77×10-6  ***
## regionsouthwest    −0.1375125 0.0338557   −4.062    5.16×10-5  ***
## bmi:smokeryes      0.0455744  0.0046633   9.773     <2×10-16 ***
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.4293 on 1328 degrees of freedom
## Multiple R-squared: 0.7835, Adjusted R-squared: 0.7821
## F-statistic: 534 on 9 and 1328 DF, p-value: <2.2×10-16
```

The *summary(stepwise_model)* output shows the final model chosen by both the forward and backward selection algorithms. In this example, all variables in the full model are retained in the final model.

Note that, while stepwise regression based on AIC or BIC is computationally easy, it is subject to heavy criticism in modern statistics. Relying solely on automated algorithms can lead to "*p*-hacking" or overfitting, where the model captures noise rather than true underlying relationships. It often results in biased parameter estimates and artificially narrow confidence intervals. Whenever possible, model selection should be guided by clinical relevance, domain knowledge, and theoretical frameworks. If automated selection must be used on high-dimensional data, penalized regression methods (like LASSO or Ridge regression) are generally preferred over stepwise approaches.

Standard Ordinary Least Squares (OLS) regression can fail when predictors are highly correlated (multicollinearity) or when there are too many variables, leading to overfitting. Penalized regression addresses this by adding a penalty term to the OLS loss function. Two primary forms of penalized regression are:

- Ridge Regression (L1 penalty): Shrinks coefficients toward zero but keeps all variables.
- Lasso Regression (L2 penalty): Shrinks some coefficients exactly to zero, performing automated variable selection.

One may try LASSO using the "*glmnet*" package [44] as in the example below (Figure 55):

```r
# Load necessary libraries
library(glmnet)
library(dplyr)

# 1. Prepare Data
# glmnet requires predictors as a matrix and outcome as a vector
# We use the 'dlm' dataset from the previous examples
x <- model.matrix(logcharges ~ age + bmi + children + smoker + region, data = dlm)[,-1]
y <- dlm$logcharges

# 2. Fit Lasso Regression (alpha = 1)
# Use cross-validation to find the optimal penalty strength (lambda)
cv_lasso <- cv.glmnet(x, y, alpha = 1)

# 3. Plot
plot(cv_lasso)
```

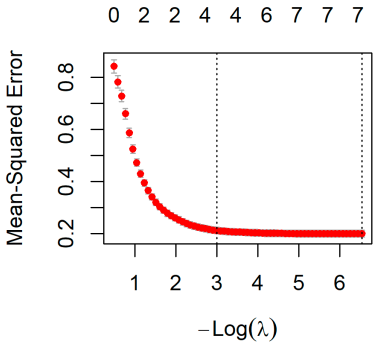


Figure 55. Model selection with LASSO. Source: Figure by author.

The plot shows the Mean Squared Error (MSE) as *lambda* changes. The first dashed line represents *lambda.min* (the value that minimizes MSE), and the second is *lambda.1se* (a more parsimonious model within one standard error of the minimum).

Coefficients may be extracted as follows.

```r
# Extract coefficients at the optimal lambda
best_lambda <- cv_lasso$lambda.min
lasso_coefs <- coef(cv_lasso, s = "lambda.min")
print(lasso_coefs)
```

```
## 8 x 1 sparse Matrix of class "dgCMatrix"
##                          lambda.1se
## (Intercept)           7.395230034
## age                   0.031644214
## bmi                   0.003388925
## children              0.062360826
## smokeryes             1.418852624
## regionnorthwest .
## regionsoutheast .
## regionsouthwest .
```

Variables with a "." ("region") in the output have been "shrunk" to zero, meaning the Lasso model has determined that "region" is not a significant predictor after accounting for others. This is highly useful for simplifying clinical models.

4.5. Model Validation

Model evaluation and selection focus on how well the model fits the data on which it was built (training data). Model validation examines the performance of a model on new (testing) data. The most common method is to split the dataset into two parts:

- Training Set: A subset of the data (e.g., 80%) used to build and train the model.
- Testing Set: The remaining portion of the data (e.g., 20%) that is held back and used only to evaluate the performance of the trained model.

While a single train/test split is conceptually simple, it can be highly dependent on how the data is randomly divided, especially in smaller datasets. Resampling approaches provide a much more robust evaluation of model performance.

- K-Fold Cross-Validation: This method splits the data into K equal-sized folds. The model is trained on K-1 folds and tested on the remaining fold, repeating this process K times.
- Bootstrapping: This involves taking repeated random samples (with replacement) from the original dataset to estimate the variability of model parameters or performance metrics.

These approaches can be done using the package "*caret*" [45].

```
## 10-fold cross-validation
library(caret)
# Define training control (10-fold cross-validation)
train_control <- trainControl(method = "cv", number = 10)

# Train the model
cv_model <- train(logcharges ~ age + bmi + smoker,
data = dlm,
method = "lm",
trControl = train_control)
print(cv_model)
```

```
## Linear Regression
##
## 1338 samples
## 3 predictor
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 1204, 1203, 1204, 1204, 1204, 1205, ...
## Resampling results:
##
##   RMSE       Rsquared   MAE
##   0.4636607  0.7462715  0.3037515
##
## Tuning parameter 'intercept' was held constant at a value of TRUE
```

The output shows that the above linear regression model evaluated using 10-fold cross-validation demonstrated good predictive performance (RMSE = 0.46, MAE = 0.30) and explained approximately 74.6% of the variance in the outcome (R^2 = 0.746), indicating a strong relationship between the predictors and the response variable.

```
## Bootstrapping
# Define training control for Bootstrapping (e.g., 100 resamples)
boot_control <- trainControl(method = "boot", number = 100)
# Train the model
boot_model <- train (log(charges) ~ age + bmi + smoker,
data = dlm,
method = "lm",
trControl = boot_control)
print(boot_model)
```

```
## Linear Regression
##
## 1338 samples
## 3 predictor
##
## No pre-processing
## Resampling: Bootstrapped (100 reps)
## Summary of sample sizes: 1338, 1338, 1338, 1338, 1338, 1338, ...
## Resampling results:
##
##       RMSE              Rsquared          MAE
##       0.4671038         0.7434627         0.3039834
##
## Tuning parameter 'intercept' was held constant at a value of TRUE
```

The output shows that the above linear regression model, evaluated using bootstrap resampling (100 repetitions), demonstrated good predictive performance. The model explained approximately 74.3% of the variance in the outcome variable ($R^2 = 0.743$), with a root mean squared error of 0.467 and a mean absolute error of 0.304, indicating relatively small prediction errors.

4.5.1. Performance Metrics

- Mean Absolute Error (MAE): the average of the absolute differences between the predicted and actual values.
- Mean Squared Error (MSE): the average of the squared prediction errors.
- Root Mean Squared Error (RMSE): the square root of the MSE (the most commonly used regression metric).
- R-squared (*R*2): the proportion of the variance in the test set's outcome that is explained by the model. A significant drop in R-squared from the training set to the test set is a strong indicator of overfitting.

Below are examples to fit the model on training data, and calculate the above performance metrics on testing data using the *"yardstick"* package [37]. First, split the data, fit the model on training data, and predict on testing data:

```
library(yardstick)

# For calculating performance metrics
# Split the data (reproducing the split for a self-contained example)
# Set a seed for reproducibility
set.seed(123)
# Create an index for the training set (80% of the data)
train_index <- sample(1:nrow(dlm), 0.8 * nrow(dlm))
# Create the training and testing datasets
train_data <- dlm[train_index, ]
test_data <- dlm[-train_index, ]
# Train the model on the training data ONLY
final_model <- lm(logcharges ~ age + sex + bmi + smoker +
                                    smoker*bmi + children + region, data
                                        =     train_data)
# Make predictions and combine with actuals
# Create a results data frame with actuals and predictions
results <- test_data %>%
    mutate(Actual = logcharges) %>%
    mutate(Predicted = predict(final_model, newdata = test_data))%>%
    select(Actual,Predicted )

# View the first few rows of the results
head(results)
```

```
##         Actual       Predicted
##    14 9.313864    9.262429
##    15 10.586881   10.276976
##    21 9.490155    9.545169
##    22 8.330800    8.475589
##    27 9.578577    9.596659
##    33 8.452718    8.485702
```

Then, calculate performance metrics using "*yardstick*" functions. The "*yardstick*" functions follow a consistent format: *metric_function(data, truth = Actual_Column_Name, estimate = Predicted_Column_Name)*.

We calculate Root Mean Squared Error (RMSE) using *rmse()*:

```
rmse_value  <-  rmse(results,  truth = Actual,  estimate = Predicted)
print(rmse_value)
```

```
## # A tibble: 1 × 3
##       .metric    .estimator    .estimate
##       <chr>      <chr>         <dbl>
## 1     rmse       standard      0.429
```

The output shows that, on average, the model's prediction of "logcharges" is off by about 0.4294596 on the testing data.

Calculate Mean Absolute Error (MAE) using *mae()*:

```
mae_value  <-  mae(results,  truth = Actual,  estimate    = Predicted)
print(mae_value)
```

```
## # A tibble: 1 × 3
##       .metric    .estimator    .estimate
##       <chr>      <chr>         <dbl>
## 1     mae        standard      0.270
```

The output shows that the average absolute error of any given prediction is 0.2695986.

We calculate R-squared using *rsq()*:

```
rsq_value <- rsq(results, truth = Actual, estimate = Predicted)
print(rsq_value)
```

```
## # A tibble: 1 × 3
##   .metric  .estimator  .estimate
##   <chr>    <chr>       <dbl>
## 1 rsq      standard    0.793
```

The output shows that the model explains approximately 0.7928342 of the variance in actual "logcharges" in the testing data. This value is very close to the R-squared from the training data. This similarity is excellent and confirms that our model is not overfit and generalizes well.

4.5.2. Calibration

Model calibration assesses the agreement between the model's predictions and the actual observed outcomes. A well-calibrated model is one where the predictions are reliable across the entire range of values. A model may have a low average error (as evaluated by the above performance metrics) but may have poor calibration, which makes the model not trustworthy for decision-making at the individual level. In other words, while RMSE and R-squared give an average error, a calibration plot shows where the model is strong and weak.

Calibration is best evaluated by a calibration plot (Figure 56):

```
ggplot(results, aes(x = Predicted, y = Actual)) +
  # Add the scatter plot of points.
  # Alpha transparency helps visualize data density.
  geom_point(alpha = 0.3, color = "gray40") +

  # Add the line of perfect calibration (y = x line).
  # This is the reference line where Predicted would equal Actual.
  geom_abline(intercept = 0, slope = 1, color = "red",
              linetype = "dashed", linewidth = 1) +

  # Add a smoothed line to show the average trend of our model's predictions. #
  This is the "calibration curve" that we will evaluate.
  geom_smooth(method = "loess", se = FALSE,
        color = "blue", linewidth = 1.2) +

  # Ensure the plot axes are scaled equally for a proper 45-degree line view
  coord_fixed(
    xlim = range(c(results$Predicted, results$Actual)), ylim =
    range(c(results$Predicted, results$Actual))
  ) +

  # Add informative labels and a clean theme
  labs(
    x = "Predicted logcharges", y =
    "Actual logcharges",
    caption = "Blue = Model's calibration curve; Red = Perfect calibration"
  ) +
  theme_minimal(base_size = 14)
```

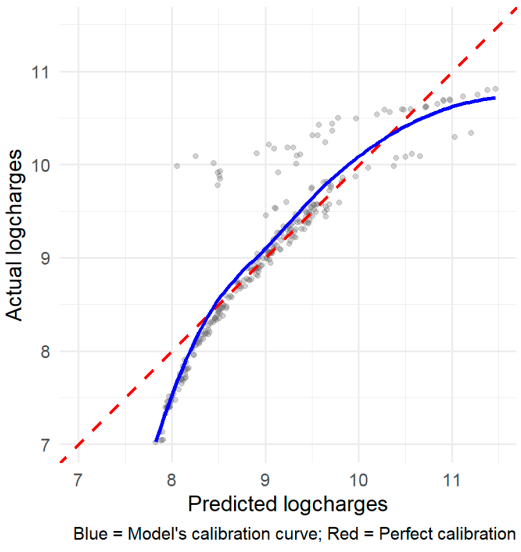


Figure 56. Calibration plot. Source: Figure by author.

In the calibration plot above, the Predicted values (x-axis) are plotted against the Actual values (y-axis). The red dashed line is the line of perfect calibration, where Predicted = Actual. The blue solid line is the calibration curve, showing the average trend of our model's predictions. A well-calibrated model would have its blue line fall directly on top of the red dashed line.

The plot shows that the model tends to underpredict for small and large "logchages" values, while predicting relatively well for middle "logchages" values.

4.6. Useful Resources

- Introduction to Statistical Learning: with Applications in R [46].
- Introduction to Linear Regression in R by UCLA Office of Advanced Research Computing Statistical Methods and Data Analytics (https://stats.oarc.ucla.edu/wp-content/uploads/2024/02/Intro-to-regression.html, accessed on 1 April 2025).
- Linear Regression Hands on R tutorial by Princeton University Library (https://libguides.princeton.edu/R-linear_regression, accessed on 1 April 2025).
- How to Do Linear Regression in R by Datacamp (https://www.datacamp.com/tutorial/linear-regression-R, accessed on 1 April 2025).
- The dataset used in some examples of this chapter are example data regarding the association of medical costs and patient information: may be downloaded from Github (https://raw.githubusercontent.com/stedy/Machine-Learning-with-R-datasets/master/insurance.csv, accessed on 1 April 2025).

4.7. Chapter Summary

- *lm()* function fits linear regression, which evaluates the relationship between a continuous dependent variable and one (for simple linear regression) or more (for multiple linear regression) independent variables;

- *summary()* function summarizes the model results;
- *coef()* to extract the model coefficients;
- *confint()* to get confidence intervals;
- *predict()* to generate predictions from the model;
- *residuals()* to access the model's residuals;
- *plot()* function applied to an lm object generates diagnostic plots;
- *influence.measures()* function to detect influential data points;
- Main assumptions of linear regression (Linear relationship, Independence, Homoscedasticity, normal distribution of the outcome) should be evaluated for the linear model;
- Model selection may be based on AIC, BIC, and may be done with *step()* function;
- Model performance metrics may be estimated using the "*yardstick*" package;
- Model calibration may be evaluated using a calibration plot (e.g., with "*ggplot2*").

5. Logistic Regression with R

This chapter introduces the readers to performing Logistic Regression in R. It starts by addressing the fundamental challenges of using a linear model for probability and showing how the Logistic function may be used for a probability constrained between 0 and 1. We frame Logistic Regression as a specific type of Generalized Linear Model (GLM), which is fitted in R using the *glm()* function with the binomial family. The practical examples focus on building, interpreting, and evaluating Univariable and Multivariable Logistic Regression models; model evaluation using key classification metrics and discrimination power using the "*ROCR*" package; and model selection and model validation.

5.1. Why Logistic Regression Is Needed?

For logistic regression, the outcome (dependent variable) is categorical, and logistic regression models the probability of an outcome occurring.

If Linear Regression serves to predict continuous variables, Logistic Regression is used to predict categorical variables or to model the probability of an outcome occurring (binary classification as "yes" or "no").

Now, load the required packages for this chapter ("*knitr*" [5], "*rmarkdown*" [6], "*tidyverse*" [11], "*arsenal*" [19], "*kableExtra*" [12], "*pscl*" [47], "*ROCR*" [48]):

```
#load multiple packages
Packages <- c("knitr", "rmarkdown", "tidyverse", "arsenal",
              "kableExtra", "pscl", "ROCR")
lapply(Packages, library, character.only = TRUE)
```

Example:

The diabetes data is used to predict the occurrence of diabetes using other variables such as BMI, skin thickness, or blood glucose, etc. This example dataset may be downloaded from GitHub.

First, we take a look at the dataset and then examine the relationship between "bmi" and "outcome" by linear regression (Figure 57).

```
url_base<-"https://raw.githubusercontent.com/plotly/"
data_path<-"datasets/master/diabetes.csv"
# Load example data
data<-read.csv(paste0(url_base, data_path))
colnames(data)<-tolower(colnames(data))
str(data)
```

```
## 'data.frame'                    : 768 obs. of 9 variables:
## $ pregnancies                   : int 6 1 8 1 0 5 3 10 2 8 ...
## $ glucose                       : int 148 85 183 89 137 116 78 115 197 125 ...
## $ bloodpressure                 : int 72 66 64 66 40 74 50 0 70 96 ...
## $ skinthickness                 : int 35 29 0 23 35 0 32 0 45 0 ...
## $ insulin                       : int 0 0 0 94 168 0 88 0 543 0 ...
## $ bmi                           : num 33.6 26.6 23.3 28.1 43.1 25.6 31 35.3 30.5 0 ...
## $ diabetespedigreefunction      : num 0.627 0.351 0.672 0.167 2.288 ...
## $ age                           : int 50 31 32 21 33 30 26 29 53 54 ...
## $ outcome                       : int 1 0 1 0 1 0 1 0 1 1 ...
```

```
ggplot(data, aes(bmi, outcome))+
  geom_point()+
  geom_smooth(method = 'lm', formula = y~x)
```

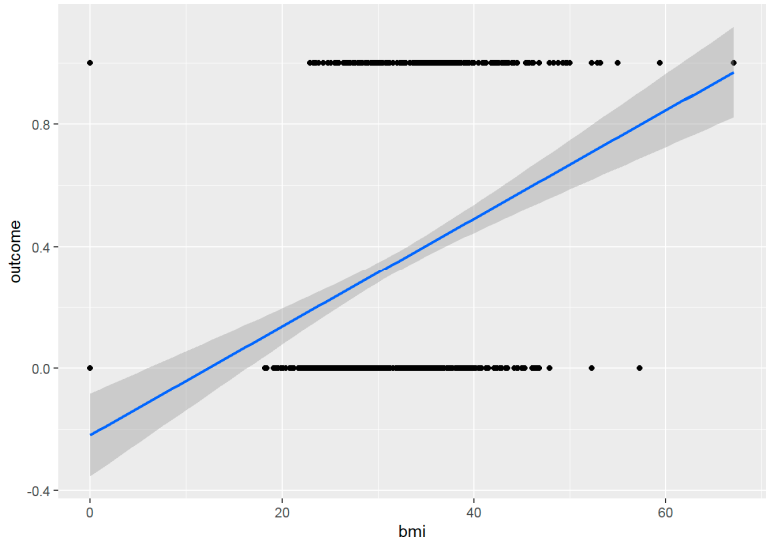


Figure 57. Model outcome vs. BMI by linear regression. Source: Figure by author.

Below are explanations of what we do above:

- 0 means there is no "outcome" (diabetes), 1 means there is the "outcome" (diabetes);
- What we try to predict is the likelihood of having the "outcome" (diabetes);
- This may be represented as follows:
  - y-axis being the Probability of having the "outcome" (diabetes);
- 0 = 0% of having the "outcome" (diabetes);
- 1 = 100% of having the "outcome" (diabetes)
  - x-axis = the "bmi" values.

The figure above shows a bad fit line, because it cannot capture the pattern of the data. It does not go through most of the data points. Furthermore:

- When "bmi" decreases, we predict negative probability!
- With higher "bmi", we even predict a probability larger than 1!
- The predictions are not sensible as the probability must fall between 0 and 1

Logistic regression provides the solution for this issue.

5.2. Logistic Regression Overview

5.2.1. Statistical Concepts

The solution for the above issue is to transform a Linear Regression line to a Logistic Regression curve, which can only go between 0 and 1 (Figure 58).

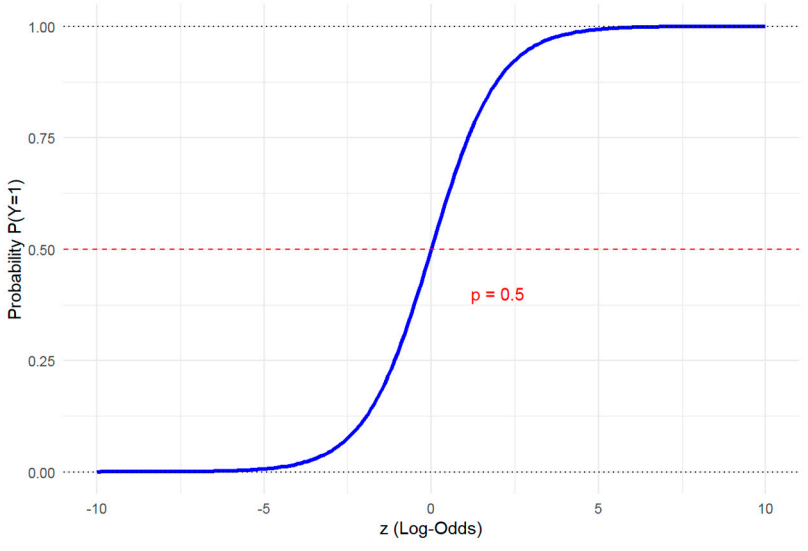


Figure 58. Logistic Regression sigmoid curve. Source: Figure by author.

Logistic function is one type of Sigmoid function [49]:

- Mathematical form: $\phi(z) = \frac{1}{1+e^{-z}}$
- Graphical form: S curve

Of note, it does not matter what value of Z we put into the logistic function; you will always get a value of 0 or 1, which just meets the requirements that we need. Therefore, the Logistic Regression function results in a probability from 0 to 1.

Logistic Regression is a specific type of Generalized Linear Model (GLM). GLM is a generalization of the concepts and abilities of regular Linear Models. Therefore, if one is familiar with Linear Models, then one may understand Logistic Regression.

A Logistic Regression will model the chance of an outcome based on individual characteristics. Because chance is a ratio, what will actually be modeled is the logarithm of the chance given by

$$log\left[\frac{y}{1-y}\right] = b_0 + b_1x_1 + b_2x_2 + b_3x_3 + \ldots + b_nx_n,$$

where

- y indicates the probability of an event;
- b_i are the regression coefficients associated with the reference group and the x_i explanatory variables;
- the reference group, represented by b_o, is constituted by those individuals presenting the reference level of each and every variable $x_{1\ldots n}$.

Of note, in a standard linear model, we predict the outcome y directly. However, in logistic regression, our outcome is a probability that must stay between 0 and 1. To achieve this, we use a link function. The link function "links" the linear combination of your predictors $b_0 + b_1x_1 + b_2x_2 + b_3x_3 + \ldots + b_nx_n$ to the mean of the response variable. For logistic regression, we use the logit link function. In R, when you write

family = binomial(link = "logit") inside the *glm()* function, you are explicitly telling R to use the binomial distribution for the error term and the logit transformation as the link between your data and the model.

Plot Predicted Probabilities from a Fitted Logistic Model

Now, we go back to our previous example, using Logistic Regression, we plot predicted probabilities of having the outcome as a function of BMI (Figure 59).

```
# Univariable model
logistic_uni <- glm(outcome ~ bmi,
                    data = data,
                    family = binomial(link = "logit"))

# 1. Create a new data frame for prediction
# We will vary 'glucose' across its range and #
hold other predictors at their mean new_data <-
data.frame(
  bmi = data$bmi,
  outcome = data$outcome)

# 2. Use the model to predict probabilities for this new data #
The 'type = "response"' argument ensures
# we get probabilities (0-1)
predicted_probs <- predict(logistic_uni,
                           newdata = new_data, type = "response")

# 3. Add these predictions to our new data frame
new_data$predicted_prob<-predicted_probs

# 4. Generate the plot
ggplot(data = new_data, aes(x = bmi, y = outcome)) +
  # Add the raw data points (0s and 1s). We use jitter to avoid overplotting.
  geom_jitter(height = 0.05, width = 0,
              alpha = 0.2, color = "gray50") +
  # Add the logistic curve from our model's predictions
  geom_line(data = new_data, aes(x = bmi, y = predicted_prob), color
            = "blue", linewidth = 1.2)+
  # Add labels and a title
  labs(x = "BMI",
    y = "Probability of outcome") +
  # Use a clean theme
  theme_bw()
```

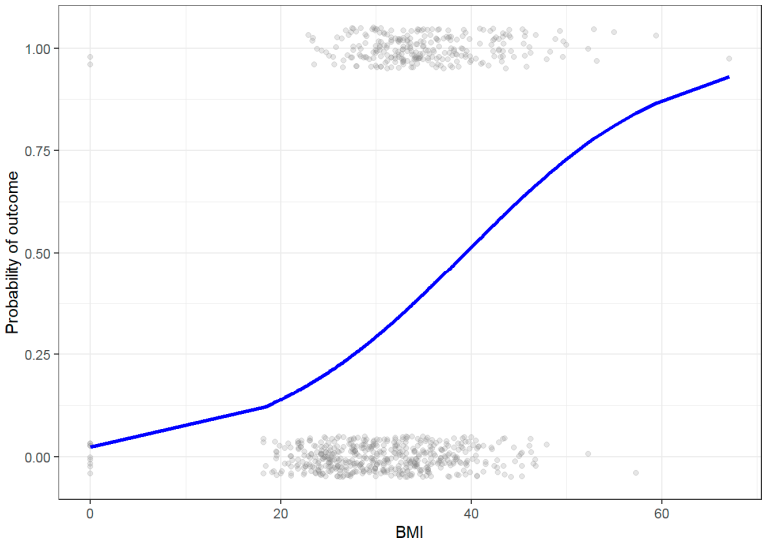


Figure 59. Predicted Probability of outcome vs. BMI. Source: Figure by author.

Coefficients

In terms of the coefficients, Logistic Regression to similar as Linear models except the coefficients are in terms of the log(odds).

Example: Given a person's profile of BMI, blood glucose, and blood pressure, our aim is to build a model that predicts the probability of that person getting an outcome (diabetes).

This is part of the coefficients from the multivariable logistic regression model fitted with *glm()* [46]:

```
# Load example data
#data<-read.csv("https://raw.githubusercontent.com/plotly/datasets/master/diabetes.csv")
#colnames(data)<-tolower(colnames(data))
data$outcome<-as.factor(data$outcome)
#  Multivariate  model
logistic_multi <- glm(outcome ~ bmi + glucose +
                        bloodpressure, data = data,
                      family = binomial(link = "logit"))
summary(logistic_multi)
```

```
## Call:
## glm(formula = outcome ~ bmi + glucose + bloodpressure, family = binomial(link = "logit"),
## data = data)
##
## Coefficients:
##                Estimate    Std. Error   z value   Pr(>|z|)
## (Intercept)    −7.182306   0.635462     −11.303   <2×10-16 ***
## bmi            0.079793    0.013565     5.882     4.05×10-9 ***
## glucose        0.035746    0.003328     10.740    <2×10-16 ***
## bloodpressure  −0.007420   0.004862     −1.526    0.127
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
```

```
##
## Null deviance: 993.48 on 767 degrees of freedom
## Residual deviance: 769.07 on 764 degrees of freedom
## AIC: 777.07
##
## Number of Fisher Scoring iterations: 4
```

- The Intercept gives the mean level of the response variable when all other variables are 0. The estimated intercept is −7.1823064, and the standard error of the estimated intercept is 0.6354617. The z value is the estimated intercept divided by the standard error. In other words, it is the number of standard deviations the estimated intercept is away from 0 on a standard normal curve.
- Each one-unit change in "bmi" will increase the log(odds) of getting the "outcome" (diabetes) by 0.0797926, and its p value indicates that it is significant in determining the "outcome".

Maximum Likelihood

For Logistic Regression, maximum likelihood is used to select the correct sigmoid function that best fits the data. In other words, maximum likelihood is used to evaluate which sigmoid function would generate the observed data with the highest probability.

The example dataset above will be used again. The univariable and multivariable logistic models above will be compared regarding their maximum likelihood to evaluate which model better fits the data.

```
# Extract the maximum log-likelihood for each model
ll_model_1 <- logLik(logistic_uni)
ll_model_2 <- logLik(logistic_multi)


# Create a summary data frame for easy comparison
model_comparison <- data.frame(
  Model = c("1: Univariable Model", "2: Multivariable Model"),
  LogLikelihood = c(ll_model_1, ll_model_2)
)

# Print the comparison table, sorted by the best fit
print(model_comparison[order(model_comparison$LogLikelihood,
                             decreasing = TRUE), ])
```

```
##       Model                    LogLikelihood
## 2   2: Multivariable Model   −384.5330
## 1   1: Univariable Model     −460.3571
```

The result output shows that the multivariable model has a larger maximum likelihood and thus better fits the data. Now, we can visualize outcome probability sigmoid curves of univariable vs. multivariable models as follows (Figure 60):

```r
# Create a data frame of the original data for plotting
plot_df <- data.frame(outcome = as.numeric(as.character(data$outcome)),
                      bmi = data$bmi,
                      glucose = data$glucose, bloodpressure =
                      data$bloodpressure)

# Predicted outcome probability of univariable with only BMI
plot_df$predicted_probs_uni <- predict(logistic_uni,
                                       newdata = data.frame(bmi = data$bmi), type
                                       = "response")

# Predicted outcome probability of multivariable with
 BMI holding glucose and bloodpressure constant at their means
plot_df$predicted_probs_multi <- predict(logistic_multi,
                                         newdata = data.frame( bmi
                                           =
                                             data$bmi,
                                           glucose =
                                             mean(data$glucose),
                                           bloodpressure = mean(data$bloodpressure)),
                                         type = "response")

ggplot(plot_df, aes(x = bmi, y = outcome)) +
  # Add the raw data points (0s and 1s) with jitter
  geom_jitter(height = 0.05, width = 0, alpha = 0.2) + # --- Add
  the two probability curves ---
  # Curve for univariable Model 1 (BMI Only)
  geom_line(aes(x = bmi, y = predicted_probs_uni, color
                = "Model 1: Univariable"))+
  # Curve for multivariable Model 2
  # (with BMI, holding glucose and bloodpressure at their means)
  geom_line(aes(x = bmi, y = predicted_probs_multi, color
                = "Model 2: Multivariable"))+
  # --- Formatting and Labels ---
  labs(
    x = "BMI",
    y = "Probability of outcomes", color
    = "Models" # Legend title
  ) +
  scale_color_manual(values = c( "Model
    1: Univariable" = "red", "Model 2:
    Multivariable" = "blue"
  )) +
  theme_minimal() + theme(legend.position
  = "bottom")
```

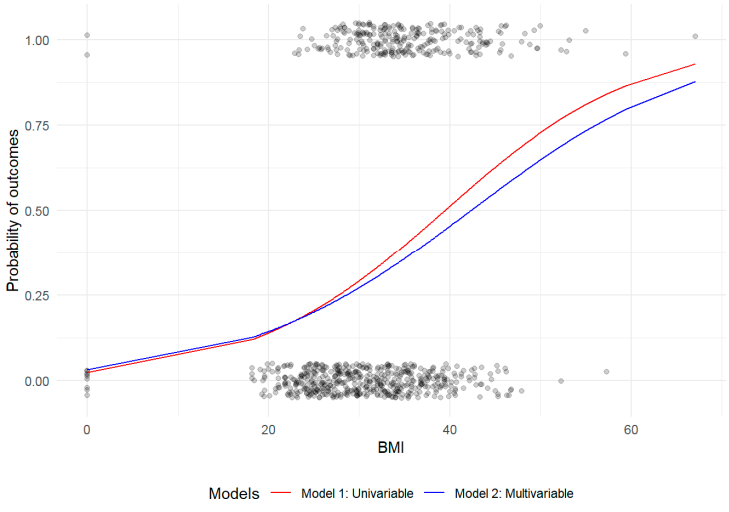


Figure 60. Sigmoid curves of univariable vs. multivariable models. Source: Figure by author.

Note that, for comparing multiple logistic regression models (e.g., a baseline univariable model vs. a multivariable model), one may use the function *compare_performance()* of the "*performance*" package [42]. The output table includes AIC, BIC, RMSE, and Sigma. By setting rank = TRUE, the function provides a "Performance Score," helping to identify which model balances complexity and fit most effectively (Figure 61).

```
library(performance)
#  Compare models side-by-side
comparison <- compare_performance(logistic_uni, logistic_multi, rank = TRUE)
print(comparison)
```

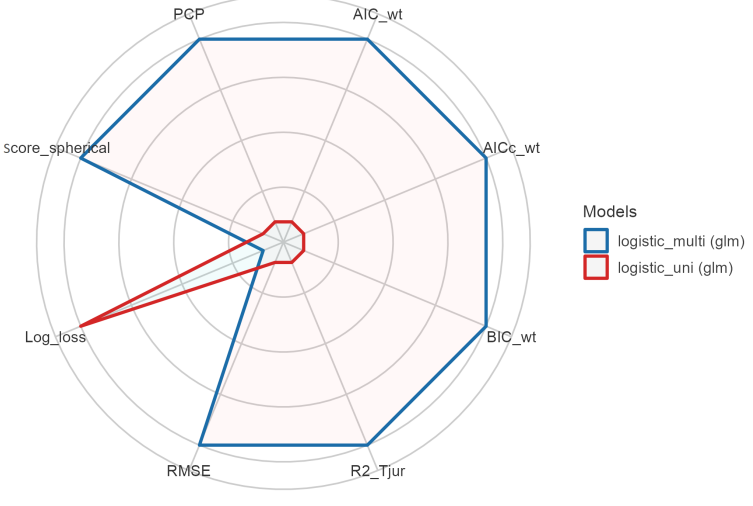


Figure 61. Model comparison using the "*performance*" package. Source: Figure by author.

In the output plot, each axis represents a model performance metric. Values are normalized to the same scale so that different metrics can be shown together:

- AIC_wt—Akaike Information Criterion weight;
- AICc_wt—corrected AIC weight;
- BIC_wt—Bayesian Information Criterion weight;
- R2_Tjur—Tjur's pseudo R^2 for logistic regression;
- RMSE—root mean square error;
- Log_loss—cross-entropy loss;
- PCP—percentage correctly predicted;
- Score_spherical—spherical scoring rule.

Radar plot interpretation rule: The farther from the center, the better the model for that metric (except for metrics where smaller is better (e.g., Log_loss)). Thus, the model with the larger polygon area generally performs better overall.

For our models, the multivariable logistic model (blue) dominates all axes. This indicates that the multivariable model provides a better fit and predictive performance than the univariate model.

R-Squared

The R-squared from linear regression (the proportion of variance explained) cannot be directly applied to logistic regression because logistic regression does not work by minimizing squared errors. Instead, it uses Maximum Likelihood Estimation (MLE). To fill this gap, several pseudo R-squared measures have been developed. One of the most commonly used is McFadden's R-squared. McFadden's R-squared measure is defined as

$$R^2_{McFadden} = 1 - \frac{\log(L_C)}{\log(L_{Null})}$$

where *Lc* denotes the (maximized) likelihood value from the current fitted model, and Lnull denotes the corresponding value for the null model (the model with only an intercept and no covariates).

The log-likelihood R^2 values go from 0, for poor models, to 1, for good models.

As a rule of thumb, values between 0.2 and 0.4 may be considered to represent a good model fit.

Below is an example to calculate McFadden's R^2 manually:

```
# Method 1: Manual Calculation (for understanding)
# 1. Fit the "null model" (intercept only)
null_model <- glm(outcome ~ 1,
                  data = data,
                  family = binomial(link = "logit"))

# 2. Extract the log-likelihood from both models
logLik_full <- logLik(logistic_multi)
logLik_null <- logLik(null_model)

# 3. Apply McFadden's formula
mcfadden_r2 <- 1 - (logLik_full / logLik_null)
cat("McFadden's R-squared (manual calculation):",
    as.numeric(mcfadden_r2), "\n")
```

```
## McFadden's R-squared (manual calculation): 0.2258898
```

Below is an example to calculate McFadden's *R*2 using the *pR2()* function of the *"pscl"* package (the *pR2()* function calculates several pseudo R-squared values) [47]:

```
pseudo_r2_values <- pR2(logistic_multi)

## fitting null model for pseudo-r2

print(pseudo_r2_values)
```

```
##   llh                 llhNull          G2             McFadden   r2ML
##   -384.5330385        -496.7419551 224.4178332 0.2258898  0.2533888
##   r2CU
##   0.3491560
```

The output shows that the multivariable model has a relatively good model fit and has a 0.2258898 better fit than the null model.

Note that one may use some other metrics for assessing the explained variance in logistic regression.

- Cox and Snell: Based on the likelihood ratio, however, its maximum value is often less than 1.
- Nagelkerke: An adjusted version of Cox and Snell that ensures a range from 0 to 1.
- Tjur's D: Specifically designed for binary outcomes; it calculates the difference between the mean predicted probabilities for the two outcome groups.

The *r2()* function of the "*performance*" package will typically return the Nagelkerke R^2 and Tjur's R^2 (Discrimination Coefficient). Tjur's R^2 is particularly intuitive: if the average predicted probability for "Success" is 0.8 and for "Failure" is 0.2, Tjur's D is 0.6.

```
library(performance)
#  Calculate various Pseudo-R2 metrics
r2(logistic_multi)
```

```
## #    R2 for Logistic Regression
##      Tjur's R2: 0.278
```

5.2.2. Univariable and Multivariable Models

In Logistic Regression, the outcome or dependent variable is binary. The predictor or independent variable is one with a univariable model and more than one with a multivariable model. In reality, most outcomes have many predictors. Hence, multivariable Logistic Regression is closer to reality.

5.3. Logistic Regression Model Building

5.3.1. Main Steps

Below are the main steps of building Logistic Regression (logit) models:

- Import data;
- Clean data and check for class bias;

- Create training and test samples;
- Compute information value to find out important variables;
- Build logit models;
- Predict on test data;
- Do model evaluation.

5.3.2. Building Model in R

We use the same example diabetes dataset above. Our aim is to build a model so that we can predict the probability of having the "outcome" (diabetes) if a profile is given.

Import Data

Now we import the dataset and take a look at the data's structure by using the function *str()* and summarize all variables in the dataset by "outcome" and overall (Table 8):

```
url_base<-"https://raw.githubusercontent.com/plotly/"
data_path<-"datasets/master/diabetes.csv"
# Load example data
data<-read.csv(paste0(url_base, data_path))
colnames(data)<-tolower(colnames(data))
str(data)
```

```
## 'data.frame': 768 obs. of 9 variables:
## $ pregnancies                : int 6 1 8 1 0 5 3 10 2 8 ...
## $glucose                     : int 148 85 183 89 137 116 78 115 197 125 ...
## $ bloodpressure              : int 72 66 64 6640 74 50 0 70 96 ...
## $ skinthickness              : int 35 29 0 23 35 0 32 0 45 0 ...
## $ insulin                    : int 0 0 0 94 168 0 88 0 543 0 ...
## $ bmi                        : num 33.6 26.6 23.3 28.1 43.1 25.6 31 35.3 30.5 0 ...
## $ diabetespedigreefunction   : num 0.627 0.351 0.672 0.167 2.288 ...
## $ age                        : int 50 31 32 21 33 30 26 29 53 54 ...
## $ outcome                    : int 1 0 1 0 1 0 1 0 1 1 ...
```

```
#convert outcome to factor
data$outcome<-as.factor(data$outcome)

#summary of all variables in the dataset using the arsenal package
my_controls <- tableby.control( test
  = TRUE,
  total = TRUE,
  numeric.stats = c("meansd", "medianq1q3","meanCI","range","Nmiss2"), cat.stats =
  c("countpct", "Nmiss2"),
  stats.labels = list( meansd =
    "Mean (SD)",
    medianq1q3 = "Median (Q1, Q3)", range =
    "Min - Max",
    Nmiss2 = "Missing", meanCI =
    "Mean (95%CI)"
  )
)
```

```
prevar<-colnames(data)[!colnames(data) %in% "outcome"] mylabels <-
as.list(prevar)
names(mylabels)<-prevar
tab <- tableby(as.formula(paste("outcome",
                                paste(prevar,collapse = "+"), sep = "~")),
               data = data,
               control = my_controls)
kable(summary(tab,labelTranslations = mylabels, text = TRUE),
      caption = "Data summary")%>%
  kable_styling(latex_options = c("scale_down", "hold_position"))
```

Table 8. Data summary.

	0 (N = 500)	**1 (N = 268)**	**Total (N = 768)**	***p*-Value**
pregnancies				<0.001
- Mean (SD)	3.298 (3.017)	4.866 (3.741)	3.845 (3.370)	
- Median (Q1, Q3)	2.000 (1.000, 5.000)	4.000 (1.750, 8.000)	3.000 (1.000, 6.000)	
- Mean (95% CI)	3.298 (3.033, 3.563)	4.866 (4.416, 5.316)	3.845 (3.606, 4.084)	
- Min–Max	0.000–13.000	0.000–17.000	0.000–17.000	
- Missing	0	0	0	
glucose				<0.001
- Mean (SD)	109.980 (26.141)	141.257 (31.940)	120.895 (31.973)	
- Median (Q1, Q3)	107.000 (93.000, 125.000)	140.000 (119.000, 167.000)	117.000 (99.000, 140.250)	
- Mean (95% CI)	109.980 (107.683, 112.277)	141.257 (137.416, 145.099)	120.895 (118.630, 123.159)	
- Min–Max	0.000–197.000	0.000–199.000	0.000–199.000	
- Missing	0	0	0	
bloodpressure				0.072
- Mean (SD)	68.184 (18.063)	70.825 (21.492)	69.105 (19.356)	
- Median (Q1, Q3)	70.000 (62.000, 78.000)	74.000 (66.000, 82.000)	72.000 (62.000, 80.000)	
- Mean (95% CI)	68.184 (66.597, 69.771)	70.825 (68.240, 73.409)	69.105 (67.734, 70.477)	
- Min–Max	0.000–122.000	0.000–114.000	0.000–122.000	
- Missing	0	0	0	
skinthickness				0.038
- Mean (SD)	19.664 (14.890)	22.164 (17.680)	20.536 (15.952)	
- Median (Q1, Q3)	21.000 (0.000, 31.000)	27.000 (0.000, 36.000)	23.000 (0.000, 32.000)	
- Mean (95% CI)	19.664 (18.356, 20.972)	22.164 (20.038, 24.291)	20.536 (19.406, 21.666)	
- Min–Max	0.000–60.000	0.000–99.000	0.000–99.000	
- Missing	0	0	0	
insulin				<0.001
- Mean (SD)	68.792 (98.865)	100.336 (138.689)	79.799 (115.244)	
- Median (Q1, Q3)	39.000 (0.000, 105.000)	0.000 (0.000, 167.250)	30.500 (0.000, 127.250)	
- Mean (95% CI)	68.792 (60.105, 77.479)	100.336 (83.656, 117.016)	79.799 (71.636, 87.963)	
- Min–Max	0.000–744.000	0.000–846.000	0.000–846.000	
- Missing	0	0	0	
bmi				<0.001
- Mean (SD)	30.304 (7.690)	35.143 (7.263)	31.993 (7.884)	
- Median (Q1, Q3)	30.050 (25.400, 35.300)	34.250 (30.800, 38.775)	32.000 (27.300, 36.600)	
- Mean (95% CI)	30.304 (29.629, 30.980)	35.143 (34.269, 36.016)	31.993 (31.434, 32.551)	
- Min–Max	0.000–57.300	0.000–67.100	0.000–67.100	
- Missing	0	0	0	
diabetespedigreefunction				<0.001
- Mean (SD)	0.430 (0.299)	0.550 (0.372)	0.472 (0.331)	
- Median (Q1, Q3)	0.336 (0.230, 0.562)	0.449 (0.263, 0.728)	0.372 (0.244, 0.626)	
- Mean (95% CI)	0.430 (0.403, 0.456)	0.550 (0.506, 0.595)	0.472 (0.448, 0.495)	
- Min–Max	0.078–2.329	0.088–2.420	0.078–2.420	
- Missing	0	0	0	

Table 8. *Cont.*

	0 (N = 500)	1 (N = 268)	Total (N = 768)	*p*-Value
age				<0.001
- Mean (SD)	31.190 (11.668)	37.067 (10.968)	33.241 (11.760)	
- Median (Q1, Q3)	27.000 (23.000, 37.000)	36.000 (28.000, 44.000)	29.000 (24.000, 41.000)	
- Mean (95% CI)	31.190 (30.165, 32.215)	37.067 (35.748, 38.386)	33.241 (32.408, 34.074)	
- Min–Max	21.000–81.000	21.000–70.000	21.000–81.000	
- Missing	0	0	0	

Source: Table by author.

We can notice that there is a greater number of "outcome" = 0 (no diabetes) than "outcome" = 1 (diabetes). From the summary table, we can see the variables showing imbalance between "outcome" = 0 (no diabetes) and "outcome" = 1 (diabetes).

Build Logistic Regression Model in R

For simplicity, we will build our model with all the data (no train-test splitting). For predictive modeling, we will need to develop a model on training data and validate the model on test data.

The *glm()* procedure with *family = "binomial"* will build the Logistic Regression model on the given formula.

```
# Univariate model
fitu <- glm(outcome ~ bmi,data = data, family = "binomial")

# Multivariate model
fitm <- glm(outcome ~ bmi + glucose + bloodpressure, data = data, family =
            "binomial")
```

- *data = data* is a data frame named data;
- *family = "binomial"* is for the Logistic Regression model;
- *outcome* is the column which contains the response variable "outcome";
- ~ help us select the predictors:
  - . means choose all the data that we have.
  - If we want to choose specific variables as predictors, we can replace . with the combination of our interest variables. For example, *~ bmi* for choosing only "bmi"; *~ bmi + glucose + bloodpressure* for choosing "bmi", "glucose", "bloodpressure".
  - You can even exclude the predictor. For example, you write *~. - glucose* to unselect variable "glucose" and end up with *fitm <- glm(outcome ~. - glucose, data = data, family = "binomial").*

Interpretation of Results

```
summary(fitm)
```

```
##
## Call:
## glm(formula = outcome ~ bmi + glucose + bloodpressure, family = "binomial", data = data)
##
## Coefficients:
##                 Estimate   Std. Error   z value   Pr(>|z|)
## (Intercept)     −7.182306   0.635462   −11.303   <2×10-16 ***
## bmi              0.079793   0.013565    5.882    4.05×10-9 ***
## glucose          0.035746   0.003328   10.740    <2×10-16 ***
## bloodpressure   −0.007420   0.004862   −1.526    0.127
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
## Null deviance: 993.48 on 767 degrees of freedom
## Residual deviance: 769.07 on 764 degrees of freedom
## AIC: 777.07
##
## Number of Fisher Scoring iterations: 4
```

- Deviance Residuals: Summary of the distribution of the deviance residuals. Deviance residuals measure how well the observations fit the model. The closer a residual is to 0, the better the fit of the observation.
- Coefficients:
  - In general, the Intercept gives the mean level of the response variable when all other variables are 0. The estimated intercept is −7.1823064, the standard error of the estimated intercept is 0.6354617, and the z value is the estimated intercept divided by the standard error.
  - Each one-unit change in "bmi" will increase the log(odds) of getting "outcome" by 0.0797926, and its p value indicates that it is significant in determining the "outcome".
  - Each unit increase in "glucose" increases the log odds of getting "outcome" by 0.0357463, and p value indicates that it is significant in determining the "outcome".
  - "bloodpressure" is not significant in determining the outcome.
- Deviance: Null deviance is the value when there is only an intercept and no variables. Residual deviance is the value when taking all the variables into account. It makes sense to consider the model good if that difference is big enough. Generally, the degrees of freedom reported on the Null deviance are always higher than the degrees of freedom reported on the Residual deviance. The difference between Null deviance and Residual deviance tells how good the model fit is: the greater the difference, the better the model.
- Akaike Information Criterion (AIC): The Akaike Information Criterion (AIC) provides a method for assessing the quality of the model through comparison of related models. It is useful for comparing models, but it is not interpretable on its own. If there are several candidate models, the model that has the smallest AIC should be selected.

Making Prediction

Let us predict if a person has a profile of "bmi" = 29, "glucose" = 148, "bloodpressure" = 96, what is the probability of that person getting "outcome" = 1.

```
x <- data.frame(bmi = 29, glucose = 148, bloodpressure = 96)
p<- predict(fitm, x, type = "response")
p
```

```
##    1
##    0.4279538
```

We see that there is 0.4279538 chance that she will get "outcome" = 1.

5.4. Model Evaluation

5.4.1. Classification Metrics

Classification results may be visualized using confusion matrix (Figure 62):

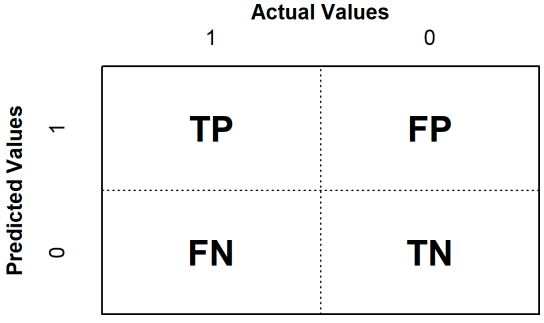


Figure 62. Confusion matrix. Source: Figure by author.

The Positive/Negative label refers to the predicted outcome of an experiment, while the True/False label refers to the actual outcome.

- Type 1 error corresponds to a False Positive.
- Type 2 error corresponds to a False Negative.

Which aspect to improve depends on specific problems. For example, for a model to identify if a breast cancer is benign or malignant, one would prefer a Type 2 error to a Type 1 error because one would not want to miss malignant cases.

From the confusion matrix, different metrics can be measured for the validity of the model:

- Accuracy (all correct/all): ACC = (TP + TN)/(TP + TN + FP + FN);
- Misclassification or Error rate (all incorrect/all): Misclassification = (FP + FN)/(TP + TN + FP + FN);
- Precision or Positive predictive value (PPV) (true positives/predicted positives): Precision = TP/(TP + FP);
- Sensitivity or Recall or true positive rate (true positives/all actual positives): Sensitivity refers to the test's ability to correctly detect ill patients who do have the condition. Sensitivity = TP/(TP + FN);

- Specificity (true negatives/all actual negatives): A positive result in a test with high specificity is useful for ruling in disease. The test rarely gives positive results in healthy patients. A positive result signifies a high probability of the presence of disease. Specificity = TN/(TN + FP).

The accuracy score might not be the best metric to use for a classification problem, especially if you deal with an unbalanced dataset.

5.4.2. Calculate, Visualize Classification Measures

We can calculate and visualize classification measures using the *"ROCR"* package [48]. First, we use *predict()* to get predicted values from a logistic regression fit object, then create a prediction object with *prediction()*, then use *performance()* to get classification measures, and use *plot()* to visualize the classification measures.

Sensitivity by Cutoff

```
library(ROCR)
predicted <- predict(fitm, type = "response")
# Create a prediction object
pred <- prediction(predicted, data$outcome)
# Sensitivity by cutoff
perf<-performance(pred,"sens","cutoff")
plot(perf)
```

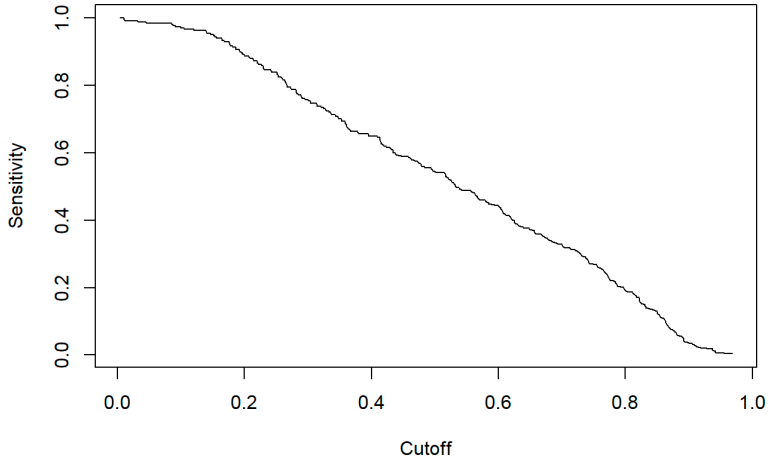


Figure 63. Sensitivity by cutoff. Source: Figure by author.

The plot above shows that as the cutoff (x-axis) decreases (we become less strict), sensitivity (y-axis) increases and vice versa (Figure 63).

Specificity by Cutoff

```
perf <- performance(pred,"spec","cutoff")
plot(perf)
```

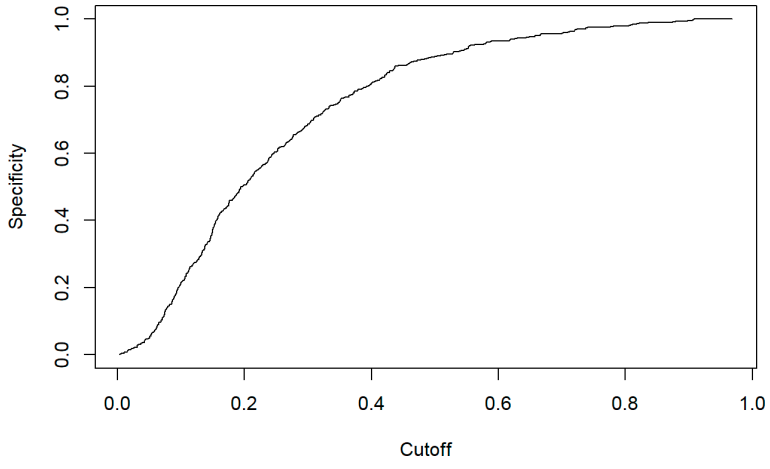


Figure 64. Specificity by cutoff. Source: Figure by author.

The plot above shows that as the cutoff (x-axis) decreases, specificity (y-axis) decreases and vice versa (Figure 64).

Sensitivity and Specificity

```
perf <- performance(pred, "sens", "spec")
plot(perf)
```

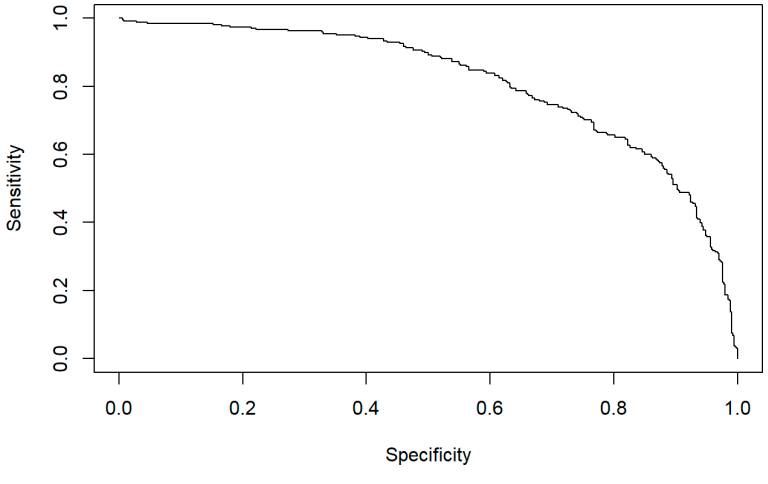


Figure 65. Sensitivity and specificity. Source: Figure by author.

The plot above shows the change in sensitivity according to specificity: as specificity increases, sensitivity decreases and vice versa (Figure 65).

The Highest Sensitivity + Specificity

```
max(perf@x.values[[1]]+perf@y.values[[1]])
```

```
## [1]  1.467522
```

The cutoff that yields the highest sensitivity + specificity:

```
perf@alpha.values[[1]][which.max(perf@x.values[[1]]+perf@y.values[[1]])]
```

```
## [1]    0.4132145
```

The cutoff that best balances sensitivity and specificity is 0.4132145.
Accuracy (all correct/all) by cutoff:

```
perf <- performance(pred, "acc","cutoff")
plot(perf)
```

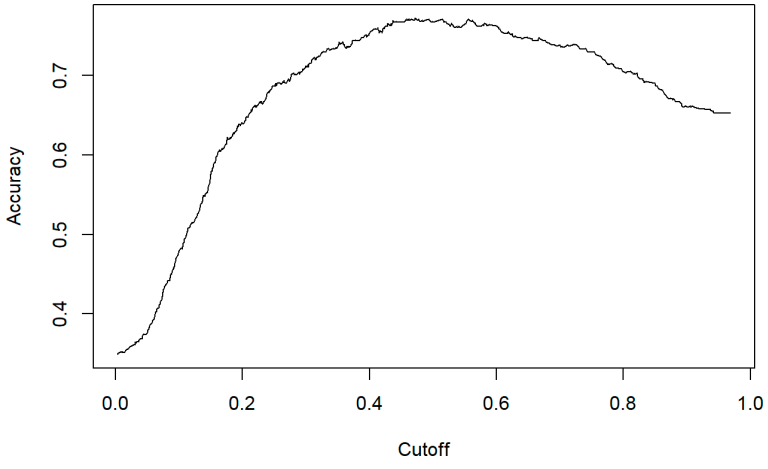


Figure 66. Accuracy by cutoff. Source: Figure by author.

The plot above shows that the model's overall accuracy is highest with a cutoff somewhere between 0.4 and 0.6 (Figure 66).

Note that Standard accuracy can be misleading if the dataset is imbalanced (e.g., 90% of patients do not have the disease). To address this, one may use:

- Balanced Accuracy: The arithmetic mean of sensitivity and specificity. It provides a better evaluation of the model's performance across both classes.
- Youden's J Statistic (Youden Index): Used to find the optimal probability threshold. It represents the vertical distance between the ROC and the diagonal line.

These can be done using the package "*caret*" [45].

```
library(caret)
# 1. Generate probabilities and apply the 0.5 threshold
# "fitm" is the model from the previous steps
probs <- predict(fitm, type = "response")
predicted_classes <- ifelse(probs > 0.5, "1", "0")
# 2. Create the Confusion Matrix
# Ensure levels match the actual outcome in the dataset
cm <- confusionMatrix(as.factor(predicted_classes),
as.factor(data$outcome),
positive = "1")
# 3. Access Balanced Accuracy
# Balanced Accuracy = (Sensitivity + Specificity) / 2
bal_acc <- cm$byClass['Balanced Accuracy']
bal_acc
```

```
##    Balanced Accuracy
##    0.7145224
```

```
# 4. Calculate Youden's J Statistic
# Youden's J = Sensitivity + Specificity - 1
sens <- cm$byClass['Sensitivity']
spec <- cm$byClass['Specificity']
youden_j <- sens + spec - 1
youden_j
```

```
##    Sensitivity
##    0.4290448
```

The outputs show that, regarding Balanced Accuracy, the model correctly classifies about 71.45% of cases on average across both classes. This value reflects the average performance for detecting positive and negative classes. Regarding the Youden Index, the model improves classification accuracy by about 42.9 percentage points above random chance. This indicates moderate discriminative ability.

While Sensitivity/Specificity, Balanced Accuracy, and Youden's J are often discussed as different measures of model performance, these three metrics are mathematically linked. Consequently, maximizing one will always maximize the others, leading to the exact same optimal cutoff point.

- Sensitivity + Specificity: This is the raw sum of the model's ability to correctly identify both positive and negative cases.
- Balanced Accuracy: This is defined as the arithmetic mean of sensitivity and specificity. Since this is simply the sum divided by a constant (2), the cutoff that maximizes the sum will also maximize the average.
- Youden's J Statistic: This metric represents the vertical distance between the ROC and the diagonal chance line: $J = Sensitivity + Specificity - 1$. Again, because this is just the sum minus a constant (1), the maximum value of J will occur at the same threshold where the sum of sensitivity and specificity is at its peak.

5.4.3. Discrimination Power

Discrimination power may be evaluated using the Receiver Operating Characteristics (ROC) Curve and the Area Under the Curve (AUC).

Receiver Operating Characteristics (ROC) Curve

Receiver Operating Characteristics (ROC) Curve traces the percentage of TPs accurately predicted by a given model as the prediction probability cutoff is lowered from 1 to 0. It plots Sensitivity (TPR) on the y-axis against 1 − Specificity (FPR) on the x-axis for all possible cutoffs. We will mostly use the R package "ROCR" for evaluation of model performance (Figure 67) [48].

```
library(ROCR)
predicted <- predict(fitm, type = "response")

# Create a prediction object
pred <- prediction(predicted, data$outcome)

# Create a performance object
perf <- performance(pred,"tpr","fpr")
# ROC curve
plot(perf)
abline(a = 0, b = 1, lty = 2, col = "blue") # Adds the y = x line
```

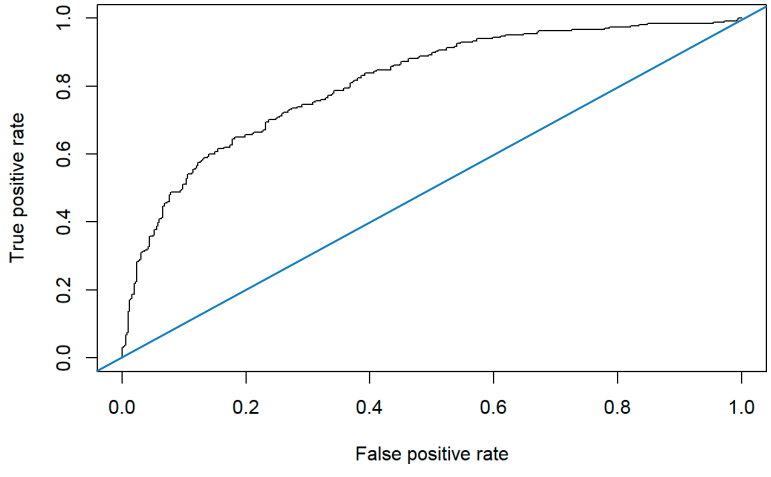


Figure 67. Receiver Operating Characteristics (ROC) Curve. Source: Figure by author.

For a good model:

- As the cutoff is lowered, it should mark more of actual label 1 as positives and fewer of actual label 0 as label 1.
- The curve should rise steeply, indicating that the TPR (Y-Axis) (or Sensitivity) increases faster than the FPR (X-Axis) (or Specificity) as the cutoff score decreases.
- A model with no discrimination power would be a 45-degree diagonal line.
- A perfect model would go straight up to the top-left corner (100% sensitivity, 100% specificity) and then across.
- Our model's curve bows far up to the top-left, showing it has good discriminatory power. It is much better than random chance.

Area Under the Curve (AUC)

The Area Under the Curve (AUC) is a single number that summarizes the entire ROC. It represents the probability that the model will rank a randomly chosen positive case higher than a randomly chosen negative case. The greater the area under the ROC (AUC), the better the predictive ability of the model.

- AUC = 1.0: Perfect model.
- AUC = 0.5: Useless model (same as random chance).
- AUC > 0.7: Generally considered a good model.

```
perf <- performance(pred,"auc") perf@y.values[[1]]
```

```
## [1]  0.8114627
```

An AUC of 0.8115 indicates good discriminatory power, and the model is very effective at distinguishing between patients with and without diabetes.

5.4.4. Model Fit Statistics

We may evaluate the logistic regression model's goodness of fit using some model fit statistics below.

Akaike Information Criterion (AIC)

A measure of model fit that penalizes complexity. Lower values are better when comparing different models. We can get AIC using *AIC()*.

```
aic_value <- AIC(logistic_uni)
cat("AIC univariable model: ", round(aic_value, 2), "\n")
```

```
## AIC univariable model:  924.71
```

```
aic_value <- AIC(logistic_multi)
cat("AIC multivariable model: ", round(aic_value, 2), "\n")
```

```
## AIC multivariable model:  777.07
```

The output shows that the multivariable model has better model fit than the univariable model.

Bayesian Information Criterion (BIC)

Similar to AIC but with a larger penalty for model complexity. The model with the lower BIC is better. We can get BIC using *BIC()*.

```
bic_value <- BIC(logistic_uni)
cat("BIC univariable model: ", round(bic_value, 2), "\n")
```

```
## BIC univariable model:  934
```

```
bic_value <- BIC(logistic_multi)
cat("BIC multivariable model: ", round(bic_value, 2), "\n")
```

```
## BIC multivariable model:  795.64
```

The output also shows that the multivariable model has better model fit than the univariable model.

McFadden's Pseudo R-Squared

Using the *pR2()* function from the "*pscl*" package [47]:

```
mcfadden_r2 <- pR2(logistic_multi)["McFadden"]

 ## fitting null model for pseudo-r2

cat("McFadden's Pseudo R-squared for multivariable model: ", round(mcfadden_r2, 4), "\n")

 ## McFadden's Pseudo R-squared for multivariable model:    0.2259
```

The output shows the proportional improvement in model fit compared to a null model. A value of 0.2259 suggests a relatively good improvement.

Note that one may use the function *compare_performance()* of the "*performance*" [42] package to create a summary table that includes the Brier Score. The Brier Score measures the accuracy of probabilistic predictions (closer to 0 is better). The visual radar chart (spider plot) produced by performance allows the reader to quickly see if Model 2 ("fitm") improves upon Model 1 ("fitu") across multiple dimensions (AIC, R2, RMSE) simultaneously. Visualization may be done together with the "*see*" package [43].

```
library(performance)
library(see)
#  Compare Model fit metrics comprehensively
model_perf <- compare_performance(fitu, fitm, metrics = "all")
plot(model_perf) # Generates a radar chart comparing models
```

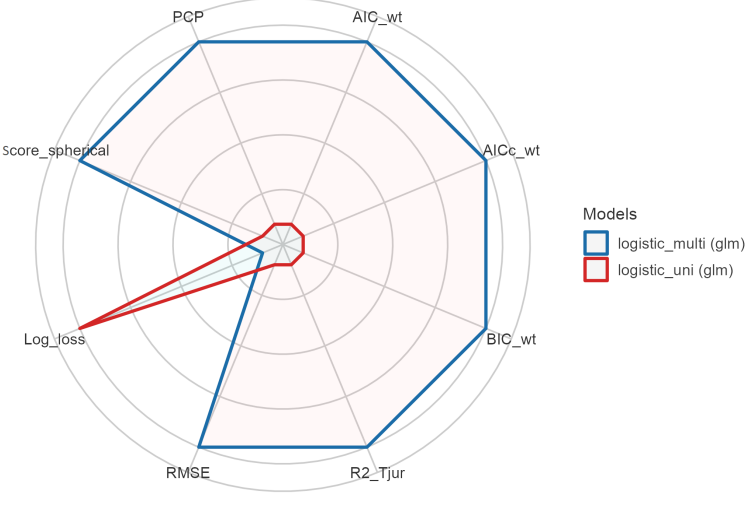


Figure 68. Radar plot from *compare_performance()*. Source: figure by author.

Similar to the above, the output plot shows that the multivariable model fits better than the univariable model (Figure 68).

5.5. Model Validation

5.5.1. Create Training and Test Samples

For model validation, the idea is that we would like to divide our original data into two groups: the training group for training our model. While we will leave aside a bunch of data to check if our model is consistent. This will be our test group. And if we obtain a much worse prediction on our test set than in our training group, then probably we are overfitting our training data. Usually, an 80–20% or 70–30% train–test split is considered reasonable.

Because there is a greater number of "outcome" = 0 (no diabetes) than "outcome" = 1 (diabetes), we can address this problem of class bias by building a model from a balance *trainingData*.

```
library(tidyverse)
data$rn<-paste("rn",1:nrow(data),sep = "") #to facilitate row selecting later
set.seed(100)  # for repeatability of samples #
Create Training data
# (sampling with the same fraction 0.8 for outcome  = 0 (no diabetes) #
and outcome  = 1 (diabetes)
trainingData<-data %>%
  group_by(outcome)  %>%
  sample_frac(size = 0.8) # sampling fraction the same
# Test Data is the remaining data that is not in training data
testData <- data[!data$rn %in% trainingData$rn,]
# drop rn
trainingData<-trainingData %>% select(!rn)
testData<-testData %>% select(!rn)
table(trainingData$outcome)
```

```
##
##   0   1
##  400  214
```

```
table(testData$outcome)
```

```
##
##   0   1
##  100  54
```

group_by(outcome) tells *sample_frac*() to operate within each outcome group separately. *sample_frac(size = 0.8)* randomly selects 80% of the rows from that group. This results in a training set with 400 "0"s and 214 "1"s, and a test set with 100 "0"s and 54 "1"s, perfectly preserving the original ratio or similar proportion of "outcome" = 0 and "outcome" = 1 between *trainingData* and *testData*.

5.5.2. Build Logitic Models on Training Data and Predict on Test Data

We build the same multivariable model above that uses "bmi", "glucose", and "bloodpressure" to predict "outcome" on *trainingData* then validate the model on *testData*. The model on *trainingData*:

```
fitm.train<- glm(outcome ~ bmi + glucose + bloodpressure,
                 data = trainingData, family = "binomial")
summary(fitm.train)
```

```
##
## Call:
## glm(formula = outcome ~ bmi + glucose + bloodpressure, family = "binomial",
## data = trainingData)
##
## Coefficients:
##                    Estimate    Std. Error   z value   Pr(>|z|)
## (Intercept)        −7.385483   0.716207     −10.312   <2×10-16 ***
## bmi                0.081722    0.015328     5.332     9.73×10-8 ***
## glucose            0.036485    0.003691     9.886     <2×10-16 ***
## bloodpressure      −0.006937   0.005440     −1.275    0.202
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
## Null deviance: 793.94 on 613 degrees of freedom
## Residual deviance: 605.86 on 610 degrees of freedom
## AIC: 613.86
##
## Number of Fisher Scoring iterations: 4
```

Now, we make a prediction and evaluate the model's performance on testData (Figure 69):

```
library(ROCR)
# predicted probability
predicted <- plogis(predict(fitm.train, testData))
# or
predicted <- predict(fitm.train, testData, type = "response")

# Create a prediction object
pred <- prediction(predicted, testData$outcome)

# Create a performance object
perf <- performance(pred,"tpr","fpr")
# ROC curve
plot(perf, main = "ROC")
abline(a = 0, b = 1, lty = 2, col = "blue") # Adds the y = x line
```

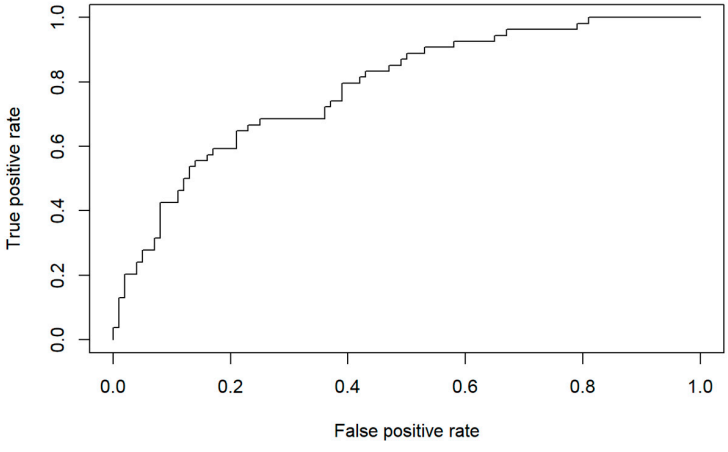


Figure 69. ROC test data. Source: Figure by author.

We can also check other performance measures similarly to the examples above (Figures 70–73):

```
# Sensitivity by cutoff
perf <- performance(pred,"sens","cutoff")
plot(perf)
```

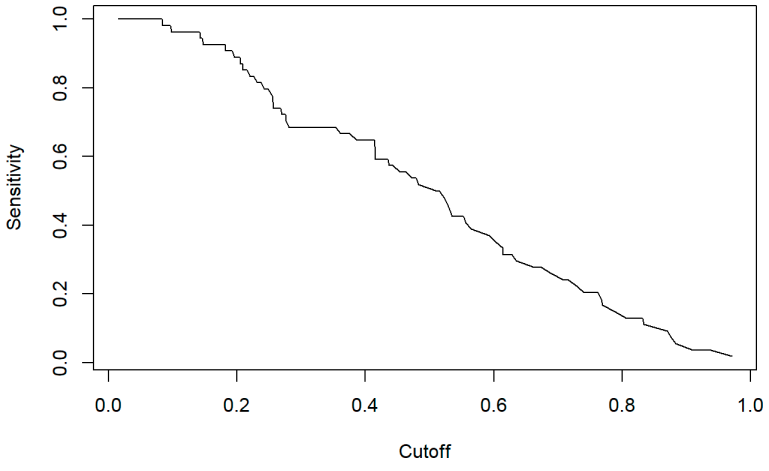


Figure 70. Sensitivity by cutoff test data. Source: Figure by author.

```
# Specificity by cutoff
perf <- performance(pred,"spec","cutoff")
plot(perf)
```

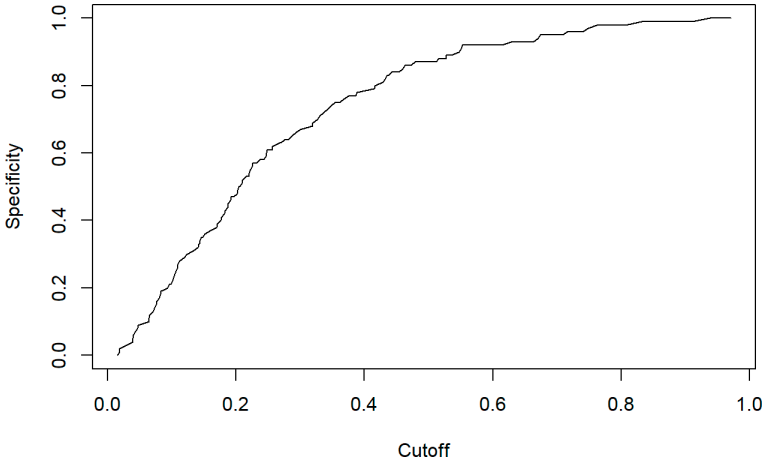


Figure 71. Specificity by cutoff test data. Source: Figure by author.

```
# Sensitivity, specificity
perf <- performance(pred, "sens", "spec")
plot(perf)
```

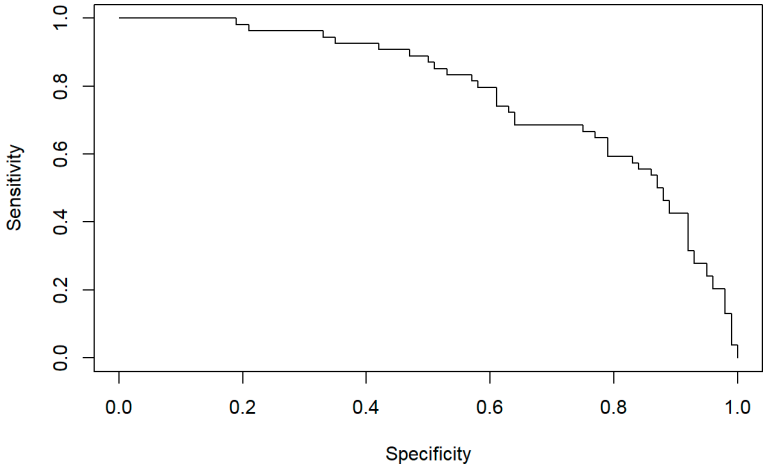


Figure 72. Sensitivity, specificity test data. Source: Figure by author.

```
# Accuracy (all correct / all) by cutoff:
perf <- performance(pred,"acc","cutoff")
plot(perf)
```

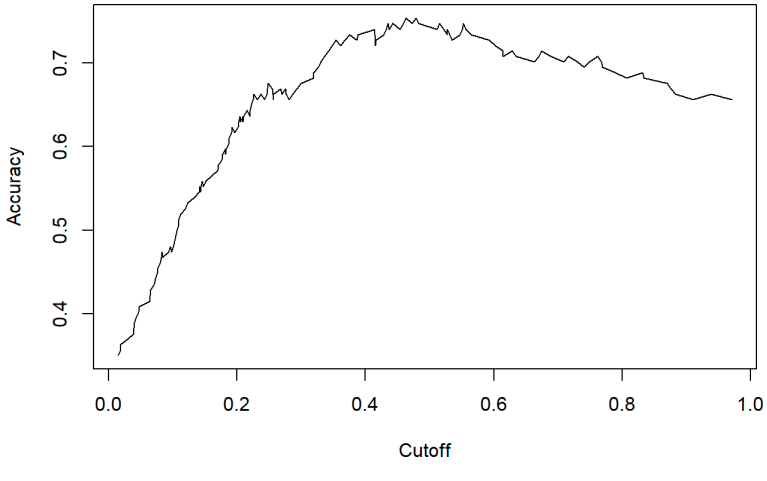


Figure 73. Accuracy by cutoff test data. Source: Figure by author.

```
# Re-create the performance object for Sensitivity and Specificity
perf_sens_spec <- performance(pred, "sens", "spec")

# The highest sensitivity + specificity
max(perf_sens_spec@x.values[[1]] + perf_sens_spec@y.values[[1]])
```

```
## [1]  1.438148
```

```
# The cutoff that yields the highest sensitivity + specificity
perf_sens_spec@alpha.values[[1]][which.max(perf_sens_spec@x.values[[1]] +
                                           perf_sens_spec@y.values[[1]])]
```

```
## [1]  0.4147078
```

```
# AUC
perf_auc <- performance(pred,"auc")
perf_auc@y.values[[1]]
```

```
## [1]  0.7809259
```

Our model's performance on *testData* is quite similar to the model's performance on training data. Specifically, the model achieved an AUC of 0.81 on the training data and an AUC of 0.78 on the test data. These two values are very close, which is an excellent result. It means our model is not overfit and generalizes well to new data, making it a reliable predictive model.

Similar to the previous chapter on linear regression, resampling approaches for logistic regression model validation, including K-Fold Cross-Validation and Bootstrapping, may provide more robust evaluation of model performance.

5.6. Useful Resources

- "ROCR" package vignette [48].
- Example diabetes data (https://raw.githubusercontent.com/plotly/datasets/master/diabetes.csv, accessed on 10 April 2025).

5.7. Chapter Summary

- Logistic Regression is used to predict categorical variables (model the logarithm of the probability of an outcome occurring);
- Logistic regression model may be fitted using the *glm()* function with *family = binomial(link = "logit");*
- Maximum likelihood *logLik(), AIC()* for Akaike Information Criterion, *BIC()* for Bayesian Information Criterion, *pR2()* for McFadden's pseudo R-squared help select the logistic regression model that best fits the data;
- Model evaluation may be done with the "*ROCR*" package (e.g., *prediction()* and *performance()* functions):
  - Classification metrics: sensitivity, specificity, accuracy, . . .
  - Discrimination power: ROC, AUC
  - Model Fit Statistics: AIC, BIC, Pseudo R-squared, . . .
- Model validation: fit the model on training data and validate on test data.

6. Survival Data Analysis with R

This chapter introduces the essential concepts and practical application of survival data analysis in R, covering key statistical concepts like the Survival Function, Hazard Function, and Censoring, etc. The practical implementation centers on the R packages "*survival*" and "*survminer*". Readers will learn how to create and visualize non-parametric Kaplan–Meier survival curves and how to test for differences between survival curves. The main analytical focus is the Cox Proportional Hazards Model to estimate Hazard Ratios (HR) while adjusting for covariates. The chapter also covers essential model validation steps, including checking the Proportional Hazards Assumption and more advanced topics such as Competing Risk Analysis and handling Time-Dependent Covariates.

6.1. Introduction to Survival Analysis

Survival analysis is the study of survival times and of the factors that influence them. Survival analysis investigates the time it takes for an event of interest to occur (time-to-event).

In many bio-medical and clinical studies, the outcome of interest is not just whether an event occurred, but when it occurred (time until an event occurs), for example:

- Time from birth until death.
- Time from entry into a clinical trial until tumor response.
- Time from treatment initiation to disease recovery.

And much more . . .
Below are some other applications of survival analysis:

- Reliability analysis.
- Duration analysis.
- Event history analysis.
- Time-to-event analysis.

Standard regression models cannot handle "censoring" (situations where a patient drops out or the study ends before the event happens). Survival analysis correctly utilizes this incomplete "time-to-event" data without biasing the results.

6.1.1. Survival Data

- Time to event: the response variable is a non-negative discrete or continuous random variable and represents the time from a well-defined origin (e.g., the date of diagnosis, the start of treatment, or birth . . .) to a well-defined event (e.g., death, recovery . . .) (distinct start time and end time).
- Survival data is often represented as a pair $(t_i; \delta_i)$, where t is the time until endpoint or last follow-up, and δ is a $0/1$ variable with 0 = "subject was censored at t" and 1 = "subject had an event at t", or in R code as Surv(time, status) [50].
- Censoring: This occurs when the event of interest is not observed for a subject.

The most common form is right-censoring, where we know that the event has not happened by a certain time, but we do not know when it will happen. For

example, a patient might still be alive at the end of a clinical trial, but we do not know when the patient might die.

6.1.2. Key Statistical Concepts

- Survival Function ($S(t)$): This function gives the probability that a subject will survive longer than a specified time t. It is a non-increasing function, starting at 1 at time 0 and approaching 0 as time increases.
- Hazard Function ($h(t)$): Also known as the hazard rate or instantaneous failure rate, the hazard function represents the instantaneous risk of the event occurring at time t, given that the subject has survived up to that time [51].

6.1.3. Clinical Case Study: Factors Affecting Lung Cancer Survival

For the chapter's examples, the "lung" dataset from the "*survival*" package will be used primarily [51].

- The Data: 228 patients with advanced lung cancer.
- The Outcome: Time until death (survival time) and the status indicator (1 = censored, 2 = dead).
- Clinical Question: What is the median survival time for this cohort, and how do age, sex, and weight loss influence the risk of death? The intention is to identify which clinical factors (age, sex, and weight loss) significantly impact the survival time of patients with advanced lung cancer.

6.2. Essential R Packages for Survival Analysis

- "*survival*": This is the essential core package for survival analysis in R. It contains functions for creating survival objects, fitting Kaplan–Meier curves, and building Cox proportional hazards models [51].
- "*survminer*": This package provides functions for creating more informative and publication-ready visualizations of survival analysis results [52], building upon the "*ggplot2*" package [18].
- "*lubridate*": This package is for working with dates (to create time-to-event data) [53].

Packages "*glmnet*" for LASSO variable selection [44] and "*cmprsk*" for competing risk analysis [54] will also be used. Some supporting packages ("*tidyverse*" [11], "*knitr*" [5], "*kableExtra*" [12]) are routinely used, as mentioned in previous chapters. First, we load the packages used in this chapter.

```
install.packages(c("survival","survminer","lubridate"))
#load multiple packages
lapply(c("survival", "survminer", "lubridate",
                "glmnet", "cmprsk", "tidyverse", "knitr", "kableExtra"), library, character.only = TRUE)
```

6.2.1. Working with Dates

Data will often come with start and end dates rather than pre-calculated survival times. The purpose of this step is to calculate the precise follow-up time required for survival objects.

- Make sure these dates are formatted as dates in R.

- Calculate survival time (time-to-event).

We first load the example data and convert dates to data using *as.Date()*. We can then calculate the number of days between the two dates using *difftime()* and convert it to a numeric value, then convert it to months by dividing by 30.42. Or, we can use the "*lubridate*" package for this with *interval()*. By using *interval()* from the "*lubridate*" package, we accurately calculate the time elapsed between study enrollment and the endpoint, converting it to months. This numerical value ("ttevent.m") becomes the response variable for subsequent models.

```
url_base<-"https://stats.idre.ucla.edu/stat/r/examples/asa/"
data_use<-"hmohiv.csv"
exdate<-read.table(paste0(url_base, data_use), sep =",", header = TRUE)
#dates with base R
# calculate the number of days between our two dates # and convert it
to a numeric value using as.numeric. # Then convert to month by
dividing by 30.42. exdateb<-exdate %>%
  mutate_at(c("entdate","enddate"), as.Date, format = "%m/%d/%Y")%>% mutate(
    ttevent.m = as.numeric( difftime(enddate,
                entdate,
                units = "days"))/30.42)
head(exdateb)
```

```
##  ID    time  age  drug  censor  entdate     enddate     ttevent.m
##  1   1  5     46   0     1       1990-05-15  1990-10-14  4.9967127
##  2   2  6     35   1     0       1989-09-19  1990-03-20  5.9829060
##  3   3  8     30   1     1       1991-04-21  1991-12-20  7.9881657
##  4   4  3     30   1     1       1991-01-03  1991-04-04  2.9914530
##  5   5  22    36   0     1       1989-09-18  1991-07-19  21.9921105
##  6   6  1     32   1     0       1991-03-18  1991-04-17  0.9861933
```

```
#with lubridate
library(lubridate)
exdatel<-exdate%>%
   mutate_at(c("entdate","enddate"), mdy)%>%
   mutate(
      ttevent.m = interval(entdate,enddate)/months(1))
head(exdatel)
```

```
##  ID    time  age  drug  censor  entdate     enddate     ttevent.m
##  1   1  5     46   0     1       1990-05-15  1990-10-14  4.9666667
##  2   2  6     35   1     0       1989-09-19  1990-03-20  6.0046729
##  3   3  8     30   1     1       1991-04-21  1991-12-20  7.9666667
##  4   4  3     30   1     1       1991-01-03  1991-04-04  3.0081967
##  5   5  22    36   0     1       1989-09-18  1991-07-19  22.0014265
##  6   6  1     32   1     0       1991-03-18  1991-04-17  0.9677419
```

```
#rename
exdatel<-exdatel%>%
   rename(status = censor)
```

6.2.2. Package "Survival" Main Functions

Below are the basic functions of the "*survival*" package [51]:

Surv()

This function is for creating a survival object.

- *Surv(time, status)*: right censored data;

- *Surv(time, endpoint = = "death")*: right censored data, where the status variable is a character or factor;
- *Surv(t1, t2, status)*: counting process data;
- *Surv(t1, ind, type = 'left')*: left censoring;
- *Surv(time, fstat)*: multiple state data, fstat is a factor.

coxph()

This function is for fitting Cox's Proportional Hazards Model.

- *coxph(Surv(time, status) ~ x, data = aml)*: standard Cox model;
- *coxph(Surv(t1, t2, stat) ~ (age + surgery)* transplant)*: time dependent covariates;
- *y <- Surv(t1, t2, stat) coxph(y~strata(inst) * sex + age + treat)*: Stratified model, with a separate baseline per institution, and institution-specific effects for sex;
- *coxph(y ~ offset(x1) + x2)*: force in a known term, without estimating a coefficient for it.

cox.zph()

This function is for computing a test of proportional hazards for the fitted Cox model.

- *zfit <- cox.zph(coxfit); plot(zfit).*

Survreg()

This function is for fitting parametric survival models.

- *survreg(Surv(time, stat) ~x, dist = "loglogistic")*: Fit a log-logistic distribution.

Survfit()

This function is for fitting a Survival Curve.

- *survfit(Surv(time, status))*: Simple Kaplan–Meier;
- *survfit(Surv(time, status) ~ rx + sex)*: Four groups;
- *fit <- coxph(Surv(time, stat) ~ rx + sex); survfit(fit, list(rx = 1, sex = 2))*: Predict the curve.

survdiff()

This function is for testing the difference between survival curves. This is one and a k-sample versions of the Fleming-Harrington G family. It includes the log-rank and Gehan-Wilcoxon as special cases.

- *survdiff(Surv(time, status) ~ sex + treat)*: Compare the 4 sub-groups formed by sex and treatment combinations.
- *survdiff(Surv(time, status) ~ offset(pred)):* One-sample test.

6.3. Survival Curves and Models

6.3.1. Survival Model and Curve for Overall Data

Before investigating how different treatments or patient characteristics (like age or smoking status) affect survival, a researcher must first understand the "global" behavior (survival) of the entire cohort.

The purpose is to estimate the "Survival Function"—the probability that a patient survives (remains event-free) past a specific time point. This analysis is to establish a baseline survival expectation for the patient population or the disease progression. This answers the question: 'What is the probability that a patient in this cohort survives at least X days?' For instance, in an oncology trial, knowing the

overall median survival of the study population is the primary step in determining the severity of the disease in that specific group.

The codes below fit a basic survival curve. In which *Surv(time, event)* creates a special survival object, *survfit()* calculates survival probabilities at every event time while handling censored observations, and ~ 1 specifies the overall survival curve (i.e., no stratification by covariates). We use the lung dataset from the survival package, which contains data on patients with advanced lung cancer.

```
fit1 <- survfit(Surv(time, status) ~ 1, data = lung)
print(fit1)
# May view summary results with: summary(fit1)
```

```
## Call: survfit(formula = Surv(time, status) ~ 1, data = lung)
##
##         n      events    median    0.95LCL    0.95UCL
## [1,]    228    165       310       285        363
```

The output shows the following:

- n = 228: This is the total number of subjects who entered the study.
- events = 165: Out of 228 patients, 165 reached the "endpoint" (death). The remaining 63 patients were censored. This is a critical distinction; the model uses the information from the 63 censored patients (who were alive at their last check-up) to calculate probabilities, even though we do not know their final date of death.
- Median survival time (310 days) and 95% CI: the time at which survival probability drops to 0.50. It is the point in time (310 days) at which the survival probability is exactly 50%. In clinical terms, 50% of the patients in this cohort are expected to survive longer than 310 days.
- 0.95LCL/0.95UCL: These are the Lower and Upper Confidence Limits. We are 95% confident that the true median survival of the population lies between 285 and 363 days.

What if one calculates median survival time as below? Is it right or wrong, and why?

```
lung %>% filter(status == 1)%>% summarise(med_surv =
   median(time))
```

```
##      med_surv
## 1    284
```

This is wrong as censoring is ignored. The correct estimate is the median time in the output from the *survfit()* object:

```
print(fit1)
```

```
## Call: survfit(formula = Surv(time, status) ~ 1, data = lung)
##
##         n      events    median    0.95LCL    0.95UCL
## [1,]    228    165       310       285        363
```

To calculate survival at a specific time point, for example, survival at 1 year, should one do as follows?

```
evy1<-lung %>%
  filter(time<= 365 & status == 1) %>%
  summarise(
    sum(as.numeric(as.character(status))))
    1-evy1/nrow(lung)
```

```
##       sum(as.numeric(as.character(status)))
## 1    0.8157895
```

The above is wrong, as censoring is ignored. The correct estimate for 1-year survival is as follows when censoring is taken into account. The function *summary(survfit object)* displays survival probabilities at each event time (or at one specific selected time point) and median survival estimates.

```
summary(fit1, times = 365)
```

```
## Call: survfit(formula = Surv(time, status) ~ 1, data = lung)
##
##   time   n.risk   n.event   survival   std.err   lower 95% CI   upper 95% CI
##   365    65       121       0.409      0.0358    0.345          0.486
```

The output shows that, at 1 year, the estimated survival probability was 40.9% (95% CI 34.5–48.6%). This indicates that fewer than half of patients survived beyond one year in this cohort.

- Time = 365 days: The survival probability is evaluated at 1 year after study entry.
- n.risk = 65: There were 65 patients still at risk (i.e., alive and not censored just prior to day 365).
- n.event = 121: A total of 121 deaths occurred before or at 365 days. This is cumulative from time 0 up to day 365.
- Survival = 0.409: The estimated probability of surviving at least 365 days is 40.9%. In practical terms, approximately 41% of patients are expected to be alive at 1 year.
- Standard Error = 0.0358: This quantifies uncertainty around the survival estimate.
- 0.95LCL/0.95UCL: We are 95% confident that the true 1-year survival probability lies between 34.5% and 48.6%.

Kaplan–Meier Estimator

The Kaplan–Meier (KM) estimator is a non-parametric statistic used to estimate the survival function from time-to-event data. The y-axis of the KM plot shows the probability of surviving (or being event-free) over time (x-axis), demonstrating the decrease in survival probability over the follow-up period. KM plot transforms the statistical table into a visual narrative that clinicians can easily interpret.

One can create a Kaplan–Meier plot using base R (Figure 74):

```
plot(fit1, mark.time  = TRUE,
       xlab = "Days",
       ylab = "Overall event-free probability")
```

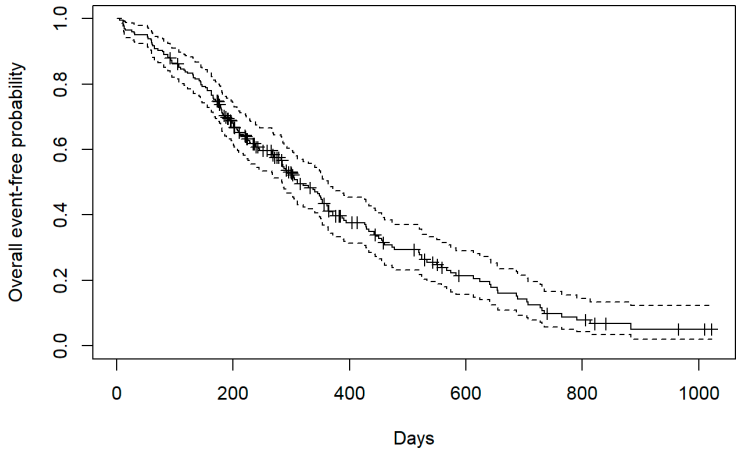


Figure 74. Kaplan–Meier plot using base R. Source: Figure by author.

Kaplan–Meier plots may be better visualized using the "*survminer*" package and its *ggsurvplot()* function. There are many options that can be used to adjust the display of the Kaplan–Meier plot (see *help* (*ggsurvplot*, *survminer)*). For example, the option *risk.table* = *TRUE* adds the number of subjects at risk over time below the main plot, or *surv.median.line* = *"hv"* to add the line that helps identify the median survival time.

```
ggsurvplot(
    fit = fit1,
    risk.table        = TRUE,
    risk.table.height = 0.3,
    surv.median.line = "hv",
    xlab = "Days",
    ylab = "Overall event-free probability")
```

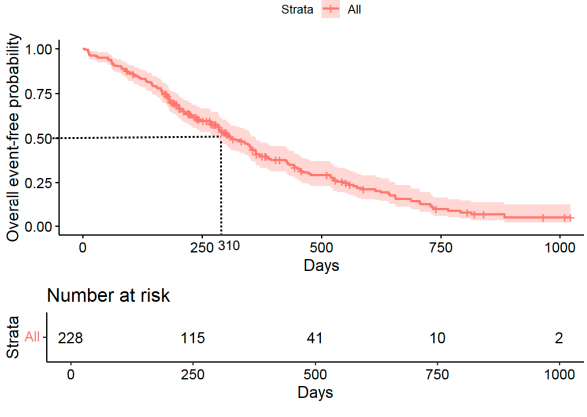


Figure 75. Kaplan–Meier plot using "*survminer*" package. Source: Figure by author.

The outputs show (Figure 75):

- The Curve: The Y-axis represents the probability of being alive. At Day 0, the probability is 1.0 (100%). As time progresses, the curve "steps" down every time a death is recorded.

- Tick Marks: Small vertical ticks often appear on the line (if enabled). These represent censored patients. A tick mark indicates a patient "dropped out" of the curve but did not experience the event.
- Shaded Area (Confidence Interval): The shading around the line represents the 95% confidence interval. Note that the shading usually gets wider as time increases. This is because fewer patients are still alive (at risk), making our estimates less precise toward the end of the study.
- Median Survival Time: The *surv.median.line* identifies the point where 50% of the population has died. In this dataset, the median survival is approximately 310 days.
- Risk table (number at risk): summary of the number of subjects who are still under observation and "at risk" of experiencing the event (e.g., death) at specific time points throughout the study.
- Interpretation: A clinician can tell a patient that, based on this cohort, the average (median) life expectancy is about 10 months, but with significant variability (indicated by the shaded confidence interval).

6.3.2. Comparing Survival Times Between Groups

This analysis is to determine if a specific characteristic (e.g., sex and a new drug) significantly changes the survival outcome. While Section 6.3.1 established a baseline for the whole group, this section moves into comparative analysis. The purpose is to provide researchers with the tools to validate clinical hypotheses—such as whether a new drug extends life compared to a placebo or if certain demographic factors (e.g., sex) or subgroups are associated with higher risk.

We will compare the survival outcomes of male and female patients in the lung dataset to see if one group has a statistically significant survival advantage. By changing the above formula from *~ 1* to *~ sex*, *survfit()* calculates separate KM estimates for each level of the sex variable (male = 1, female = 2).

```
fit2 <- survfit(Surv(time, status) ~ sex, data = lung)
print(fit2)
# May view summary results with: summary(fit2)
```

```
## Call: survfit(formula = Surv(time, status) ~ sex, data = lung)
##
##             n  events  median  0.95LCL  0.95UCL
## sex = 1   138  112     270     212      310
## sex = 1    90  53      426     348      550
```

The output shows that the estimated median survival time is 270 days for males (*sex = 1*) and 426 days for females (*sex = 2*), suggesting a large difference in survival time in favor of females.

Testing the Difference

The most common way to compare survival times between groups is the log-rank test (or Mantel–Haenszel test), which equally weights observations over the entire follow-up time (rho = 0 for the log-rank test (default), rho = 1 for the Peto–Wilcoxon test).

The log-rank test is equivalent to the Score test from a Cox model with the group as a factor variable.

```
survdiff(Surv(time, status) ~ sex, data = lung)
```

```
## Call:
## survdiff(formula = Surv(time, status) ~ sex, data = lung)
##
##             N  Observed  Expected  (O-E)^2/E  (O-E)^2/V
## sex = 1   138       112      91.6       4.55       10.3
## sex = 2    90        53      73.4       5.68       10.3
##
## Chisq = 10.3 on 1 degrees of freedom, p = 0.001
```

```
survdiff(Surv(time, status) ~ sex, rho = 1, data = lung)
```

```
## Call:
## survdiff(formula = Surv(time, status) ~ sex, data = lung, Rho = 1)
##             N  Observed  Expected  (O-E)^2/E  (O-E)^2/V
## sex = 1   138      70.4      55.6       3.95       12.7
## sex = 2    90      28.7      43.5       5.04       12.7
##
## Chisq = 12.7 on 1 degrees of freedom, p = 0.0004
```

The output shows that the p-values from the log-rank test ($p = 0.001$) and the Peto–Wilcoxon test ($p = 0.0004$) are both well below the commonly used significance level of $\alpha = 0.05$. If we had pre-specified a more stringent significance level, such as $\alpha = 0.01$, these results would still provide sufficient evidence to reject the null hypothesis. Based on these results, we conclude that there is a statistically significant difference in the survival distributions between males and females in this cohort.

Note that the output also shows Observed (N)—the actual number of deaths that occurred in each group, and Expected (E)—the number of deaths that would be expected in each group if the null hypothesis (no difference between groups) were true.

The Kaplan–Meier curve between groups and the p-value comparing between groups may be visualized using the "*survminer*" package as below. The plot shows where exactly the survival curves diverge (e.g., do they separate early or only after several months?):

```
ggsurvplot(
    fit = fit2,
    conf.int =        TRUE,
    pval = TRUE,
    risk.table        = TRUE,
    risk.table.height = 0.3,
    xlab = "Days",
    ylab = "Overall event-free probability")
```

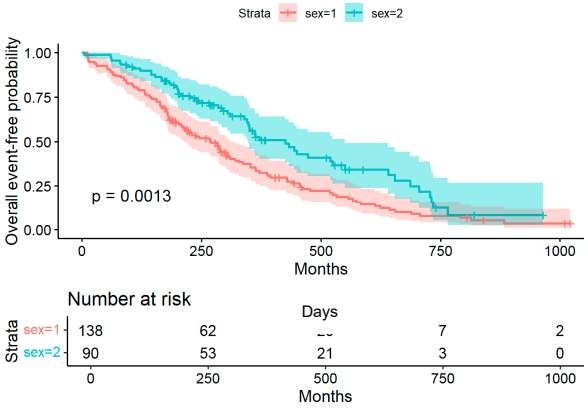


Figure 76. Kaplan–Meier curve between groups with "*survminer*". Source: Figure by author.

The output shows the following (Figure 76):

- *p*-value: The plot displays a *p*-value ($p = 0.0013$), meaning that the difference between men and women is statistically significant.
- Clinical Conclusion: Looking at the curves, the female line (blue) stays consistently above the male line (yellow). This indicates that females have a significantly better prognosis than males in this lung cancer cohort.
- Note that, if the curves cross, it suggests that the "advantage" of one group changes over time. This is a vital observation because it may violate the "Proportional Hazards" assumption required for more advanced models like the Cox model (discussed in 6.3.3). For the risk table, at later time points, the survival estimate for the group with five people becomes much less reliable, which will be reflected by a wider confidence interval (the shaded area).

The clinical question ("*Do men and women survive lung cancer differently?*") has been transformed into a quantifiable statistical result. By combining the log-rank *p*-value with the visual evidence of the Kaplan–Meier plot, the researcher can report whether gender is a significant prognostic factor in this population.

The Cox Proportional Hazards Model

While the log-rank test tells us if there is a difference in survival between groups, it does not quantify the magnitude of that difference.

While KM curves look at one factor at a time, the Cox proportional hazards model can examine the effect of multiple variables simultaneously. The Cox proportional hazards model is a semi-parametric model that is used to investigate the relationship between survival outcome and one or more covariates. The Cox Proportional Hazards (PH) model provides a Hazard Ratio (HR)—a precise numerical measure of how much a specific factor increases or decreases the "hazard" (risk of the event). Unlike the log-rank test, the Cox model is highly flexible; it can handle continuous variables (like age) and allows for multivariable analysis where one can control for confounding factors.

Some key assumptions of the Cox model are non-informative censoring and proportional hazards (the effect of the covariates on the hazard is constant over

time) [55]. It makes no assumptions about the "baseline hazard" (the risk over time for a subject with all predictors at zero) but assumes that the effect of a predictor is constant throughout the study period. This is known as the Proportional Hazards Assumption.

The Cox model is expressed as $h(t \mid X) = h_0(t) \times \exp(\beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p)$, where:

- $h(t \mid X)$ is the hazard rate at time t for an individual with covariates X.
- $h_0(t)$ is the baseline hazard function, which is the hazard when all covariates are zero.
- $exp(\beta_p)$ is the hazard ratio (HR) associated with the covariate X_p [46].

The Cox model is fitted with *coxph()*, and summarized with *summary(Cox model object)*, which reports coefficients (β), HR = exp(β), standard error, z-statistic, and *p*-value, etc.

In this example, we use the *coxph()* function from the "*survival*" package to determine exactly how much the risk of death differs between males and females in the lung cancer dataset.

```
cfit1 <- coxph(Surv(time, status) ~ sex, data = lung)
print(cfit1, digits = 3)
```

```
## Call:
## coxph(formula = Surv(time, status) ~ sex, data = lung)
##
##          coef      exp(coef)   se(coef)   z        p
## sex      -0.531    0.588       0.167      -3.18    0.0015
##
## Likelihood ratio test = 10.6 on 1 df, p = 0.00111
## n = 228, number of events = 165
```

```
# print routine gives a short summary and
# summary gives a longer one
summary(cfit1, digits = 3)
```

```
## Call:
## coxph(formula = Surv(time, status) ~ sex, data = lung)
##
## n = 228, number of events = 165
##
##       coef      exp(coef)  se(coef)   z           Pr(>|z|)
## sex  -0.5310 0.5880    0.1672    -3.176    0.00149 **
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##       exp(coef) exp(-coef) lower .95 upper .95
## sex  0.588     1.701       0.4237    0.816
##
## Concordance = 0.579 (se = 0.021 )
## Likelihood ratio test = 10.63 on 1 df, p = 0.001
## Wald test = 10.09 on 1 df, p = 0.001
## Score (logrank) test = 10.33 on 1 df, p = 0.001
```

The output shows the following:

- coef (−0.5310): This is the log-hazard ratio (β). A negative coefficient indicates that as the variable increases (from 1 = male to 2 = female), the risk of death decreases.
- exp(coef) (0.5880): This is the Hazard Ratio (HR). Since the HR is 0.588, being female is associated with a 41.2% reduction in the risk of death (calculated as 1 − 0.588 = 0.412) compared to being male.
- exp(−coef) (1.701): This is the reciprocal HR. It indicates that males (the reference group) have a 1.7 times higher risk of death than females.
- se(coef) (0.1672): The standard error of the coefficient, which measures the precision of our estimate.
- z (−3.176) and Pr(>|z|) (0.00149): These are the Wald test statistics. The *p*-value (0.00149) is far below 0.05, meaning the effect of sex on survival is statistically significant.
- 95% CI (0.4237, 0.816): The 95% confidence interval for the HR. Because this entire range is below 1.0 (it does not cross 1), it provides further evidence that females have a significantly lower hazard.
- Likelihood ratio/Wald/Score tests: These "Global" tests at the bottom of the output assess the model as a whole. In a univariable model, they usually align. Here, all three show a *p*-value < 0.002, confirming that the model significantly improves our understanding of survival over a model with no predictors.

A likelihood ratio test for the significance of a variable (e.g., "sex") may be done as below. First, fit the Cox model without "sex" *cfit0*, and then fit the model with "sex" *cfit1*, then use the function *anova(cift1, cfit0)* or simply *anova(cift1)* to compare the two models, *cfit1* and *cfit0*.

```
#likelihood ratio test
anova(cfit1)
```

```
## Analysis of Deviance Table
## Cox model: response is Surv(time, status)
## Terms added sequentially (first to last)
##
##             loglik          Chisq       Df     Pr(>|Chi|)
## NULL       -749.91
## sex        -744.59       10.634        1       0.001111 **
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
cfit0 <- coxph(Surv(time, status) ~ 1, data = lung)
anova(cfit1, cfit0)
```

```
## Analysis of Deviance Table
## Cox model: response is Surv(time, status)
## Model 1: ~ sex
## Model 2: ~ 1
##           loglik          Chisq       Df     Pr(>|Chi|)
## 1        -744.59
## 2        -749.91       10.634        1       0.001111 **
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

6.3.3. Hazard Ratios

- Hazard ratio (HR) represents the ratio of hazards between two groups at any particular point in time.
- Interpreted as the instantaneous rate of occurrence of the event of interest in those who are still at risk for the event. It is not a risk, though it is commonly interpreted as such. $HR = exp(\beta)$ (exponentiation of coefficients of Cox model output).
- A HR < 1 indicates a reduced hazard of the event, whereas an HR > 1 indicates an increased hazard of the event.

The output shows that HR is 0.588, which implies that the ratio of the hazard of event (death) of sex = 2 (female) over the hazard of event (death) of sex = 1 (male) is 0.588.

6.3.4. Multivariable Cox Model

The multivariable Cox model is used to quantify the relative risk (Hazard Ratio) of multiple predictors simultaneously. In the example below, it answers the question: "How much increase in the risk of death between females and males, while holding age constant?"

A multivariable Cox model may be done by adding covariates to the Cox model, for example, "sex" and "age".

```
# covariates
cfit2 <- coxph(Surv(time, status) ~ sex + age, data = lung)
print(cfit2, digits = 3)
```

```
## Call:
## coxph(formula = Surv(time, status) ~ sex + age, data = lung)
##
##          coef          exp(coef)     se(coef)     z         p
## sex      -0.51322      0.59857       0.16746      -3.06     0.0022
## age      0.01705       1.01719       0.00922      1.85      0.0646
##
## Likelihood ratio test = 14.1 on 2 df, p = 0.000857
## n = 228, number of events = 165
```

```
summary(cfit2, digits = 3)
```

```
## Call:
## coxph(formula = Surv(time, status) ~ sex + age, data = lung)
##
## n = 228, number of events = 165
##
##          coef          exp(coef)     se(coef)     z             Pr(>|z|)
## sex      -0.513219     0.598566      0.167458     -3.065        0.00218 **
## age      0.017045      1.017191      0.009223     1.848         0.06459 .
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##          exp(coef)     exp(-coef)    lower .95    upper .95
## sex      0.5986        1.6707        0.4311       0.8311
## age      1.0172        0.9831        0.9990       1.0357
##
## Concordance = 0.603 (se = 0.025)
## Likelihood ratio test = 14.12 on 2 df, p = 0.0009
## Wald test = 13.47 on 2 df, p = 0.001
## Score (logrank) test = 13.72 on 2 df, p = 0.001
```

The output shows:

- Hazard Ratio (exp(coef)): This is the most critical value.
  - *sex* (HR ≈ 0.60): Since the value is less than 1.0, the "hazard" (risk of death) for females is 40% lower than for males, even when adjusting for age. This is statistically significant ($p = 0.00218$).
  - age (HR ≈ 1.02): For every one-unit increase in age, the risk of death increases by 2%. This is not statistically significant ($p = 0.06459$).
- Concordance Index: A score of 0.603 suggests that the model correctly predicts who will die first in about 60.3% of random patient pairs.

6.3.5. Stratified Cox Model

In a standard Cox Proportional Hazards (PH) model, we assume that the ratio of hazards between groups remains constant over time for all variables. However, in complex bio-medical data—especially multi-site clinical trials or consortium studies—certain variables may violate this assumption. The stratified Cox model allows certain "nuisance" variables to have their own unique baseline hazard functions without requiring them to satisfy the proportional hazards assumption. It produces more accurate estimates for the variables of primary interest (e.g., treatment effect) by "stratifying" the analysis across different groups, such as different hospitals, study sites, or demographic categories that show fundamentally different risk patterns over time.

- Adding one or more strata terms to the model formula.
- Each subject is compared only to subjects within their own stratum for computing the partial likelihood, and then the final results are summed over the strata.
- Variable included as a stratum is adjusted for in the most general way, without having an estimate of its effect.
- Common use of strata: to adjust for the enrolling institution in a multi-center study.

The following example adds *strata()* to the Cox model and compares the coefficients of the Cox models with and without *strata()*.

```
# Get complete dataset with no missing values
lung_complete <- na.omit(lung)
# Example
cfit <- coxph(Surv(time, status) ~ age + sex + wt.loss,
                          data = lung_complete)
# may view results with: summary(cfit)
cfits <- coxph(Surv(time, status) ~ age + sex + wt.loss + strata(inst),
                            data = lung_complete)
# may view results with: summary(cfits)
# compare coefficients between simple and stratified models
round(cbind(simple = coef(cfit), stratified = coef(cfits)), 4)
```

```
##          simple   stratified
## age      0.0174   0.0229
## sex      -0.4491 -0.5346
## wt.loss-0.0012 -0.0034
```

This comparison serves as a diagnostic check. The fact that the coefficients changed notably (e.g., sex moving from -0.44 to -0.53) indicates that stratification was necessary. It confirms that the underlying baseline hazards were different enough across groups that a "one-size-fits-all" simple Cox model would have underestimated the true prognostic power of "age" and "sex".

The stratified Cox model is often used when:

- Violation of Proportional Hazards: If a diagnostic test (like *cox.zph()*, discussed in Section 6.4.1) shows that a specific covariate violates the PH assumption, that variable is a prime candidate for stratification.
- Multicenter Trials: To account for the inherent variability between different hospitals without making them a primary focus of the analysis.

6.4.1. Checking Assumptions for Cox Model

The Cox Proportional Hazards (PH) model relies on specific mathematical assumptions. If these are violated, the resulting Hazard Ratios (HR) may be misleading or incorrect. Cox proportional hazards models should be checked for the following assumptions:

- Proportional hazards: The hazard ratio between groups stays constant over time.
- Linearity: Continuous variables (like age) have a linear relationship with the log-hazard.
- Additivity: predictors act independently.

Checking Proportional Hazards Assumption

The Cox model assumes that the effect of a covariate remains constant over time (Proportional Hazards) and that continuous variables have a linear relationship with the log-hazard. If these are violated, the Hazard Ratios are invalid. The purpose here is diagnostic verification to check if the hazard ratio changes over time. This helps avoid reporting a single "average" HR for a variable that actually behaves differently across the study duration.

Checking Proportional Hazards Assumption may be done using *cox.zph(Cox model fit)* and plot the results.

```
zp1 <- cox.zph(cfit)
zp1
```

```
##             chisq  df  p
## age         0.668  1   0.41
## sex         1.003  1   0.32
## wt.loss     0.351  1   0.55
## GLOBAL      1.868  3   0.60
```

```
plot(zp1[1], resid = FALSE)
```

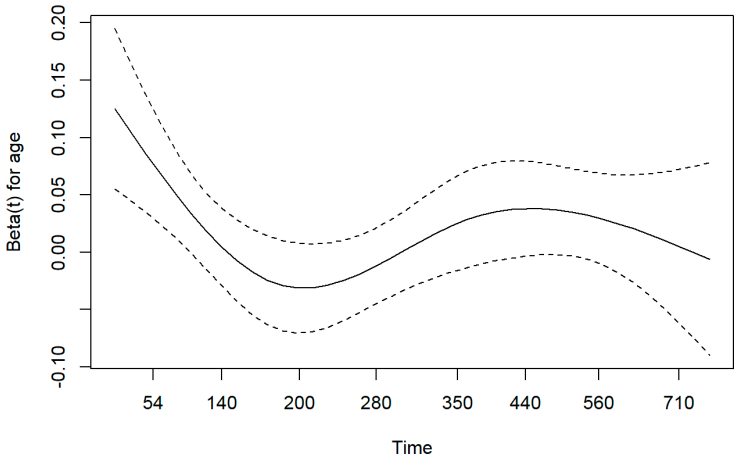


Figure 77. Check Proportional Hazards. Source: Figure by author.

The output shows a test for each covariate and a global test (Figure 77). All *p*-values are not significant; thus, we cannot reject the proportional hazards assumption (the PH assumption is met).

- *p*-values: If a variable has a *p*-value less than 0.05, it violates the proportional hazards assumption. This means the risk factor's effect changes over time.
- GLOBAL *p*-value: This is a summary of the whole model. If the Global *p*-value is >0.05 ($p = 0.60$), it suggests that the model as a whole is stable and the PH assumption is satisfied.
- Graph of the Scaled Schoenfeld Residuals against time: A horizontal line centered around zero indicates the PH assumption holds (if the line has a clear upward or downward slope, the HR is not constant).

The visual assessment provided by the Schoenfeld residual plot is in strong agreement with the statistical results of the *cox.zph()* test.

- Statistical Agreement: The *cox.zph()* test yielded non-significant *p*-values for all individual covariates. This indicates that we fail to reject the null hypothesis of proportional hazards.
- Visual Evidence: As shown in the figure above, the smoothed line for the residuals of "age" (the solid black line) remains relatively flat and horizontal across the entire follow-up period.
- Zero Line Alignment: Because the solid line does not exhibit a significant upward or downward slope and stays centered around the zero line, it confirms visually that the hazard ratio for this variable is constant over time.

Checking Additivity (No Interaction)

When building a multivariable Cox model, we typically assume that the predictors are "additive" and that the relationships in our model are truly independent. This means the effect of one variable (e.g., age) does not change based on the level of another variable (e.g., sex). If an interaction exists, the "Additivity" assumption is violated; in such cases, reporting a single Hazard Ratio for a variable would be misleading because its effect is actually dependent on another

factor. Additivity is often checked by testing for a significant interaction term (e.g., "age*sex"). In this example, we test whether the biological effect of age on lung cancer survival differs between men and women.

```
cfitm<-update(cfit, . ~ . + age*sex)
anova(cfit, cfitm)
```

```
## Analysis of Deviance Table
## Cox model: response is Surv(time, status)
## Model 1: ~ age + sex + wt.loss
## Model 2: ~ age + sex + wt.loss + age:sex
##       loglik        Chisq    Df    Pr(>|Chi|)
## 1    -503.66
## 2    -503.54    0.238     1      0.6257
```

The output shows that the interaction term is not significant; thus, the additivity assumption cannot be rejected. (If additivity is violated, you cannot say "Age increases risk by X%", but instead say "For men, age increases risk by X%, but for women, it increases risk by Y%".)

Checking Linearity

For continuous covariates like "age", the Cox model assumes a linear relationship between the covariate and the log-hazard. Checking is to determine if the "risk" of death increases at a constant rate with every year of age. The purpose is to identify if a variable should be transformed (e.g., using a log scale or a polynomial) to better fit the data.

This is checked using restricted cubic splines (via *pspline()* and the *termplot()* function). In the example below, *pspline(age)* models age non-linearly, and *termplot()* plots the estimated effect of age on the log-hazard.

```
cfitl<-coxph(Surv(time, status) ~ pspline(age) + sex + wt.loss,
                                  lung_complete)
print(cfitl, digit = 3)
```

```
## Call:
## coxph(formula = Surv(time, status) ~ pspline(age) + sex + wt.loss, data = lung_complete)
##
##                                coef          se(coef)      se2          Chisq      DF
## pspline(age), linear           0.01611       0.01061       0.01061      2.30460    1.00
## pspline(age), nonlin                                                    4.92319    3.05
## sex                            -0.49026      0.20007       0.19958      6.00493    1.00
## wt.loss                        -0.00129      0.00678       0.00676      0.03621    1.00
##                                p
## pspline(age), linear           0.129
## pspline(age), nonlin           0.183
## sex                            0.014
## wt.loss                        0.849
##
## Iterations: 4 outer, 12 Newton-Raphson
## Theta = 0.751
## Degrees of freedom for terms = 4 1 1
## Likelihood ratio test = 15.4 on 6.04 df, p = 0.02
## n = 167, number of events = 120
```

```
#Plot effect of age (quite linear)
termplot(cfitl, term = 1, se = TRUE)
```

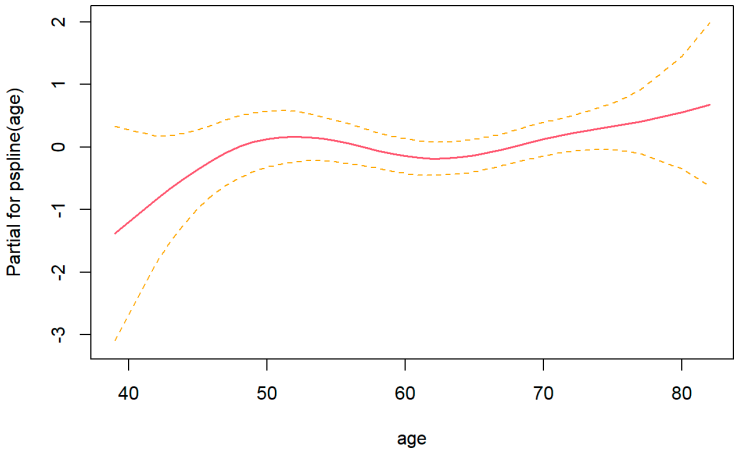


Figure 78. Check linearity. Source: Figure by author.

The output shows that non-linearity ($p = 0.183$) is not significant (Figure 78). The figure shows the estimated effect (solid line) and its 95% confidence bounds (dashed lines). If the solid line is mostly linear and the confidence band is narrow and covers the linear trend, the linearity assumption is met. The plotted line is mostly straight, indicating that the assumption of linearity is satisfied. (If the line shows a distinct "U-shape" or "S-curve," it suggests that the risk does not increase linearly. In such cases, the researcher should consider categorizing the variable.

6.4.2. Evaluating Model Accuracy

A good model should accurately discriminate between individuals with different risks. The most common measure of this discriminatory power for survival models is the Concordance Index (C-index). It is analogous to the Area Under the Curve (AUC) in logistic regression. As a common rule, a C-index of 1.0 indicates perfect prediction, a C-index of 0.5 indicates that the model is no better than random chance, and a C-index > 0.7 is generally considered to indicate a reasonably good model.

The output of the *summary(Cox model)* displays the C-index:

```
lung_complete<-na.omit(lung)
cfit <- coxph(Surv(time, status) ~ age + sex + wt.loss,
                        data = lung_complete)
summary(cfit)
```

```
## Call:
## coxph(formula = Surv(time, status) ~ age + sex + wt.loss, data = lung_complete)
##
## n = 167, number of events = 120
##
##              coef          exp(coef)     se(coef)      z             Pr(>|z|)
## age          0.017400      1.017553      0.010836      1.606         0.1083
## sex          -0.449121     0.638189      0.197983      -2.268        0.0233 *
## wt.loss      -0.001213     0.998788      0.006823      -0.178        0.8589
```

```
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##             exp(coef)  exp(-coef)  lower .95  upper .95
## age         1.0176     0.9828      0.9962     1.0394
## sex         0.6382     1.5669      0.4329     0.9407
## wt.loss     0.9988     1.0012      0.9855     1.0122
##
## Concordance = 0.602 (se = 0.032)
## Likelihood ratio test = 8.91 on 3 df, p = 0.03
## Wald test = 8.5 on 3 df, p = 0.04
## Score (logrank) test = 8.62 on 3 df, p = 0.03
```

The output shows that the C-index is 0.6016187, suggesting the model performs slightly better than random chance. A value above 0.7 is generally considered to indicate a reasonably good model.

6.4.3. Residual Analysis for Goodness-of-Fit

Residuals are used to identify potential outliers and non-linear patterns.

- Martingale Residuals: Used to check the functional form (linearity) of continuous covariates. If plotted against a covariate (e.g., age), a non-random pattern suggests the need for a non-linear term (like a spline or polynomial) or data transformation.
- Deviance Residuals: Used to identify subjects whose observed survival time is unusually long or short compared to the model's prediction (i.e., potential outliers).

We can extract Martingale and Deviance Residuals using *residuals()*:

```
lung_complete$martingale_res <- residuals(cfit, type = "martingale")
lung_complete$deviance_res <- residuals(cfit, type = "deviance")
```

The example below plots Martingale residuals against a continuous predictor (age) to check for non-linear patterns: *residuals(cfit, type = "martingale")* extracts the martingale residuals, and *geom_smooth(method = "loess")* plots a smoothed line of the relationship.

```
ggplot(lung_complete, aes(x = age, y = martingale_res)) + geom_point() +
    geom_smooth(method = "loess") +
    labs(x = "Age", y = "Martingale Residual") + theme_minimal()
```

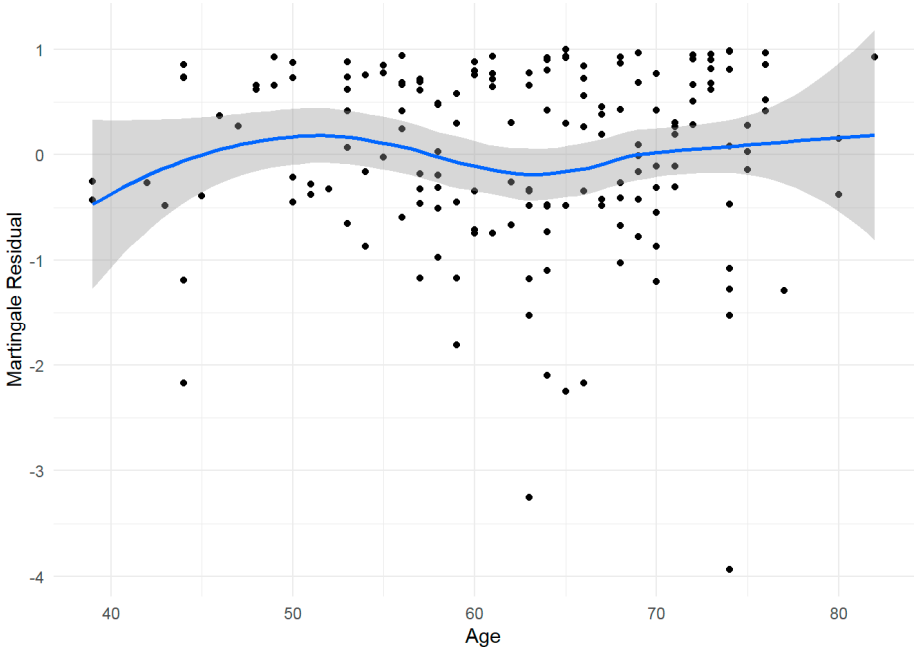


Figure 79. Martingale Residuals vs. age. Source: Figure by author.

The above plot shows the Martingale Residuals vs. "Age" (Figure 79). A horizontal smoothed line (around 0) suggests the functional form for "Age" is correct. If the line bends, it suggests non-linearity.

The example below plots Deviance Residuals to identify potential outliers. We can see a few outliers (in black) (Figure 80):

```
ggplot(lung_complete, aes(x = 1:nrow(lung_complete),
                          y = deviance_res)) +
geom_point(aes(color = ifelse(abs(deviance_res) > 2.5, 1, 2))) +
labs(x= "Index", y = "Deviance Residual") +
theme_minimal() +
theme(legend.position = "none")
```

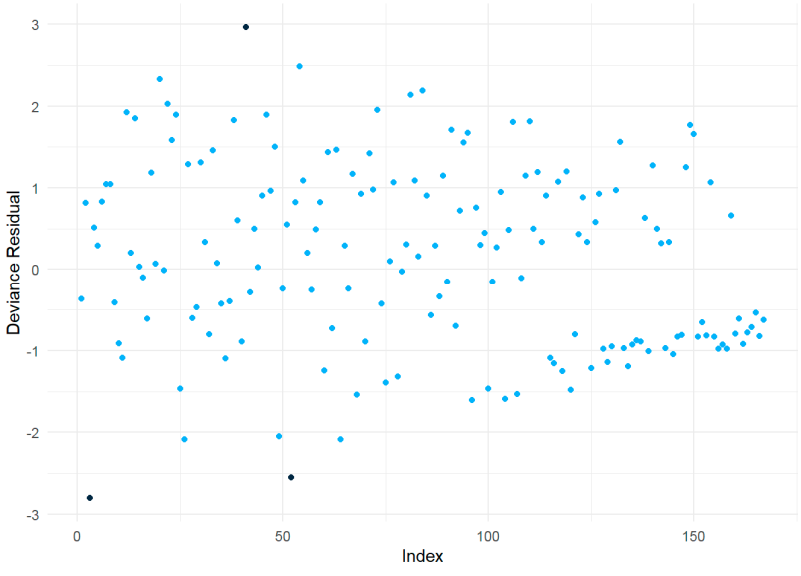


Figure 80. Deviance Residuals. Source: Figure by author.

6.5. Model Selection

6.5.1. Partial Likelihood Hypothesis Tests

We can use three forms of test for β: H0: $\beta = 0$.

- Wald test: commonly used test;
- Score test: equivalent to the log-rank test;
- Likelihood ratio test: key advantage of this test over the other two is that it is invariant to monotonic transformations of β.

The likelihood ratio test is used to compare nested models. Likelihood ratio tests may be done using the *anova()* function:

```
anova(cfit)
```

```
## Analysis of Deviance Table
## Cox model: response is Surv(time, status)
## Terms added sequentially (first to last)
##
##              loglik      Chisq     Df  Pr(>|Chi|)
## NULL         -508.12
## age          -506.36     3.5236    1   0.06050
## sex          -503.68     5.3571    1   0.02064 *
## wt.loss      -503.66     0.0318    1   0.85839
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The *anova()* function shows likelihood ratio tests for each term in a model, added sequentially (compared to the null model *cfit0*). The output shows that the *p*-value for "sex" is significant.

6.5.2. Information Criteria

The Akaike Information Criterion (AIC) with *AIC()* and the Bayesian Information Criterion (BIC) with *BIC()* may be used to compare non-nested models. A lower value indicates a better model.

```
AIC(cfit)
```

```
## [1] 1013.321
```

```
AIC(cfits)
```

```
## [1] 401.7227
```

6.5.3. Model Selection Approaches

With a large number of potential factors, the model needs to be pruned so that only necessary covariates are included.

Stepwise Variable Selection

- Backward Elimination: Starts with a "full" model containing all candidate predictors. In each step, it removes the predictor whose removal results in the largest improvement (lowest value) of the AIC. The process stops when no further removals improve the model.
- Forward Selection: Starts with an "empty" model (intercept only). In each step, it adds the predictor that provides the best improvement in the AIC.
- Stepwise (Bidirectional): A combination of forward and backward steps.

Stepwise selection automatically adds or removes variables based on their statistical contribution to the model. It uses Akaike Information Criterion (AIC) as the decision metric to find a balance between model fit and model complexity. AIC "penalizes" the model for every additional variable added; therefore, the model with the lowest AIC is considered the most efficient. Stepwise model selection may be done using the *step()* function:

```
lung_complete<-na.omit(survival::lung)
fitall2<-coxph(Surv(time, status) ~ age + sex + wt.loss+ inst+ ph.ecog+ ph.karno+
                 meal.cal,
               data = lung_complete)
aicstep2<-step(fitall2, scope = list(upper = ~age +sex + wt.loss+ inst+
                                       ph.ecog+ ph.karno+ meal.cal, lower =
                                     ~age))
```

```
## Start: AIC = 998.33
## Surv(time, status) ~ age + sex + wt.loss + inst + ph.ecog + ph.karno + meal.cal
##
##                          Df     AIC
## - meal.cal               1      996.38
##                          <none> 998.33
## - wt.loss                1      1000.12
## - ph.karno               1      1001.23
## - inst                   1      1002.27
## - sex                    1      1005.05
## - ph.ecog                1      1015.67
##
## Step: AIC = 996.38
## Surv(time, status) ~ age + sex + wt.loss + inst + ph.ecog + ph.karno
##
##                          Df     AIC
## <none>                   996.38
## - wt.loss                1      998.16
## + meal.cal 1             998.33
## - ph.karno 1             999.27
## - inst                   1      1000.29
## - sex                    1      1003.07
## - ph.ecog                1      1013.83
```

```
aicstep2
```

```
##  Call:
##  coxph(formula = Surv(time, status) ~ age + sex + wt.loss + inst +
##  ph.ecog + ph.karno, data = lung_complete)
##
##                coef          exp(coef)     se(coef)     z        p
##  age           0.013784      1.013880      0.011717     1.176    0.23943
##  sex           −0.572214     0.564275      0.199575     −2.867   0.00414
##  wt.loss       −0.014682     0.985425      0.007765     −1.891   0.05866
##  inst          −0.031333     0.969152      0.013087     −2.394   0.01665
##  ph.ecog       0.993069      2.699507      0.229849     4.321    0.0000156
##  ph.karno      0.024618      1.024923      0.011472     2.146    0.03189
##
##  Likelihood ratio test = 31.85 on 6 df, p = 0.0000174
##  n = 167, number of events = 120
```

The output shows the final selected model after *Call*.

Of note, while automated stepwise selection is a computationally efficient way to simplify a model, it has several notable limitations. It can lead to biased parameter estimates, artificially narrow confidence intervals, and may fail to include clinically important variables that do not meet the strict statistical threshold for entry. Furthermore, the final model is highly sensitive to the order in which variables are added or removed and may capture random noise rather than true biological relationships. Therefore, wherever possible, model selection should be guided by prior clinical knowledge and theoretical frameworks rather than relying solely on automated algorithms.

Penalized Regression (LASSO)

For situations with a very large number of predictors (high-dimensional data), traditional stepwise methods can be unstable. Penalized regression, particularly LASSO (Least Absolute Shrinkage and Selection Operator), is a more modern and robust alternative.

LASSO adds a penalty term to the model-fitting process that is proportional to the absolute value of the coefficients. This penalty forces the coefficients of less important variables to shrink towards, and often exactly to, zero. Therefore, LASSO performs both coefficient estimation and variable selection simultaneously. It is particularly effective at preventing overfitting.

The LASSO approach may be done using the *cv.glmnet()* function of the "*glmnet*" package [44]:

```
library(glmnet)

# glmnet requires a matrix of predictors (x) and a survival object (y)
x_vars <- model.matrix( ~ age + sex + wt.loss+ inst+ ph.ecog+
                                          ph.karno+ meal.cal,
                                    data      =     lung_complete)[,-1]
y_surv <- Surv(lung_complete$time, lung_complete$status)

# Use cross-validation to find the optimal penalty parameter (lambda)
cv_lasso <- cv.glmnet(x_vars, y_surv, family = "cox", alpha = 1)

# The optimal lambda that minimizes cross-validated error
optimal_lambda <- cv_lasso$lambda.min

# Extract the coefficients at the optimal lambda
lasso_coefs <- coef(cv_lasso, s= optimal_lambda)
print(lasso_coefs)
```

```
##    7 x 1 sparse Matrix of class "dgCMatrix"
##
##    age          0.0005103152
##    sex          −0.3236625195
##    wt.loss      −0.0018539813
##    inst         −0.0088237865
##    ph.ecog      0.3846742971
##    ph.karno     .
##    meal.cal     .
```

The output shows that LASSO has eliminated "ph.karno" and "meal.cal" by shrinking their coefficients to exactly zero (.), and selected a model with "age", "sex", "wt.loss", "inst", and "ph.ecog".

6.6. Transform Survival Curves

The purpose of transforming survival curves is to:

- Investigate the shape of the hazard function.
- Assess assumptions required by the Cox proportional hazards model.
- Improve interpretability of survival dynamics across time.

Transformations convert the survival function into alternative scales where theoretical relationships become easier to assess visually.

6.6.1. Cumulative Incidence Function

While the survival curve shows the probability of remaining event-free, clinicians are often interested in the probability that the event has occurred by time t. This is the cumulative incidence. The Cumulative Events Function (Cumulative Incidence Function (CIF)) is a simple transformation of the Kaplan–Meier estimate: $F(t) = 1 - S(t)$, where $F(t)$ is the cumulative probability of the event occurring at or before time t (Figure 81).

```
fit2 <- survfit(Surv(time, status) ~ sex, data = lung)
#plot cumulative event (incidence)
ggsurvplot(fit2, conf.int = TRUE,
           fun = "event", # Key argument for the CIF transformation
           risk.table = TRUE, risk.table.col = "strata")
```

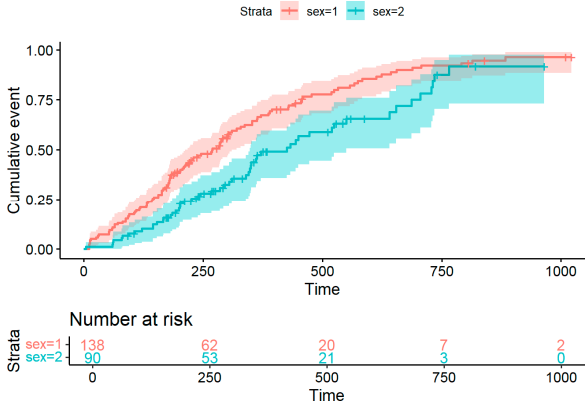


Figure 81. Cumulative Incidence Function. Source: Figure by author.

The curve starts at 0 because no events occur at time zero and increases monotonically over time (steeper increases indicate higher event occurrence).

For example, if the cumulative incidence of sex = 1 reaches 0.50 at 270 days, this implies that approximately 50% of patients have experienced the event (e.g., death) by 270 days. This representation is often easier for clinical interpretation than survival probability.

6.6.2. Cumulative Hazard Function

The Cumulative Hazard Function (CHF) (Nelson–Aalen Estimator) represents the total accumulated risk up to time t. Unlike survival probability, cumulative hazard increases continuously, relates directly to regression modeling, and connects survival analysis with generalized linear modeling. It is calculated by integrating the hazard function (h(t), the instantaneous risk of an event at time t, given survival up to t) from 0 to t (Figure 82).

```
ggsurvplot(fit2, conf.int = TRUE,
           fun = "cumhaz", # Key argument for plotting cumulative hazard
           risk.table = TRUE, risk.table.col = "strata")
```

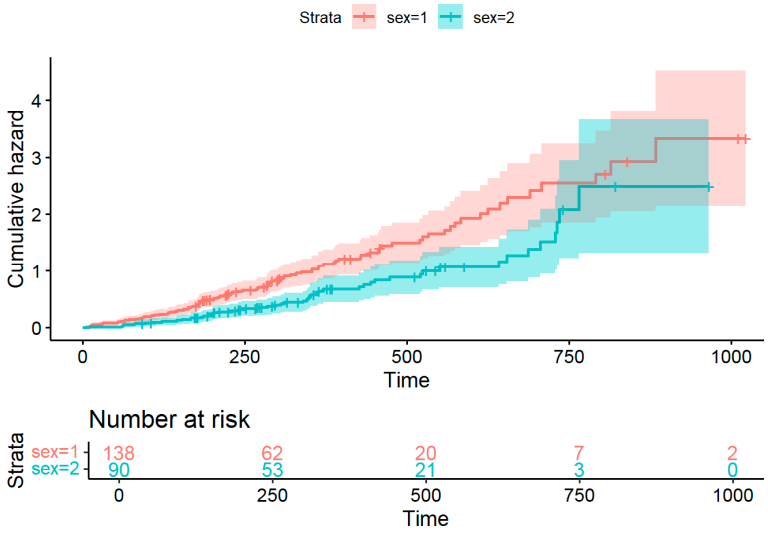


Figure 82. Cumulative hazard function. Source: Figure by author.

The output shows a straight-line pattern, suggesting a constant hazard rate. (Curvature indicates changing hazard over time, e.g., if the curve becomes steeper later, it suggests the risk of death increases as follow-up progresses. If the curve flattens, it suggests hazard decreases over time, possibly reflecting survival of lower-risk individuals.)

6.7. More Advanced Aspects in Survival Analysis

6.7.1. Competing Risk Analysis

In many clinical studies, patients are at risk for more than one type of event, and the occurrence of one event can prevent another from happening. This scenario is known as a competing risk setting. For example, patients with a pre-malignant condition may either progress to full-blown cancer or die from an unrelated cause before progression can occur. In this example, cancer progression is the event of interest, and death due to unrelated causes is a competing event.

The most common mistake in analyzing competing risks data is to treat the competing events as if they were standard censoring. This is incorrect because it violates the core assumption of non-informative censoring. When a patient who died of a heart attack is censored, that patient is no longer at risk of ever progressing to cancer. Treating this as censoring implicitly assumes the patient is still at risk, which leads to a biased and artificially inflated probability of the event of interest. The standard Kaplan–Meier method will overestimate the true incidence of the primary event in the presence of competing risks.

The examples below use the *mgus2* dataset. The question is: what factors (like age, sex, and hemoglobin levels) predict the cumulative incidence of cancer progression, properly accounting for death as a competing risk.

Now, we get the data ready for competing risk analysis:

```
# Load the mgus2 dataset
data(mgus2)

# For simplicity, remove rows with missing hemoglobin
mgus_complete <- mgus2%>% filter(!is.na(hgb))

# The data has time to progression/death ('futime'),
# progression status ('pstat'), and death status ('dstat').
# We need to create a single competing risk status variable:
# 0   =   censored (alive and no progression at last follow-up)
# 1   =   event of interest (progression to cancer)
# 2   =   competing event (death without progression)

mgus_complete <- mgus_complete %>%
  mutate(
    status_cr = case_when(
      pstat == 1 ~ 1, # Progression
      pstat == 0 & death == 1 ~ 2,          # Death without progression
      TRUE ~ 0        # Otherwise, they are censored
    )
  )

# Check the event codes
table(mgus_complete$status_cr)
```

```
##
##   0     1     2
##  402   114   855
```

Cumulative Incidence Function (CIF)

CIF would be a better approach for competing risk, as the CIF for a specific event k is the probability that event k has occurred by time t, accounting for the fact that other competing events could have occurred instead. CIF may be calculated using *cuminc()*, and CIF curves may be visualized using the *ggcompetingrisks()* function (Figure 83):

```
# Calculate the CIF using the cuminc() function
cif_sex  <-  cuminc(ftime = mgus_complete$futime,
                    fstatus = mgus_complete$status_cr,
                    group = mgus_complete$sex,
                    cencode = 0)

# Visualize the CIF curves
ggcompetingrisks(cif_sex,
                 multiple_panels = FALSE,
                 palette = "jco",
                 ggtheme = theme_minimal(),
                 conf.int = TRUE)
```

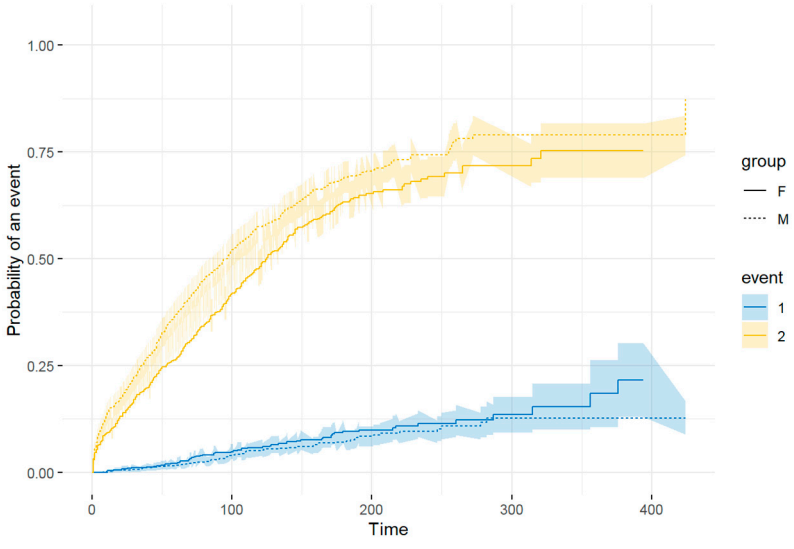


Figure 83. Cumulative incidence of cancer progression and death by sex. Source: Figure by author.

The plot shows a much higher incidence of death than of cancer progression, highlighting the importance of the competing risk (death) in this elderly population.

Regression Modeling (Fine–Gray Model)

For assessing covariate effects, the Fine–Gray Subdistribution Hazard Model is the most common approach for clinical prediction. It directly models the effect of covariates on the cumulative incidence (CIF). The result is the Subdistribution Hazard Ratio (SHR). An SHR > 1 for a covariate means that an increase in that covariate is associated with a higher cumulative incidence of the event of interest [54,56].

The example below uses the Fine–Gray subdistribution hazards model to quantify the effect of "age", "sex", and hemoglobin ("hgb") on the cumulative incidence of cancer progression, accounting for death as a competing event, using the same *mgus* dataset. This may be done using the "*cmprsk*" package [54]:

```
# Prepare the covariate matrix for the crr() function
# The function requires a matrix, not a formula
cov_matrix <- model.matrix(~ age + sex + hgb,
                           data = mgus_complete)[,-1]

# Fit the Fine-Gray subdistribution hazards model
# We specify failcode = 1 because progression is our event of interest
fit_fg <- crr(ftime = mgus_complete$futime,
              fstatus = mgus_complete$status_cr,
              cov1 = cov_matrix,
              failcode = 1,
              cencode = 0)

summary(fit_fg)
```

```
## Competing Risks Regression
##
## Call:
## crr(ftime = mgus_complete$futime, fstatus = mgus_complete$status_cr,
## cov1 = cov_matrix, failcode = 1, cencode = 0)
##
##          coef        exp(coef)     se(coef)     z          p-value
## age      −0.0174     0.983         0.00579      −3.012     0.0026
## sexM     −0.2374     0.789         0.18788      −1.264     0.2100
## hgb      −0.0110     0.989         0.04481      −0.244     0.8100
##
##          exp(coef)   exp(−coef)    2.5%         97.5%
## age      0.983       1.02          0.972        0.994
## sexM     0.789       1.27          0.546        1.140
## hgb      0.989       1.01          0.906        1.080
##
## Num. cases = 1371
## Pseudo Log-likelihood = −767
## Pseudo likelihood ratio test = 7.01 on 3 df
```

The *crr()* function outputs SHRs instead of standard HRs. The output shows that SHR for age is 0.98 with a *p*-value of 0.0026. This means that for every one-year increase in age at diagnosis, the subdistribution hazard of progressing to cancer decreases by about 0.02%. This might seem counterintuitive, but it makes sense in a competing risks framework: older patients have a much higher risk of the competing event (death), leaving them less time and opportunity to progress to cancer.

6.7.2. Time Dependent Covariates

Log-rank tests and Cox regression examine the association between survival outcomes and covariates of interest at baseline (before follow-up time and fixed at the start of the study).

Time-dependent covariates (TDC) are variables whose values for a subject can change at different points during the observation period.

The example below uses the heart transplant *heart* data [57] of the survival package. Patients are enrolled in the study when they are accepted as candidates for a transplant. The primary research question is: Does receiving a heart transplant improve survival for these patients? The key challenge is that "transplant status" is not a baseline characteristic. Every patient enters the study without a transplant. They are at risk of dying while waiting for a donor heart. At some point, a patient may receive a transplant. From that moment forward, their transplant status changes from "no" to "yes," and they continue to be at risk of dying. A simple Cox model like *coxph(Surv(futime, fustat) ~ transplant)* would be flawed and lead to a spurious and overly optimistic estimate of the transplant's benefit because it suffers from immortal time bias. It implicitly assumes that patients in the "transplant" group were somehow immortal from the start of the study until the moment they received their transplant.

Modeling for time-dependent covariates may be done in the framework of the classical Cox proportional hazards model, but important adjustments to the data are required: pre-processing the data into "start-stop" format. The *heart* dataset in R is conveniently pre-processed into this (*start*, *stop*, and *event*) format.

```
fit_tdc <- coxph(Surv(start, stop, event) ~ age + year + transplant,
                 data = heart)

summary(fit_tdc)
```

```
##  Call:
##  coxph(formula = Surv(start, stop, event) ~ age + year + transplant, data = heart)
##
##  n = 172, number of events = 75
##
##                    coef  exp(coef)  se(coef)  z      Pr(>|z|)
##  age            0.02682  1.02718  0.01409  1.903    0.0570 .
##  year          -0.17870  0.83636  0.07044 -2.537    0.0112 *
##  transplant1   -0.03083  0.96964  0.31751 -0.097    0.9226
##  ---
##  Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##                exp(coef)  exp(-coef) lower .95 upper .95
##  age              1.0272    0.9735    0.9992    1.0559
##  year             0.8364    1.1957    0.7285    0.9602
##  transplant1      0.9696    1.0313    0.5204    1.8066
##
##  Concordance = 0.618 (se = 0.034 )
##  Likelihood ratio test = 11.67 on 3 df, p = 0.009
##  Wald test= 11.4 on 3 df, p = 0.01
##  Score (logrank) test = 11.57 on 3 df, p = 0.009
```

Using Surv(start, stop, event) ~ transplant, the Cox model reveals that, at any given point in time, the instantaneous risk of death for a patient who has received a transplant is not significantly different from a patient still waiting ($p = 0.9226$) when assessed as a simple constant variable across the entire follow-up period. The above model assumes the effect of the transplant is constant over time, which may not be true. One may assume that the benefit of a new heart might be different in the first month post-transplant compared to five years later. This may be tested by creating an interaction term between the transplant status and a function (e.g., logarithm) of time.

```
library(broom) # for tidy output display

fit_tt <- coxph(Surv(start, stop, event) ~ age + year + transplant +
                transplant * log(stop),
                data = heart)
# Tidy summary with confidence intervals
tidy_fit <- tidy(fit_tt, exponentiate = TRUE, conf.int = TRUE)

# Rename & select useful columns
tidy_fit <- tidy_fit %>%
  transmute(
    Term = term,
    HR = estimate,
    `95% CI Lower` = conf.low,
    `95% CI Upper` = conf.high,
    `Std. Error` = std.error,
    `z value` = statistic,
    `p value` = p.value
  ) %>%
  mutate(across(where(is.numeric), ~ round(.x, 4))) # Round neatly
```

```
# Print as a nice table
kable(tidy_fit,
      caption = 'Cox model with interaction term')%>%
      kable_styling(latex_options = c("scale_down",  "hold_position"))
```

Table 9. Cox model with interaction term.

Term	HR	95% CI Lower	95% CI Upper	Std. Error	z Value	*p*-Value
age	0.9881	0.9463	1.0317	0.0221	−0.5444	0.5862
year	0.8434	0.6701	1.0616	0.1174	−1.4508	0.1468
transplant1	10.5581	0.3549	314.0688	1.7310	1.3616	0.1733
log(stop)	0.0000	0.0000	0.0000	44.7434	−3.4412	0.0006
transplant1:log(stop)	0.7852	0.3356	1.8372	0.4337	−0.5576	0.5771

Source: Table by author.

The output shows that the effect of "transplant" does not significantly change over the logarithm of time (Table 9).

6.8. Useful Resources

- Tutorial for the essential R package, which is the base of most packages related to survival analysis: survival [51].
- Comprehensive book: Applied Survival Analysis Using R [55].
- "*survminer*": Survival Analysis and Visualization package tutorial [52].

6.9. Chapter Summary

- "*survival*" (for most analysis and visualization), "*survminer*" (for publication-ready visualization), and "*lubridate*" (to work with date data) are the main R packages for survival analysis.
- *survfit(Surv(time, status))* to fit a Kaplan–Meier survival curve, *ggsurvplot()* to visualize a survival curve, and *survdiff()* to test the difference between survival curves.
- *coxph(Surv(time, status) ~...* for the standard Cox model, *coxph(Surv(t1, t2, stat)~...* for time-dependent covariates.
- *cox.zph()* to check proportional hazards.
- *step()* function for stepwise variable selection, *cv.glmnet()* function from the glmnet package for performing LASSO penalized regression.
- "*cmprsk*" is the main package for competing risk analysis, with the main function *crr()* to fit the Fine–Gray subdistribution hazards model.
- *ggcompetingrisks()* to visualize the cumulative Incidence Function (CIF).

7. Longitudinal Data Analysis with R

This chapter provides a practical guide to analyzing longitudinal data. It begins by exploring exploratory visualization techniques, such as spaghetti plots, to understand individual trajectories before introducing the two primary statistical frameworks for longitudinal analysis: Generalized Estimating Equations (GEE) and Generalized Linear Mixed-Effects Models (GLMM). The chapter distinguishes between the population-average focus of GEE models and the subject-specific focus of Mixed-Effects models, estimating fixed effects (population trends) and random effects (individual variations). The readers will gain the skills to implement these models using the key R packages "*geepack*" and "*lme4*". Using a real-world dataset of Bangladeshi infants, the chapter walks through the entire workflow: from reshaping data and selecting appropriate correlation structures to fitting models and interpreting model outputs.

7.1. Longitudinal Data

Longitudinal data are repeated measurements on the same individuals over time. Analyzing longitudinal data requires specialized statistical techniques because the repeated observations within an individual are typically correlated, violating the independence assumption of standard regression models. Failure to account for this within-subject correlation can lead to incorrect standard errors and misleading conclusions. Specialized statistical models (like Generalized Estimation Equation (GEE) or Mixed-Effects) instead of standard linear regression help handle "within-subject correlation".

The main R packages for longitudinal data analysis introduced in this chapter are "*lme4*" for Linear Mixed Effect Models [41], "*geepack*" for Generalized Estimation Equation models [58], and "*ggplot2*" for visualization [18].

We may also explore some supporting packages such as "*mgcv*" [59], "*gee*" [60], "*sjPlot*" [61], "*sjlabelled*" [62], "*sjmisc*" [63], and "*MASS*" [64]. Some other supporting packages ("*rio*" [14], "*tidyverse*" [11], "*knitr*" [5], "*kableExtra*" [12]) are routinely used, as mentioned in previous chapters. First, we load the required packages for longitudinal data analysis.

```
#load multiple packages
packages<-c("rio","tidyverse","knitr", "kableExtra", "lme4","mgcv",
                        "geepack","gee","ggplot2",
                        "sjPlot","sjlabelled","sjmisc","MASS")
lapply(packages, library, character.only = TRUE)
```

7.2. Example: The Bangladesh Data

7.2.1. About Example Data

We will use the same dataset as in the "Practice Review" section of Chapter 2, "Introduction to R". The dataset comes from a cohort study of 50 healthy Bangladeshi infants. This dataset is truly longitudinal, containing 996 stool samples and anthropometric data collected from these 50 infants over time.

Detailed data description may be found elsewhere [13].

We can load the example data from the GitHub repository:

```
# Define the URL for the Excel file
url_base <- "https://github.com/nhanhocu/metamicrobiomeR/raw/master/" data_path <-
"inst/extdata/QIIME_outputs/Bangladesh/nature13421-s2.xlsx"

# Import sheet 2 from the URL
sh2 <- import(paste0(url_base, data_path), sheet = 2, skip = 1) str(sh2)

#Rename column names
oldname<-colnames(sh2)
newname<-c("cohort",
                "family.id",
                "child.id",
                "sample.id",
                "age.d",
                "age.m",
                "whz","waz","haz",
                "breast.milk", "formula",
                "solid", "diarrhea",
                "antibiotic.7d",
                "medication",
                "no.sequence", "serun.id",
                "barcode")
sh2<-sh2%>% rename_at(vars(oldname), ~newname)
#colnames(sh2) #remove
unused rows sh2<-sh2%>%
  filter(!is.na(family.id))
```

7.2.2. Exploratory Data Analysis

Longitudinal Data Visualization

Before modeling, we should visually examine the longitudinal data. The best way to start is with a "spaghetti plot," which draws a separate line for each individual subject. This helps us see both the overall trend and the individual-level trajectory.

Below is an example spaghetti plot for continuous variable weight-for-age z-scores "waz" over time "age.m" using "*ggplot2*". Note that *group = child.id* tells *ggplot()* to group the data by "child.id"; thus, *geom_line()* knows to connect only the dots that belong to the same child, creating the characteristic "spaghetti" effect.

```
sh2%>% filter(!is.na(waz)) %>% ggplot()+
  geom_point(aes(x = age.m, y = waz, group = child.id, colour = cohort))+
  geom_line(aes(x = age.m, y = waz, group = child.id, colour = cohort), size = 0.3)
```

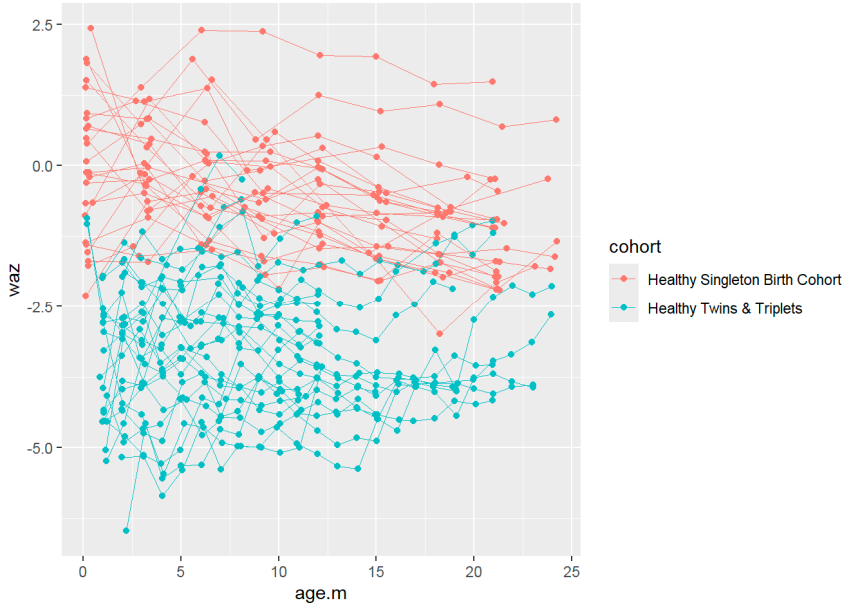


Figure 84. waz spaghetti plot. Source: Figure by author.

The plot above shows the data's structure (Figure 84). We can see individual lines for each child, revealing that some children have consistently higher or lower "waz" scores than others. We also see the correlation: a child's "waz" at one time point is clearly related to their score at the next. The two cohort groups (singletons vs. twins/triplets) appear to have very different trajectories, with the "Healthy Twins & Triplets" (blue) generally having lower "waz" scores than the "Healthy Singleton Birth Cohort" (red).

Plot the Means

To see the average population trend, we can plot the mean "waz" score at each month for each group.

The plot below shows, on average (Figure 85):

- Trend: Weight for age z score ("waz") decreases over time for singletons; no change in twins-triplets;
- Cross-sectional: singletons have larger "waz" than twins-triplets at every age;
- Longitudinal trend comparison: Decrease in average "waz" is larger for singletons than for twins and triplets.

```
d1<-sh2%>%
  filter(!is.na(waz)) %>%
  mutate(age.m.r = round(age.m,0)) %>% group_by(age.m.r, cohort)%>% summarise(mean.waz =
  mean(waz,na.rm = TRUE))
d1%>%
  ggplot()+
  geom_point(aes(x = age.m.r, y = mean.waz, colour = cohort))+ geom_line(aes(x = age.m.r, y =
  mean.waz, colour = cohort), size = 0.3)
```

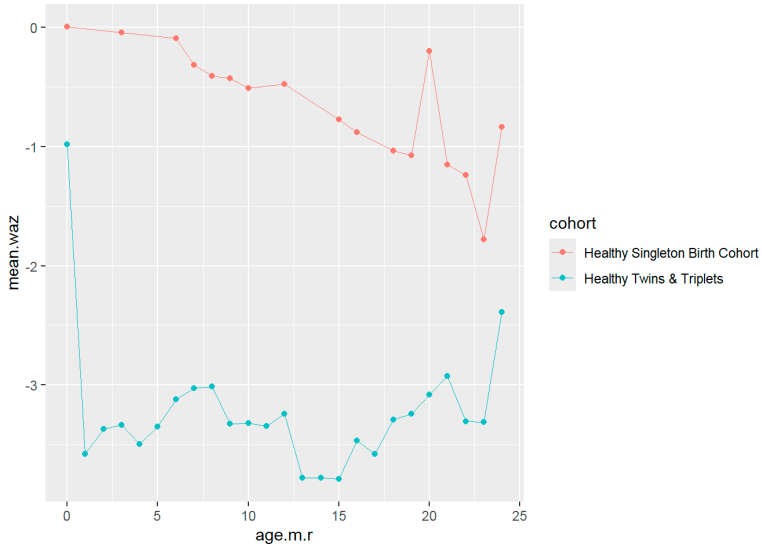


Figure 85. waz mean plot. Source: Figure by author.

7.3. Options for Analysis of Change

Our main research question is: Does the mean change differ across groups?

7.3.1. Pre-Post Analysis

One simple (but flawed) approach is to only use the first and last measurements for each child and see how much their "waz" changed.

- Same baseline (e.g., randomization): analysis of post or change;
- Different baseline: analysis of post or change, adjusting for baseline.

This is generally not a good idea because it ignores all the valuable data from intermediate time points and fails to properly model the within-subject correlation.

7.3.2. Linear Model for Time (the "naïve" Approach)

Let us first try to answer our question using a standard linear regression *lm()* model to demonstrate why it is problematic. We are treating all 573 valid data points as independent observations.

The Mean Model includes:
time = age or "age.m" in this data:

$$E[Y_{ij} \mid age_{ij}] = \beta_0 + \beta_1 age_{ij}$$

The rate of Change is Slope β_1, which is a constant rate of change.
Other models for time may be considered:

- Quadratic;
- Linear spline;
- Cubic spline
- Higher-order polynomials.

Research Question

- What is the rate of change?

  Which is = time slope $E[waz_{ij} \mid x_{ij} = (age.m, cohort)] = \beta_0(x_{ij}) + \beta_1(x_{ij})time_{ij}$

```
fitlm1<-lm(waz~age.m, data = sh2)
summary(fitlm1)
```

```
## Call:
## lm(formula = waz ~ age.m, data = sh2)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
##  −4.134   −1.446   −0.114    1.333    4.758
##
## Coefficients:
##             Estimate  Std. Error  t value  Pr(>|t|)
## (Intercept) −2.323415  0.134893  −17.224  <2×10-16 ***
## age.m       −0.005831  0.011508  −0.507   0.613
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.742 on 571 degrees of freedom
## (423 observations deleted due to missingness)
## Multiple R-squared: 0.0004494, Adjusted R-squared: −0.001301
## F-statistic: 0.2567 on 1 and 571 DF, p-value: 0.6126
```

The output shows that "age.m" is not significant ($p = 0.613$). This model is clearly not good, as it is averaging the decreasing red trend and the stable blue trend of the two cohorts.

So, another question is:

- Does the rate of change differ between singletons vs. twins-triplets? This can be tested with an interaction in the model: $E[Y_{ij}] = \beta_0 + \beta_1 age_{ij} + \beta_2 cohort_i + \beta_3 age_{ij} * cohort_i$

```
fitlm2<-lm(waz~age.m +cohort +age.m*cohort, data = sh2)
summary(fitlm2)
```

```
## Call:
## lm(formula = waz ~ age.m + cohort + age.m * cohort, data = sh2)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
##  −3.1604  −0.7730  −0.1336   0.7111   3.4986
##
## Coefficients:
##                                  Estimate  Std. Error  t value  Pr(>|t|)
## (Intercept)                      0.11060   0.14276     0.775    0.438815
## age.m                            −0.05676  0.01110     −5.113   4.34×10-7 ***
## cohortHealthy Twins & Triplets   −3.41607  0.17838     −19.150  <2×10-16 ***
## age.m:cohortHealthy Twins &      0.05487   0.01469     3.734    0.000207 ***
Triplets
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.096 on 569 degrees of freedom
## (423 observations deleted due to missingness)
## Multiple R-squared: 0.6055, Adjusted R-squared: 0.6034
## F-statistic: 291.1 on 3 and 569 DF, p-value: <2.2×10-16
```

Parameter Interpretation

- $\beta_0 = 0.11060$ = predicted "waz" for the reference group (Singletons) at age.m = 0.
- $\beta_1 = -0.05676$ = expected "waz" change (per month or 1 unit of "age.m") for singletons (reference group).
- $\beta_2 = -3.41607$ = expected difference in "waz" comparing "Healthy Twins & Triplets" to singletons at first visit (newborn).
- $\beta_3 = 0.05487$ = expected difference in "waz" change (per month) between "Healthy Twins & Triplets" and singletons.
- Calculated Slope for "Healthy Twins & Triplets": Slope(Twins) = Slope(Singletons) + Slope Difference = $-0.05676 + 0.05487 = -0.0019$.
- The interaction is highly significant ($p = 0.0002$), confirming this difference in slopes between the two cohorts is real.

A linear regression line may be added to the plot with *stat_smooth(. . ., method = "glm")*. Of note, in the *stat_smooth()* function, *method = "glm"* is utilized to add the regression lines to the plot. While a standard linear model (*method = "lm"*) would produce an identical result for this continuous Gaussian data as introduced in previous chapters, *glm* is used here to remain consistent with the Generalized Linear Model framework introduced in this chapter. By default, *glm* uses the Gaussian family with an identity link, making it functionally equivalent to *lm* in this context, while providing a unified syntax that the reader can later adapt for non-normal outcomes (such as binary or count data) by simply specifying a different family. For example, for binary data, one may specify *stat_smooth(method = "glm", method.args = list(family = "binomial"))* (Figure 86).

```
sh2%>% filter(!is.na(waz)) %>% ggplot()+
    geom_point(aes(x = age.m, y = waz, group = child.id, colour = cohort))+
    stat_smooth(aes(x = age.m, y = waz,
                                colour = cohort), method = 'glm', size = 1)+
geom_point(aes(x = age.m.r, y   = mean.waz,
                               colour = cohort), size = 4, pch = 17, data = d1)+
geom_line(aes(x = age.m.r, y =mean.waz,
                              colour = cohort), size = 1, pch = 17, data = d1)
```

Figure 86. waz linear regression plot. Source: Figure by author.

7.3.3. Issues of Linear Model

The following are not accounted for in the linear model:
Dependence and Correlation

- Response variables measured on the same subject are correlated.
- Observations are dependent or correlated when one variable predicts the value of another variable ("waz" at month 1 may predict "waz" at month 2).
- The correlation is a measure of dependence that takes values between -1 and $+1$.
- A correlation of 0 implies that two measures are unrelated (linearly).
- A correlation of 1 implies that the two measures fall perfectly on a line (one exactly predicts the other!).

Variance

The variance measures the average distance that an observation falls away from the mean.

Covariance

The covariance measures whether, on average, departures in one variable 'go together with' departures in a second variable.

With the above issues of the linear regression model, we need a method that correctly handles longitudinal data.

7.4. Generalized Estimating Equations (GEE)

7.4.1. Introduction to GEE

The GEE model contrasts average outcome values across populations of individuals defined by covariate values, while accounting for correlation. GEE is an extension of generalized linear models (GLMs) and is particularly useful when the primary interest is in the effect of covariates on the average response in the population, rather than on individual-level trajectories [65].

The assumptions of GEE are:

- Observations are independent across subjects.
- Observations may be correlated within subjects: longitudinal correlation structure is a nuisance feature of the data. The GEE model is specified by:
- A mean model: A regression model for the average outcome, e.g., linear or logistic;
- A correlation model: A model for longitudinal correlation, which describes how the repeated measurements within a subject are correlated. Common choices for the working correlation structure include:
    - Independence: correlation is assumed to be zero (observations over time are independent). This is appropriate with the use of a robust variance estimator (large n).
    - Exchangeable: correlation is assumed to be constant (all observations over time have the same correlation). This is more appropriate for clustered data.
    - Auto-regressive: correlation is assumed to depend on time or distance (correlation decreases as a power of how many timepoints apart two observations are). This is more appropriate for equally spaced longitudinal data.

- Unstructured: correlation is assumed to be distinct for each pair (correlation between all timepoints may be different). This is only appropriate for short series (small m) on many subjects (large n).

An advantage of GEE is that the estimates of the regression coefficients are generally consistent even if the working correlation structure is mis-specified, although the efficiency of the estimates is improved with a correctly specified correlation structure.

7.4.2. GEE Example: The Bangladesh Data

Objectives

In this example, we characterize "waz" (continuous outcome) change among singletons and twins-triplets to:

- Estimate the average "waz" change among all children.
- Estimate the "waz" change for individual children.
- Characterize the degree of heterogeneity across children.
- Identify factors that predict "waz" change.

GEE Approach

- Examine "waz" change among twins_triplets and singletons: $E[Y_{ij}] = \beta_0 + \beta_1 age_{ij} + \beta_2 cohorti + \beta_3 age_{ij} * cohort_i$
- Consider various specifications for the 'working' correlation structure. Selection of a working correlation structure should be guided by prior knowledge and/or exploratory analysis. Different Correlation Structures may be tried with the GEE models. In the examples below, we refit the GEE models for different specified correlation structures. We also refit the linear model for comparison.

We will use the *geeglm()* function from the "*geepack*" package [58] to fit GEE models.

- *id*: id of subjects (which observations belong to the same cluster);
- *family = gaussian*: for continuous outcome (*family = binomial* for binary outcome, *family = poisson* with *offset = log(time)* for count outcome);
- *corstr*: for correlation structure.

```
library(geepack)
waz.nona<-sh2%>%
   filter(!is.na(waz))
g.i<-geeglm(waz~age.m +cohort +age.m*cohort, id = child.id,
                    family = gaussian,
                    corstr = "independence",
                    data = waz.nona)
summary(g.i)
```

```
## Call:
## geeglm(formula = waz ~ age.m + cohort + age.m * cohort, family = gaussian,
## data = waz.nona, id = child.id, corstr = "independence")
##
## Coefficients:                          Estimate      Std.err
## (Intercept)                            1.11×10^1     4.46×10^-16
## age.m                                  −5.68×10^2    4.17×10^-17
## cohortHealthy Twins & Triplets         −3.42         7.19×10^-17
```

```
## age.m:cohortHealthy Twins & Triplets          5.49×10^2     3.33×10^-18
##                                               Wald          Pr(>|W|)
## (Intercept)                                   6.14×10^28    <2×10^-16 ***
## age.m                                         1.85×10^30    <2×10^-16 ***
## cohortHealthy Twins & Triplets                2.26×10^33    <2×10^-16 ***
## age.m:cohortHealthy Twins & Triplets          2.72×10^32    <2×10^-16 ***
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Correlation structure = independence
## Estimated Scale Parameters:
##
##                                               Estimate      Std.err
## (Intercept)                                   1.19          3.98×10^-17
## Number of clusters: 1 Maximum cluster size: 573
```

```
g.e<-geeglm(waz~age.m +cohort +age.m*cohort,
            id = child.id,
            family = gaussian,
            corstr = "exchangeable",
            data = waz.nona)
summary(g.e)
```

```
##
## Call:
## geeglm(formula = waz ~ age.m + cohort + age.m * cohort, family = gaussian,
## data = waz.nona, id = child.id, corstr = "exchangeable")
##
## Coefficients:
##                                               Estimate      Std.err       Wald
## (Intercept)                                   0.1106        NaN           NaN
## age.m                                         −0.0568       NaN           NaN
## cohortHealthy Twins & Triplets                −3.4161       NaN           NaN
## age.m:cohortHealthy Twins & Triplets          0.0549        NaN           NaN
##                                               Pr(>|W|)
## (Intercept)                                   NaN
## age.m                                         NaN
## cohortHealthy Twins & Triplets                NaN
## age.m:cohortHealthy Twins & Triplets          NaN
##
## Correlation structure = exchangeable
## Estimated Scale Parameters:
##
##                                               Estimate      Std.err
## (Intercept)                                   1.19          1.04×10^-16
##                                               Link = identity
##
## Estimated Correlation Parameters:
##                                               Estimate      Std.err
## alpha                                         −0.00175      3.39×10^-19
## Number of clusters: 1 Maximum cluster size: 573
```

```
g.a<-geeglm(waz~age.m +cohort +age.m*cohort,
            id = child.id,
            family = gaussian,
            corstr = "ar1",
            data = waz.nona)
summary(g.a)
```

```
##
## Call:
## geeglm(formula = waz ~ age.m + cohort + age.m * cohort, family = gaussian,
## data = waz.nona, id = child.id, corstr = "ar1")
##
## Coefficients:                                Estimate        Std.err
## (Intercept)                                  −4.99×10-02     1.96×10-6
## age.m                                        −5.45×10-02     1.23×10-9
## cohortHealthy Twins & Triplets               −3.51           2.38×10-6
## age.m:cohortHealthy Twins & Triplets         8.53×10-02      4.48×10-8
##                                              Wald            Pr(>|W|)
## (Intercept)                                  6.47×108        <2×10-16 ***
## age.m                                        1.96×1015       <2×10-16 ***
## cohortHealthy Twins & Triplets               2.17×1012       <2×10-16 ***
## age.m:cohortHealthy Twins & Triplets         3.62×1012       <2×10-16 ***
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Correlation structure = ar1
## Estimated Scale Parameters:
##
##                                              Estimate        Std.err
## (Intercept)                                  1.23            3.08×10-7
## Link = identity
##
## Estimated Correlation Parameters:
##                                              Estimate        Std.err
## alpha                                        0.827           2.13×10-7
## Number of clusters: 1 Maximum cluster size: 573
```

```
# Linear model
# = > SE are larger
fit.ols<-lm(waz~age.m +cohort +age.m*cohort,
                  data = waz.nona)
summary(fit.ols)
```

```
##
## Call:
## lm(formula = waz ~ age.m + cohort + age.m * cohort, data = waz.nona)
##
## Residuals:
##                                              Min         1Q          Median      3Q      Max
##                                              −3.16       −0.773      −0.134      0.711   3.499
##
## Coefficients:
##                                              Estimate    Std. Error  t value
## (Intercept)                                  0.1106      0.1428      0.77
## age.m                                        −0.0568     0.0111      −5.11
## cohortHealthy Twins & Triplets               −3.4161     0.1784      −19.15
## age.m:cohortHealthy Twins & Triplets         0.0549      0.0147      3.73
##                                              Pr(>|t|)
## (Intercept)                                  0.43882
## age.m                                        4.3×10-7 ***
## cohortHealthy Twins & Triplets               <2×10-16 ***
## age.m:cohortHealthy Twins & Triplets         0.00021 ***
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.1 on 569 degrees of freedom
## Multiple R-squared: 0.605, Adjusted R-squared: 0.603
## F-statistic: 291 on 3 and 569 DF, p-value: <2×10-16
```

The outputs show that the standard errors (SE) of GEE models are smaller than those of linear models. The interpretation of the coefficients from GEE models would

be similar to that from a linear model (see Chapter: Linear regression with R, or see the example using a linear regression model above).

The correlation structure is selected based on the model with the smallest Quasi Information Criterion (QIC) using *QIC()*.

```
QIC(g.i)
```

```
##    QIC          QICu         Quasi Lik    CIC           params   QICC
##    6.84×10²     6.92×10²     −3.42×10²    3.03×10⁻²⁹    4.00     6.84×10²
```

```
QIC(g.e)
```

```
##    QIC          QICu         Quasi Lik    CIC           params   QICC
##    6.84×10²     6.92×10²     −3.42×10²    −1.25×10⁻¹⁴   4.00     6.85×10²
```

```
QIC(g.a)
```

```
##    QIC          QICu         Quasi Lik    CIC           params   QICC
##    7.02×10²     7.10×10²     −3.51×10²    8.65×10⁻¹⁰    4.00     7.04×10²
```

The output shows that QIC is smallest in the models with correlation structure either "independence" or "exchangeable".

7.4.3. GEE Example: Other Types of Outcomes

Our previous example focuses on continuous growth data (Gaussian). However, many clinical outcomes are binary (e.g., recovery vs. no recovery) or counts (e.g., number of seizures). Using GEE for these outcomes—rather than standard Logistic or Poisson regression—can account for the "clustering" of data within subjects. If we ignore that a patient's health status at Visit 2 is correlated with their status at Visit 1, our *p*-values will be inaccurate, leading to inaccurate conclusions. GEE allows us to model the population-average probability or rate while providing "robust" standard errors.

Binary Outcome

The example below uses the *respiratory* dataset from the "*geepack*" package [58]. Our research question is: On average, what is the effect of an active treatment A (vs. placebo P) on the likelihood of a good respiratory outcome (1 = good, 0 = poor), accounting for age and visit number? Data is clustered by patient id (*id=id*). We tell *geeglm()* to perform a longitudinal logistic regression with *family = binomial*.

```
library(geepack)

# Load the built-in respiratory dataset
data("respiratory")
str(respiratory)
```

```
## 'data.frame'  : 444 obs. of 8 variables:
## $ center      :int 1 1 1 1 1 1 1 1 1 1 ...
## $ id          :int 1 1 1 1 2 2 2 2 3 3 ...
## $ treat       : Factor w/ 2 levels "A","P": 2 2 2 2 2 2 2 2 1 1 ...
## $ sex         : Factor w/ 2 levels "F","M": 2 2 2 2 2 2 2 2 2 2 ...
```

```
## $ age          : int 46 46 46 46 28 28 28 28 23 23 ...
## $ baseline     : int 0 0 0 0 0 0 0 0 1 1 ...
## $ visit        : int 1 2 3 4 1 2 3 4 1 2 ...
## $ outcome      : int 0 0 0 0 0 0 0 0 1 1 ...
```

```
respiratory$treat<-factor(respiratory$treat, levels = c("P","A"))

# Fit the GEE model with an exchangeable correlation structure
# Exchangeable assumes the correlation between
# any two visits is the same for a child.
gee_binomial <- geeglm(
    outcome ~ age + treat + visit,
    id = id,
    data = respiratory,
    family = binomial,
    corstr = "exchangeable"
)

# Display the summary
summary(gee_binomial)
```

```
##
                    ## Call:
                    ## geeglm(formula = outcome ~ age + treat + visit, family = binomial,
                    ## data = respiratory, id = id, corstr = "exchangeable")
##
## Coefficients:
##                  Estimate          Std.err         Wald        Pr(>|W|)
## (Intercept)      0.3328            0.4630          0.52        0.4722
## age              −0.0121           0.0116          1.08        0.2986
## treatA           0.9804            0.3128          9.83        0.0017 **
## visit            −0.0625           0.0656          0.91        0.3412
## ---
                    ## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
                    ## Correlation structure = exchangeable ## Estimated Scale Parameters:
##
##                  Estimate          Std.err
## (Intercept)      1                 0.0548
## Link = identity
##
                    ## Estimated Correlation Parameters:
##                  Estimate          Std.err
## alpha            0.491             0.0653
                    ## Number of clusters: 111 Maximum cluster size: 4
```

The coefficients are in log-odds scale. We must exponentiate them to get odds ratios.

```
exp(coef(gee_binomial))
```

```
## (Intercept)  age      treatA    visit
## 1.395        0.988    2.665     0.939
```

The output shows that the odds ratio is 2.665, or the odds of having a good respiratory outcome for a child receiving the active treatment (A) are 2.665 times the odds of a child receiving the placebo.

Count Outcome

The example below uses the "*epil*" dataset from the "*MASS*" package [64]. Our research question is: On average, does a new drug ("treat" = "progabide") reduce the rate of seizures compared to a placebo after accounting for baseline seizure count ("base")? We specify a model for count data by *family = poisson*, and a crucial step for modeling rates is *offset = log(period)*. By including the log of the observation period as an offset, the model is no longer predicting the raw count (*y*) but the rate (*y/period*).

```
library(geepack)
library(MASS) # For the epil dataset

# Load and inspect the data
data("epil") str(epil)
```

```
## 'data.frame'    : 236 obs. of 9 variables:
## $ y             : num 5 3 3 3 3 5 3 3 2 4 ...
## $ trt           : Factor w/ 2 levels "placebo", "progabide": 1 1 1 1 1 1 1 1 1 1 ...
## $ base          : int 11 11 11 11 11 11 11 11 6 6 ...
## $ age           : int 31 31 31 31 30 30 30 30 25 25 ...
## $ V4            : int 0 0 0 1 0 0 0 1 0 0 ...
## $ subject       : int 1 1 1 1 2 2 2 2 3 3 ...
## $ period        : int 1 2 3 4 1 2 3 4 1 2 ...
##$ lbase          : num -0.756 -0.756 -0.756 -0.756 -0.756 ...
## $ lage          : num 0.1142 0.1142 0.1142 0.1142 0.0814 ...
```

```
# Fit the Poisson GEE model
# Note the use of offset() to model the rate of seizures, # not just the count.
gee_poisson <- geeglm( y ~ base + trt,
   id = subject,
   data = epil,
   family = poisson,
   corstr = "ar1",
   offset = log(period) # Model rate: seizures per week
)
# Display the summary
summary(gee_poisson)
```

```
##
## Call:
## geeglm(formula = y ~ base + trt, family = poisson, data = epil,
## offset = log(period), id = subject, corstr = "ar1")
##
## Coefficients:
##                 Estimate     Std.err     Wald        Pr(>|W|)
## (Intercept)     0.301614     0.126810    5.657       0.0174 *
## base            0.021796     0.001036    442.883     <2×10-16 ***
## trtprogabide    -0.236570    0.170400    1.927       0.1650
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Correlation structure = ar1
## Estimated Scale Parameters:
##
##                 Estimate     Std.err
## (Intercept)     10.46        2.104
## Link = identity
##
## Estimated Correlation Parameters:
##                 Estimate     Std.err
## alpha           0.353        0.0728
## Number of clusters: 59 Maximum cluster size: 4
```

The coefficients are in log scale. We must exponentiate them to get interpretable Incident Rate Ratios (IRRs).

```
# Exponentiate to get Incident Rate Ratios (IRR)
exp(coef(gee_poisson))
```

```
#     (Intercept)    base    trtprogabide
##    1.352          1.022   0.789
```

The output shows that the IRR is 0.789, or, on average, the seizure rate for patients in the progabide group is 0.789 times the seizure rate for patients in the placebo group. Clinically, this means the treatment reduces the population-average seizure rate by 21%.

7.4.4. GEE Summary Main Characteristics

- Primary focus of the analysis is a marginal mean regression model that corresponds to any GLM.
- Longitudinal correlation is secondary and is treated as a nuisance feature.
- Need 'working' correlation model.
- Semi-parametric: Only the mean and correlation models are specified.
- Hypothesis testing with GEE uses Wald statistics.
- Working correlation model does not need to be correctly specified to obtain a consistent estimator for β or valid standard errors for $\hat{\beta}$, but efficiency gains are possible if the correlation model is correct.

Issues.

- Only one source of correlation is addressed: longitudinal or cluster.
- Any missing data are required to be missing completely at random.
- Issues arise with time-dependent exposures and covariance weighting.

7.5. Generalized Linear Mixed-Effects Models (GLMM)

7.5.1. GLMM Overview

Linear mixed-effects models (LME) contrast outcomes both within and between individuals [66].

LME models include the following components:

- Fixed Effects: These are the conventional regression coefficients and are assumed to be constant, or "fixed," across the entire population. They represent the population-average relationship between a predictor and the outcome. For the Bangladesh data example, we estimate the average "waz" change by a unit increase in age.
- Random Effects: These are effects that are specific to each individual or cluster (e.g., each subject). Random effects capture the heterogeneity across subjects, acknowledging that each individual may have a unique trajectory.

LME models decompose the total variance in the outcome into several components:

- Between-Subject Variance: This is the variability captured by the random effects. It explains why some subjects consistently have higher or lower values than

others (random intercept) or why some subjects respond more strongly to a predictor than others (random slope).

- Within-Subject Variance (Residual Variance): This is the variability of a subject's measurements around their own individual trajectory. It represents measurement error and other unexplained time-specific fluctuations.

Generalized Linear Mixed-Effects Models (GLMM) are a powerful extension of the Linear Mixed-Effects (LME) model. GLMM handles non-normal outcome variables (e.g., binary, count) by using a link function to transform the outcome and an appropriate error distribution (called a "family").

7.5.2. Linear Mixed Effect (LME) Model Example

The Bangladesh Data

We go back to our previous data example to characterize "waz" change among singletons and twins–triplets:

- Estimate the average "waz" change among all children;
- Estimate the "waz" change for individual children;
- Characterize the degree of heterogeneity across children;
- Identify factors that predict "waz" change.

The main R package for the LME model is *"lme4"*. We use the *lmer()* function from the *"lme4"* package for continuous outcomes (LME is a specific type of GLMM) [41].

Linear Mixed Effect (LME) Model with Random Intercept

This model assumes a common slope for everyone in a cohort but allows each child to have their own unique baseline (intercept). The random effect formula is specified by *(1 | child.id)*. It reads: "fit a random intercept (the 1) for each level of *child.id*."

```
library(lme4)
library(lmerTest) #for p-value
waz.nona<-sh2%>%
   filter(!is.na(waz))
r.in<-lmer(waz ~ age.m*cohort+ (1 | child.id), data = waz.nona)
summary(r.in)
```

```
## Linear mixed model fit by REML ['lmerMod']
## Formula: waz ~ age.m * cohort + (1 | child.id)
## Data: waz.nona
##
## REML criterion at convergence: 1344
##
## Scaled residuals:
##                                  Min      1Q      Median   3Q      Max
##                                 -4.437   -0.565   -0.025   0.552   4.973
##
## Random effects:
## Group Name                       Variance  Std.Dev.
## child.id (Intercept)             0.721     0.849
## Residual                         0.464     0.681
## Number of obs: 573, groups: child.id, 50
##
```

```
## Fixed effects:
##                                    Estimate   Std. Error   t value
## (Intercept)                        0.09962    0.19176      0.52
## age.m                              -0.05498   0.00697      -7.89
## cohortHealthy Twins & Triplets     -3.47286   0.26529      -13.09
## age.m:cohortHealthy Twins & Triplets 0.06998  0.00969      7.23
##
## Correlation of Fixed Effects:
##                                    (Intr)     age.m        chHT&T
## age.m                              -0.386
## chrtHlthT&T                        -0.723     0.279
## ag.m:chHT&T                        0.277      -0.719       -0.354
```

Note that the package "*lmerTest*" should be loaded to get *p*-values for fixed effects. The output shows random effects and fixed effects:

Random effects: *child.id (Intercept)*: The Variance is 0.721. This is the between-subject variance. It quantifies the variability in the intercepts from child to child. *Residual*: The Variance is 0.464. This is the within-subject variance, or the unexplained "noise" of measurements around each child's own line.

Fixed effects: The coefficients (Intercept), "age.m", etc., are the population-average estimates. They are interpreted just like the *lm()* and GEE models:

- Intercept coefficient: which means that the population-average "waz" of the reference group (singleton) is 0.1 when age.m is zero (the start of the study).
- "age.m" coefficient: which means that the estimated slope (rate of waz change) for one unit increase in age.m of the reference group is -0.055.
- "cohortHealthy Twins & Triplets" coefficient: which means that the estimated difference in waz between the Healthy Twins & Triplets group and the reference (singleton) group at age.m = 0 is -3.473.
- "age.m:cohortHealthy Twins & Triplets" coefficient, which means that the difference in slopes between the Healthy Twins & Triplets group and the reference group is 0.07.

The slope for Healthy Twins & Triplets group may be calculated by = $-0.055 + 0.07 = 0.015$.

LME Model with Random Intercept and Random Slope

This model is more complex. It allows each child to have both their own baseline (intercept) and their own unique rate of change (slope). We specify in the model formula *(age.m | child.id)*: This is shorthand for *(1 + age.m | child.id)*. It reads: "fit a random intercept and a random slope for "age.m", grouping by "child.id" ".

```
r.s<-lmer(waz ~ age.m*cohort+ (age.m | child.id), data = waz.nona)
summary(r.s)
```

```
## Linear mixed model fit by REML ['lmerMod']
## Formula: waz ~ age.m * cohort + (age.m | child.id)
## Data: waz.nona
##
## REML criterion at convergence: 1239
##
## Scaled residuals:
```

```
##                                          Min       1Q        Median    3Q      Max
##                                          −2.998    −0.510    0.007     0.525   4.531
##
## Random effects:
## Groups Name                              Variance  Std.Dev.  Corr
## child.id (Intercept)                     0.9505    0.975
## age.m                                    0.0126    0.112     −0.44
## Residual                                 0.2939    0.542
## Number of obs: 573, groups: child.id, 50
##
## Fixed effects:
##                                          Estimate  Std. Error  t value
## (Intercept)                              0.1027    0.2077      0.49
## age.m                                    −0.0549   0.0232      −2.37
## cohortHealthy Twins & Triplets           −3.6032   0.2913      −12.37
## age.m:cohortHealthy Twins & Triplets     0.0997    0.0332      3.01
##
## Correlation of Fixed Effects:
##                                          (Intr)    age.m       chHT&T
## age.m                                    −0.475
## chrtHlthT&T                              −0.713    0.339
## ag.m:chHT&T                              0.332     −0.699      −0.479
## optimizer (nloptwrap) convergence code: 0 (OK)
## Model failed to converge with max|grad| = 0.00410411
## (tol = 0.002, component 1)
```

The output shows random effects and fixed effects:

Random effects: *child.id (Intercept)*: Variance of the intercepts = 0.9505. *age.m*: Variance of the "age.m" slopes = 0.0126. This tells us that the slopes for "age.m" vary from child to child. There is a "moderate" correlation (Corr: −0.44) between random slopes and intercepts. This means that a subject's baseline "waz" is moderately correlated with the change of "waz" over age (children who started with a higher-than-average "waz" (positive intercept) tended to have a more-negative-than-average slope (a faster decline)). If a strong correlation is observed, it is "over-parameterized" (too many parameters). In such cases, either the intercept or slope is implemented as a random effect.

Fixed effects are interpreted as above.

We can also designate that intercepts and slopes are determined independently.

```
r.isi<-lmer(waz ~ age.m*cohort+ (1 | child.id)+
                  (0+age.m | child.id),
                  data = waz.nona)
summary(r.isi)
```

```
## Linear mixed model fit by REML ['lmerMod']
## Formula:
## waz ~ age.m * cohort + (1 | child.id) + (0 + age.m | child.id)
## Data: waz.nona
##
## REML criterion at convergence: 1248
##
## Scaled residuals:
##                                          Min       1Q        Median    3Q      Max
##                                          −3.369    −0.515    0.002     0.532   4.608
##
## Random effects:
## Groups Name                              Variance  Std.Dev.
## child.id (Intercept)                     0.8735    0.935
## child.id.1 age.m                         0.0105    0.102
```

```
## Residual                                  0.2990      0.547
## Number of obs: 573, groups: child.id, 50
##
## Fixed effects:
##                                           Estimate    Std. Error    t value
## (Intercept)                               0.1019      0.2003        0.51
## age.m                                     -0.0548     0.0213        -2.58
## cohortHealthy Twins & Triplets            -3.5907     0.2807        -12.79
## age.m:cohortHealthy Twins & Triplets      0.0972      0.0305        3.19
##
## Correlation of Fixed Effects:
##                                           (Intr)      age.m         chHT&T
## age.m                                     -0.081
## chrtHlthT&T                               -0.714      0.058
## ag.m:chHT&T                               0.057       -0.698        -0.086
```

Comparing the model with random intercept vs. the model with random intercept and slope: Is the more complex random slope model (*r.s*) significantly better than the simpler random intercept model (*r.in*)? We use a Likelihood Ratio Test (*anova()*).

```
anova(r.in,r.s)
```

```
## Data: waz.nona
## Models:
## r.in: waz ~ age.m * cohort + (1 | child.id)
## r.s: waz ~ age.m * cohort + (age.m | child.id)
##          npar  AIC   BIC   logLik  -2*log(L)  Chisq  Df  Pr(>Chisq)
## r.in     6     1337  1363  -662    1325
## r.s      8     1240  1275  -612    1224       100    2   <2×10-16 ***
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The output shows that the random slope is significant *p*-value (<0.000 . . .), meaning that the model with random intercept and random slope (*r.s*) better fits the data than the *r.in* model (with only a random intercept). This indicates that children not only start at different baselines, but their "waz" also changes at different rates.

Extract Effects

This section demonstrates how to pull specific parameters—namely, Fixed Effects (FE) and Random Effects (RE)—out of the complex model object. By extracting fixed effects, we quantify the average impact of age and cohort across the entire study. By extracting random effects, we "peek" into the unique biological baseline and growth rate of each individual child. This is essential to identify specific subjects who are "outliers" (e.g., children who are failing to catch up).

We can get population-average (fixed) coefficients and subject-specific (random) effects:

```
#fixed effects
fixef(r.s)
```

```
##                                           (Intercept)
##                                           0.1027
##                                           age.m
##                                           -0.0549
## cohortHealthy Twins & Triplets
##                                           -3.6032
```

```
## age.m:cohortHealthy Twins & Triplets
##                                      0.0997
```

```
#random effects
ranef(r.s)
```

```
##     $child.id
##                 (Intercept)  age.m
##     Bgsng7018   0.1813       −0.01865
##     Bgsng7035   −1.4131      0.02076
##     Bgsng7052   −0.1744      0.02352
##     Bgsng7063   0.8686       −0.04558
##     Bgsng7071   1.0960       −0.01241
##     Bgsng7082   −0.7357      0.04352
##     Bgsng7090   0.2326       −0.00878
##     Bgsng7096   −0.3517      −0.02508
##     Bgsng7106   0.0299       −0.02547
##     Bgsng7114   −0.5781      0.03357
##     Bgsng7115   0.5440       −0.01088
##     Bgsng7128   −1.1239      0.03577
##     Bgsng7131   −0.3477      −0.04017
##     Bgsng7142   0.5960       −0.06901
##     Bgsng7149   −0.0745      0.02303
##     Bgsng7150   0.6207       −0.01086
##     Bgsng7155   0.2284       −0.07241
##     Bgsng7173   −0.9953      0.02941
##     Bgsng7177   0.4226       −0.07760
##     Bgsng7178   −1.1764      0.07566
##     Bgsng7192   −0.9135      −0.00481
##     Bgsng7202   0.5255       0.00144
##     Bgsng7204   0.1668       0.01788
##     Bgsng8064   1.1785       0.08485
##     Bgsng8169   1.1933       0.03230
##     Bgtw1.T1    0.3700       −0.04224
##     Bgtw1.T2    0.7327       −0.09384
##     Bgtw10.T1   1.4475       −0.10310
##     Bgtw10.T2   0.2469       0.02256
##     Bgtw11.T1   0.2106       −0.09735
##     Bgtw11.T2   0.6967       −0.06167
##     Bgtw12.T1   0.1434       −0.19251
##     Bgtw12.T2   0.6882       −0.17715
##     Bgtw2.T1    −2.0567      0.02965
##     Bgtw2.T2    −0.9605      −0.02606
##     Bgtw3.T1    1.6332       −0.02464
##     Bgtw3.T2    0.5931       0.02242
##     Bgtw4.T1    −0.0146      −0.03925
##     Bgtw3.T2    0.5931       0.02242
##     Bgtw4.T2    0.3240       −0.07395
##     Bgtw4.T3    0.5405       −0.00948
##     Bgtw5.T1    −2.0004      0.06607
##     Bgtw5.T2    0.2573       −0.10866
##     Bgtw6.T1    −0.4264      0.40051
##     Bgtw6.T2    −1.2643      0.41722
##     Bgtw7.T1    0.2326       −0.11179
##     Bgtw7.T2    −0.6600      −0.06444
##     Bgtw8.T1    1.5170       0.03427
##     Bgtw8.T2    0.8968       −0.00457
##     Bgtw9.T1    −1.3654      0.08765
##     Bgtw9.T2    −1.7823      0.15036
##
## with conditional variances for “child.id”
```

The outputs of *fixef(r.s)* show a table of coefficients. These values are "fixed" because they represent the general law governing the data, similar to the output of a standard linear regression.

- Intercept: The average Weight-for-Age Z-score (WAZ) for the reference group at the start of the study.
- Age Coefficient: The average monthly rate of change in WAZ for the population.
- Cohort Effect: The average difference in baseline WAZ between singletons and twins.

The output of *ranef(r.s)* shows the "subject-specific" part of the model:

- Individual Intercepts (Individual Baseline): You will see a value for each id. If a child has a random intercept of +0.45, it means they started 0.45 units higher than the population average.
- Individual Slopes (Growth Rate): If the model includes a random slope for age, a value of -0.02 means that a specific child is growing slightly slower than the population average.

Understanding these extracted effects allows us to quantify Inter-individual Variability. If the variance of the random intercepts is large compared to the fixed intercept, it tells us that children vary wildly in their starting weights, which might be more clinically important than the average effect of the cohort itself.

One may also do summary of mixed model results as a table using *tab_model()* (Table 10):

```
tab_model(r.s)
```

Table 10. Mixed model summary table.

		waz	
Predictors	**Estimates**	**CI**	***p***
(Intercept)	0.10	−0.31–0.51	0.621
age m	−0.05	−0.10–−0.01	0.018
cohort [Healthy Twins & Triplets]	−3.60	−4.18–−3.03	<0.001
age m × cohort [Healthy Twins & Triplets]	0.10	0.03–0.16	0.003
Random Effects			
σ^2		0.29	
$\tau_{00\ child.id}$		0.95	
$\tau_{11\ child.id.age.m}$		0.01	
$\rho_{01\ child.id}$		−0.44	
ICC		0.85	
$N_{child.id}$		50	
Observations		573	
Marginal R^2/Conditional R^2		0.442/0.918	

Source: Table by author.

Visualization of Results

We can visualize with three types of plots: Coefficients, Marginal Effects, and Model Diagnostics using *plot_model()* (with specified *type*) of the package *"sjPlot"* [61]

in combination with "*ggplot2*". Other packages are also useful for plotting results, such as "*sjlabelled*" [62] and "*sjmisc*" [63].

Coefficients

Estimates: the plot shows the fixed-effects coefficients and their confidence intervals (Figure 87).

```
install.packages("glmmTMB")
library(sjPlot)
library(sjlabelled)
library(sjmisc)
library(ggplot2)
theme_set(theme_sjplot()) plot_model(r.s)
```

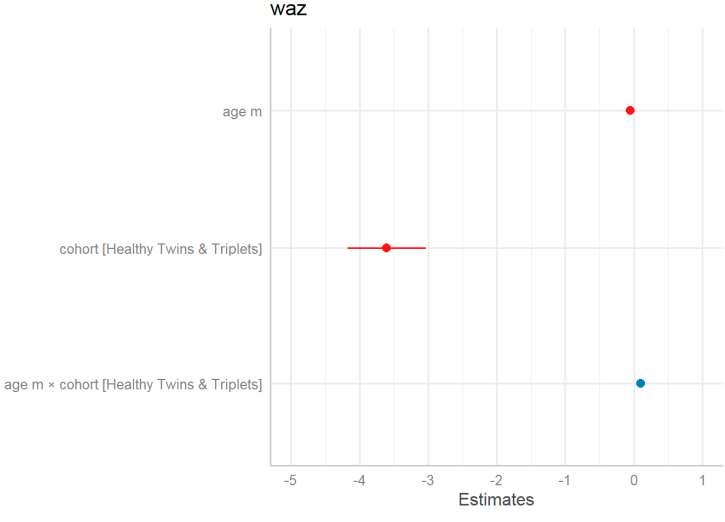


Figure 87. Coefficient estimates for waz. Source: Figure by author.

Random effects (*type* = "*re*"): the (Caterpillar) plot shows the estimated intercept (right panel) and slope (left panel) deviation for every single child in the study. This helps rank and visualize the individual deviations (Random Effects) from the population average and, thus, helps identify "High-Performers" or "At-Risk" individuals. If the error bars do not cross the zero vertical line, that child is statistically "different" from the average (e.g., the 7th and 8th child from the top for "age.m" are significant "High-Performers") (Figure 88).

```
plot_model(r.s, type = "re")
```

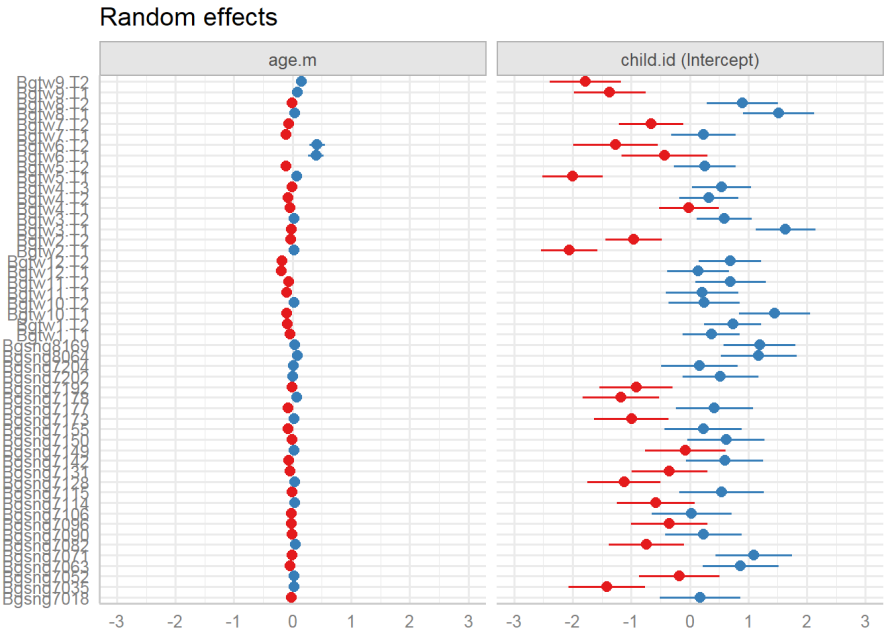


Figure 88. Random effects. Source: Figure by author.

Marginal effects

The plot shows predicted values (marginal effects) for specific model terms (*type= "pred"*) (Figures 89 amd 90):

```
plot_model(r.s, type = "pred", show.data = TRUE)
```

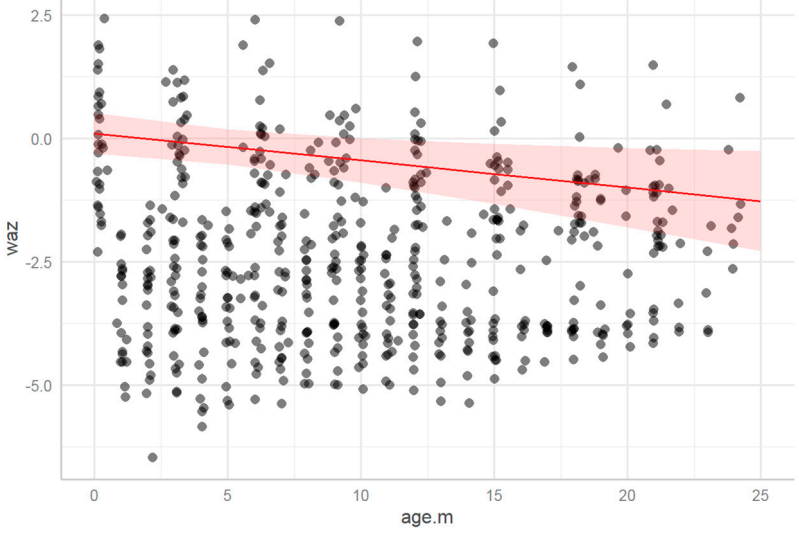


Figure 89. Predicted values "waz" vs. "age.m". Source: Figure by author.

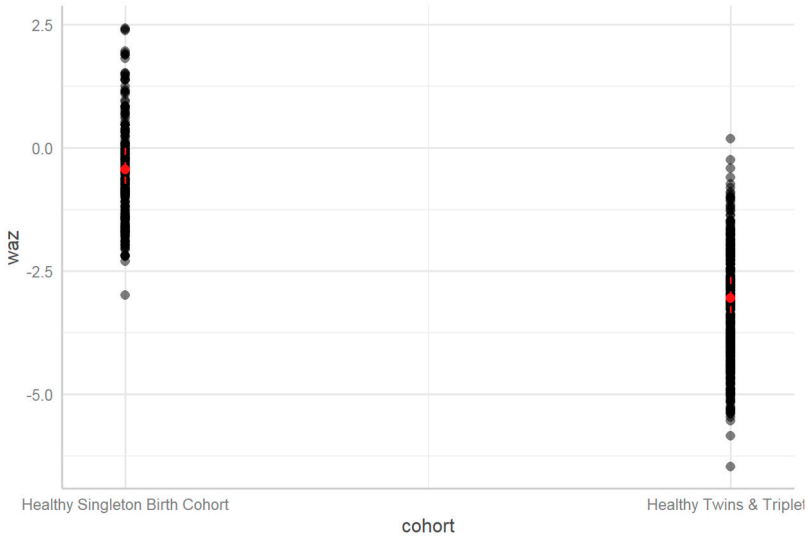


Figure 90. Predicted values of waz vs. cohort. Source: Figure by author.

Marginal effects of interaction terms (*type* = *"int"*): This visualizes the fixed-effect interaction, showing the average predicted slope for the "Singleton" cohort and the average predicted slope for the "Twins & Triplets" cohort, along with the raw data (Figure 91).

```
plot_model(r.s, type = "int", show.data = TRUE)
```

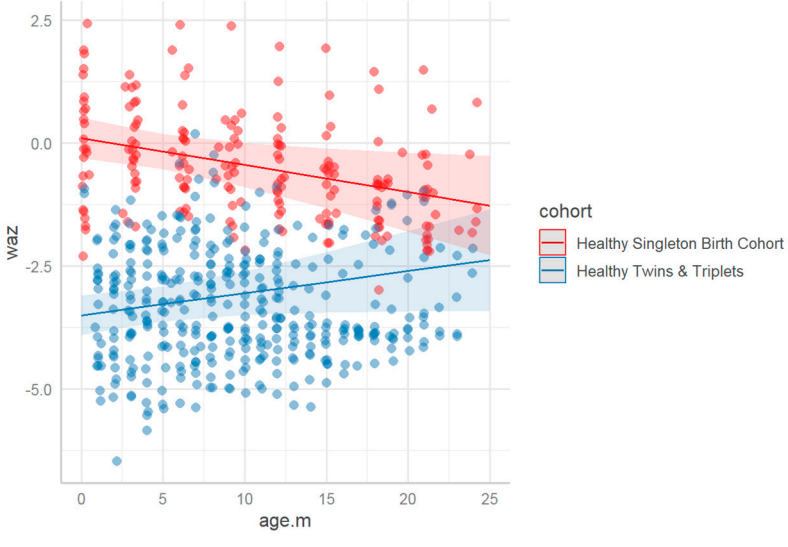


Figure 91. Interaction. Source: Figure by author.

The output shows the population-average trajectory. The singleton line goes down while the twin line is rising, suggesting the "catch-up growth" hypothesis for twins.

Model Diagnostics

Linear Mixed-Effects (LME) model assumes a specific mathematical structure: residuals are normally distributed, variances are constant (homoscedasticity), and random effects follow a normal distribution.

This can be done using the same *plot_model()* function above, with *type = "slope"* for slopes or *type = "resid"* for residuals. The plot below shows the slope of coefficients for every single predictor against the response (linear relationship between each model term and response) (Figure 92).

```
plot_model(r.s, type = "slope")
```

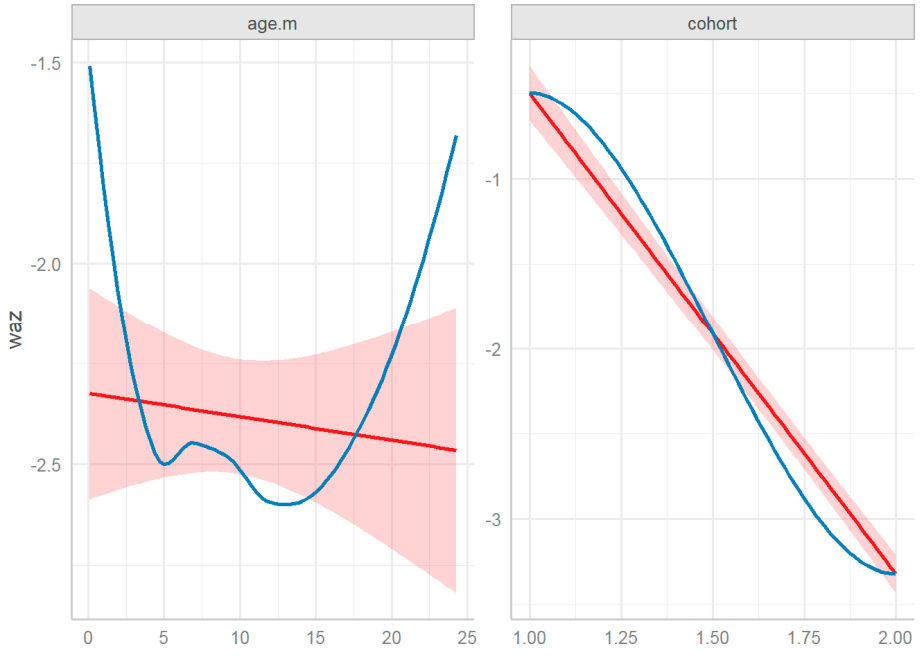


Figure 92. Slopes of coefficients. Source: Figure by author.

The plot below shows the slope of coefficients for every single predictor against the residuals (linear relationship between each model term and residuals) (Figure 93).

```
plot_model(r.s, type = "resid")
```

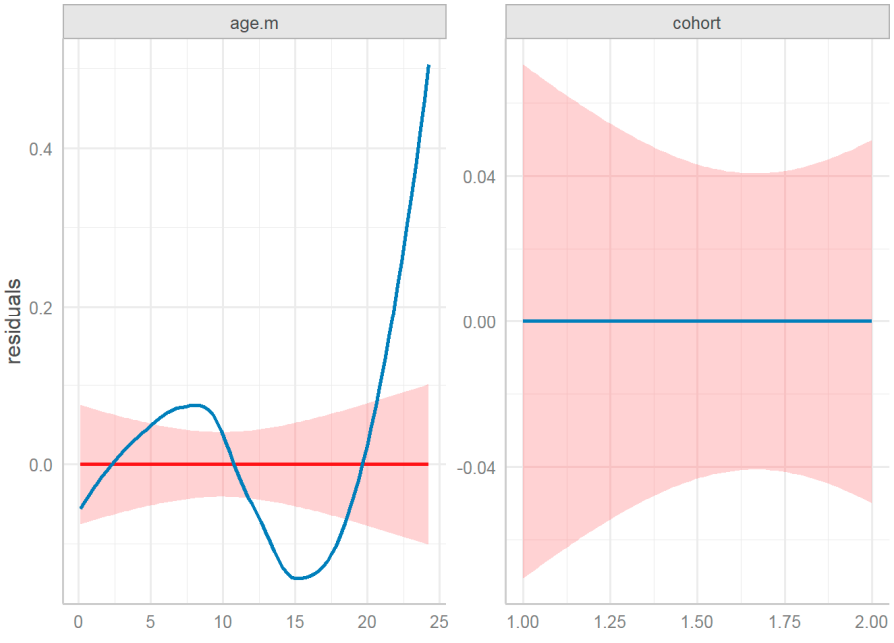


Figure 93. Residuals. Source: Figure by author.

One may also do mixed model diagnostics using the "*performance*" package, which provides a cohesive visual summary of all major LME assumptions in a single output. This can be done together with the "*see*" package. The *check_model()* function provides automated "success/fail" visual cues (e.g., using green and red colors) and handles the complexities of the *lmerMod* object automatically. It calculates the Standardized Residuals, which are necessary in LME because raw residuals can be misleading when observations are clustered within individuals.

```
library(performance)
library(see)
pd<-check_model(r.s)
plot(pd)
```

The output shows (Figure 94):

- Posterior Predictive Check (Model Fit): Overlays the distribution of the "observed" data (WAZ scores) with the distribution of the "simulated" data from the model.
    - If the green lines (model) closely follow the black line (observed data), the model has captured the general shape of the growth distribution. Significant deviations suggest that the model structure (linear) might not be sufficient to capture the real-world complexity of the data.
- Linearity and Homoscedasticity (Residuals vs. Fitted): A scatter plot of the fitted values against the residuals.
    - Linearity: We look for a flat, horizontal reference line. (If the line is curved, the relationship between age and WAZ might be non-linear.)

- Homoscedasticity: The points should be spread evenly. (If they form a "funnel" shape (wider at one end), it indicates that the variance in weight increases as children get older, which violates the constant variance assumption.)

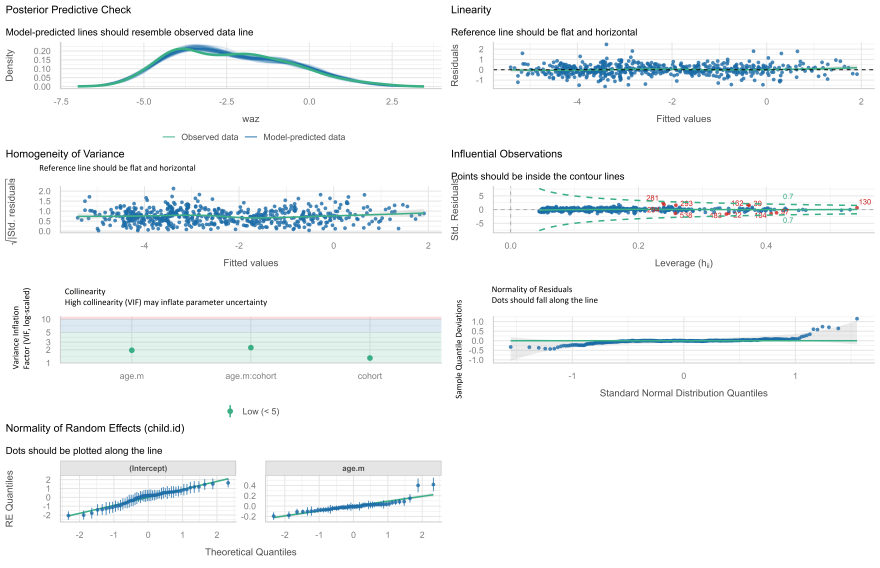


Figure 94. Diagnostic plot for mixed model using the "*performance*" package. Source: Figure by author.

- Normality of Residuals (Q-Q Plot): Plots the quantiles of the model residuals against the quantiles of a theoretical normal distribution. The points should fall along the diagonal 45-degree line.
  - If the points "curve away" at the ends, the model is struggling to predict extreme values (very heavy or very light children).
  - If residuals are not normal, the standard errors for our cohort effects might be untrustworthy.
- Normality of Random Effects (Q-Q Plot for Subjects): Since LME assumes that individual deviations (random intercepts/slopes) follow a normal distribution, this plot checks the subjects (id).
  - This is unique to longitudinal data. If the dots (representing individual children) do not follow the straight line, it suggests that the population contains distinct "sub-groups" that the model has not accounted for or that there are "outlier" children whose growth patterns are fundamentally different from the rest of the cohort.
- Influential Observations (Cook's Distance): Identifies individual data points or subjects that have a disproportionately large impact on the model results.
  - Points outside the dashed lines (contour lines) are "influential. Removing these influential points and re-running the model is a standard "sensitivity analysis".

7.5.3. GLMM Examples

GLMM may be fitted using *glmer()* function with *link* and *family* specification.

Continuous Outcome (Gaussian GLMM)

The below example uses the same Bangladesh data as above. *family = gaussian(link = "identity")* specifies model for continuous outcome.

```
r.s.glmm<-glmer(waz ~ age.m*cohort+ (age.m | child.id),
                data = waz.nona,
                family = gaussian(link = "identity"))
summary(r.s.glmm)
```

```
## Linear mixed model fit by REML ['lmerMod']
## Formula: waz ~ age.m * cohort + (age.m | child.id)
##Data: waz.nona
##
## REML criterion at convergence: 1239
##
## Scaled residuals:
##                                    Min      1Q      Median   3Q      Max
##                                    -2.998   -0.510  0.007    0.525   4.531
##
## Random effects:
## Groups Name                        Variance Std.Dev. Corr
## child.id (Intercept)               0.9505   0.975
## age.m                              0.0126   0.112    -0.44
## Residual                           0.2939   0.542
## Number of obs: 573, groups: child.id, 50
##
## Fixed effects:
##                                    Estimate Std. Error t value
## (Intercept)                        0.1027   0.2077    0.49
## age.m                              -0.0549  0.0232    -2.37
## cohortHealthy Twins & Triplets     -3.6032  0.2913    -12.37
## age.m:cohortHealthy Twins & Triplets 0.0997 0.0332    3.01
##
## Correlation of Fixed Effects:
##                                    (Intr)   age.m     chHT&T
## age.m                              -0.475
## chrtHlthT&T                        -0.713   0.339
## ag.m:chHT&T                        0.332    -0.699    -0.479
## optimizer (nloptwrap) convergence code: 0 (OK)
## Model failed to converge with max|grad| = 0.00410411
## (tol = 0.002, component 1)
```

This is equivalent to the LME model using *lmer()* function above.

Binary Outcome (Binomial GLMM)

The example below uses a simulated dataset.

- Response (a binary 0/1 variable). The 1 indicates a patient has achieved a "clinically significant response" based on a depression scale; 0 indicates they have not.
- Research Question: Does the likelihood of a patient responding to treatment change over time, and more importantly, does this change differ between the new drug and a placebo?

First, we take a look at our generated dataset:

```
## 'data.frame':        400 obs. of 8 variables:
## $ subject_id         : Factor w/ 100 levels "1","2","3","4", ..: 1 1 1 1 2 2 2 2 3 3 ...
## $ group              : Factor w/ 2 levels "Placebo","Drug": 1 1 1 1 1 1 1 1 1 1 ...
## $ rand_intercept     : num -0.841 -0.841 -0.841 -0.841 -0.345 ...
## $ month              : int 0 1 2 3 0 1 2 3 0 1 ...
## $ is_drug            : num 0 0 0 0 0 0 0 0 0 0 ...
## $ log_odds           : num -3.34 -3.04 -2.74 -2.44 -2.85 ...
## $ probability        : num 0.0342 0.0456 0.0606 0.0801 0.0549 ...
## $ response           : int 0 1 0 0 0 0 0 0 0 1 ...
```

Now, we will analyze this data with *glmer()*. *family = binomial(link = "logit")* tells *glmer()* that the outcome is binary and that it should model the "logit" (log-odds) of a successful response.

```
# Fit the Binomial GLMM with a random intercept for subject
# This accounts for patients having different baseline propensities to respond.
glmm_binary <- glmer(response ~ month * group + (1 | subject_id), data =
                       long_df_binary,
                       family = binomial(link = "logit"))

# Display the summary
summary(glmm_binary)
```

```
## Generalized linear mixed model fit by maximum likelihood
## (Laplace Approximation) [glmerMod]
## Family: binomial (logit)
## Formula: response ~ month * group + (1 | subject_id)
## Data: long_df_binary
##
##                                AIC       BIC       logLik    -2*log(L)          df.resid
##                                410       430       -200      400                395
##
## Scaled residuals:
##                                Min       1Q        Median    3Q                 Max
##                                -1.814    -0.454    -0.333    0.551              2764
##
## Random effects:
## Groups Name                    Variance  Std.Dev.
## subject_id (Intercept)         0.7       0.837
## Number of obs: 400, groups: subject_id, 100
##
## Fixed effects:
##                                Estimate  Std. Error  z value   Pr(>|z|)
## (Intercept)                    -2.250    0.410       -5.48     0.000000042 ***
## month                          0.157     0.188       0.84      0.4035
## groupDrug                      0.563     0.508       1.11      0.2686
## month:groupDrug                0.767     0.253       3.03      0.0024 **
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Correlation of Fixed Effects:
##                                (Intr)    month     grpDrg
## month                          -0.750
## groupDrug                      -0.735    0.598
## mnth:grpDrg                    0.475     -0.734    -0.753
```

The output shows coefficients on the log-odds scale, so we exponentiate the coefficients to get Odds Ratios.

```
# Calculate odds ratios
exp(fixef(glmm_binary))
```

```
##(Intercept)monthgroupDrugmonth:groupDrug
##0.105       1.170 1.755       2.153
```

The output shows the following:

- OR for "month" (Placebo slope) is 1.17. For patients in the Placebo group, the odds of having a successful response multiply by 1.17 each month. This is statistically significant ($p = 0.023$).
- OR for "groupDrug" (baseline difference) is 1.755. At Month 0, the odds of response for a patient in the Drug group were 1.755 times the odds for a patient in the Placebo group. This difference was not statistically significant.
- OR for "month:groupDrug" (Interaction) is 2.153. This is the most important result. It is a ratio of odds ratios. It means that the monthly increase in odds for the Drug group is 2.153 times greater than the monthly increase in odds for the Placebo group. This interaction is highly significant.

Calculating the Slope for the Drug Group: The odds of response for the Drug group multiply by 1.17 * 2.153 = 2.519 times each month.

Count Outcome (Poisson GLMM)

This is used for outcomes that are counts of events (e.g., number of lesions, seizures, hospital visits). We will use the dataset *epil* from the "*MASS*" package [64].

- Outcome Variable: y (the number of epileptic seizures).
- Research Question: Does a new drug (treat) reduce the rate of seizures, accounting for baseline seizure count (base), time (visit), and patient-specific proneness to seizures?

family = poisson(. . .) and *offset = log(period)* are used to model the rate of seizures, just as in the GEE example above.

```
# Load necessary packages library(lme4)
library(MASS)

# Load and prepare data
data(epil) str(epil)
```

```
## 'data.frame'    :236 obs. of 9 variables:
## $ y             : num 5 3 3 3 3 5 3 3 2 4 ...
## $ trt           : Factor w/ 2 levels "placebo", "progabide": 1 1 1 1 1 1 1 1 1 1 ...
## $ base          : int 11 11 11 11 11 11 11 11 6 6 ...
## $ age           : int 31 31 31 31 30 30 30 30 25 25 ...
## $ V4            : int 0 0 0 1 0 0 0 1 0 0 ...
## $ subject       : int 1 1 1 1 2 2 2 2 3 3 ...
## $ period        : int 1 2 3 4 1 2 3 4 1 2 ...
## $ lbase         : num −0.756 −0.756 −0.756 −0.756 −0.756 ...
## $ lage          : num 0.1142 0.1142 0.1142 0.1142 0.0814 ...
```

```
# The 'period'  variable is the duration over which seizures were counted.
# It is essential to include this as an offset to model the RATE of seizures.
epil$age_num <- as.numeric(epil$age) # Treat age as a numeric predictor

# Fit the Poisson GLMM
glmm_poisson <- glmer(y ~ base + trt + age+
                        (1 | subject), #random intercept
                      data = epil,
                      family = poisson(link = "log"), #Poisson distribution.
                      offset = log(period)) # offset to model the rate

# Display the summary
summary(glmm_poisson)
```

```
## Generalized linear mixed model fit by maximum likelihood
## (Laplace Approximation) [glmerMod]
## Family: poisson (log)
## Formula: y ~ base + trt + age + (1 | subject)
## Data: epil
## Offset: log(period)
##
##                    AIC          BIC          logLik       −2*log(L)    df.resid
##                    1956         1974         −973         1946         231
##
## Scaled residuals:
##                    Min          1Q           Median       3Q           Max
##                    −5.294       −1.093       −0.066       1.290        13.015
##
## Random effects:
## Groups Name        Variance     Std.Dev.
## subject (Intercept) 0.286       0.534
## Number of obs: 236, groups: subject, 59
##
## Fixed effects:
##                    Estimate     Std. Error   z value      Pr(>|z|)
## (Intercept)        −0.3942      0.4072       −0.97        0.33
## base               0.0272       0.0028       9.73         <0.0001 ***
## trtprogabide       −0.2611      0.1541       −1.69        0.09
## age                0.0143       0.0126       1.14         0.26
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Correlation of Fixed Effects:
##                    (Intr)       base         trtprg
## base               −0.395
## trtprogabid        −0.291       −0.006
## age                −0.937       0.190        0.114
```

The output shows the coefficients on a log scale, so we exponentiate the coefficients to get Incident Rate Ratios (IRRs):

```
# Calculate Incident Rate Ratios
exp(fixef(glmm_poisson))
```

```
##  (Intercept) base   trtprogabide   age
##  0.674       1.028  0.770          1.014
```

The output shows that the IRR for "trtprogabide" is 0.77. The rate of seizures for patients on the new drug ("progabide") is 0.77 of the rate for patients on the placebo. In other words, the treatment is associated with a 0.23 reduction in seizures, though this effect is not statistically significant.

7.6. GEE or Mixed-Effects Model

The two approaches have different targets for inferences and address subtly different questions about longitudinal change.

- GEE: Marginal (population-average): address the correlation among repeated measurements by robust variance estimation.
- Mixed effect: Conditional (subject-specific): provide an explanation for the source of correlation at different levels.

Below are example scenarios for choosing GEE or GLME.
Scenario 1: Public Health Policy

- Question: Does a city-wide smoking ban reduce the overall prevalence of respiratory illness over 5 years?
- Choice: GEE. The interest is in the effect on the entire city's population, not in predicting a specific individual's illness trajectory.

Scenario 2: Child Development Study

- Question: What is the growth trajectory of reading ability in children from ages 6 to 10? Are the starting ability (intercept) and the rate of learning (slope) correlated?
- Choice: Mixed effect. The focus is explicitly on modeling the individual growth curves and understanding the sources of variation between children.

7.7. Useful Resources

7.7.1. Courses

- SISCR Course 2016: "Introduction to Longitudinal Data Analysis" by Benjamin French, PhD (University of Pennsylvania), and Colleen Sitlani, PhD (University of Washington).

7.7.2. Online Tutorials

- Fitting Linear Mixed-Effects Models Using lme4 [41].
- Introduction To Generalized Linear Mixed Models (https://stats.idre.ucla.edu/other/mult-pkg/introduction-to-generalized-linear-mixed-models/, accessed on 1 January 2026).
- Generalized Estimating Equations (GEE) (https://rlbarter.github.io/Practical-Statistics/2017/05/10/generalized-estimating-equations-gee/, accessed on 1 January 2026).
- To GEE or not to GEE [67].

7.7.3. Books, Other Resources

- Applied Longitudinal Data Analysis [65].
- Linear Mixed-Effects Models Using R [66].

7.8. Chapter Summary

- "*lme4*" and "*geepack*" are the main packages for longitudinal data analysis;
- *geeglm()* function to fit GEE model, which focuses on population-average and addresses correlation of repeated measures as nuisance:

- *family = gaussian*: for continuous outcome, *family = binomial* for binary outcome, *family = poisson* with *offset = log(time)* for count outcome;
- *corstr*: for correlation structure (*QIC()* to select the best correlation structure);

- *lmer()* function to fit Linear Mixed Effect (LME) models, which include:
  - Fixed Effects: population-average relationship between a predictor and the outcome;
  - Random effects: heterogeneity across subjects (as random intercepts (*1 | subject_id*) and random slopes (*time | subject_id*));
- "*sjPlot*", "*sjlabelled*", and "*sjmisc*" are packages to help visualize LME model results
- *glmer()* function to fit Generalized Linear Mixed-Effects Models (GLMM) (an extension of the Linear Mixed-Effects (LME) model) that handle various types of outcomes.
  - *family = gaussian(link = "identity"):* for continuous outcomes;
  - *family = binomial(link = "logit"):* for binary outcomes;
  - *family = poisson(link = "log")* with *offset = log(time)*: for count outcomes.

8. Analysis of Large Consortium Data with R

In bio-medical research, especially in epidemiology, it is common for research groups from around the world to pool their data into a large consortium. A consortium combines multiple individual studies to create a single, massive dataset. This approach has several key advantages: (1) increased statistical power: by dramatically increasing the total sample size, consortia can detect small but important risk factors that individual studies would miss; (2) ability to study rare outcomes: for diseases like liver cancer, which are rare in the general population, individual cohorts may only have a small number of cases, pooling data aggregates thousands of cases, making meaningful analysis possible; and (3) diverse populations: consortia often include data from different countries and ethnicities, allowing researchers to see if findings are consistent across diverse populations or if they are unique to a specific group.

Analyzing consortium data, however, is more complex than analyzing a single study. One must account for the fact that the data comes from different sources, which may have different baseline disease rates, follow-up procedures, and population characteristics.

This chapter will guide you through the analyses to generate similar results to those of the original paper "Diabetes is associated with increased liver cancer incidence and mortality in adults: A report from Asia Cohort Consortium" published in *the International Journal of Cancer* by Nhan Thi Ho et al. [68]. Using time-to-event (survival) analysis as the main analysis example, the chapter will provide examples for: (1) pooled analysis: analyzing all data together while using stratification to account for differences between studies; (2) study-specific analysis: running the analysis separately within each individual study; (3) meta-analysis: formally combining the results from the individual studies to get a robust, summary estimate; and (4) sensitivity analysis: checking if the overall result is overly influenced by any single study.

This chapter will provide examples for practical R skills, including writing R functions, and loops to perform repeated analysis more efficiently, and provide R code examples to generate clear, report-ready tables and plots. This chapter focuses on how to do specific analyses and result display using R, not on result output interpretation. One may read the paper mentioned above for more information about the actual outcome and exposure, actual results, detailed explanation, and interpretation [68].

8.1. About Data

The example dataset used in this chapter was generated to mimic the real data used in the above-mentioned paper. The generated example dataset includes data from multiple cohort studies (identified by "studycode") regarding the association between a certain type of cancer ("cancer"), time to event ("ttcancer"), and a main exposure of interest ("main.exposure"), as well as several covariates like age, gender, BMI, etc. Interested readers may read the original paper for more detailed information about the cancer event, "main.exposure", and further information about the original data, results, and interpretation [68].

8.1.1. Load Required Packages and Example Data

The example dataset is hosted in GitHub: *load(url(...))* will load the .rda (R Data) file from the provided GitHub URL into our R environment. The data is stored in an object named "candat". The main R packages used in this chapter include "*lubridate*" [53], "*survival*" [51], "*cmprsk*" [54], "*survminer*" [52], "*gridExtra*" [69], "*Hmisc*" [70], "*psych*" [71], "*gmodels*" [72], "*gtsummary*" [33], "*meta*" [73], "*patchwork*" [74], "*rio*" [14], and "*arsenal*" [19]. Other supporting packages ("*knitr*" [5], "*rmarkdown*" [6], "*kableExtra*" [12], and "*tidyverse*" [11]) are used routinely as in previous chapters. Now, we load the packages and the example data.

```
#load multiple packages
Packages <- c("knitr","rmarkdown","kableExtra","tidyverse", "rio","arsenal",
                "lubridate","survival","cmprsk", "survminer","gridExtra","Hmisc",
                "psych", "gmodels", "gtsummary", "meta", "patchwork")
lapply(Packages, library, character.only = TRUE)
# Data path
url_base <- "https://raw.githubusercontent.com/nhanhocu/"
data_path <- "biodata-r/main/candat.rda"
data_url <- paste0(url_base, data_path)
#load data
con <- url(data_url) load(con)
close(con)
```

8.1.2. Summary of Data

The first step is to examine the data by summarizing the main variables by groups. This is the diagnostic phase of the consortium analysis.

Summary of Data Overall and by Studies

We use the *tableby()* function from the "*arsenal*" package [19] to create a comprehensive descriptive table, showing the characteristics of each study side-by-side with the overall total. This table allows researchers to examine the overall pooled data and identify heterogeneity between cohorts. The tables are presented in a format that is both clinically meaningful and ready for peer-reviewed publication.

First, we define summary options and make a summary table (Table 11):

```
# Define summary options
my_controls<-tableby.control(
    test = FALSE,
    total = TRUE,
    numeric.test = "kwt",
    cat.test = "chisq",
    numeric.stats = c("meansd","medianq1q3","range","Nmiss2"),
    cat.stats = c("countpct","Nmiss2"),
    stats.labels =list(
        meansd = "Mean (SD)",
        medianq1q3 = "Median (Q1, Q3)",
        range = "Min-Max",
        Nmiss2 = "Missing"
    )
)
sumvar<-c("cancer.f", "ttcancer", "main.exposure",
                "age", "age.cat", "gender",
                "bmi", "bmi.cat",
                "smoking", "alcoholuse")
mylabels <-as.list(c("Cancer",
                                "Time-to-cancer (year)",
                                "Exposure of interest",
                                "Age (year)", "Age categories",
                                "Gender",
                                "BMI", "BMI categories",
                                "Smoking", "Alcohol use"))
names(mylabels)<-sumvar
tab1 <- tableby(as.formula(paste("studycode",
                                                paste(sumvar,collapse = "+"),
                                                sep = "~")),
                        data =  candat,
                        control = my_controls)
kable(summary(tab1,
              labelTranslations = mylabels,
              text = TRUE),
              caption = "Summary of data by study and overall")%>%
    kable_styling(latex_options = c("scale_down", "hold_position"))
```

Table 11. Summary of data by study and overall.

	Study1 (N = 33,529)	Study2 (N = 31,552)	Study3 (N = 20,636)	Study4 (N = 42,751)	Study5 (N = 47,605)	Study6 (N = 31,345)	Study7 (N = 51,252)	Study8 (N = 52,883)	Study9 (N = 63,257)	Total (N = 374,810)
Cancer										
- No	33,325 (99.4%)	31,397 (99.5%)	20,332 (98.6%)	39,528 (99.6%)	47,134 (99.0%)	31,229 (99.6%)	50,825 (99.2%)	52,145 (98.6%)	62,736 (99.2%)	368,651 (99.2%)
- Yes	204 (0.6%)	155 (0.5%)	299 (1.4%)	150 (0.4%)	471 (1.0%)	116 (0.4%)	427 (0.8%)	738 (1.4%)	521 (0.8%)	3081 (0.8%)
- Missing	0	0	5	3073	0	0	0	0	0	3078
Time-to-cancer (year)										
- Mean (SD)	11.508 (5.134)	13.507 (4.232)	13.872 (4.670)	9.301 (3.276)	21.435 (6.039)	7.600 (2.606)	10.702 (4.304)	21.889 (10.281)	13.872 (3.839)	14.220 (7.378)
- Median (Q1, Q3)	15.165 (7.485, 15.165)	15.578 (15.140, 15.578)	13.864 (10.891, 17.443)	9.254 (6.606, 12.044)	24.583 (20.931, 24.583)	8.999 (7.556, 8.999)	13.246 (9.251, 13.246)	21.916 (10.916, 32.000)	15.165 (12.591, 16.657)	13.246 (8.999, 16.925)
- Min–Max	0.000–15.502	0.000–15.578	0.027–21.804	0.022–15.403	0.038–24.583	0.016–8.999	0.000–13.246	0.000–38.916	0.008–17.892	0.000–38.916
- Missing	0	0	5	3073	0	0	0	3055	0	6133
Exposure of interest										
- No	22,048 (95.3%)	30,115 (95.4%)	18,224 (95.4%)	39,033 (93.8%)	45,566 (95.7%)	29,558 (94.3%)	47,845 (93.4%)	40,766 (92.8%)	57,561 (91.0%)	330,716 (93.7%)
- Yes	1076 (4.7%)	1437 (4.6%)	885 (4.6%)	2559 (6.2%)	2039 (4.3%)	1787 (5.7%)	3407 (6.6%)	3181 (7.2%)	5696 (9.0%)	22067 (6.3%)
- Missing	10,405	0	1527	1159	0	0	0	8936	0	22,027
Age (year)										
- Mean (SD)	56.424 (11.408)	55.959 (12.889)	54.118 (14.320)	50.009 (9.224)	52.141 (7.538)	57.324 (11.345)	60.511 (10.302)	52.249 (13.684)	56.505 (8.016)	55.064 (11.227)
- Median (Q1, Q3)	54.000 (47.000, 64.000)	54.667 (44.854, 65.087)	56.000 (45.000, 64.000)	50.000 (43.000, 56.000)	53.000 (45.000, 59.000)	56.000 (48.000, 65.000)	62.000 (53.000, 68.000)	51.400 (41.800, 61.800)	56.000 (50.000, 63.000)	55.000 (47.000, 63.000)
- Min–Max	40.000–103.000	35.425–101.051	15.000–91.000	16.000–85.000	40.000–64.000	40.000–98.000	40.000–80.000	19.300–98.700	43.000–83.000	15.000–103.000
- Missing	0	0	5	0	0	0	0	3055	0	3060
Age categories										
- <40 years old	0 (0.0%)	3123 (9.9%)	3188 (15.5%)	5373 (12.6%)	0 (0.0%)	0 (0.0%)	0 (0.0%)	10,122 (20.3%)	0 (0.0%)	21,806 (5.9%)
- ≥40&<60 years old	21,564 (64.3%)	16,781 (53.2%)	8931 (43.3%)	30,091 (70.4%)	37,138 (78.0%)	19,140 (61.1%)	21,097 (41.2%)	25,069 (50.3%)	40,852 (64.6%)	220,663 (59.4%)
- ≥60 years old	11,965 (35.7%)	11,648 (36.9%)	8512 (41.3%)	7287 (17.0%)	10,467 (22.0%)	12,205 (38.9%)	30,155 (58.8%)	14,637 (29.4%)	22,405 (35.4%)	129,281 (34.8%)
- Missing	0	0	5	0	0	0	0	3055	0	3060
Gender										
- Female	17,783 (53.0%)	17,125 (54.3%)	12,399 (60.1%)	21,326 (49.9%)	24,769 (52.0%)	17,353 (55.4%)	26,679 (52.1%)	32,493 (61.4%)	35,303 (55.8%)	205,230 (54.8%)

Table 11. *Cont.*

	Study1 (N = 33,529)	Study2 (N = 31,552)	Study3 (N = 20,636)	Study4 (N = 42,751)	Study5 (N = 47,605)	Study6 (N = 31,345)	Study7 (N = 51,252)	Study8 (N = 52,883)	Study9 (N = 63,257)	Total (N = 374,810)
- Male	15,746 (47.0%)	14,427 (45.7%)	8232 (39.9%)	21,425 (50.1%)	22,836 (48.0%)	13,992 (44.6%)	24,573 (47.9%)	20,390 (38.6%)	27,954 (44.2%)	169,575 (45.2%)
- Missing	0	0	5	0	0	0	0	0	0	5
BMI										
- Mean (SD)	22.101 (2.976)	22.206 (2.896)	23.571 (3.285)	23.859 (2.997)	23.644 (2.984)	23.254 (3.443)	23.532 (3.324)	35.318 (2912.005)	23.116 (3.274)	24.890 (1085.530)
- Median (Q1, Q3)	21.971 (20.047, 23.875)	22.052 (20.269, 23.957)	23.415 (21.259, 25.606)	23.723 (21.778, 25.716)	23.438 (21.514, 25.433)	22.959 (21.077, 25.104)	23.309 (21.403, 25.391)	21.504 (19.628, 23.805)	23.112 (21.094, 24.655)	22.880 (20.830, 24.889)
- Min–Max	10.644–56.960	9.934–57.398	12.955–63.267	11.204–51.554	11.687–54.861	7.171–64.930	8.002–82.305	−3.556–650000.000	9.778–68.966	−3.556–650000.000
- Missing	1306	1845	1620	342	2737	1791	3542	3055	0	16,238
BMI categories										
- <23	20,706 (64.3%)	18,760 (63.2%)	8645 (45.5%)	17,044 (40.2%)	19,763 (44.0%)	14,859 (50.3%)	21,883 (45.9%)	0	30,366 (48.0%)	152,026 (49.2%)
- ≥23	11,517 (35.7%)	10,947 (36.8%)	10,371 (54.5%)	25,365 (59.8%)	25,105 (56.0%)	14,695 (49.7%)	25,827 (54.1%)	0	32,891 (52.0%)	156,718 (50.8%)
- Missing	1306	1845	1620	342	2737	1791	3542	52,883	0	66,066
Smoking										
- No	15,285 (49.3%)	14,927 (50.9%)	12,819 (63.2%)	22,339 (54.3%)	20,139 (50.0%)	12,704 (57.0%)	22,310 (51.4%)	28,408 (56.2%)	43,930 (69.4%)	192,861 (56.5%)
- Yes	15,723 (50.7%)	14,391 (49.1%)	7471 (36.8%)	18,799 (45.7%)	20,103 (50.0%)	9597 (43.0%)	21,097 (48.6%)	22,102 (43.8%)	19,327 (30.6%)	148,610 (43.5%)
- Missing	2521	2234	346	1613	7363	9044	7845	2373	0	33,339
Alcohol use										
- No	10,318 (34.4%)	0	11,754 (58.1%)	13,442 (32.2%)	17,255 (41.1%)	9637 (39.4%)	19,408 (43.3%)	32,189 (60.9%)	51,384 (81.2%)	165,387 (51.8%)
- Yes	19,682 (65.6%)	0	8462 (41.9%)	28,341 (67.8%)	24,767 (58.9%)	14,803 (60.6%)	25,423 (56.7%)	20,694 (39.1%)	11,873 (18.8%)	154,045 (48.2%)
- Missing	3529	31,552	420	968	5583	6905	6421	0	0	55,378

Source: Table by author.

Summary of Data by Main Exposure Status

Next, we look at the pooled data, comparing the characteristics of the "exposed" ("Yes") versus "unexposed" ("No") groups. This table helps examine participant characteristics between the "exposed" and "unexposed" groups that may affect their later outcome and provides baseline information for later multivariable (adjusted) analysis (Table 12).

```
# Define summary options
my_controls<-tableby.control(
    test = FALSE,
    total = FALSE,
    numeric.test = "kwt",
    cat.test = "chisq",
    numeric.stats = c("meansd","medianq1q3","range","Nmiss2"),
    cat.stats = c("countpct","Nmiss2"),
    stats.labels = list(
        meansd = "Mean (SD)",
        medianq1q3    = "Median (Q1, Q3)",
        range = "Min-Max",
        Nmiss2 = "Missing"
    )
)
sumvar<-c("cancer.f", "ttcancer",
                    "age", "age.cat", "gender",
                    "bmi", "bmi.cat",
                    "smoking","alcoholuse")
mylabels <-as.list(c("Cancer",
                                    "Time-to-cancer (year)",
                                    "Age (year)", "Age categories",
                                    "Gender",
                                    "BMI", "BMI categories",
                                    "Smoking", "Alcohol use"))
names(mylabels)<-sumvar
tab1 <- tableby(as.formula(paste("main.exposure",
                                                        paste(sumvar, collapse = "+"),
                                                        sep = "~")),
                                data = candat, control = my_controls)
kable(summary(tab1,
                labelTranslations = mylabels,
                text = TRUE),
                caption = "Summary of data by main exposure")%>%
    kable_styling(latex_options = c("scale_down", "hold_position"))
```

Table 12. Summary of data by main exposure.

	No (N = 330,716)	**Yes (N = 22,067)**
Cancer		
- No	325,309 (99.2%)	21,524 (98.3%)
- Yes	2523 (0.8%)	373 (1.7%)
- Missing	2884	170
Time-to-cancer (year)		
- Mean (SD)	14.229 (7.177)	12.264 (6.831)
- Median (Q1, Q3)	13.246 (8.999, 16.975)	12.493 (7.748, 15.578)
- Min–Max	0.000–38.916	0.000–38.916
- Missing	4667	339

Table 12. *Cont.*

	No (N = 330,716)	**Yes (N = 22,067)**
Age (year)		
- Mean (SD)	54.725 (11.037)	59.338 (9.905)
- Median (Q1, Q3)	54.000 (46.500, 62.000)	60.000 (52.000, 66.000)
- Min–Max	15.000–103.000	15.000–94.200
- Missing	1786	169
Age categories		
- <40 years old	18,969 (5.8%)	422 (1.9%)
- ≥40&<60 years old	200,318 (60.9%)	10,452 (47.7%)
- ≥60 years old	109,643 (33.3%)	11,024 (50.3%)
- Missing	1786	169
Gender		
- Female	179,833 (54.4%)	10,069 (45.6%)
- Male	150,880 (45.6%)	11,998 (54.4%)
- Missing	3	0
BMI		
- Mean (SD)	25.130 (1153.341)	23.857 (3.580)
- Median (Q1, Q3)	22.880 (20.889, 24.889)	23.508 (21.671, 25.778)
- Min–Max	–3.556–650,000.000	10.644–121.528
- Missing	13,070	831
BMI categories		
- <23	137,894 (49.5%)	7134 (39.1%)
- ≥23	140,769 (50.5%)	11,090 (60.9%)
- Missing	52,053	3843
Smoking		
- No	170,992 (56.6%)	10,118 (49.5%)
- Yes	131,083 (43.4%)	10,341 (50.5%)
- Missing	28,641	1608
Alcohol use		
- No	143,094 (51.1%)	105,32 (54.0%)
- Yes	137,004 (48.9%)	8954 (46.0%)
- Missing	50,618	2581

Source: Table by author.

8.2. Analysis Approaches

8.2.1. Estimate Incidence of Cancer by Person Years

We first calculate the crude cancer incidence rate, which is the number of new cancer cases divided by the total "person-years" of follow-up. Incidence of cancer (per 100,000 person_year) overall and by "main.exposure" status (with *group_by()*) for pooled data of all studies is calculated as follows: *person.year = sum(ttcancer, na.rm = TRUE)*: this code calculates total person-years by summing the follow-up time (*ttcancer*) for all participants; *cancer.case* = sum of all cancer cases; *cancer.incidence* = . . . * *100000*. This standardizes the rate to be "per 100,000 person-years", which is a standard epidemiological metric.

```
# Overall
py<- candat %>%
    filter(!is.na(main.exposure)) %>%
    summarise(person.year = sum(ttcancer, na.rm = TRUE),
                          cancer.case = sum(as.numeric(as.character(cancer)),
                                                na.rm = TRUE),
                           cancer.incidence = round((cancer.case/person.year)*100000,2))
py
```

```
##      person.year   cancer.case   cancer.incidence
## 1    4,905,681     2896          59
```

```
# By main exposure
py.be<-candat %>%
    filter(!is.na(main.exposure)) %>%
    group_by(main.exposure)%>%
    summarise(person.year = sum(ttcancer, na.rm = TRUE),
                          cancer.case = sum(as.numeric(as.character(cancer)), na.rm = TRUE),
                          cancer.incidence = round((cancer.case/person.year)*100000,2))
py.be
```

```
##      # A tibble: 2 × 4
##      main.exposure   person.year   cancer.case   cancer.incidence
##      <fct>           <dbl>         <dbl>         <dbl>
## 1    No              4,639,216.    2523          54.4
## 2    Yes             266,465.      373           140.
```

The example below calculates the incidence of cancer (per 100,000 person_year) for each study (with *group_by(studycode)*) and summarizes the results as a table (Table 13):

```
inc.study<-candat %>%
    filter(!is.na(main.exposure)) %>%
    group_by(studycode)%>%
    summarise(py.study = sum(ttcancer, na.rm = TRUE),
                          cancer = sum(as.numeric(as.character(cancer)), na.rm = TRUE),
                          cancer.inc = round((cancer/py.study)*100000,2))
kable(inc.study, caption = "Incidence by study")%>%
    kable_styling(latex_options = c("scale_down", "hold_position"))
```

Table 13. Incidence by study.

studycode	py.study	cancer	cancer.inc
study1	269,191	122	45.3
study2	426,187	155	36.4
study3	267,830	281	104.9
study4	356,565	145	40.7
study5	1,020,397	471	46.2
study6	238,214	116	48.7
study7	548,492	427	77.8
study8	901,276	658	73.0
study9	877,529	521	59.4

Source: Table by author.

The example below calculates the incidence of cancer (per 100,000 person_year) for each study and by main exposure status (with *group_by(main.exposure, studycode)*) and summarizes the results as a table (Table 14):

```
inc.ex.study<-candat  %>%
   filter(!is.na(main.exposure))  %>%
   group_by(main.exposure,studycode)%>%
   summarise(py.ex.study = sum(ttcancer, na.rm = TRUE),
                        cancer    = sum(as.numeric(as.character(cancer)), na.rm = TRUE),
                     cancer.inc = round((cancer/py.ex.study)*100000,2))
kable(inc.ex.study,
          caption = "Incidence by main exposure status by study")%>% kable_styling(latex_options =
   c("scale_down",  "hold_position"))
```

Table 14. Incidence by main exposure status by study.

main.exposure	studycode	py.ex.study	cancer	cancer.inc
No	study1	258,159	103	39.9
No	study2	408,875	137	33.5
No	study3	257,285	267	103.8
No	study4	335,294	118	35.2
No	study5	982,283	419	42.7
No	study6	225,515	95	42.1
No	study7	515,011	374	72.6
No	study8	846,288	581	68.7
No	study9	810,505	429	52.9
Yes	study1	11,032	19	172.2
Yes	study2	17,312	18	104.0
Yes	study3	10,545	14	132.8
Yes	study4	21,271	27	126.9
Yes	study5	38,114	52	136.4
Yes	study6	12,698	21	165.4
Yes	study7	33,481	53	158.3
Yes	study8	54,988	77	140.0
Yes	study9	67,023	92	137.3

Source: Table by author.

8.2.2. Kaplan–Meier Plot of Time to Cancer

The example below creates a Kaplan–Meier plot by "main.exposure" status for the pooled data using *ggsurvplot()* of the package "*survminer*" [52]: *Surv(ttcancer, cancer)* creates the survival object, where *ttcancer* is the time and *cancer* is the event indicator (1 = event, 0 = censored); ~ *main.exposure*: asks *survfit* to calculate separate curves for the "Yes" and "No" exposure groups; *ggsurvplot(...)* of the *survminer* package creates a publication-ready plot of the *survfit* object. Note that *risk.table* = *TRUE* and *cumevents* = *TRUE* show the risk table and the cumulative event table at the bottom of the plot (Figure 95).

```
candat$cancer<-as.numeric(as.character(candat$cancer))
candat$main.exposure<-as.factor(candat$main.exposure)
#  Fit the model
fit<-survfit(Surv(ttcancer,cancer)~main.exposure, data = candat)
#  Plot survfit object
ggsurvplot(fit, linetype = "strata",
           conf.int = TRUE, pval = TRUE, censor = FALSE,
           palette = "redblue",
           risk.table = TRUE,
           cumevents = TRUE,
           tables.height = 0.2,
           break.x.by = 5,
           ylim = c(0.7, 1),
           xlim = c(0,40),
           font.x = 6, font.y = 6,
           font.legend = 6,
           font.tickslab = 6,
           pval.coord = c(1,0.8),
           xlab = "Follow-up year",
           ylab = "Liver Cancer Free Probability",
                  tables.theme = theme_bw() +
             theme(
               axis.text = element_text(size = 6),          # smaller axis text axis.title
               = element_text(size = 6),                    # smaller axis titles
               strip.text = element_text(size = 6),         # smaller facet/strip text
               legend.text = element_text(size = 6)
             )
           )
```

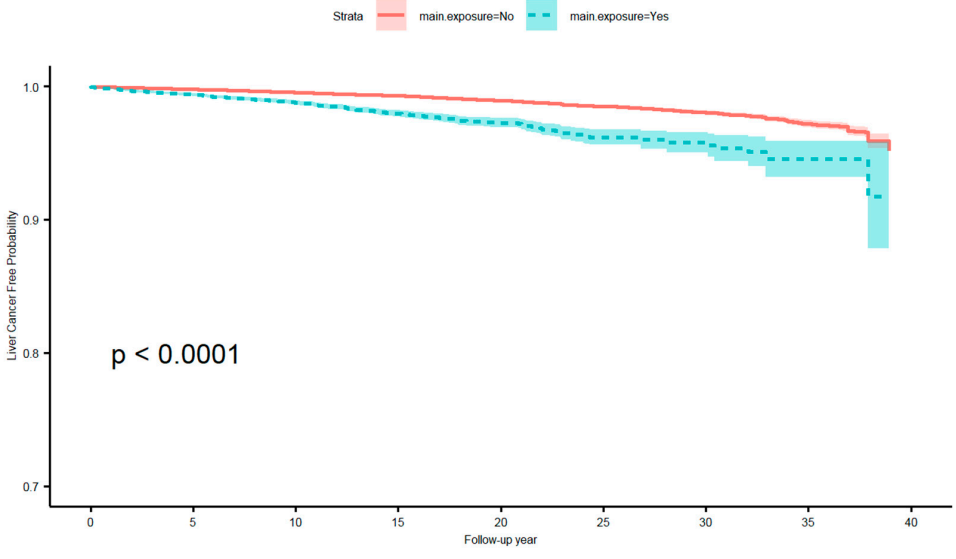


Figure 95. *Cont.*

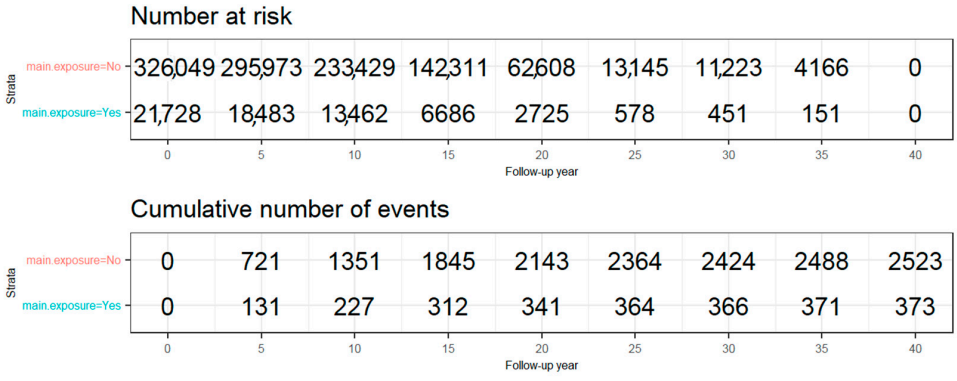


Figure 95. Kaplan–Meier curve overall pooled data. Source: Figure by author.

The KM plot shows that the "main.exposure=Yes" group (blue line) has a lower cancer-free probability over time compared to the "No" group (red line). The $p < 0.0001$ (from a log-rank test) confirms this difference in the pooled data is highly statistically significant.

8.2.3. Cox Models

Unadjusted Cox Model

First, we perform a univariable (unadjusted) Cox model to: 1. Establish a Baseline Effect to determine the "raw" association between the main exposure and cancer outcome without the influence of other variables; 2. Provide a reference point to observe how Hazard Ratios (HR) shift when confounders like age and sex, etc., are added later. This shift helps researchers understand the magnitude of confounding within the consortium data.

We can run a simple (unadjusted) Cox model on the pooled data as below and summarize the results in a table (Table 15).

```
cfit<-coxph(Surv(ttcancer,cancer)~main.exposure, data = candat)
summary(cfit)
```

```
## Call:
## coxph(formula = Surv(ttcancer, cancer) ~ main.exposure, data = candat)
##
## n= 347,777, number of events= 2896
## (27,033 observations deleted due to missingness)
##
##                    coef      exp(coef)   se(coef)    z          Pr(>|z|)
## main.exposureYes   0.97837   2.66011     0.05556     17.61      <2×10-16 ***
## ---
## Signif. codes:     0 '***'   0.001 '**'  0.01 '*'    0.05 '.'   0.1 ' ' 1
##
##                              exp(coef)   exp(−coef)  lower .95  upper .95
## main.exposureYes             2.66        0.3759      2.386      2.966
##
## Concordance = 0.543 (se = 0.004)
## Likelihood ratio test = 244.1 on 1 df, p < 2×10-16
## Wald test = 310.1 on 1 df, p < 2×10-16
## Score (logrank) test = 335.6 on 1 df, p <2×10-16
```

```
rtab<-c(summary(cfit)$coefficients,
        summary(cfit)$conf.int[,2:4],
        summary(cfit)$logtest [3])
rtab<-ifelse(rtab>= 0.0001,round(rtab,4),"<0.0001")
rtab<-c(rownames(summary(cfit)$coefficients),rtab)
names(rtab)<-c("com.grp",colnames(summary(cfit)$coefficients),
                         names(summary(cfit)$conf.int[,2:4]),
                         "lr.p")
kable(rtab, caption = "Summary of unadjusted Cox model")%>%
   kable_styling(latex_options = c("scale_down",  "hold_position"))
```

Table 15. Summary of unadjusted Cox model.

x	
com.grp	**main.exposureYe**
s coef	0.9784
exp(coef)	2.6601
se(coef)	0.0556
z	17.6099
Pr(>|z|)	<0.0001
exp(−coef)	0.3759
lower .95	2.3857
upper .95	2.9661
lr.p	<0.0001

Source: Table by author.

The output shows that the Hazard Ratio (HR) is 2.66. This is interpreted as: "In the pooled, unadjusted analysis, the exposed group has 2.66 times the hazard (or instantaneous risk) of developing cancer at any given time point compared to the unexposed group."

Adjusted Cox Model

Now we fit a multivariable Cox model adjusted for covariates that may affect the association between "main.exposure" and cancer outcome (adding covariates "gender + age + bmi +smoking + alcoholuse" to the above Cox model) and stratified by study (+ *strata(studycode)*). The + *strata(studycode)* tells the Cox model to assume a different, unique baseline hazard for each study. It does not estimate an effect for "*studycode*". This is a method for accounting for heterogeneity between studies (Table 16).

```
cfit<-coxph(Surv(ttcancer,cancer)~main.exposure  +
                        gender  +  age  +  bmi  +
                        smoking  +  alcoholuse  +
                        strata(studycode), data  =  candat)
rtab<-cbind(summary(cfit)$coefficients,
                    summary(cfit)$conf.int[,2:4])
rtab<-ifelse(rtab>= 0.0001, round(rtab,4),"<0.0001")
rtab<-cbind(rownames(summary(cfit)$coefficients),rtab)
rownames(rtab)<-NULL
kable(rtab, caption = "Summary of adjusted Cox model")%>%
   kable_styling(latex_options = c("scale_down",  "hold_position"))
```

Table 16. Summary of adjusted Cox model.

	coef	HR (or exp(coef))	se(coef)	z	Pr(>|z|)	exp(-coef)	HR.lower .95	HR.upper .95
main.exposureYes	0.7502	2.1175	0.0596	12.5936	<0.0001	0.4723	1.8842	2.3798
genderMale	0.835	2.3049	0.0598	13.9747	<0.0001	0.4339	2.0502	2.5913
age	0.0556	1.0572	0.0021	25.9292	<0.0001	0.9459	1.0528	1.0617
bmi	<0.0001	1	<0.0001	0.0727	0.942	1	0.9999	1.0001
smokingYes	0.4785	1.6136	0.0551	8.6845	<0.0001	0.6197	1.4485	1.7976
alcoholuseYes	0.0975	1.1024	0.0479	2.0342	0.0419	0.9071	1.0036	1.211

Source: Table by author.

The output shows that after adjusting for confounders and stratifying by study, the HR for "main.exposureYes" is 2.1175, which is slightly smaller than the HR from the univariable Cox model.

8.2.4. Subgroup Analysis

New Function for Subgroup Analysis, Heterogeneity Test

For a complex analysis and result summary that will be done multiple times, writing a new function for that analysis would be more efficient. Below is an example function *subhe()* written to perform and summarize neatly the results of the Cox model for subgroups, heterogeneity test, and Cox model unadjusted or adjusted for the subgroup variable, with and without having "studycode" as strata in the model. With this function, *subhe(),* one just needs to specify the variables corresponding to time to event ("ttevent"), event ("event"), comparison variable ("comvar"), variables to be adjusted ("adjustvar"), subgroup variable ("subvar"), and the dataset to be used ("data"), and the function will do all the computation and summarize the results.

The *subhe()* function is a custom-built utility that streamlines the evaluation of whether the effect of a primary exposure is consistent across different clinical or demographic subgroups. The purpose of this function is as follows:

- Automation: To fit multiple Cox Proportional Hazards models across all strata of a subgroup variable with a single command.
- Interaction Testing: To automatically calculate the *p*-value for interaction ("p.heterogeneity"), which determines if the difference between subgroup effects is statistically significant.
- Standardized Output: To produce a clean, row-based summary of Hazard Ratios (HR) and Confidence Intervals (CI) for both unadjusted and adjusted models.

The results of the following example will be summarized and displayed as rows:

- The Cox model results compare the groups of the comparison variable (*comvar*) for each stratum of the subgroup variable (*subvar*). For example, for *comvar*= *"main.exposure"* and *subvar*= *"gender"*, the univariable Cox model results comparing between "main.exposureYes" (vs. "main.exposureNo") will be displayed for each subgroup (stratum) "Female" and "Male" (rows "gender.stratified Female main.exposureYes" and "gender.stratified Male main.exposureYes").
- The results of the adjusted Cox model include the comparison variable (*comvar*) and the subgroup variable (*subvar*) for each of these two variables. For example, for *comvar*= *"main.exposure"* and *subvar*= *"gender"*, the adjusted Cox model includes "main.exposure" and "gender", and the results for each of

these two variables will be displayed (rows "main.exposure.gender.adjusted main.exposureYes" and "main.exposure.gender.adjusted genderMale").

- The results of the adjusted Cox model include the comparison variable (*comvar*), the subgroup variable (*subvar*), and *strata(studycode)*. For example, for *comvar= "main.exposure"* and *subvar= "gender"*, the adjusted Cox model includes "main.exposure", "gender" and *strata(studycode)*, and the results for "main.exposure" and "gender" will be displayed (rows "main.exposure.gender.adjusted.strata main.exposureYes" and "main.exposure.gender.adjusted.strata genderMale").
- The results of the unadjusted (univariable) Cox model including the comparison variable (*comvar*) and the results of the Cox model including the comparison variable (*comvar*) and *strata(studycode)* using the same dataset with non-missing values as the adjusted Cox models above. For example, for *comvar= "main.exposure"* and *subvar= "gender"*, the unadjusted Cox model comparing between "main.exposure" groups without and with *strata(studycode)* using the same dataset with non-missing values for "main.exposure" and "gender" will be displayed in result (rows "main.exposure.gender.unadjusted main.exposureYes" and "main.exposure.gender.unadjusted.strata main.exposureYes").

The output table columns display the number of studies with available data included in the analysis ("number.study"), the number of subjects included in the analysis ("number.subject"), Cox model summarized results such as HR ("exp(coef)") and 95% CI ("lower.95", "upper.95"), *p*-value from likelihood ratio test for the significance of a variable ("lr.p"), and *p*-value for heterogeneity ("p.heterogeneity") (interaction term between the comparison variable (*comvar*) and the subgroup variable(*subvar*)).

```
subhe<-function(ttevent, event, comvar, adjustvar, subvar, data){
# The subhe() function logic:
# 1. Stratified Analysis
# The function splits the 'data' by the 'subvar' (e.g., Male/Female)
# For each stratum, it fits a Cox model: Surv(ttevent, event) ~ comvar
# 2. Test for interaction between comvar and subvar
# It fits Cox model without and with interaction term to test for heterogeneity
# Extract the results of both models
# Surv(ttevent, event) ~ comvar + subvar
# Surv(ttevent, event) ~ comvar + subvar+ comvar * subvar
# 3. Fit stratified Cox model with +strata(studycode) and extract the results
# 4. To facilitate comparison between Cox models, it also fit the univariable Cox models
    with the same non-missing data with the data used in multivariable Cox model
# 5. Result Compilation: It extracts: HR and 95% CI for each subgroup row, the
    Interaction P-value from the global model, Adjusted HRs for the comparison variable
    require(survival)
    data[,subvar]<-as.factor(data[,subvar]) fit<-
    list()
    fit.re<-NULL
    #For each stratum, fits a Cox model
    for (i in 1:nlevels(data[,subvar])){
      datsub<-subset(data,data[,subvar] %in% levels(data[,subvar])[i]) nstudy<-
      length(unique(datsub[,"studycode"][!is.na(datsub[,subvar])])) nsubj<-
```

```
        nrow(na.omit(datsub[,c(comvar,ttevent,event)]))
        fit[[i]]<-coxph(as.formula(paste(paste("Surv(",ttevent,",",event,")",
                                               sep = ""),
                                         comvar,sep = "~")),
                        data = datsub)
        rtab<-c(paste(subvar,"stratified",sep = "."),levels(data[,subvar])[i],
                rownames(summary(fit[[i]])$coefficients),
                nstudy,nsubj, round(summary(fit[[i]])$coefficients,4),
                round(summary(fit[[i]])$conf.int[,2:4],4),
                round(summary(fit[[i]])$logtest[3],6))
          names(rtab)<-c("varname","subgroup","comvar",
                         "number.study","number.subject",
                         colnames(summary(fit[[i]])$coefficients),
                         names(summary(fit[[i]])$conf.int[,2:4]),
                         "lr.p")
          fit.re<-rbind(fit.re,rtab)
        }
        #full dataset #heterogeneity test
        f<-coxph(as.formula(paste(paste("Surv(",ttevent,",",event,")",sep = ""),
                                  paste(comvar,subvar,sep = "+"),sep = "~")),
                 data = data)
        fs<-coxph(as.formula(paste(paste("Surv(",ttevent,",",event,")",
                                         sep = ""), paste(comvar,subvar,
                                         paste(comvar,subvar, sep = "*"),
                                         sep = "+"), sep = "~")),
                                         data = data)
ano<-anova(f,fs)
        p.he<-round(ano$`Pr(>|Chi|)`[2],6)
        fit.he<-cbind(fit.re,p.heterogeneity = p.he)
        #adjusted for subgroup var
        datad<-na.omit(data[,c("studycode",comvar,
                               subvar,ttevent,event)])
        nstudy<-length(unique(datad["studycode"])) nsubj<-
        nrow(datad)
        fa<-coxph(as.formula(paste(paste("Surv(",ttevent,",",event,")",
                                         sep = ""), paste(comvar,subvar,sep
                                   = "+"), sep = "~")),
                  data = data)
        atab<-cbind(paste(comvar,subvar,"adjusted",sep = "."),"",
                    rownames(summary(fa)$coefficients), nstudy,nsubj,
                    round(summary(fa)$coefficients,4),
                    round(summary(fa)$conf.int[,2:4],4),
                    round(summary(fa)$logtest[3],6))
        colnames(atab)<-c("varname","subgroup","comvar",
                          "number.study","number.subject",
                          colnames(summary(fa)$coefficients),
                          colnames(summary(fa)$conf.int[,2:4]), "lr.p")
        #adjusted and stratified by study
        fas<-coxph(as.formula(paste(paste("Surv(",ttevent,",",event,")",
                                          sep = ""),
                                    paste(comvar,subvar,"strata(studycode)",
                                          sep = "+"),
                                    sep = "~")),
                   data = data)
```

```
datad<-na.omit(data[,c("studycode",comvar,
                       subvar,ttevent,event)])
nstudy<-length(unique(datad["studycode"])) nsubj<-
nrow(datad)
fa<-coxph(as.formula(paste(paste("Surv(",ttevent,",",event,")",
                                 sep = ""), paste(comvar,subvar,sep
                                 = "+"), sep = "~")),
          data = data)
atab<-cbind(paste(comvar,subvar,"adjusted",sep = "."),"",
            rownames(summary(fa)$coefficients), nstudy,nsubj,
            round(summary(fa)$coefficients,4),
            round(summary(fa)$conf.int[,2:4],4),
            round(summary(fa)$logtest[3],6))
colnames(atab)<-c("varname","subgroup","comvar",
                  "number.study","number.subject",
                  colnames(summary(fa)$coefficients),
                  colnames(summary(fa)$conf.int[,2:4]), "lr.p")
#adjusted and stratified by study
fas<-coxph(as.formula(paste(paste("Surv(",ttevent,",",event,")",
                                  sep = ""),
                            paste(comvar,subvar,"strata(studycode)",
                                  sep = "+"),
                            sep = "~")),
           data = data)
astab<-cbind(paste(comvar,subvar,"adjusted.strata",sep = "."),"", rownames(summary(fas)$coefficients),
             nstudy,nsubj, round(summary(fas)$coefficients,4),
             round(summary(fas)$conf.int[,2:4],4),
             round(summary(fas)$logtest[3],6))
colnames(astab)<-c("varname","subgroup","comvar",
                   "number.study","number.subject",
                   colnames(summary(fas)$coefficients),
                   colnames(summary(fas)$conf.int[,2:4]), "lr.p")
# add unadjusted analysis with the same # non-
missing sample size with adjusted
fu<-coxph(as.formula(paste(paste("Surv(",ttevent,",",event,")",
                                 sep = ""),
                           comvar,sep = "~")),
          data = datad)
utab<-c(paste(comvar,subvar,"unadjusted",sep = "."),"",
            rownames(summary(fu)$coefficients), nstudy,nsubj,
            as.vector(round(summary(fu)$coefficients,4)),
            as.vector(round(summary(fu)$conf.int[,2:4],4)),
            as.vector(round(summary(fu)$logtest[3],6)))
names(utab)<-c("varname","subgroup","comvar",
               "number.study","number.subject",
               colnames(summary(fu)$coefficients),
               names(summary(fu)$conf.int[,2:4]), "lr.p")
#unadjusted but stratified by study
fus<-coxph(as.formula(paste(paste("Surv(",ttevent,",",event,")",
                                  sep = ""), paste(comvar,"strata(studycode)",
                                  sep = "+"),
                            sep = "~")),
           data = datad)
ustab<-c(paste(comvar,subvar,"unadjusted.strata",sep = "."),"", rownames(summary(fus)$coefficients),
```

```
                nstudy,nsubj, as.vector(round(summary(fus)$coefficients,4)),
                as.vector(round(summary(fus)$conf.int[,2:4],4)),
                as.vector(round(summary(fus)$logtest[3],6)))
    names(ustab)<-c("varname","subgroup","comvar",
                    "number.study","number.subject",
                    colnames(summary(fus)$coefficients),
                    names(summary(fus)$conf.int[,2:4]), "lr.p")
    #combine results
    austab<-rbind(atab,astab,utab,ustab) austab<-
    as.data.frame(austab) fit.he<-
    as.data.frame(fit.he)
    fit.all<-plyr::rbind.fill(fit.he,austab)
  fit.all
}
```

Now, we apply the above function *subhe()* for multiple subgroup variables ("gender", "age.cat", "bmi.cat", "smoking", and "alcoholuse") by writing a loop. The loop will perform the *subhe()* function for each of the subgroup variables and add the results to a table. The output will be a big table combining the results for each of all variables (Table 17). Interested readers should read the original paper by Nhan Thi Ho et al. for further details about the actual result interpretation and explanation [68].

```
subvec<-c("gender","age.cat","bmi.cat","smoking","alcoholuse")
candat[,subvec]<-lapply(candat[,subvec], as.factor)
tabsub<-NULL
for (i in 1:length(subvec)){
  tabv<-subhe(ttevent = "ttcancer",
              event = "cancer",
              comvar = "main.exposure",
              subvar = subvec[i], data = candat)
  tabsub<-rbind(tabsub,tabv)
}
kable(tabsub, caption = "Summary of subgroup analysis results")%>%
  kable_styling(latex_options = c("scale_down", "hold_position"))
```

Table 17. Summary of subgroup analysis results.

Varname	Subgroup	Comvar	Number Study	Number Subject	coef	exp(coef)	se(coef)	z	Pr(>|z|)	exp(-coef)	Lower .95	Upper .95	p LR	p Heterogeneity
gender.stratified	Female	main.exposureYes	9	186,759	0.9527	2.5927	0.11	8.6644	0	0.3857	2.0901	3.2162	0	0.564023
gender.stratified	Male	main.exposureYes	9	161,018	0.8742	2.3969	0.0644	13.5643	0	0.4172	2.1125	2.7196	0	0.564023
main.exposure.gender.adjusted		main.exposureYes	9	347,777	0.8954	2.4484	0.0556	16.0982	0	0.4084	2.1955	2.7304	0	NA
main.exposure.gender.adjusted		genderMale	9	347,777	1.0901	2.9747	0.0409	26.657	0	0.3362	2.7455	3.2229	0	NA
main.exposure.gender.adjusted.strata		main.exposureYes	9	347,777	0.8888	2.4322	0.0558	15.9387	0	0.4111	2.1804	2.7131	0	NA
main.exposure.gender.adjusted.strata		genderMale	9	347,777	1.1192	3.0625	0.041	27.3137	0	0.3265	2.8262	3.3186	0	NA
main.exposure.gender.unadjusted		main.exposureYes	9	347,777	0.9784	2.6601	0.0556	17.6099	0	0.3759	2.3857	2.9661	0	NA
main.exposure.gender.unadjusted.strata		main.exposureYes	9	347,777	0.9621	2.6173	0.0558	17.2573	0	0.3821	2.3463	2.9195	0	NA
age.cat.stratified	<40 years old	main.exposureYes	4	19,151	0.2324	1.2617	0.5104	0.4553	0.6489	0.7926	0.4639	3.4311	0.660216	0.008634
age.cat.stratified	≥40&<60 years old	main.exposureYes	9	208,460	1.0488	2.8541	0.0827	12.6844	0	0.3504	2.4271	3.3562	0	0.008634
age.cat.stratified	≥60 years old	main.exposureYes	9	120,166	0.6929	1.9995	0.0761	9.1055	0	0.5001	1.7224	2.3211	0	0.008634
main.exposure.age.cat.adjusted		main.exposureYes	9	347,777	0.8494	2.3383	0.0558	15.2184	0	0.4277	2.096	2.6086	0	NA
main.exposure.age.cat.adjusted		age.cat ≥ 40&<60 years old	9	347,777	0.8253	2.2826	0.1054	7.8329	0	0.4381	1.8567	2.8062	0	NA
main.exposure.age.cat.adjusted		age.cat ≥ 60 years old	9	347,777	1.6624	5.272	0.1073	15.4946	0	0.1897	4.2722	6.5058	0	NA
main.exposure.age.cat.adjusted.strata		main.exposureYes	9	347,777	0.8349	2.3046	0.056	14.9142	0	0.4339	2.0651	2.5719	0	NA
main.exposure.age.cat.adjusted.strata		age.cat ≥ 40&<60 years old	9	347,777	1.0183	2.7684	0.1064	9.5694	0	0.3612	2.2473	3.4104	0	NA
main.exposure.age.cat.adjusted.strata		age.cat ≥ 60 years old	9	347,777	1.7948	6.0184	0.1088	16.504	0	0.1662	4.8631	7.4482	0	NA
main.exposure.age.cat.unadjusted		main.exposureYes	9	347,777	0.9784	2.6601	0.0556	17.6099	0	0.3759	2.3857	2.9661	0	NA
main.exposure.age.cat.unadjusted.strata		main.exposureYes	9	347,777	0.9621	2.6173	0.0558	17.2573	0	0.3821	2.3463	2.9195	0	NA
bmi.cat.stratified	<23	main.exposureYes	8	143,748	0.9911	2.6942	0.1053	9.4113	0	0.3712	2.1918	3.3119	0	0.725015
bmi.cat.stratified	≥23	main.exposureYes	8	150,120	1.0468	2.8484	0.0804	13.0211	0	0.3511	2.4332	3.3345	0	0.725015
main.exposure.bmi.cat.adjusted		main.exposureYes	8	293,868	1.0257	2.789	0.0639	16.0621	0	0.3586	2.4609	3.1608	0	NA
main.exposure.bmi.cat.adjusted		bmi.cat ≥ 23	8	293,868	0.0761	1.079	0.0438	1.7377	0.0823	0.9268	0.9903	1.1757	0	NA
main.exposure.bmi.cat.adjusted.strata		main.exposureYes	8	293,868	1.0117	2.7504	0.0641	15.7794	0	0.3636	2.4256	3.1187	0	NA

Table 17. *Cont.*

Varname	Subgroup	Comvar	Number Study	Number Subject	coef	exp(coef)	se(coef)	z	Pr(>|z|)	exp(-coef)	Lower .95	Upper .95	p LR	p Heterogeneity
main.exposure.bmi.cat.adjusted.strata		bmi.cat ≥ 23	8	293,868	0.0555	1.0571	0.0441	1.2582	0.2083	0.946	0.9695	1.1526	0	NA
main.exposure.bmi.cat.unadjusted		main.exposureYes	8	293,868	1.0338	2.8118	0.0637	16.2335	0	0.3556	2.4818	3.1856	0	NA
main.exposure.bmi.cat.unadjusted.strata		main.exposureYes	8	293,868	1.0175	2.7664	0.064	15.9113	0	0.3615	2.4405	3.1358	0	NA
smoking.stratified	No	main.exposureYes	9	178,259	1.0493	2.8558	0.1007	10.4248	0	0.3502	2.3445	3.4786	0	0.117492
smoking.stratified	Yes	main.exposureYes	9	139,527	0.8577	2.3578	0.069	12.4224	0	0.4241	2.0593	2.6994	0	0.117492
main.exposure.smoking.adjusted		main.exposureYes	9	317,786	0.917	2.5017	0.057	16.0914	0	0.3997	2.2373	2.7973	0	NA
main.exposure.smoking.adjusted		smokingYes	9	317,786	0.9709	2.6402	0.0407	23.8569	0	0.3788	2.4378	2.8594	0	NA
main.exposure.smoking.adjusted.strata		main.exposureYes	9	317,786	0.9006	2.4611	0.0572	15.7537	0	0.4063	2.2002	2.7528	0	NA
main.exposure.smoking.adjusted.strata		smokingYes	9	317,786	1.041	2.832	0.0413	25.2072	0	0.3531	2.6118	3.0708	0	NA
main.exposure.smoking.unadjusted		main.exposureYes	9	317,786	0.9824	2.6708	0.0569	17.2544	0	0.3744	2.3888	2.9861	0	NA
main.exposure.smoking.unadjusted.strata		main.exposureYes	9	317,786	0.9697	2.637	0.0571	16.9704	0	0.3792	2.3577	2.9495	0	NA
alcoholuse.stratified	No	main.exposureYes	8	151,202	0.984	2.6751	0.0868	11.332	0	0.3738	2.2565	3.1714	0	0.741064
alcoholuse.stratified	Yes	main.exposureYes	8	143,403	0.9479	2.5803	0.0779	12.1674	0	0.3875	2.2149	3.006	0	0.741064
main.exposure.alcoholuse.adjusted		main.exposureYes	8	294,605	0.9653	2.6255	0.058	16.6495	0	0.3809	2.3435	2.9414	0	NA
main.exposure.alcoholuse.adjusted		alcoholuseYes	8	294,605	0.3914	1.479	0.0394	9.9437	0	0.6761	1.3692	1.5976	0	NA
main.exposure.alcoholuse.adjusted.strata		main.exposureYes	8	294,605	0.9502	2.5862	0.0582	16.3378	0	0.3867	2.3076	2.8985	0	NA
main.exposure.alcoholuse.adjusted.strata		alcoholuseYes	8	294,605	0.4861	1.6259	0.0418	11.6388	0	0.615	1.4981	1.7646	0	NA
main.exposure.alcoholuse.unadjusted		main.exposureYes	8	294,605	0.9621	2.6171	0.058	16.5919	0	0.3821	2.3359	2.9321	0	NA
main.exposure.alcoholuse.unadjusted.strata		main.exposureYes	8	294,605	0.9564	2.6023	0.0582	16.4374	0	0.3843	2.3218	2.9167	0	NA

Source: Table by author.

8.2.5. Analysis of Each Study in the Consortium

While previous sections focused on the consortium as a single pooled entity, the intention of this section is to decompose the data and analyze each participating study independently. This "study-by-study" approach is a prerequisite for any rigorous meta-analysis. The purpose of this analysis is to:

- Assess Heterogeneity: To visually and statistically determine if the effect of the primary exposure is consistent across different geographical locations or clinical settings (represented by different studies).
- Verify Robustness: To ensure that the "pooled" effect is not being driven by a single large study while the others show no effect.
- Prepare for Meta-Analysis: To generate the study-specific effect sizes (Log–Hazard Ratios) and standard errors (SE) required for the fixed- and random-effects meta-analyses.

Kaplan–Meier Curve and Univariable Cox Model for Each Study

We can write a loop to perform the same analysis (Kaplan–Meier curve and univariable Cox model) for each study to gain efficiency. Then we combine all the plots into a multi-panel plot (using *wrap_plots()*) (Figure 96) and combine the results of univariable Cox model of each of all studies in a table (Table 18).

```
cname<-unique(candat$studycode[!is.na(candat$main.exposure)]) fitl<-
list()
splotl<-list()
cfitl<-list()
rtabl<-list()
cfit.re<-NULL
for (i in 1:length(cname)){
  # Survival curves
  fitl[[i]]<-survfit(Surv(ttcancer,cancer)~main.exposure,
                     data = subset(candat,studycode = = cname[i]))
  sp<-ggsurvplot(fitl[[i]], linetype = "strata",
                           conf.int = TRUE,pval = TRUE,censor = FALSE,
                           palette = "grey", ylim = c(0.7, 1), pval.coord
                           = c(1,0.8),
                           xlab = "Follow-up year",
                           ylab = "Cancer Free Probability", title
                           = paste(cname[i]))
  splotl[[i]]<-sp$plot
  # cox model
  cfitl[[i]]<-coxph(Surv(ttcancer,cancer)~main.exposure, data
= subset(candat,studycode = = cname[i]))
  rtabl[[i]]<-cbind(matrix(cname[i],ncol = 1),
                    matrix(rownames(summary(cfitl[[i]])$coefficients), ncol
                           = 1),
                    matrix(prettyNum(summary(cfitl[[i]])$coefficients), ncol =
                           5),
                    matrix(prettyNum(summary(cfitl[[i]])$conf.int[,2:4]), ncol
                           = 3),
                    matrix(prettyNum(summary(cfitl[[i]])$logtest[3]), ncol
                           = 1))
  cfit.re<-rbind(cfit.re,rtabl[[i]])
}
```

```
# Combine all plots into a single figure (grid layout)
combined_plot <- wrap_plots(splotl, ncol = 2) +
  plot_annotation(title = "Kaplan-Meier curves for each of all studies")

# Print the combined figure
combined_plot
```

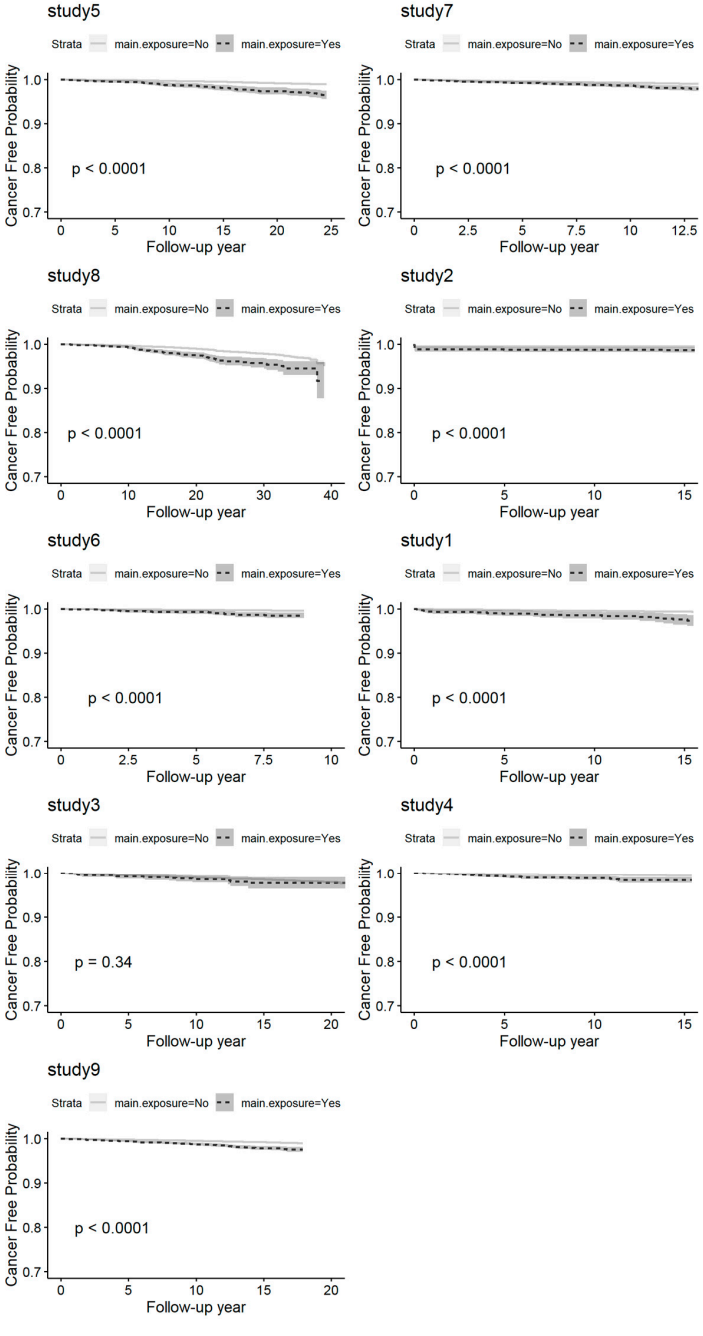


Figure 96. Kaplan–Meier curve for each of the studies. Source: Figure by author.

Now we combine univariable Cox model results for each of all studies in a table.

```
colnames(cfit.re)<-c("study","com.grp",
                     "coef","hr","se.coef","z", "z.p",
                     "exp.minuscoef","ll.95.hr","ul.95.hr",
                     "lr.p")
cfit.re<-as.data.frame(cfit.re)
#calculate SE of HR
cfit.re[,3:11]<-lapply(cfit.re[,3:11],as.character)
cfit.re[,3:11]<-lapply(cfit.re[,3:11],as.numeric)
kable(cfit.re, caption = "Univariable Cox model results of each study")%>%
  kable_styling(latex_options = c("scale_down", "hold_position"))
```

Table 18. Univariable Cox model results of each study.

study	com.grp	coef	HR (or exp(coef)	se.coef	z	z.p	exp.minuscoef	LL.95.HR	UL.95.HR	lr.p
study5	main.exposureYes	1.200	3.32	0.147	8.160	0.000	0.301	2.490	4.43	0.000
study7	main.exposureYes	0.779	2.18	0.147	5.300	0.000	0.459	1.630	2.91	0.000
study8	main.exposureYes	0.827	2.29	0.122	6.800	0.000	0.437	1.800	2.90	0.000
study2	main.exposureyes	1.030	2.81	0.251	4.120	0.000	0.356	1.720	4.59	0.000
study6	main.exposureYes	1.370	3.95	0.241	5.700	0.000	0.253	2.460	6.34	0.000
study1	main.exposureYes	1.460	4.32	0.250	5.860	0.000	0.231	2.650	7.06	0.000
study3	main.exposureYes	0.259	1.30	0.274	0.944	0.345	0.772	0.757	2.22	0.363
study4	main.exposureYes	1.280	3.60	0.213	6.010	0.000	0.277	2.370	5.48	0.000
study9	main.exposureYes	0.978	2.66	0.115	8.500	0.000	0.376	2.120	3.33	0.000

Source: Table by author.

Multivariable Cox Model for Each Study

Similar to the above, we can write a loop to perform the same multivariable Cox model for each study and combine the results of the multivariable Cox model of each of all studies in a table. Note that we write an *if* condition to perform the Cox model only for the study with available data for all variables included in the multivariable model (Table 19).

```
cfitl<-list()
rtabl<-list()
cfit.re.m<-NULL
for (i in 1:length(cname)){
  # cox model
  datas<-subset(candat,studycode = = cname[i]) if
  (nrow(na.omit(datas)) = = 0){
     cfit.re.m<-cfit.re.m
  }
  if (nrow(na.omit(datas))>0){
      cfitl[[i]]<-coxph(Surv(ttcancer,cancer)~main.exposure +
                gender + age + bmi +
                smoking + alcoholuse, data
                    = datas)
  rtabl[[i]]<-cbind(matrix(cname[i],ncol = 1,
                           nrow = nrow(summary(cfitl[[i]])$coefficients)),
                    matrix(rownames(summary(cfitl[[i]])$coefficients),
                           ncol = 1),
                    matrix(prettyNum(summary(cfitl[[i]])$coefficients), ncol =
                           5),
                    matrix(prettyNum(summary(cfitl[[i]])$conf.int[,2:4]), ncol
                           = 3))
```

```
  cfit.re.m<-rbind(cfit.re.m,rtabl[[i]])
  }
}
colnames(cfit.re.m)<-c("study","com.grp",
                       "coef","hr","se.coef","z", "z.p",
                       "exp.minuscoef","ll.95.hr","ul.95.hr")
cfit.re.m<-as.data.frame(cfit.re.m)
cfit.re.m[,3:10]<-lapply(cfit.re.m[,3:10],as.character)
cfit.re.m[,3:10]<-lapply(cfit.re.m[,3:10],as.numeric)
kable(cfit.re.m,
      caption = "Multivariable Cox model results of each study")%>% kable_styling(latex_options
  = c("scale_down", "hold_position"))
```

Table 19. Multivariable Cox model results of each study.

study	com.grp	coef	HR (or exp(coef)	se.coef	z	z.p	exp.minuscoef	LL.95.HR	UL.95.HR
study5	main.exposureYes	0.840	2.320	0.162	5.190	0.000	0.432	1.690	3.18
study5	genderMale	0.696	2.010	0.175	3.980	0.000	0.498	1.420	2.83
study5	age	0.075	1.080	0.007	10.100	0.000	0.928	1.060	1.09
study5	bmi	0.070	1.070	0.016	4.240	0.000	0.932	1.040	1.11
study5	smokingYes	0.343	1.410	0.145	2.370	0.018	0.710	1.060	1.87
study5	alcoholuseYes	0.540	1.720	0.148	3.640	0.000	0.583	1.280	2.29
study7	main.exposureYes	0.646	1.910	0.157	4.130	0.000	0.524	1.400	2.59
study7	genderMale	0.337	1.400	0.178	1.900	0.058	0.714	0.989	1.99
study7	age	0.058	1.060	0.006	9.720	0.000	0.944	1.050	1.07
study7	bmi	0.019	1.020	0.015	1.230	0.220	0.982	0.989	1.05
study7	smokingYes	0.620	1.860	0.164	3.780	0.000	0.538	1.350	2.57
study7	alcoholuseYes	0.161	1.170	0.143	1.120	0.262	0.852	0.887	1.55
study6	main.exposureYes	1.020	2.770	0.307	3.310	0.001	0.362	1.510	5.05
study6	genderMale	0.661	1.940	0.343	1.930	0.054	0.516	0.989	3.79
study6	age	0.063	1.070	0.011	5.980	0.000	0.939	1.040	1.09
study6	bmi	−0.001	0.999	0.035	−0.021	0.983	1.000	0.933	1.07
study6	smokingYes	0.550	1.730	0.315	1.750	0.080	0.577	0.936	3.21
study6	alcoholuseYes	−0.137	0.872	0.286	−0.477	0.634	1.150	0.498	1.53
study1	main.exposureYes	1.180	3.260	0.255	4.630	0.000	0.306	1.980	5.39
study1	genderMale	0.972	2.640	0.316	3.080	0.002	0.378	1.420	4.91
study1	age	0.052	1.050	0.009	5.810	0.000	0.950	1.030	1.07
study1	bmi	0.045	1.050	0.032	1.390	0.165	0.956	0.982	1.11
study1	smokingYes	0.586	1.800	0.290	2.020	0.043	0.557	1.020	3.17
study1	alcoholuseYes	−0.094	0.911	0.242	−0.387	0.698	1.100	0.567	1.46
study3	main.exposureYes	0.112	1.120	0.276	0.407	0.684	0.894	0.651	1.92
study3	genderMale	0.992	2.700	0.189	5.250	0.000	0.371	1.860	3.90
study3	age	0.047	1.050	0.005	8.770	0.000	0.954	1.040	1.06
study3	bmi	−0.016	0.984	0.020	−0.792	0.428	1.020	0.945	1.02
study3	smokingYes	0.293	1.340	0.173	1.690	0.090	0.746	0.955	1.88
study3	alcoholuseYes	0.319	1.380	0.148	2.160	0.031	0.727	1.030	1.84
study4	main.exposureYes	0.608	1.840	0.235	2.580	0.010	0.545	1.160	2.91
study4	genderMale	1.380	3.990	0.311	4.460	0.000	0.250	2.170	7.34
study4	age	0.064	1.070	0.010	6.650	0.000	0.938	1.050	1.09
study4	bmi	0.069	1.070	0.030	2.260	0.024	0.933	1.010	1.14

Table 19. *Cont.*

study	com.grp	coef	HR (or exp(coef)	se.coef	z	z.p	exp.minuscoef	LL.95.HR	UL.95.HR
study4	smokingYes	0.049	1.050	0.228	0.215	0.830	0.952	0.672	1.64
study4	alcoholuseYes	0.149	1.160	0.248	0.600	0.548	0.862	0.714	1.89
study9	main.exposureYes	0.751	2.120	0.117	6.440	0.000	0.472	1.690	2.66
study9	genderMale	1.100	3.000	0.113	9.700	0.000	0.334	2.400	3.74
study9	age	0.069	1.070	0.006	12.100	0.000	0.934	1.060	1.08
study9	bmi	0.071	1.070	0.011	6.540	0.000	0.931	1.050	1.10
study9	smokingYes	0.508	1.660	0.101	5.030	0.000	0.602	1.360	2.03
study9	alcoholuseYes	−0.096	0.909	0.110	−0.869	0.385	1.100	0.733	1.13

Source: Table by author.

8.2.6. Meta-Analysis Across Studies Based on the Results of Each Study

After calculating the independent hazard ratios for each study, a meta-analysis is performed to perform a statistical synthesis of these results. Meta-analysis will help:

- Generate a Pooled Estimate: To provide a single "Global Hazard Ratio" that represents the collective evidence from all participating studies.
- Quantify Heterogeneity: To measure how much the results vary between studies (using the I^2 statistic) and test for statistical significance.
- Increase Statistical Power: By combining thousands of patients across multiple studies, the meta-analysis can detect small but clinically significant effects that individual studies might miss.

Univariable Cox Model

Now that we have the Cox model results of all studies from the above analysis. We can perform meta-analysis and forest plot of the results of the univariable Cox model of all studies above using the "*meta*" package [73]: *metagen(TE = coef, seTE = se.coef, . . .)* is a meta-analysis function, *TE* stands for "Treatment Effect" (our log-HR), and *seTE* is its standard error, *sm = "HR"* specifies the summary measure is a Hazard Ratio, and *forest(. . .)* generates the forest plot (Figure 97).

```
mcfit<-metagen(TE = coef,
                        seTE = se.coef,
studlab = study, sm = "HR",
                        data = cfit.re)
# Save current margin settings
old_mar <- par("mar")
# Increase margins to avoid cropping
par(mar = c(5, 6, 4, 2))
forest(mcfit,
          smlab = "HR", fontsize = 8)
# Reset margins to original settings
par(mar = old_mar)
```

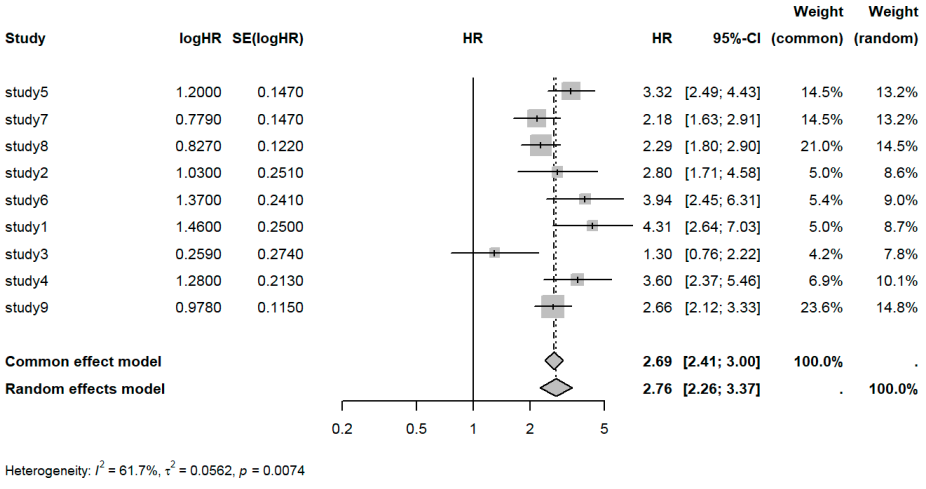


Figure 97. Meta-analysis univariable Cox model. Source: Figure by author.

The output figure shows the following:

- The squares represent the HR from each individual study. The horizontal lines are their 95% CIs.
- The "Common effect model" (HR=2.69) is the pooled estimate if we assume the true effect is identical in all studies.
- The "Random effects model" (HR=2.76) is the pooled estimate allowing for the true effect to vary between studies.
- Heterogeneity: The statistics (I^2 = 61.7, p = 0.0074) show that there is significant heterogeneity. This means the random effects model is the more appropriate and realistic choice.

Multivariable Cox Model

We can perform a similar meta-analysis and forest plot of the results of the multivariable Cox model of all studies above (note that we only do meta-analysis of the results of "main.exposure"):

```
mcfit.m<-metagen(TE = coef,
                 seTE = se.coef,
                 studlab = study, sm =
                 "HR",
                 data = subset(cfit.re.m, com.grp == "main.exposureYes"))
# Save current margin settings
old_mar <- par("mar")
# Increase margins to avoid cropping
par(mar = c(5, 6, 4, 2))
forest(mcfit.m, smlab = "HR", fontsize = 8)
# Reset margins to original settings
par(mar = old_mar)
```

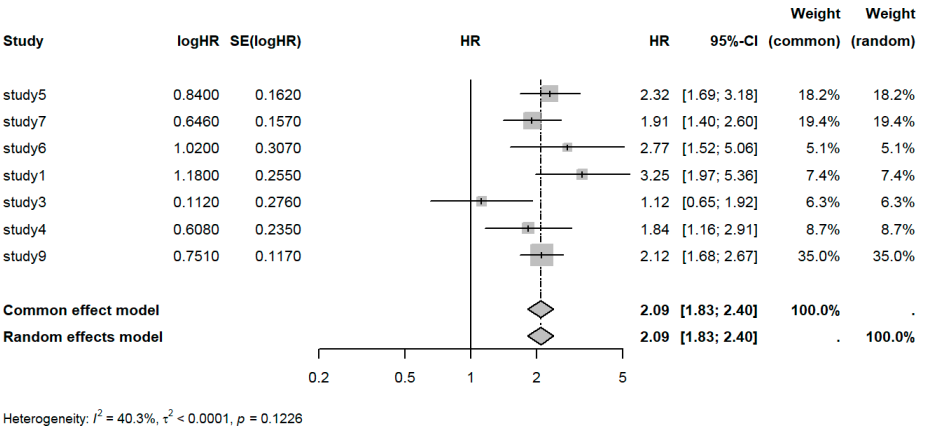


Figure 98. Meta-analysis multivariable Cox model. Source: Figure by author.

Result interpretation is similar to the above for the univariable model. Note that the multivariable model shows a smaller HR after adjusting for confounders, and heterogeneity is no longer statistically significant (Figure 98).

8.2.7. Sensitivity Analysis (Leave One Study Out Validation)

In large consortium data, there is a risk that the overall result is being disproportionately driven by one very large study or one study with extreme findings. Thus, we need to conduct a "stability audit" of the meta-analysis results.

The purpose of sensitivity analysis is to evaluate whether any specific study strongly affects the meta-analysis results. The approach is to remove a study and re-perform the meta-analysis and repeat the procedure for all studies (again, we can write a loop for this repeated analysis). Then, the meta-analysis results after removing each of the studies are combined and summarized in a plot. The purpose of the "Leave-One-Out" sensitivity analysis is:

- Identify Influential Studies: To determine if the pooled Hazard Ratio (HR) remains statistically significant after removing each study one by one.
- Assess Robustness: If the pooled effect size stays consistent regardless of which study is excluded, the findings are considered "robust."
- Detect Bias: To uncover whether a single study's unique clinical protocol or population demographic is responsible for the entire consortium's "discovery".

Below is an example of a leave-one-out study for validation of the univariable Cox model. First, we remove a study and re-perform the meta-analysis and display the forest plot without the study that has been removed (Figures 99–107):

```
# Save current margin settings
old_mar <- par("mar")
# Increase margins to avoid cropping
par(mar      =    c(5, 6, 4, 2))
cname<-unique(cfit.re$study)
datv<-list()
mcfitv<-list() hr.fixed<-
NULL ll.fixed<-NULL
ul.fixed<-NULL hr.random<-
NULL ll.random<-NULL
ul.random<-NULL
   for (i in 1:length(cname)){
   datv[[i]]<-cfit.re[!cfit.re$study %in% cname[i],]
   mcfitv[[i]]<-metagen(TE = coef,
                                seTE  =  se.coef,
                                studlab = study,
                                sm  =  "HR",
                                data  =  datv[[i]])
   # Forest plot
   forest(mcfitv[[i]],smlab = paste("Uni","No",cname[i],sep = "_"), fontsize =8)
   hr.fixed[i]<-exp(mcfitv[[i]]$TE.fixed)
   ll.fixed[i]<-exp(mcfitv[[i]]$TE.fixed-1.96*mcfitv[[i]]$seTE.fixed)
   ul.fixed[i]<-exp(mcfitv[[i]]$TE.fixed+1.96*mcfitv[[i]]$seTE.fixed)
   hr.random[i]<-exp(mcfitv[[i]]$TE.random)
   ll.random[i]<-exp(mcfitv[[i]]$TE.random-1.96*mcfitv[[i]]$seTE.random)
   ul.random[i]<-exp(mcfitv[[i]]$TE.random+1.96*mcfitv[[i]]$seTE.random)
}
```

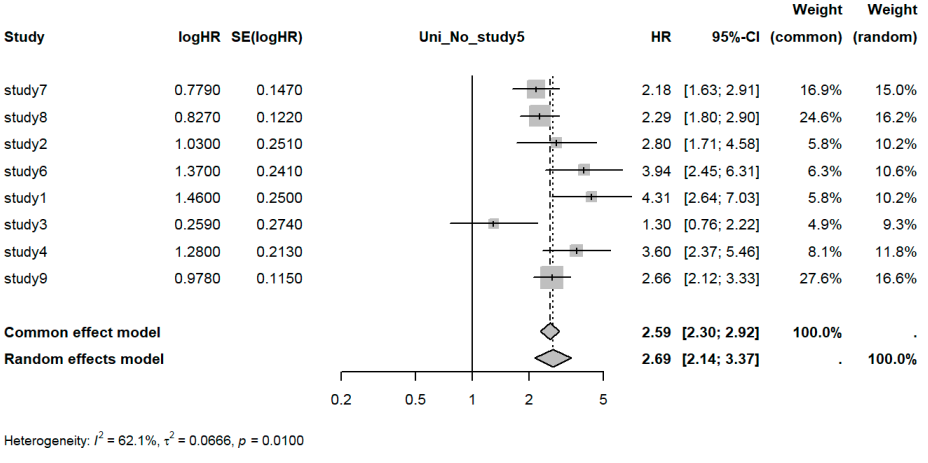


Figure 99. Study left out: study5. Source: Figure by author.

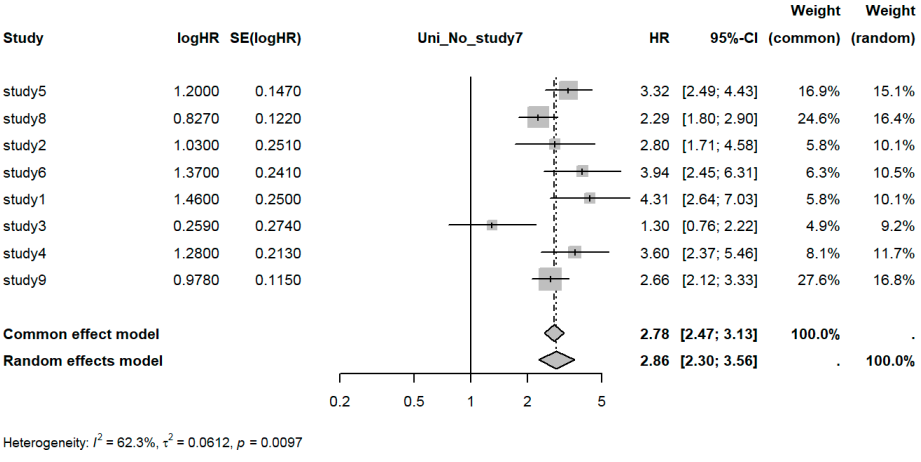


Figure 100. Study left out: study7. Source: Figure by author.

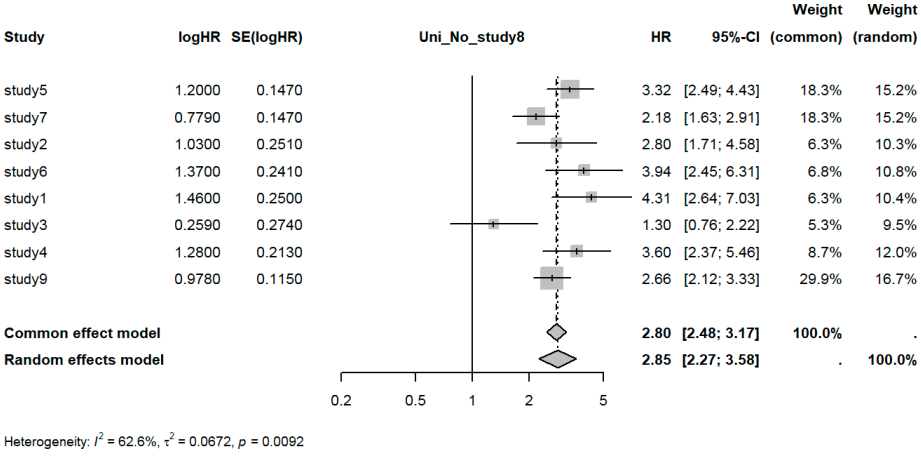


Figure 101. Study left out: study8. Source: Figure by author.

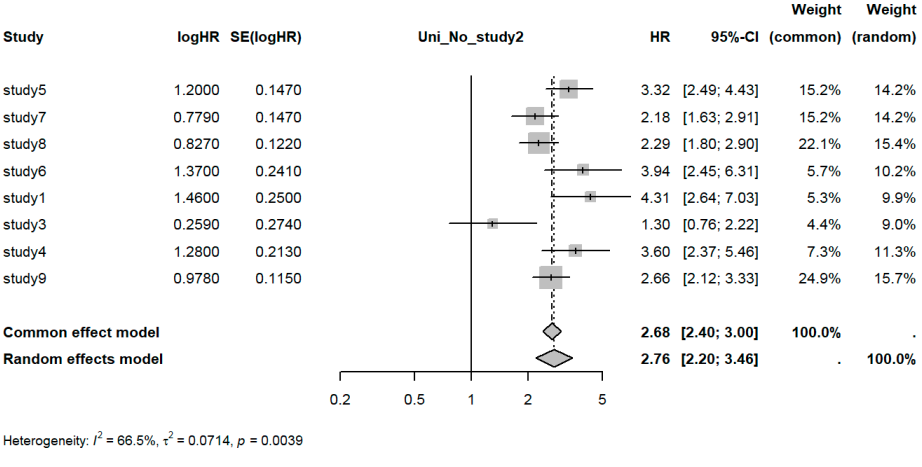


Figure 102. Study left out: study2. Source: Figure by author.

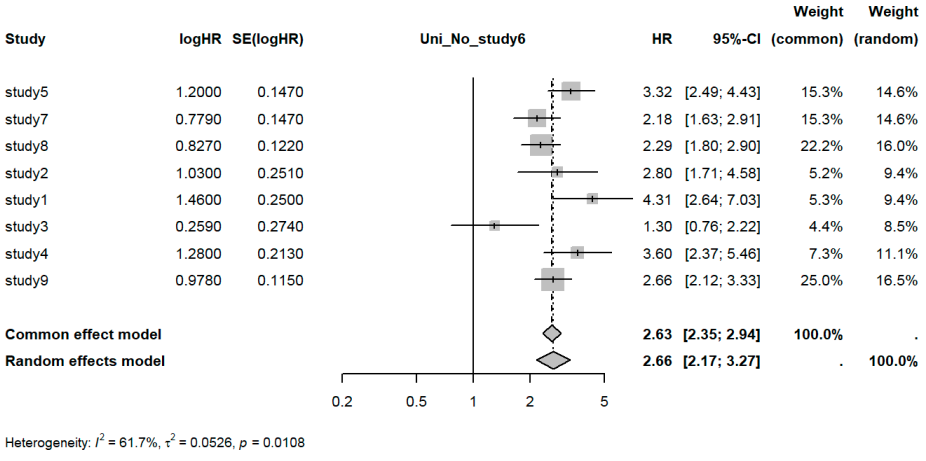


Figure 103. Study left out: study6. Source: Figure by author.

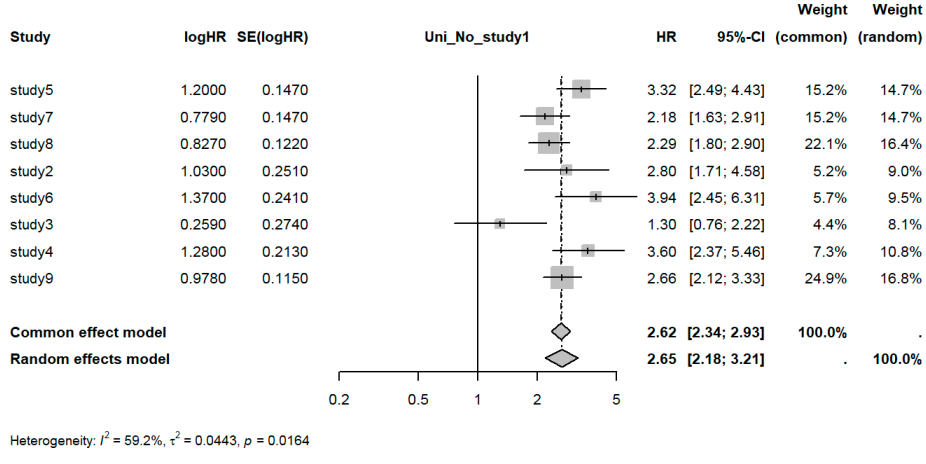


Figure 104. Study left out: study1. Source: Figure by author.

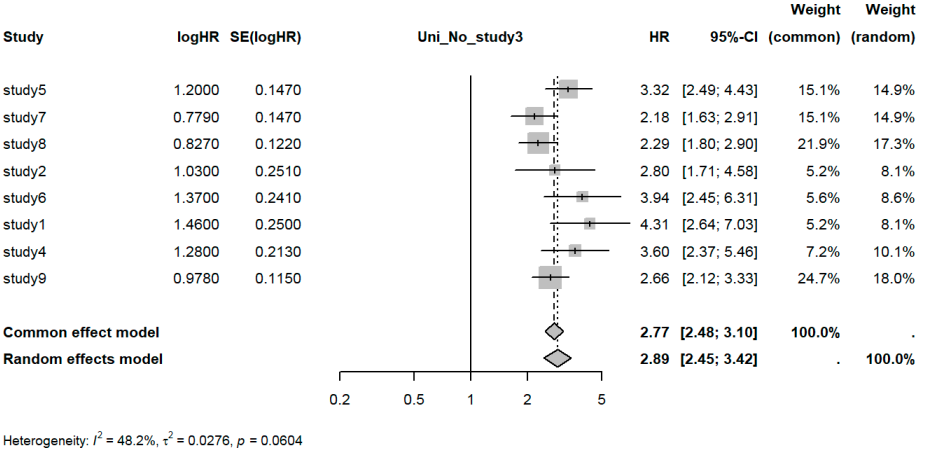


Figure 105. Study left out: study3. Source: Figure by author.

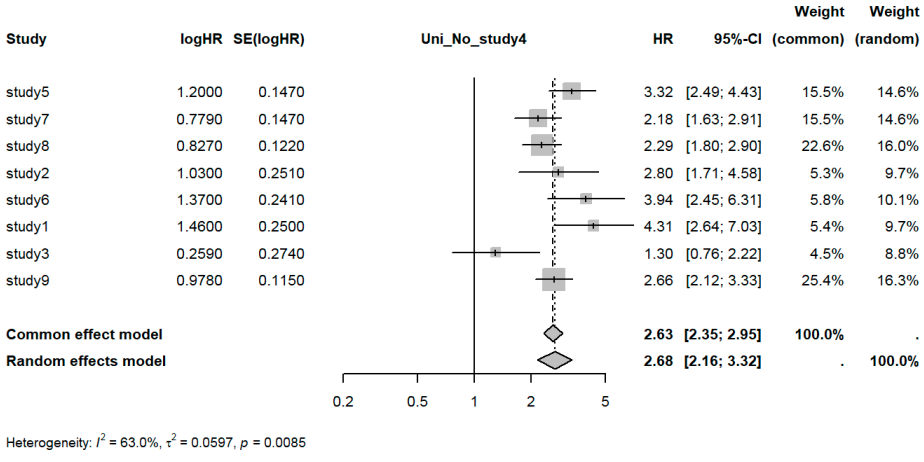


Figure 106. Study left out: study4. Source: Figure by author.

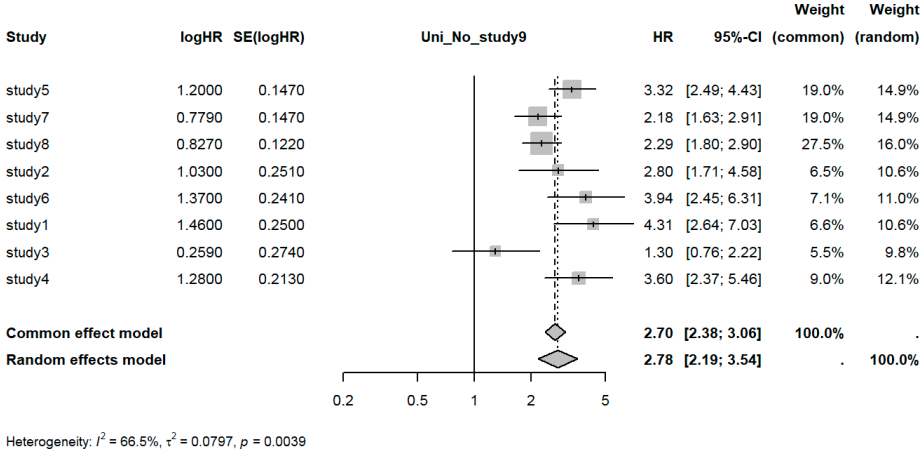


Figure 107. Study left out: study9. Source: Figure by author.

Now, we combine the results from the above leave-one-study-out meta-analysis.

```
v.mat<-as.data.frame(cbind(cname,hr.fixed,ll.fixed,ul.fixed,
                           hr.random,ll.random,ul.random)) v.mat$cname<-
paste("No", v.mat$cname, sep = " ") v.mat[,c("hr.fixed","ll.fixed","ul.fixed",
            "hr.random","ll.random","ul.random")]<-
  lapply(v.mat[,c("hr.fixed","ll.fixed","ul.fixed",
                  "hr.random","ll.random","ul.random")],as.numeric)
v.mat[,c("hr.fixed","ll.fixed","ul.fixed",
            "hr.random","ll.random","ul.random")]<-
  round(v.mat[,c("hr.fixed","ll.fixed","ul.fixed",
                 "hr.random","ll.random","ul.random")],2)
# Reset margins to original settings
par(mar = old_mar)
```

Then, we create a summary plot for leave-one-study-out validation for random-effects meta-analysis. The vertical dash line shows the original meta-analysis results of all studies (Figure 108).

```
ggplot(data = v.mat, aes(x = hr.random,y = cname))+
  geom_point() +
  scale_y_discrete()+
  geom_errorbar(aes(xmin = ll.random,xmax = ul.random))+
  geom_vline(xintercept = exp(mcfit$TE.random),
             linetype = "dashed")+ xlab("Random
  effect HR (95%CI)")+
  ylab("Study left out")+
  xlim(1.8,4)
```

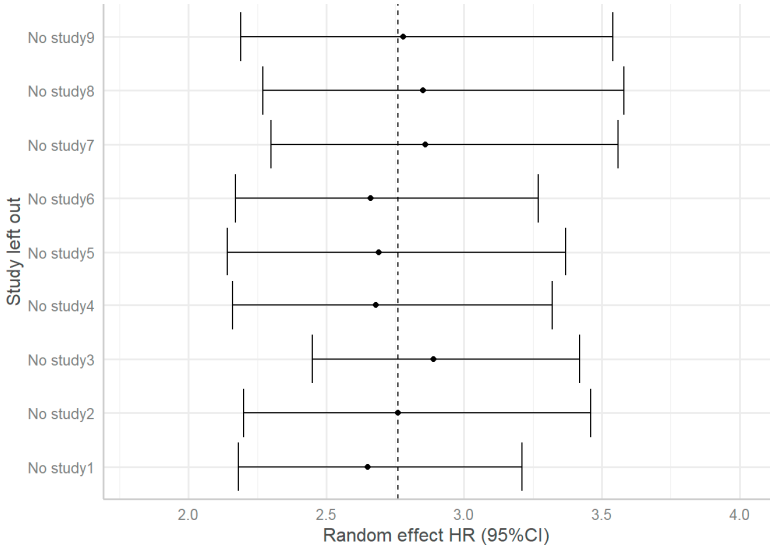


Figure 108. Sensitivity analysis summary: univariable. Source: Figure by author.

The interpretation of a sensitivity plot focuses on the "vertical alignment" of the results around the vertical dashed line of the original meta-analysis results of all studies. We can see the consistency of the results and the stability of the Point Estimate with no "highly influential" study.

Note that one may also try the *metainf()* function from the "*meta*" package [73] for leave-one-study-out sensitivity analysis.

This chapter provides example R codes for analysis and meta-analysis of large consortium data with example R functions and loops for more efficient analysis and result summary. One may apply and elaborate on the example R codes in this chapter for their large, complex data.

8.3. Useful Resources

The paper "Diabetes is associated with increased liver cancer incidence and mortality in adults: A report from Asia Cohort Consortium" was published in the *International Journal of Cancer* by Nhan Thi Ho et al. [68]. One may read this paper for more information about the actual outcome and exposure, actual results, explanation, and interpretation.

8.4. Chapter Summary

- *Tableby()* function of the "*arsenal*" package is handy for data summary;

- *"survival"* and *"survminer"* packages are essential for survival analysis;
- Newly created *subhe()* function may be used for subgroup analysis, heterogeneity test and result summary as a table for consortium data from multiple studies;
- Univariable and multivariable model may be done for each study and results are summarized as a table;
- Loops and functions may be used to make repeated analysis more efficient;
- *"meta"* package is used for meta-analysis across studies;
- Sensitivity analysis (leave one study out validation) to evaluate if any specific study strongly affects the overall results.

9. Analysis of Child Growth Data with R

Child growth data requires special analysis approaches. This chapter will guide through the analyses to generate similar results to those of the paper "Overweight & obesity epidemic, temporal trends and regional disparities in physical growth of Vietnamese children" by Nhan Thi Ho et al., published in *Scientific Reports* [75]. The example anthropometric dataset used in this chapter was generated to mimic the real data used in the paper. The data include height, weight, and BMI of children from 18 months to 18 years measured annually during 7 years. This is a longitudinal dataset with repeated measures of the same subjects. The data will be compared with the World Health Organization (WHO) 2007 child growth reference data [76,77]. WHO 2007 child growth reference data are provided in this chapter as examples.

This chapter provides example R codes for child growth-specific analysis, including summary of anthropometric measures and comparing between groups, plotting study anthropometric data with corresponding WHO reference data for visual comparison, calculating Z-scores for study anthropometric data based on WHO references using both available R packages and newly created R functions dedicated to this work, performing nutritional status classification (e.g., obesity, wasting, and stunting, etc.) based on Z-scores and estimating corresponding prevalence, and constructing growth curves based on the study data using the WHO Lambda-Mu-Sigma (LMS) method. Alongside, there are examples of longitudinal data analysis for temporal trends, as well as examples for writing R functions and loops for more efficient analysis and result display. This chapter focuses on how to do specific analyses and result display using R, not on statistical method explanation or result output interpretation. Interested readers should read the original paper published by the author mentioned above for more details about the intention and purpose of the analysis, methodology description, result description, explanation, and interpretation [75].

9.1. Data Summary

9.1.1. Load Required Packages and Example Data

The main R packages introduced in this chapter include:

- "*anthro*" [78] & "*zscorer*" [79]: Specialized packages that contain functions to easily calculate child growth Z-scores based on WHO standards.
- "*DescTools*" [80] & "*confintr*" [81]: Provide functions to calculate robust confidence intervals for medians (*MedianCI()*) and for the difference between medians (*ci_median_diff()*), which are better for skewed growth data than mean-based statistics.
- "*gamlss*" [82]: The Generalized Additive Models for Location, Scale, and Shape package. We use this for the advanced WHO LMS method to construct growth curves based on study data.
- "*geepack*" [58]: For GEE models to analyze longitudinal trends. "*sjPlot*" [61]: For plotting GEE model results.
- "*ggplot2*": for visualization [18].

Some other supporting R packages (e.g., "*knitr*" [5], "*rmarkdown*" [6], "*kableExtra*" [12], "*tidyverse*" [11], "*rio*" [14], "*arsenal*" [19], "*gridExtra*" [69],

"*hrbrthemes*" [83], "*MASS*" [64], "*viridis*" [84], "*emmeans*" [85], "*ggpubr*" [20], "*ggpmisc*" [74,86], "*patchwork*" [74], and "*stringr*" [87]) will also be used.

```
# load multiple packages
Packages <- c("knitr", "rmarkdown",
              "kableExtra", "tidyverse",
              "rio", "arsenal", "gridExtra",
             "anthro", "zscorer", "hrbrthemes",
             "MASS", "gamlss", "viridis", "confintr",
             "DescTools", "geepack", "emmeans", "sjPlot",
             "ggpubr", "ggpmisc", "patchwork", "stringr")
lapply(Packages, library, character.only = TRUE)

# Data path
url_base <- "https://raw.githubusercontent.com/nhanhocu/"
data_path <- "biodata-r/main/growdat.rda"
data_url <- paste0(url_base, data_path)

# load datasets
con      <- url(data_url) load(con)
close(con)

# quick look at example data
str(dat)
# quick look at WHO 2007 reference data for children 0-5 years old
names(who5)
str(who5[[2]])
# quick look at WHO 2007 reference data for children 5-19 years old
names(who619)
str(who619[[2]])
```

9.1.2. Data Summary by Study Sites

Similar to previous chapters, we use the R package "*arsenal*" for data summary by study sites to have an overview of the data (Table 20) [19].

```
my_controls          <- tableby.control(
    test = TRUE,
    total = TRUE,
    numeric.test = "kwt",
    cat.test = "chisq",
    numeric.stats = c("meansd", "medianq1q3","meanCI","range","Nmiss2"),
    cat.stats = c("countpct", "Nmiss2"),
    stats.labels = list(
        meansd = "Mean (SD)",
        medianq1q3 = "Median (Q1, Q3)",
        range = "Min - Max",
        Nmiss2 = "Missing",
        meanCI = "Mean (95%CI)"
    )
)
svar <-c("age","age.cat","sex","year")
mylabels        <-as.list(svar)
names(mylabels) <-svar
tabs       <- tableby(as.formula(paste("site",paste(svar[!svar %in% "site"],
                                                    collapse="+"), sep="~")),
                      data = dat,
                      control = my_controls)
kable(summary(tabs,labelTranslations = mylabels, text=TRUE),
          caption="Data summary by site")%>%
          kable_styling(latex_options = c("scale_down", "hold_position"))
```

Table 20. Data summary by study site.

	Site1 (N = 30,355)	Site2 (N = 58,191)	Site3 (N = 8791)	Total (N = 97,337)	*p*-Value
age					<0.001
- Mean (SD)	9.372 (3.822)	8.696 (3.917)	9.404 (3.786)	8.971 (3.891)	
- Median (Q1, Q3)	9.200 (6.400, 12.200)	8.400 (5.500, 11.600)	9.400 (6.400, 12.200)	8.800 (5.900, 11.900)	
- Mean (95% CI)	9.372 (9.329, 9.415)	8.696 (8.664, 8.728)	9.404 (9.324, 9.483)	8.971 (8.946, 8.995)	
- Min–Max	1.500–18.300	1.500–18.200	1.500–17.900	1.500–18.300	
- Missing	0	0	0	0	
age.cat					<0.001
- age 0–5 years	4521 (14.9%)	12,323 (21.2%)	1322 (15.0%)	18,166 (18.7%)	
- age 6–11 years	15,445 (50.9%)	29,125 (50.1%)	4380 (49.8%)	48,950 (50.3%)	
- age ≥ 12 years	10,389 (34.2%)	16,743 (28.8%)	3089 (35.1%)	30,221 (31.0%)	
- Missing	0	0	0	0	
sex					<0.001
- Female	15,142 (49.9%)	27,703 (47.6%)	4010 (45.6%)	46,855 (48.1%)	
- Male	15,213 (50.1%)	30,488 (52.4%)	4781 (54.4%)	50,482 (51.9%)	
- Missing	0	0	0	0	
year					<0.001
- 0	2550 (8.4%)	506 (0.9%)	405 (4.6%)	3461 (3.6%)	
- 1	4204 (13.8%)	4200 (7.2%)	738 (8.4%)	9142 (9.4%)	
- 2	3952 (13.0%)	10,652 (18.3%)	1331 (15.1%)	15,935 (16.4%)	
- 4	8570 (28.2%)	14,244 (24.5%)	2760 (31.4%)	25,574 (26.3%)	
- 5	5612 (18.5%)	14,781 (25.4%)	1838 (20.9%)	22,231 (22.8%)	
- 6	5467 (18.0%)	13,808 (23.7%)	1719 (19.6%)	20,994 (21.6%)	
- Missing	0	0	0	0	

Source: Table by author.

9.2. Analysis Approaches

9.2.1. Compare Median of Anthropometric Measure for Age by Gender

Define New Functions for "ggplot2"

To get a general sense of the population, medians (with interquartile range (IQR) or 95% confidence intervals (95% CI)) are used for age by gender while minimizing the influence of extreme outliers (severely wasted or exceptionally large children).

Below is a new function, *iqr()*, to calculate median and range at given quantiles. This function supports easy plotting of the quantiles of choice with "*ggplot2*". The default is to plot median and interquartile (IQR) range (*lower = 0.25, upper = 0.75*).

```
iqr = function(z, lower = 0.25, upper = 0.75) {
      data.frame(
      y = median(z),
      ymin = quantile(z, lower),
      ymax = quantile(z, upper)
   )
}
```

Below is a new function *medci()* to calculate median and 95% confidence interval (95% CI) and support easy plotting the median and 95% CI with "*ggplot2*".

```
medci <-function(z,ci=0.95){
      data.frame(
      y = median(z),
      ymin = MedianCI(z, conf.level = ci, na.rm=TRUE,
                              method="exact")[2],
      ymax = MedianCI(z, conf.level = ci, na.rm=TRUE,
                              method="exact")[3]
   )
}
```

Plot the Median, 95% CI Applying the New Function Between Study Sites

This is one of the steps to examine the difference in growth indices in the population between geographical sites.

The above functions are designed to work with "*ggplot2*"'s *stat_summary()* layer. We now plot the median 95% CI of BMI by age (in years), sex, and study site with "*ggplot2*" and use the function *medci*() created above: *aes(. . ., group = interaction(sex, site))* tells *ggplot()* to create a separate line and summary for each unique combination of sex and site; *stat_summary(fun.data = medci)* calls our *medci()* function for each group at each age point. It uses the returned *y*, *ymin*, and *ymax* values to plot the median (as a point) and its 95% CI (as a vertical error bar) (Figure 109).

```
ggplot(dat,  aes(x=age.y,
                        y=bmi,
                        color=sex,
                        shape=site,
                        group=interaction(sex,site)))  +
   stat_summary(
      fun =  median,
      geom='line') +
   stat_summary(
      fun=median,
      geom='point') +
   stat_summary(
      fun.data = medci)+
   theme_bw()+
   ggtitle("c")+
   theme(plot.title = element_text(face = "bold"))+
   labs(x="Age (year)", y="BMI (kg/m2)",
            color="Sex", shape="Site")
```

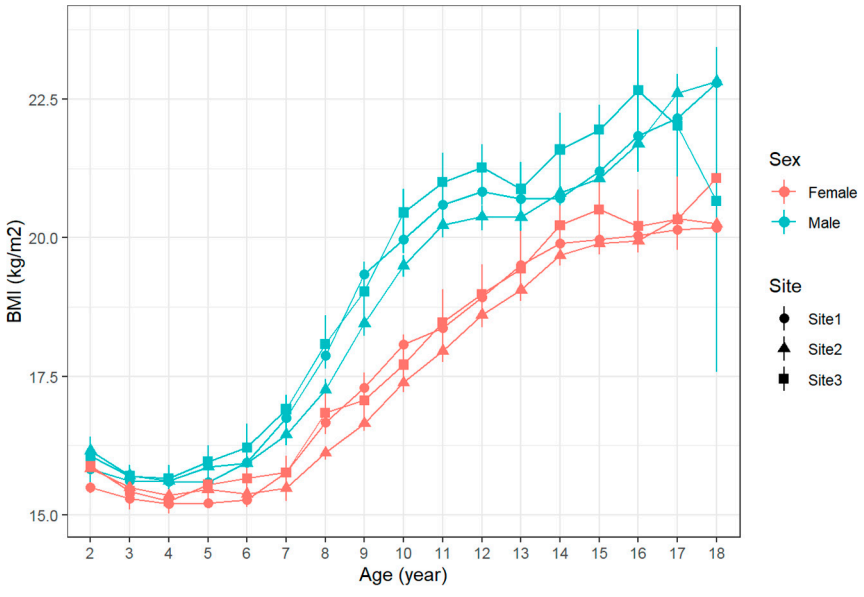


Figure 109. BMI median, 95% CI between study sites. Source: Figure by author.

One may plot median and quantile range using the *iqr()* function above in place of the *medci()* function.

Calculate Median, 95% CI of Height

Median, 95% CI of anthropometric measures may be calculated using the function *MedianCI()* of the "*DescTools*" package [80]. The example below calculates the median and 95% CI of height for males at each age by writing a loop (repeating the calculation for each age) and combining the results into a table (Table 21).

```
datme <-dat[dat$sex %in% "Male",]
hk  <-NULL
for (k in 2:18){
   h <-MedianCI(datme$height[datme$age.y %in% k], conf.level = 0.95, na.rm=TRUE, method="exact")
   hk  <-rbind(hk,h)
   }
age <-as.character(2:18)
hk <-round(hk,2)
hk <-cbind(age, hk)
rownames(hk) <-NULL
hb <- as.data.frame(cbind(age=age, median.ci=paste(hk[,2],"  (",hk[,3],",  ",hk[,4],")", sep="")))
kable(hb, caption="Height median, 95\\%CI of males")%>%
   kable_styling(latex_options  =  c("scale_down",  "hold_position"))
```

Table 21. Height median, 95% CI of males.

Age	Median (95% CI)
2	87 (87, 87.5)
3	95 (95, 95)
4	102.5 (102, 102.6)
5	110 (109.5, 110)
6	117 (117, 117)
7	123.5 (123, 123.6)
8	129.3 (129, 129.5)
9	134.8 (134.5, 135)
10	140 (140, 140.4)
11	146 (145.6, 146)
12	153 (152.6, 153)
13	161 (160.8, 161.4)
14	166.35 (166, 166.7)
15	169.5 (169.1, 170)
16	171.2 (171, 171.8)
17	172.7 (172, 173)
18	172.75 (172, 174)

Source: Table by author.

From the output table, for example, we can see that the median height of males at 18 years old is 172.75, and the 95% CI is 172 to 174.

Calculate Median Difference (95% CI of Difference) Between Sites

In addition to plotting, the actual difference in medians of growth indices between sites may be calculated. The difference between medians and 95% CI of the difference may be calculated by a two-sided 95% bootstrap confidence interval for the population value of median(x)-median(y) based on 999 bootstrap replications and the bias-corrected and accelerated (BCa) method implemented in the function *ci_median_diff()* of the "*confintr*" package [81]. The example below calculates the median height difference between two sites ("Site2" vs. "Site3") and the 95% CI for the median difference ("median.diff.ci"). We write a loop to repeat the calculation for each age and combine the results in a table (Table 22).

```
datd <-subset(datme, site %in% c("Site2","Site3"))
a2 <-sort(unique(datd$age.y[datd$site %in% "Site2"]))
a3 <-sort(unique(datd$age.y[datd$site %in% "Site3"]))
ageu <-sort(a2[a2%in% a3])
hk <-NULL
for (k in ageu){
  h <-ci_median_diff(datd$height[datd$age.y %in% k & datd$site %in% "Site2"],
                     datd$height[datd$age.y %in% k & datd$site %in% "Site3"],
                     probs = c(0.025, 0.975),
                     type = "bootstrap",
                     boot_type = "basic",  #CI by basic bootstrap method
                     R = 999)
  hd <-cbind(median.diff=h$estimate,
             l2.5=h$interval [1],
             l97.5=h$interval[2])
  hk <-rbind(hk,hd)
}
hk <-round(hk,2)
rownames(hk) <-as.character(ageu)
hkb <- as.data.frame(cbind(age=as.character(ageu),
                           median.diff.ci=paste(hk[,1]," (",hk[,2],", ",hk[,3],")",
                                                sep="")))
kable(hkb,
      caption="Height median difference, 95\\%CI of males")%>% kable_styling(latex_options =
  c("scale_down", "hold_position"))
```

Table 22. Height median difference, 95% CI of males.

Age	Median.diff.ci
2	0.25 (−1.3, 1.35)
3	−0.5 (−1.7, −0.05)
4	−0.5 (−1, 0.5)
5	−1 (−1.5, 0)
6	−0.3 (−0.9, 0.2)
7	0.2 (−1.1, 1.4)
8	0.1 (−1.3, 0.2)
9	0.5 (0, 1.6)
10	0.25 (−0.9, 0.8)
11	1 (0.4, 2.5)
12	0.4 (−1.2, 1.4)
13	−0.4 (−1.85, 0.7)
14	−0.25 (−1.7, 0.8)
15	0.5 (−0.7, 1.9)
16	1.3 (0.9, 2.9)
17	1.5 (0.65, 3.9)
18	−0.5 (−4.8, 4.7)

Source: Table by author.

From the output table, for example, the median height of males at 17 years old in Site 2 is 1.5 cm taller than that in Site 3, and this is statistically significant (95% CI = (0.65, 3.9)).

9.2.2. Temporal Trend

Growth indices may change over calendar years due to many reasons, such as environment and nutrition, etc. For example, height for age has increased largely in Japanese or Korean as compared to decades ago. Therefore, examining temporal trends over calendar years to evaluate population trends over time is important. As children are measured repeatedly, the correlation between measurements should be accounted.

Temporal trends over calendar years may be evaluated using the Generalized Estimating Equation (GEE) model with *geeglm()* of the "*geepack*" package, which evaluates average slope and allows accounting for repeated measures within subjects [58]. The correlation structure is selected based on the smallest Quasi Information Criterion (QIC). First, we fit the GEE model to evaluate how child height by age is changing over calendar year ("yearvisit"). The model includes "height" as the dependent variable, "yearvisit", "age.y", and the interaction term between "yearvisit" and "age.y" (*age.y*yearvisit*) as independent variables. With the interaction term, we want to examine if the temporal trend ("yearvisit" slope) is different between the child's age group ("age.y"). The example GEE model below uses an "*independent*" correlation structure. Then we use the function *QIC()* to evaluate the QIC of the model.

```
datme$age.y <-droplevels(datme$age.y)
ghm <-geeglm(height ~ yearvisit +age.y+age.y*yearvisit,
                    corstr = "independence",
                    id=cid, data=datme)
summary(ghm)
```

```
##
## Call:
## geeglm(formula = height ~ yearvisit + age.y + age.y * yearvisit,
##                         data = datme, id = cid, corstr = "independence")
##
## Coefficients:
##                  Estimate      Std.err      Wald          Pr(>|W|)
## (Intercept)      88.6409       0.2473       1.28×10^5     <2×10^-16 ***
## yearvisit        −0.4663       0.0636       5.37×10       2.4×10^-13 ***
## age.y3           7.3611        0.3045       5.85×10^2     <2×10^-16 ***
## age.y4           14.8465       0.3053       2.37×10^3     <2×10^-16 ***
## age.y5           21.9698       0.3279       4.49×10^3     <2×10^-16 ***
## age.y6           29.2578       0.3129       8.75×10^3     <2×10^-16 ***
## age.y7           35.5397       0.3199       1.23×10^4     <2×10^-16 ***
## age.y8           41.6244       0.3207       1.68×10^4     <2×10^-16 ***
## age.y9           46.5552       0.3481       1.79×10^4     <2×10^-16 ***
## age.y10          51.6295       0.3512       2.16×10^4     <2×10^-16 ***
## age.y11          57.3249       0.3785       2.29×10^4     <2×10^-16 ***
## age.y12          63.3965       0.4291       2.18×10^4     <2×10^-16 ***
## age.y13          71.2617       0.4646       2.35×10^4     <2×10^-16 ***
## age.y14          76.7558       0.4397       3.05×10^4     <2×10^-16 ***
## age.y15          80.6519       0.5054       2.55×10^4     <2×10^-16 ***
## age.y16          82.6985       0.5720       2.09×10^4     <2×10^-16 ***
## age.y17          83.2308       0.7019       1.41×10^4     <2×10^-16 ***
## age.y18          83.0227       1.4546       3.26×10^3     <2×10^-16 ***
```

```
## yearvisit:age.y3          0.2112          0.0782          7.30          0.00690 **
## yearvisit:age.y4          0.2111          0.0764          7.63          0.00574 **
## yearvisit:age.y5          0.2434          0.0816          8.90          0.00285 **
## yearvisit:age.y6          0.2253          0.0794          8.05          0.00456 **
## yearvisit:age.y7          0.2417          0.0801          9.11          0.00255 **
## yearvisit:age.y8          0.1850          0.0806          5.27          0.02168 *
## yearvisit:age.y9          0.3752          0.0840          1.99×10          8.0×10-06 ***
## yearvisit:age.y10         0.4814          0.0851          3.20×10          1.5×10-08 ***
## yearvisit:age.y11         0.5269          0.0912          3.34×10          7.6×10-09 ***
## yearvisit:age.y12         0.7030          0.1002          4.93×10          2.2×10-12 ***
## yearvisit:age.y13         0.6274          0.1087          3.33×10          7.9×10-09 ***
## yearvisit:age.y14         0.6022          0.1024          3.46×10          4.1×10-09 ***
## yearvisit:age.y15         0.5010          0.1143          1.92×10          1.2×10-05 ***
## yearvisit:age.y16         0.4676          0.1242          1.42×10          0.00017 ***
## yearvisit:age.y17         0.6111          0.1442          1.80×10          2.3×10-05 ***
## yearvisit:age.y18         0.7037          0.2848          6.11          0.01348 *
## ---
## Signif. codes: 0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1
##
## Correlation structure = independence
## Estimated Scale Parameters:
##
##                          Estimate          Std.err
## (Intercept)              35.9              0.574
## Number of clusters:      16,863            Maximum cluster size: 10
```

```
# Examine QIC
QIC(ghm)
```

```
##              QIC           QICu      Quasi Lik    CIC    params          QICC
##    1,804,358.3    1,804,356.1    −902,144.0    35.1      34.0    1,804,358.4
```

One may refit the GEE model with another correlation structure and compare the QIC between GEE models to select the correlation structure with the smallest QIC. In this case, the “independence” structure has the smallest QIC, and thus it is chosen.

Next, we get slopes and 95% CI for trend over years by age using *lstrends()* and display results in a table (Table 23).

```
myModelSlopes <- lstrends(ghm, "age.y", var="yearvisit") kable(myModelSlopes,
caption="Slopes raw table")%>%
  kable_styling(latex_options = c("scale_down", "hold_position"))
```

Table 23. Slopes raw table.

Age.y	Yearvisit.Trend	SE	df	Lower.CL	Upper.CL
2	−0.466	0.064	50,247	−0.591	−0.342
3	−0.255	0.060	50,247	−0.372	−0.138
4	−0.255	0.047	50,247	−0.347	−0.163
5	−0.223	0.050	50,247	−0.321	−0.125
6	−0.241	0.045	50,247	−0.329	−0.153
7	−0.225	0.046	50,247	−0.316	−0.134
8	−0.281	0.046	50,247	−0.372	−0.191
9	−0.091	0.055	50,247	−0.198	0.016
10	0.015	0.056	50,247	−0.096	0.126
11	0.061	0.065	50,247	−0.067	0.189
12	0.237	0.077	50,247	0.085	0.388
13	0.161	0.088	50,247	−0.012	0.334
14	0.136	0.080	50,247	−0.021	0.293
15	0.035	0.095	50,247	−0.151	0.221
16	0.001	0.107	50,247	−0.208	0.210
17	0.145	0.129	50,247	−0.109	0.398
18	0.237	0.278	50,247	−0.307	0.781

Source: Table by author.

The output table shows the slope for the temporal trend for height for each age in years from 2 years old to 18 years old. There seem to be increasing trends (positive slopes) from calendar year 0 to calendar year 6 for ages 10 to 18. However, most of these trend slopes are not statistically significant (95% CI includes 0). Whereas, the slopes for age from 2 years old to 9 years old show significant decreasing trends (negative slopes and 95% CI do not include 0).

Then we can create a clean table containing only the useful results that we want to display (slopes and 95% CI) (Table 24).

```
sld <-as.data.frame(myModelSlopes)
sld[,c(2,3,5,6)] <-round(sld[,c(2,3,5,6)],2)
slc.hm  <-as.data.frame(cbind(Age  =  as.character(sld[,"age.y"]),
                              Slope.95CI = paste(sld[,"yearvisit.trend"], " (",
                                                 sld[,"lower.CL"], "; ",
                                                 sld[,"upper.CL"], ")",
                                                 sep=""")))
kable(slc.hm,
      caption="GEE slopes and 95\\%CI")%>%
      kable_styling(latex_options  =  c("scale_down",  "hold_position"))
```

Table 24. GEE slopes and 95% CI.

Age	Slope.95CI
2	−0.47 (−0.59; −0.34)
3	−0.26 (−0.37; −0.14)
4	−0.26 (−0.35; −0.16)
5	−0.22 (−0.32; −0.13)
6	−0.24 (−0.33; −0.15)

Table 24. *Cont.*

Age	Slope.95CI
7	−0.22 (−0.32; −0.13)
8	−0.28 (−0.37; −0.19)
9	−0.09 (−0.2; 0.02)
10	0.02 (−0.1; 0.13)
11	0.06 (−0.07; 0.19)
12	0.24 (0.09; 0.39)
13	0.16 (−0.01; 0.33)
14	0.14 (−0.02; 0.29)
15	0.03 (−0.15; 0.22)
16	0 (−0.21; 0.21)
17	0.14 (−0.11; 0.4)
18	0.24 (−0.31; 0.78)

Source: Table by author.

Next, we can turn the table above into a plot object (using *geom* = *"table"*) to combine with another plot object later in *"ggplot2"* (Figure 110):

```
gg.slc.hm <- ggplot() + theme_void() +
  annotate(geom = "table",
           x = 1,
           y = 1,
           label = list(slc.hm))
gg.slc.hm
```

Age	Slope.95CI
2	-0.47 (-0.59; -0.34)
3	-0.26 (-0.37; -0.14)
4	-0.26 (-0.35; -0.16)
5	-0.22 (-0.32; -0.13)
6	-0.24 (-0.33; -0.15)
7	-0.22 (-0.32; -0.13)
8	-0.28 (-0.37; -0.19)
9	-0.09 (-0.2; 0.02)
10	0.02 (-0.1; 0.13)
11	0.06 (-0.07; 0.19)
12	0.24 (0.09; 0.39)
13	0.16 (-0.01; 0.33)
14	0.14 (-0.02; 0.29)
15	0.03 (-0.15; 0.22)
16	0 (-0.21; 0.21)
17	0.14 (-0.11; 0.4)
18	0.24 (-0.31; 0.78)

Figure 110. GEE slopes and 95% CI. Source: Figure by author.

Next, we visualize the trend slopes and 95% CI with the function *plot_model()* of the *sjPlot* package (Figure 111) [61].

```
gee.hm <-plot_model(ghm, type="int",
                    show.data=TRUE,
                    show.values = TRUE,
                    show.p = TRUE,
                    axis.title =c("Year","Height (cm)"),
                    title = "Males",
                    legend.title = "Age (year)",
                    colors = as.character(seq(2,18,by=1)))
gee.hm
```

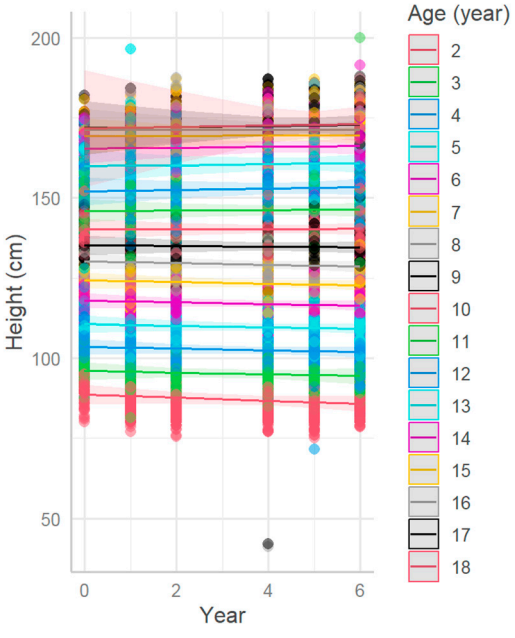
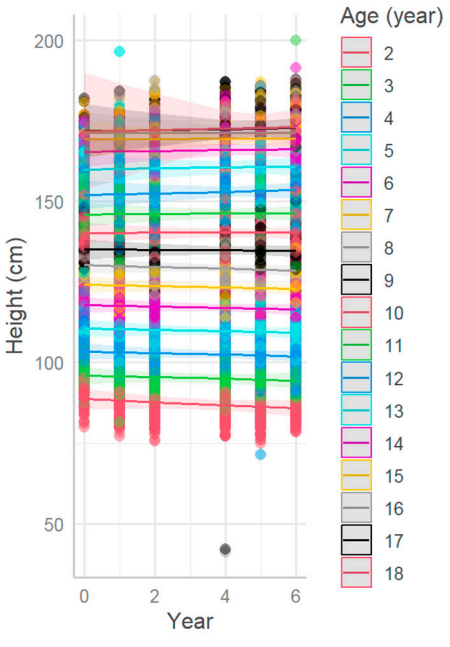


Figure 111. Trend slopes, 95% CI for males. Source: Figure by author.

Then, we can combine the table plot object above with the slope plot object into one plot using *ggarrange()*. This is a good way to combine multiple plot panels and result tables into a publication-ready compact figure. Readers may consult the original paper for the actual multi-panel figure combining many plot objects and table objects (Figure 112) [75].

```
gee_hwb <-ggarrange(gee.hm, gg.slc.hm,
                    ncol=2, nrow=1, legend="right")
gee_hwb
```



Age	Slope.95CI
2	-0.47 (-0.59; -0.34)
3	-0.26 (-0.37; -0.14)
4	-0.26 (-0.35; -0.16)
5	-0.22 (-0.32; -0.13)
6	-0.24 (-0.33; -0.15)
7	-0.22 (-0.32; -0.13)
8	-0.28 (-0.37; -0.19)
9	-0.09 (-0.2; 0.02)
10	0.02 (-0.1; 0.13)
11	0.06 (-0.07; 0.19)
12	0.24 (0.09; 0.39)
13	0.16 (-0.01; 0.33)
14	0.14 (-0.02; 0.29)
15	0.03 (-0.15; 0.22)
16	0 (-0.21; 0.21)
17	0.14 (-0.11; 0.4)
18	0.24 (-0.31; 0.78)

Figure 112. Trend slopes, 95% CI combined for males. Source: Figure by author.

Note that for longitudinal growth trajectory of individuals, linear mixed-effect models or other models such as polynomial mixed-effects models or spline-based mixed-effects models may be used. This chapter is not intended to discuss these methods.

9.2.3. Comparison with WHO 2007 Growth References

The World Health Organization (WHO) 2007 Growth References were developed by the World Health Organization to describe normal growth patterns for children $\leq$ 5 years old and adolescents aged 5–19 years. These references are widely used in clinical practice and public health research to evaluate whether a child's growth trajectory falls within the expected range for their age and sex.

Comparing the growth measurements in a study dataset with these international standards allows researchers to determine whether children in the study population are growing normally or whether deviations from the reference distribution are present. Such deviations may indicate nutritional issues that may affect child development.

Thus, the objective of this section is to demonstrate how anthropometric measurements in a dataset can be evaluated in comparison with the WHO 2007 reference standards using R.

Plot Study Data Against WHO References

First, we set the color code, categories for height, weight, BMI, and nutritional status classification:

```
colors2 <-c("Study median" = "red", "WHO median"="green",
                    "+4SD"="blue",
                    "+3SD"="blue",
                    "+2SD"="blue",
                    "+1SD"="blue",
                    "-1SD"="purple",
                    "-2SD"="purple",
                    "-3SD"="purple",
                    "-4SD"="purple")
colors2i <-c("Observed" = "red",
                    "WHO median"="green","+4SD"="blue",
                    "+3SD"="blue",
                    "+2SD"="blue",
                    "+1SD"="blue",
                    "-1SD"="purple",
                    "-2SD"="purple",
                    "-3SD"="purple",
                    "-4SD"="purple")
 hl <-c("height >+4SD",
            "+3SD < height~<~=+4SD",
            "+2SD < height~<~=+3SD",
            "+1SD < height~<~=+2SD",
            "-1SD <= height~<~=+1SD",
            "-2SD <= height~<~-1SD",
            "-3SD <= height~<~-2SD",
            "-4SD <= height~<~-3SD", "height~<~-
            4SD")
 wl <-c("weight >+4SD",
            "+3SD < weight~<~=+4SD",
            "+2SD < weight~<~=+3SD",
            "+1SD < weight~<~=+2SD",
            "-1SD <= weight~<~=+1SD",
            "-2SD <= weight~<~-1SD",
            "-3SD <= weight~<~-2SD",
            "-4SD <= weight~<~-3SD", "weight~<~-
            4SD")
 bl <-c("BMI >+4SD",
            "+3SD < BMI~<~=+4SD",
            "+2SD < BMI~<~=+3SD",
            "+1SD < BMI~<~=+2SD",
            "-1SD <= BMI~<~=+1SD",
            "-2SD <= BMI~<~-1SD",
            "-3SD <= BMI~<~-2SD",
            "-4SD <= BMI~<~-3SD",
            "BMI~<~-4SD")
 whl <-c("WFH >+4SD",
              "+3SD < WFH~<~=+4SD",
              "+2SD < WFH~<~=+3SD",
              "+1SD < WFH~<~=+2SD",
              "-1SD <= WFH~<~=+1SD",
              "-2SD <= WFH~<~-1SD",
              "-3SD <= WFH~<~-2SD",
              "-4SD <= WFH~<~-3SD", "WFH~<~-
              4SD")
 stl <-c("Extreme tallness","Normal HFA","Stunting")
 w5l <-c("Obesity","Overweight","Normal WFH","Wasting")
 t18L <-c("Obesity","Overweight","Normal BMI","Thinness")
```

WHO growth reference data are provided for $\leq$5 years old and $\geq$5 to 19 years old. Thus, in the example below, we get data for boys $\leq$ 5 years old from the total study dataset, get WHO reference height for age data for boys $\leq$ 5, and summarize by age in months:

```
dat05b <-dat[dat$age <=5 & dat$sex %in% "Male",]
hfa.boy.z.05 <-who5$`lhfa-boys-zscore-expanded-tables`
hfa.boy.z.05$age.m <-round(hfa.boy.z.05$day/30,0)
hfa.boy.z.05.m <-hfa.boy.z.05%>%
  group_by(age.m)%>%
  summarise(across(everything(),  median))
```

Then, we plot the study data median, 95% CI, and actual data points against the WHO median and standard deviation (SD) using our new function *medci()* and the color code created previously. This is to evaluate the overall study population data in comparison with the WHO references (Figure 113).

```
who.5hm <-ggplot() +
  geom_point(dat05b, mapping=aes(x=age.m, y=height), size=0.1)+
  stat_summary(dat05b, mapping=aes(x=age.m, y=height,
                                                    color="Study median"),
               fun = median,
               geom='line') +
  stat_summary(dat05b, mapping=aes(x=age.m, y=height), fun=median,
               geom='point', colour="red") +
  stat_summary(dat05b, mapping=aes(x=age.m, y=height),
               fun.data=medci, geom='errorbar',
               width=0.2, colour="red") +
  geom_line(hfa.boy.z.05.m, mapping=aes(x=age.m, y=sd0, color="WHO median"))+
  geom_line(hfa.boy.z.05.m, mapping=aes(x=age.m, y=sd1, color="+1SD"))+
  geom_line(hfa.boy.z.05.m, mapping=aes(x=age.m, y=sd2, color="+2SD"))+
  geom_line(hfa.boy.z.05.m, mapping=aes(x=age.m, y=sd3, color="+3SD"))+
  geom_line(hfa.boy.z.05.m, mapping=aes(x=age.m, y=sd4, color="+4SD"))+
  geom_line(hfa.boy.z.05.m, mapping=aes(x=age.m, y=sd1neg, color="-1SD"))+
  geom_line(hfa.boy.z.05.m, mapping=aes(x=age.m, y=sd2neg, color="-2SD"))+
  geom_line(hfa.boy.z.05.m, mapping=aes(x=age.m, y=sd3neg, color="-3SD"))+
  geom_line(hfa.boy.z.05.m, mapping=aes(x=age.m, y=sd4neg, color="-4SD"))+
  theme_bw()+
  scale_color_manual(name = "", values = colors2,
                                    breaks = names(colors2)[c(1:10)])+
  theme_bw()+
  labs(x = "Age (month)", y="Height (cm)")+
  coord_cartesian(xlim = c(0, 60), ylim=c(30,150))+
  scale_x_discrete(limits =seq(0, 60, by = 6))+
  ggtitle("Males, <= 5 years old")
  who.5hm
```

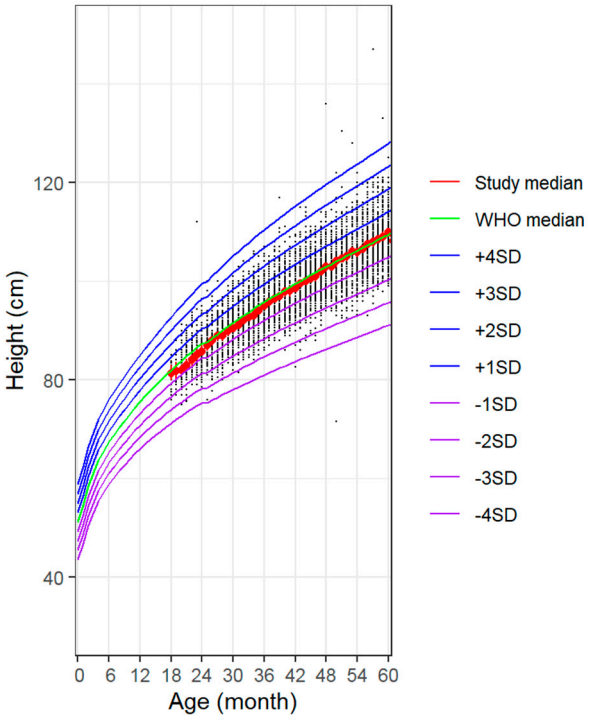


Figure 113. Study data height of males $\leq$ 5 years old vs. WHO references. Source: Figure by author.

The output plot shows that the study population median is relatively consistent with the WHO reference median.

Plot Individual Data Against WHO References

We can plot to examine the growth trajectory of an individual as compared to the WHO reference. For example, we plot the data of a child with "cid" ="c23782". First, we get the data of that child, then we plot that child's data on top of the WHO reference lines, similarly to the above (Figure 114).

```
# get an individual data
datin <-dat05b[dat05b$cid %in% "c23782",]
who.inh <- ggplot()+
   geom_point(datin, mapping=aes(x = age.m, y = height,
                                 group = cid,
                                 colour="Observed"))+
    geom_line(datin, mapping=aes(x = age.m, y = height,
                                 group = cid, colour="Observed"),
                                 size=0.3)+
    geom_line(hfa.boy.z.05.m, mapping=aes(x=age.m, y=sd0, color="WHO median"))+
    geom_line(hfa.boy.z.05.m, mapping=aes(x=age.m, y=sd1, color="+1SD"))+
    geom_line(hfa.boy.z.05.m, mapping=aes(x=age.m, y=sd2, color="+2SD"))+
    geom_line(hfa.boy.z.05.m, mapping=aes(x=age.m, y=sd3, color="+3SD"))+
    geom_line(hfa.boy.z.05.m, mapping=aes(x=age.m, y=sd4, color="+4SD"))+
    geom_line(hfa.boy.z.05.m, mapping=aes(x=age.m, y=sd1neg, color="-1SD"))+
    geom_line(hfa.boy.z.05.m, mapping=aes(x=age.m, y=sd2neg, color="-2SD"))+
```

```
    geom_line(hfa.boy.z.05.m, mapping=aes(x=age.m, y=sd3neg, color="-3SD"))+
    geom_line(hfa.boy.z.05.m, mapping=aes(x=age.m, y=sd4neg, color="-4SD"))+
    theme_bw()+
    scale_color_manual(name = "", values = colors2i,
                                    breaks = names(colors2i)[c(1:10)])+
    theme_bw()+
    labs(x = "Age (month)", y="Height (cm)")+
    coord_cartesian(xlim = c(0, 60), ylim=c(30,150))+
    scale_x_discrete(limits =seq(0, 60, by = 6))+
    ggtitle("Individual vs. WHO")
who.inh
```

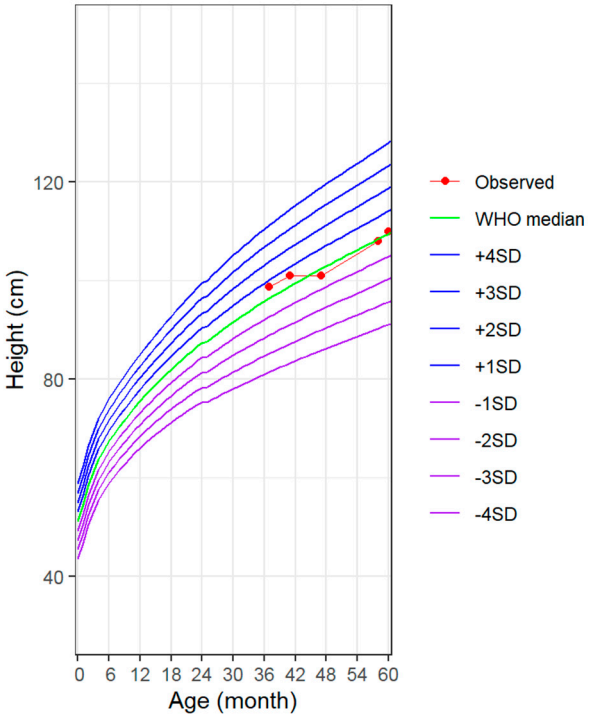


Figure 114. Individual height vs. WHO references. Source: Figure by author.

The output shows that the height growth of this child is quite consistent with the WHO median.

Nutritional Status Classification According to WHO References

One way to do this is to merge study data and WHO reference data by age in months, then create a new height classification variable (using *case_when()* of "*tidyverse*") [11]. This is to classify the growth measures into categories according to the WHO reference SD, for example, between −1SD to +1SD, >+1 SD, >+2 SD, and <−2SD, etc. These are used to classify nutritional status of the child, e.g., stunting if height-for-age < −2SD, obesity of BMI-for-age > +2SD, etc.

```
dat05b.h.o <-merge(dat05b, hfa.boy.z.05.m, by="age.m", all.x=TRUE)
dat05b.h.o <- dat05b.h.o %>%
  mutate(
    height.cat = case_when(height>=sd1neg & height <=sd1 ~ "-1SD <= height <= +1SD",
                           height>sd1 & height <= sd2 ~ "+1SD < height <=+2SD",
                           height>sd2 & height <= sd3 ~ "+2SD < height <=+3SD",
                           height>sd3 & height <= sd4 ~ "+3SD < height <=+4SD",
                           height>sd4 ~"height >+4SD",
                           height>=sd2neg & height < sd1neg ~ "-2SD <= height < -1SD",
                           height>=sd3neg & height < sd2neg ~ "-3SD <= height < -2SD",
                           height>=sd4neg & height < sd3neg ~ "-4SD <= height < -3SD",
                           height < sd4neg ~"height < -4SD"))
dat05b.h.o$height.cat <-factor(dat05b.h.o$height.cat, levels=hl)
tbl <- table(dat05b.h.o$height.cat)
```

Then we can calculate the prevalence and 95% CI for the prevalence of height classification using Glaz & Sison's method for simultaneous confidence intervals for multinomial proportions using the function *MultinomCI()* of the *"DescTools"* package [80] and display the results in a table (Table 25).

```
pci <-round(MultinomCI(tbl,
                       conf.level=0.95, method="sisonglaz"),5)
tblb <- as.data.frame(cbind(count=tbl, p.95ci=paste(pci[,1], " (",
                                                    pci[,2], ", ",
                                                    pci[,3], ")",
                                                    sep="")))
kable(tblb,
      caption="Height classification prevalence and 95\\%CI")%>%
      kable_styling(latex_options = c("scale_down", "hold_position"))
```

Table 25. Height classification prevalence and 95% CI.

	Count	**Prevalence (95% CI)**
height > +4SD	10	0.00104 (0, 0.01044)
+3SD < height $\leq$ +4SD	25	0.00261 (0, 0.012)
+2SD < height $\leq$ +3SD	208	0.02171 (0.01242, 0.03111)
+1SD < height $\leq$ +2SD	1025	0.10699 (0.0977, 0.11639)
$-$1SD $\leq$ height $\leq$ +1SD	6486	0.67704 (0.66775, 0.68643)
$-$2SD $\leq$ height < $-$1SD	1559	0.16273 (0.15344, 0.17213)
$-$3SD $\leq$ height < $-$2SD	245	0.02557 (0.01628, 0.03497)
$-$4SD $\leq$ height < $-$3SD	20	0.00209 (0, 0.01148)
height < $-$4SD	2	0.00021 (0, 0.0096)

Source: Table by author.

We can classify height into nutritional status categories following WHO references and calculate the prevalence and 95% CI of these categories (e.g., stunting) (Table 26).

```
dat05b.h.o <- dat05b.h.o %>%
   mutate(
      height.cats = case_when(height> sd3 ~"Extreme tallness",
                              height>=sd2neg & height <= sd3 ~"Normal HFA",
                              height  <  sd2neg  ~"Stunting"))
dat05b.h.o$height.cats  <-factor(dat05b.h.o$height.cats,  levels=stl)
tbl <- table(dat05b.h.o$height.cats)
pci  <-round(MultinomCI(tbl,
                        conf.level=0.95, method="sisonglaz"),5)
tblb <- as.data.frame(cbind(count=tbl, p.95ci=paste(pci[,1],
                                                    " (",
                                                    pci[,2],
                                                    ", ",
                                                    pci[,3],
                                                    ")",
                                                    sep="")))
kable(tblb,
      caption="Prevalence, 95\\%CI of stunting")%>%
      kable_styling(latex_options = c("scale_down", "hold_position"))
```

Table 26. Prevalence, 95% CI of stunting.

	Count	**Prevalence (95% CI)**
Extreme tallness	35	0.00365 (0.00042, 0.00702)
Normal HFA	9278	0.96848 (0.96524, 0.97184)
Stunting	267	0.02787 (0.02463, 0.03124)

Source: Table by author.

The prevalence and 95% CI for BMI, weight, height classification as overweight, obesity, wasting or thinness, and stunting may be done similarly.

9.2.4. Calculate Z-Scores and Prevalence with "Anthro" Package

It is often more handy to calculate standardized Z-scores for growth indicators based on WHO reference distributions and use these Z-scores for nutritional classification or further analysis.

One may use the "*anthro*" package [78] to make it easier to standardize the measurements of height, weight, and BMI into Z-scores (or Standard Deviation Scores) by comparing them to WHO references. Then one may classify the Z-scores as overweight, obesity, wasting, or thinness following the WHO recommendation and calculate their prevalence.

Calculate Z-Scores for a Dataset

We can calculate Z-scores for all subjects and many anthropometric indices in a dataset at a time. The *anthro_zscores()* function requires age in days; '*lenhei*' is the column for length/height, and '*weight*' is for weight. Note that the package "*anthro*" calculates Z-scores for age $\leq$ 5 years old and may produce NA for age > 5 years old. It is important that age should be in days and sex should be numeric, with *1 = male* and *2 = female*.

```
#  turn sex to numeric
dat <- dat %>%
   mutate(sex_numeric = case_when(
          sex == "Male" ~ 1,
          sex == "Female" ~ 2,
          TRUE ~ NA_real_ # Handles any other cases
      )
   )
# get data for age~<~=5
dat5 <-dat[dat$age.d <=365*5,]
z5 <-with(dat5,
      anthro_zscores(
      sex = sex_numeric,
      age = age.d,
      weight = weight,
      lenhei = height
   )
)
str(z5)
```

```
## 'data.frame':    17,763 obs. of 20 variables:
## $ clenhei        : num 112 106 104 106 102 ...
## $ cbmi           : num 17.5 16 18.7 18.2 17.5 ...
## $ cmeasure       : chr NA NA NA NA ...
## $ csex           : int 1 2 2 2 2 2 1 1 2 2 ...
## $ zlen           : num 0.52 −0.45 −0.91 −0.59 −0.83 0.08 −0.63 −0.88 −0.1 1.39 ...
## $ flen           : int 0 0 0 0 0 0 0 0 0 0 ...
## $ zwei           : num 1.37 0.08 0.83 0.86 0.46 1.32 −0.78 −0.34 0.05 0.93 ...
## $ fwei           : int 0 0 0 0 0 0 0 0 0 0 ...
## $ zwfl           : num 1.46 0.53 2.15 1.88 1.49 1.91 −0.68 0.3 0.14 −0.03 ...
## $ fwfl           : int 0 0 0 0 0 0 0 0 0 0 ...
## $ zbmi           : num 1.58 0.49 1.97 1.74 1.39 1.86 −0.62 0.34 0.14 0.21 ...
## $ fbmi           : int 0 0 0 0 0 0 0 0 0 0 ...
## $ zhc            : num NA NA NA NA NA NA NA NA NA NA ...
## $ fhc            : int NA NA NA NA NA NA NA NA NA NA ...
## $ zac            : num NA NA NA NA NA NA NA NA NA NA ...
## $ fac            : int NA NA NA NA NA NA NA NA NA NA ...
## $ zts            : num NA NA NA NA NA NA NA NA NA NA ...
## $ fts            : int NA NA NA NA NA NA NA NA NA NA ...
## $ zss            : num NA NA NA NA NA NA NA NA NA NA ...
## $ fss            : int NA NA NA NA NA NA NA NA NA NA ...
```

```
# To add the z-scores to study dataset
dz5  <-as_tibble(dat5)  %>%
   drop_na() %>% # needs to have no NA values
   mutate(anthro_zscores(sex = sex_numeric,
                                    age = age.d,
                                    is_age_in_month = FALSE,
                                          weight = weight,
                                          lenhei = height))
    str(dz5)
```

```
## tibble [17,419 x 35] (S3: tbl_df/tbl/data.frame)
## $ cid            : Factor w/ 88678 levels "c1","c2","c3",..: 828 1230 1684 1726 1726 2132 2694 2845
                      3077 3182 ...
## $ sex            : chr [1:17419] "Male" "Female" "Female" "Female" ...
## $ weight         : num [1:17419] 22 18 20.2 20.5 18.2 21.9 16.5 16.8 17.7 20.6 ...
## $ height         : num [1:17419] 112 106 104 106 102 ...
## $ bmi            : num [1:17419] 17.5 16 18.7 18.2 17.5 ...
## $ site           : Factor w/ 3 levels "Site1","Site2",..: 2 2 2 2 2 2 2 2 2 2 ...
## $ age            : num [1:17419] 4.9 4.8 4.8 4.9 4.4 4.9 5 4.6 4.7 4.9 ...
## $ age.d          : num [1:17419] 1807 1750 1758 1789 1621 ...
## $ age.m          : num [1:17419] 59 57 58 59 53 58 60 55 56 58 ...
## $ age.y          : Factor w/ 18 levels "2","3","4","5",..: 4 4 4 4 3 4 4 4 4 4 ...
## $ yearvisit      : num [1:17419] 0 0 0 0 0 0 1 0 0 1 ...
```

```
## $ year            : Factor w/ 6 levels "0","1","2","4",..: 1 1 1 1 1 1 2 1 1 2 ...
## $ age.cat         : Factor w/ 3 levels "age 0–5 years",..: 1 1 1 1 1 1 1 1 1 1 ...
## $ bmi.cat         : Factor w/ 8 levels "BMI<=10","10<BMI <=15",..: 3 3 3 3 3 3 2 3 3 3 ...
## $ sex_numeric     : num [1:17419] 1 2 2 2 2 2 1 1 2 2 ...
## $ clenhei         : num [1:17419] 112 106 104 106 102 ...
## $ cbmi            : num [1:17419] 17.5 16 18.7 18.2 17.5 ...
## $ cmeasure        : chr [1:17419] NA NA NA NA ...
## $ csex            : int [1:17419] 1 2 2 2 2 2 1 1 2 2 ...
## $ zlen            : num [1:17419] 0.52 −0.45 −0.91 −0.59 −0.83 0.08 −0.63 −0.88 −0.1 1.39 ...
## $ flen            : int [1:17419] 0 0 0 0 0 0 0 0 0 0 ...
## $ zwei            : num [1:17419] 1.37 0.08 0.83 0.86 0.46 1.32 −0.78 −0.34 0.05 0.93 ...
## $ fwei            : int [1:17419] 0 0 0 0 0 0 0 0 0 0 ...
## $ zwfl            : num [1:17419] 1.46 0.53 2.15 1.88 1.49 1.91 −0.68 0.3 0.14 −0.03 ...
## $ fwfl            : int [1:17419] 0 0 0 0 0 0 0 0 0 0 ...
## $ zbmi            : num [1:17419] 1.58 0.49 1.97 1.74 1.39 1.86 −0.62 0.34 0.14 0.21 ...
## $ fbmi            : int [1:17419] 0 0 0 0 0 0 0 0 0 0 ...
## $ zhc             : num [1:17419] NA NA NA NA NA NA NA NA NA NA ...
## $ fhc             : int [1:17419] NA NA NA NA NA NA NA NA NA NA ... ## $ zac
                     : num [1:17419] NA NA NA NA NA NA NA NA NA NA ...
## $ fac             : int [1:17419] NA NA NA NA NA NA NA NA NA NA ... ## $ zts
                     : num [1:17419] NA NA NA NA NA NA NA NA NA NA ...
## $ fts             : int [1:17419] NA NA NA NA NA NA NA NA NA NA ... ## $ zss
                     : num [1:17419] NA NA NA NA NA NA NA NA NA NA ...
## $ fss             : int [1:17419] NA NA NA NA NA NA NA NA NA NA ...
```

The output shows that the calculated z-scores are combined with the original dataset (last columns).

Calculate Z-Scores for Individuals

We can also use the *anthro_zscores()* function to calculate Z-scores for an individual. For example, we calculate Z-scores for a male 700 days old with a weight = 9 kg, height = 85 cm.

```
zi <-anthro_zscores(sex = 1,
       age = 700,
       weight = 9,
       lenhei = 85
   )
zi
```

```
##      clenhei   cbmi   cmeasure   csex   zlen   flen   zwei   fwei   zwfl   fwfl
## 1    85        12.5   <NA>       1      −0.65  0      −2.5   0      −3.12   0
##      zbmi      fbmi   zhc        fhc    zac    fac    zts    fts    zss    fss
## 1    −3.26     0      NA         NA     NA     NA     NA     NA     NA     NA
```

9.2.5. Calculate Prevalence of Z-Score Categories with "anthro" Package

We can calculate the prevalence of Z-Score Categories using the *anthro_prevalence()* function of the *"anthro"* package [78]. However, their default prevalence calculation for age groups is not very flexible for various research purposes.

```
pz5 <-with( dat5,
   anthro_prevalence(
      sex = sex_numeric,
      age = age.d,
      weight = weight,
      lenhei = height
   )
)
head(pz5[,1:5])
```

```
##      Group                                HAZ_pop  HAZ_unwpop  HA_3_r  HA_3_se
## 1    All                                  17,642   17,642      0.249   0.0376
## 2    Age  group:  00–05  mo  0           0        NA          NA
## 3    Age  group:  06–11  mo  0           0        NA          NA
## 4    Age  group:  12–23  mo  990         990      0.909       0.3017
## 5    Age  group:  24–35  mo  3959        3959     0.177       0.0668
## 6    Age  group:  36–47  mo  5781        5781     0.346       0.0772
```

9.2.6. Calculate Prevalence of Nutritional Status Manually

Prevalence helps evaluate the public health significance of nutritional status. If one does not like the default prevalence calculation of the *"anthro"* package, one may calculate prevalence manually from the Z-scores generated from the *anthro_zscores()* function above. First, we get nutritional status by categorizing Z-scores (e.g., of weight-for-height "zwfl") following WHO recommendations using the function *cut()*.

```
dz5 <-dz5%>% mutate(wh.catw= cut(
      zwfl,
      breaks = c(-Inf,-2, 2, 3, Inf), # Define break points
      labels = c("Wasting", "Normal WFH", "Overweight","Obesity"), # labels
      right  =  TRUE
   ))%>%
   mutate(height.cats= cut( zlen,
      breaks = c(-Inf,-2, 3, Inf), # Define break points
      labels = c("Stunting", "Normal HFA", "Extreme tallness"), # labels
      right  =  TRUE))
```

Then we calculate the prevalence and 95% CI of prevalence of wasting, and obesity based on weight-for-height z-score categories and display the results in a table (function *MultinomCI()* with *method="sisonglaz"*). This would give the researchers more flexibility to work with their data (Table 27).

```
w5l <-c("Obesity","Overweight","Normal WFH","Wasting")
dz5$wh.catw <-factor(dz5$wh.catw, levels=w5l)
tbl  <-  table(dz5$wh.catw)
pci  <-round(MultinomCI(tbl,
                        conf.level=0.95,
                        method="sisonglaz"),5)
tblb <- cbind(count=tbl, p.95ci=paste(pci[,1],
                                      " (",
                                      pci[,2],
                                      ", ",
                                      pci[,3],
                                      ")",
                                      sep=""))
kable(tblb, caption="Prevalence of wasting, obesity based on z-scores")%>%
   kable_styling(latex_options = c("scale_down", "hold_position"))
```

Table 27. Prevalence of wasting and obesity based on z-scores.

	Count	Prevalence (95% CI)
Obesity	389	0.02237 (0.01852, 0.02632)
Overweight	870	0.05003 (0.04618, 0.05398)
Normal WFH	15,901	0.91443 (0.91058, 0.91838)
Wasting	229	0.01317 (0.00932, 0.01712)

Source: Table by author.

The output table shows the prevalence and 95% CI of each nutritional status category.

Similarly, we can calculate the prevalence of stunting based on height-for-age z-score categories and display the results in a table (Table 28).

```
stl <-c("Extreme tallness","Normal HFA","Stunting")
dz5$height.cats <-factor(dz5$height.cats, levels=stl)
tbl      <- table(dz5$height.cats)
pci <-round(MultinomCI(tbl, conf.level=0.95, method="sisonglaz"),5)
tblb <- cbind(count=tbl, p.95ci=paste(pci[,1],
                                      " (",
                                      pci[,2],
                                      ", ",
                                      pci[,3],
                                      ")",
                                      sep=""))
kable(tblb, caption="Prevalence of stunting based on z-scores")%>%
kable_styling(latex_options = c("scale_down", "hold_position"))
```

Table 28. Prevalence of stunting based on z-scores.

	Count	Prevalence (95% CI)
Extreme tallness	46	0.00264 (0, 0.00533)
Normal HFA	16,778	0.9632 (0.96056, 0.96589)
Stunting	595	0.03416 (0.03152, 0.03685)

Source: Table by author.

9.2.7. Calculate Z-Scores Manually

The packages "*Anthro*" [78] and "*zscorer*" [79] have built-in functions to calculate Z-scores (see examples above). However, they produce errors or NA for data of children > 5 years old and lack flexibility. For those with some R programming skills, it may be handier to download the WHO reference data from the WHO website and do some programming to calculate Z-scores, especially for children > 5 years old. This will help researchers get over the lack of flexibility or limitations of the above-mentioned existing R-packages.

Note that there are two sets of WHO reference data for children 0 to 5 years old (age in days) and for children 5 to 19 years old (age in months). Now, look at the WHO reference dataset that the "*zscorer*" package uses. The dataset is named "wgsData". It contains the columns "indicator" (a vector of indices "*hfa*", "*wfa*", and "*wfh*"), "*sex*" (as numeric with one for male and two for female), "*given*" (either age in days or height in cm), "*l*", "*m*", and "*s*".

```
data(wgsData)
str(wgsData)
```

```
## 'data.frame':    1746 obs. of 6 variables:
## $ indicator: Factor w/ 3 levels "hfa","wfa","wfh": 3 3 3 3 3 3 3 3 3 3 ...
## $ sex      : num 1 1 1 1 1 1 1 1 1 1 ...
## $ given    : num 45 45.1 45.2 45.3 45.4 45.5 45.6 45.7 45.8 45.9 ...
## $ l        : num -0.352 -0.352 -0.352 -0.352 -0.352 ...
## $ m        : num 2.44 2.46 2.47 2.49 2.51 ...
## $ s        : num 0.0918 0.0918 0.0917 0.0916 0.0916 ...
```

Prepare WHO Reference Data

First, we create a combined WHO reference dataset for height, weight, and BMI for age and weight for height by sex (numeric with *1 = male*, *2 = female*), similarly to the WHO reference dataset "wgsData" of the "*zscorer*" package. We combined separate WHO reference datasets for children 0–5 years old ("whoref5") and for children 5–19 years old ("whoref19"). Note that age should be in days for children 0–5 years old and age should be in months for children 5–19 years old. Both datasets contain the crucial L, M, and S parameters for each age and sex.

```
# Combine WHO reference data for children >5 years old
zn <-names(who619)[grep("z",names(who619))]
whoref19 <-NULL
for (i in zn){
   test <-as.data.frame(who619[[i]])
   indicator      <- str_extract(i, paste0("^[^", "_", "]+"))
   test <-cbind(indicator=indicator, test)
   test$sex <-ifelse(grepl("boys", i) ==TRUE,1,2)
   colnames(test)[colnames(test) %in% "month"] <-"given"
   whoref19 <-plyr::rbind.fill(whoref19,test)
}
whoref19$indicator[whoref19$indicator %in% "bmi"] <-"bfa"
whoref19 <-as.data.frame(whoref19)
str(whoref19)
```

```
## 'data.frame'    : 792 obs. of 17 variables:
## $ indicator     : chr "bfa" "bfa" "bfa" "bfa" ...
## $ given         : num 61 62 63 64 65 66 67 68 69 70 ...
## $ l             : num -0.739 -0.762 -0.786 -0.809 -0.832 ...
## $ m             : num 15.3 15.3 15.3 15.3 15.3 ...
## $ s             : num 0.0839 0.0841 0.0844 0.0846 0.0849 ...
## $ sd4neg        : num 11.2 11.2 11.2 11.2 11.2 ...
## $ sd3neg        : num 12.1 12.1 12.1 12.1 12.1 ...
## $ sd2neg        : num 13 13 13 13 13 ...
## $ sd1neg        : num 14.1 14.1 14.1 14.1 14.1 ...
## $ sd0           : num 15.3 15.3 15.3 15.3 15.3 ...
## $ sd1           : num 16.6 16.6 16.7 16.7 16.7 ...
## $ sd2:          : num 18.3 18.3 18.3 18.3 18.3 ...
## $ sd3           : num 20.2 20.2 20.2 20.3 20.3 ...
## $ sd4           : num 22.1 22.1 22.2 22.2 22.3 ...
## $ sex           : num 1 1 1 1 1 1 1 1 1 1 ...
## $ stdev         : num NA NA NA NA NA NA NA NA NA NA ...
## $ sd5neg        : num NA NA NA NA NA NA NA NA NA NA ...
```

```
# study data > 5 years old
dat18 <-dat[dat$age.m>=61,]

# Combine WHO reference data for children <=5 years old
zn5 <-names(who5)[grep("z",names(who5))]
zn5 <-zn5[grep("expanded", zn5)]
whoref5 <-NULL for (i in zn5){
    test <-as.data.frame(who5[[i]])
    indicator          <- str_extract(i, paste0("^[^", "-", "]+"))
    test <-cbind(indicator=indicator, test)
    test$sex <-ifelse(grepl("boys", i) ==TRUE,1,2)
    colnames(test)[colnames(test) %in% "day"] <-"given"
    whoref5 <-plyr::rbind.fill(whoref5,test)
}
whoref5 <-whoref5%>%
    mutate(indicator=case_when(!indicator %in% "lhfa" ~ indicator,
                                                    indicator %in% "lhfa" ~ "hfa"))%>%
    mutate(given=case_when(!indicator %in% c("wfh","wfl")~ given,
                                            indicator %in% "wfh" ~ height,
                                            indicator %in% "wfl" ~ length))
whoref5 <-whoref5[,!names(whoref5) %in% c("height", "length")]
whoref5 <-as.data.frame(whoref5)
str(whoref5)
# study data <= 5 years old
dat5 <-dat[dat$age.m <=60,]
```

```
## 'data.frame':        13,546 obs. of 15 variables:
## $ indicator: chr     "bfa" "bfa" "bfa" "bfa" ...
## $ given              : num 0 1 2 3 4 5 6 7 8 9 ...
## $ l                  : num -0.3053 -0.1867 -0.0681 0.0505 0.169 ...
## $ m                  : num 13.4 13.4 13.4 13.4 13.4 ...
## $ s                  : num 0.0956 0.096 0.0963 0.0967 0.0971 ...
## $ sd4neg             : num 9.23 9.15 9.06 8.96 8.86 ...
## $ sd3neg             : num 10.18 10.12 10.06 9.99 9.92 ...
## $ sd2neg             : num 11.1 11.1 11.1 11 11 ...
## $ sd1neg             : num 12.2 12.2 12.2 12.1 12.1 ...
## $ sd0                : num 13.4 13.4 13.4 13.4 13.4 ...
## $ sd1                : num 14.8 14.8 14.7 14.7 14.7 ...
## $ sd2                : num 16.3 16.3 16.3 16.2 16.2 ...
## $ sd3                : num 18.1 18 17.9 17.8 17.8 ...
## $ sd4                : num 19.9 19.7 19.6 19.5 19.3 ...
## $ sex                : num 1 1 1 1 1 1 1 1 1 1 ...
```

Write New Function for z-Score Calculation

Second, we write a new function to calculate height, weight, BMI for age, and weight for height z-scores.

Note that age should be in months for children >5 years old and age should be in days for children $\leq$ 5 years old, with no decimal digit. Height should be in cm with 1 decimal digit.

To simplify, height will be used for all index calculations: Height = supine length$-$0.7 cm (if standing = 2 = supine). If the standing status is unknown and the child is <731 days old, then the measure of the child is regarded as supine length. If the standing status is unknown and the child is $\geq$731 days old, then the measure of the child is regarded as standing height.

Our new function *zcal()* below is designed to:

- Accept a study data frame ("*data*"), the correct reference table ("*ref.dat*"), and column names.
- Correct for supine length vs. standing height.
- Calculate BMI from the (potentially corrected) height.
- Join the study data to the WHO reference table by sex and age (either days or months).
- Calculate the Z-scores for height for age (HFA), weight for age (WFA), BMI for age (BFA), and weight for height (WFH) using the official LMS formula.
- Handle extreme Z-scores as the WHO recommends (by extrapolating from the SD bands). The WHO growth standards define a valid Z-score range of -3 to $+3$. When calculated Z-scores fall outside this interval, the function adjusts them using WHO-recommended extrapolation methods. This prevents unrealistic values and maintains comparability across studies.

The implementation of this *zcal()* function follows the WHO growth standard methodology based on the LMS (Lambda Mu Sigma) transformation, which accounts for skewness in the reference distribution.

The function returns a dataset that contains the original study variables and the calculated Z-scores for each selected index. These standardized z-scores can then be used for subsequent analyses.

```
zcal <-function(data=dat5,
                        ref.dat=whoref5,
                        sex.numeric= "sex_numeric", #sex numeric 1=male, 2= female
                        age.month=NULL, #age in months, for age >5 years old
                        age.day=NULL, #age in days, for age 0-5 years old
                        length.height.cm="height", # length or height in cm
                        weight.kg="weight", # weight in kg index=c("hfa","wfa","bfa","wfh"),
                        # calculate these indices standing.var=NULL, #1=standing, 2= supine,
                        3=unknown
                        age.group=c(" <=5 years",">5 years")){
  # Calculate age in months or age in days
  # if required but not available
  if (age.group==" <=5 years" & is.null(age.day)){
    data$age.day  <-round(data$age.month*30.4,0)
  }
    if (age.group==">5 years" & is.null(age.month)){ data$age.month  <-
    round(data$age.day/30.4,0)
  }
  if  (!is.null(age.month)){
    data$age.month <-round(data[,age.month],0)
    }
   if (!is.null(age.day)){
      data$age.day <-round(data[,age.day],0)
   }
   data$sex.numeric <-data[,sex.numeric]
   data$length.height.cm <-round(data[,length.height.cm],1) data$weight.kg <-
   data[,weight.kg]
   data$standing.var <-data[,standing.var]
   # corrected height (heightc) corresponding to standing status
```

```
if (age.group==“>5 years”){ data <-
    data %>%
      mutate(heightc=length.height.cm)
}
if (age.group==“ <=5 years” & is.null(standing.var)){
    data <-data %>%
      mutate(heightc=case_when(age.day < 731 ~ length.height.cm -0.7,
                               age.day>=731 ~ length.height.cm))
}
if (age.group==“ <=5 years” & !is.null(standing.var)){
    data <-data %>%
      mutate(heightc=case_when(standing.var==1 ~
                               length.height.cm,
                             standing.var==2 ~
                               length.height.cm-0.7,
                               standing.var==3 & age.day < 731 ~
                               length.height.cm -0.7, standing.var==3 &
                             age.day>=731 ~
                               length.height.cm))
}
data <-data %>% mutate(bmi=(weight.kg/((heightc/100)^2)))
#Height z-scores
if (age.group==“>5 years”){
    datc <-left_join(data,subset(ref.dat,indicator %in% “hfa”),
                     by = c(age.month = “given”, sex.numeric=“sex”))
}
    if (age.group==“ <=5 years”){
      datc <-left_join(data,subset(ref.dat,indicator %in% “hfa”),
                     by = c(age.day = “given”, sex.numeric=“sex”))
}
datc <-datc %>%
    mutate(sd23pos = sd3 - sd2,
           sd23neg = sd2neg - sd3neg)%>%
    mutate(hfaz = (((heightc/m)^l) - 1)/(l *s)) %>%
    #manage extreme z values
    mutate(hfaz =case_when(hfaz >= -3 & hfaz <=3 ~ hfaz,
                           hfaz>3 ~ 3 + ((heightc - sd3)/sd23pos),
                           hfaz <-3 ~ ((heightc - sd3neg)/sd23neg)))
datz <- datc[,!colnames(datc) %in% c(colnames(ref.dat),
                                     “sd23pos”,”sd23neg”)]
#Weight z-scores
if (age.group==“>5 years”){
      datc <-left_join(datz,subset(ref.dat,indicator %in% “wfa”),
                     by = c(age.month = “given”, sex.numeric=“sex”))
}
    if (age.group==“ <=5 years”){
      datc <-left_join(datz,subset(ref.dat,indicator %in% “wfa”),
                     by = c(age.day = “given”, sex.numeric=“sex”))
}
datc <-datc %>%
    mutate(sd23pos = sd3 - sd2,
           sd23neg = sd2neg - sd3neg)%>%
    mutate(wfaz = (((weight.kg/m)^l) - 1)/(l *s)) %>%
```

```
    #manage extreme z values
    mutate(wfaz =case_when(wfaz >= -3 & wfaz <=3 ~ wfaz,
                           wfaz>3 ~ 3 + ((weight.kg - sd3)/sd23pos),
                           wfaz <-3 ~ ((weight.kg - sd3neg)/sd23neg)))
  datz <- datc[,!colnames(datc) %in% c(colnames(ref.dat),
                                       "sd23pos","sd23neg")]
  #BMI z-scores
  if (age.group==">5 years"){
      datc <-left_join(datz,subset(ref.dat,indicator %in% "bfa"),
                       by = c(age.month = "given", sex.numeric="sex"))
  }
    if (age.group==" <=5 years"){
      datc <-left_join(datz,subset(ref.dat,indicator %in% "bfa"),
                       by = c(age.day = "given", sex.numeric="sex"))
  }
  datc <-datc %>%
    mutate(sd23pos = sd3 - sd2,
           sd23neg = sd2neg - sd3neg)%>%
           mutate(bfaz = (((bmi/m)^l) - 1)/(l *s)) %>% #manage
    extreme z values
    mutate(bfaz =case_when(bfaz >= -3 & bfaz  <=3 ~ bfaz,
                           bfaz>3 ~ 3 + ((bmi - sd3)/sd23pos),
                           bfaz <-3 ~ ((bmi - sd3neg)/sd23neg)))
  datz <- datc[,!colnames(datc) %in% c(colnames(ref.dat),
                                       "sd23pos","sd23neg")]
  #WFH z-scores
    if (age.group==">5 years"){ datz$wfhz <-
      NA
  }
   if (age.group==" <=5 years"){
     datc <-left_join(datz, subset(ref.dat, indicator %in% "wfh"),
                      by = c(heightc = "given", sex.numeric="sex"))
     datc  <-datc  %>% mutate(sd23pos =
       sd3 - sd2,
        sd23neg = sd2neg - sd3neg)%>%
       mutate(wfhz = (((weight.kg/m)^l) - 1)/(l *s)) %>%
       #manage extreme z values
       mutate(wfhz =case_when(wfhz >= -3 & wfhz  <=3 ~ wfhz,
                              wfhz>3 ~ 3 + ((weight.kg - sd3)/sd23pos),
                              wfhz < -3 ~ ((weight.kg - sd3neg)/sd23neg)))
     datz <- datc[,!colnames(datc) %in% c(colnames(ref.dat),
                                          "sd23pos","sd23neg")]
   }
  return(datz)
}
```

Apply the New Function for z-Score Calculation

Third, we apply the function *zcal()* written above to calculate and add Z-scores to the study dataset of children. Below is an example application for a dataset of children > 5 years old.

```
datz18 <-zcal(data=dat18,
                    ref.dat=whoref19,
                    sex.numeric= "sex_numeric",
                    age.month="age.m",
                    age.day="age.d", length.height.cm="height",
                    weight.kg="weight",
                    standing.var=NULL,
                    age.group=">5 years")
str(datz18)
```

```
## 'data.frame':        79,224 obs. of 25 variables:
## $ cid                : Factor w/ 88678 levels "c1","c2","c3", ..: 1 21 21 22 27 34 37 39 46 53 ...
## $ weight             : num 33.1 56.4 65 77 60.7 77.8 44.6 82.9 41.2 52.3 ...
## $ height             : num 135 154 157 166 171 ...
## $ bmi                : num 18.2 23.8 26.2 28.1 20.8 ...
## $ site               : Factor w/ 3 levels "Site1","Site2", ..: 2 2 2 2 2 2 2 2 2 2 ...
## $ age                : num 8.6 12.9 14 16.5 15.6 13.9 15 17.6 10.3 9.9 ...
## $ age.d              : num 3124 4726 5122 6026 5706 ...
## $ age.m              : num 103 155 168 198 187 167 180 212 123 119 ...
## $ age.y              : Factor w/ 18 levels "2","3","4","5",..: 8 12 13 15 15 13 14 17 9 9 ...
## $ yearvisit          : num 1 0 1 1 1 1 1 1 1 1 ...
## $ year               : Factor w/ 6 levels "0","1","2","4",..: 2 1 2 2 2 2 2 2 2 2 ...
## $ age.cat            : Factor w/ 3 levels "age 0–5 years",..: 2 3 3 3 3 3 3 3 2 2 ...
## $ bmi.cat            : Factor w/ 8 levels "BMI<=10","10<BMI <=15",..: 3 4 5 5 4 4 3 5 3 4 ...
## $ sex_numeric        : num 2 1 1 1 1 1 2 1 2 1 ...
## $ age.month          : num 103 155 168 198 187 167 180 212 123 119 ...
## $ age.day            : num 3124 4726 5122 6026 5706 ...
## $ sex.numeric        : num 2 1 1 1 1 1 2 1 2 1 ...
## $ length.height.cm   : num 135 154 157 166 171 ...
## $ weight.kg          : num 33.1 56.4 65 77 60.7 77.8 44.6 82.9 41.2 52.3 ...
## $ standing.var       :'data.frame': 79224 obs. of 0 variables
## $ heightc            : num 135 154 157 166 171 ...
## $ hfaz               : num 0.8369 −0.1936 −0.7516 −1.1178 −0.0985 ...
## $ wfaz               : num 1.2 NA NA NA NA ...
## $ bfaz               : num 1.04 1.81 2.06 1.96 0.24 ...
## $ wfhz               : logi NA NA NA NA NA NA ...
```

The output shows that the calculated z-scores are added to the study dataset (the last 4 columns).

Similarly, the example below applies the function *zcal()* written above to calculate and add Z-scores to a study dataset of children $\leq$ 5 years old.

```
datz5 <-zcal(data=dat5,
                    ref.dat=whoref5,
                    sex.numeric= "sex_numeric",
                    age.month="age.m",
                    age.day="age.d", length.height.cm="height",
                    weight.kg="weight",
                    standing.var=NULL,
                    age.group="  <=5  years")
str(datz5)
```

```
## 'data.frame':        18,113 obs. of 25 variables:
## $ cid                : Factor w/ 88678 levels "c1","c2","c3",..: 828 1181 1230 1684 1726 1726
                          2132 2694 2845 3077 ...
## $ weight             : num 22 17.5 18 20.2 20.5 18.2 21.9 16.5 16.8 17.7 ...
## $ height             : num 112 108 106 104 106 ...
## $ bmi                : num 17.5 15.1 16 18.7 18.2 ...
## $ site               : Factor w/ 3 levels "Site1","Site2",..: 2 2 2 2 2 2 2 2 2 2 ...
## $ age                : num 4.9 5 4.8 4.8 4.9 4.4 4.9 5 4.6 4.7 ...
## $ age.d              : num 1807 1827 1750 1758 1789 ...
## $ age.m              : num 59 60 57 58 59 53 58 60 55 56 ...
## $ age.y              : Factor w/ 18 levels "2","3","4","5",..: 4 4 4 4 4 3 4 4 4 4 ...
## $ yearvisit          : num 0 0 0 0 0 0 0 1 0 0 ...
```

```
## $ year                : Factor w/ 6 levels "0","1","2","4",..: 1 1 1 1 1 1 1 2 1 1 ...
## $ age.cat             : Factor w/ 3 levels "age 0–5 years",..: 1 1 1 1 1 1 1 1 1 1 ...
## $ bmi.cat             : Factor w/ 8 levels "BMI<= 10","10<BMI <= 15",..: 3 3 3 3 3 3 3 2 3 3 ...
## $ sex_numeric         : num 1 1 2 2 2 2 2 1 1 2 ...
## $ age.month           : num 59 60 57 58 59 53 58 60 55 56 ...
## $ age.day             : num 1807 1827 1750 1758 1789 ...
## $ sex.numeric         : num 1 1 2 2 2 2 2 1 1 2 ...
## $ length.height.cm    : num 112 108 106 104 106 ...
## $ weight.kg           : num 22 17.5 18 20.2 20.5 18.2 21.9 16.5 16.8 17.7 ...
## $ standing.var        : 'data.frame': 18,113 obs. of 0 variables
## $ heightc             : num 112 108 106 104 106 ...
## $ hfaz                : num 0.517 −0.535 −0.448 −0.905 −0.589 ...
## $ wfaz                : num 1.3744 −0.3483 0.0841 0.8347 0.8611 ...
## $ bfaz                : num 1.5754 −0.0365 0.4927 1.965 1.7386 ...
## $ wfhz                : num 1.464 −0.102 0.534 2.145 1.875 ...
```

The output shows that the calculated z-scores are added to the study dataset (the last 4 columns).

The above *zcal()* function may also be applied to calculate Z-scores for an individual. The following example calculates Z-scores for a female 71 months old with a weight = 19 kg and a height = 115 cm. We first need to create a data frame for an individual, then apply the function to the data frame.

```
dati  <-as.data.frame(cbind(sex.numeric=2, age.month=71,
                            height.cm=115, weight.kg=19))
datzi <-zcal(data=dati,
             ref.dat=whoref19,
             sex.numeric="sex.numeric", age.month="age.month",
             age.day=NULL,
             length.height.cm="height.cm", weight.kg="weight.kg",
             age.group=">5  years",
             standing.var=NULL)
datzi[,7:12]
```

```
##      height   bmi    hfaz      wfaz      bfaz      wfhz
## 1    115      14.4   0.0703    −0.347    −0.617    NA
```

The example below calculates z-scores for a 31-month-old male with weight = 11 kg, height = 105 cm, supine.

```
dati  <-as.data.frame(cbind(sex.numeric=1,
                            age.month=31,
                            length.cm=105,
                            weight.kg=11, standing=2))
datzi <-zcal(data=dati,
             ref.dat=whoref5,
             sex.numeric="sex.numeric", age.month="age.month",
             age.day=NULL,
             length.height.cm="length.cm", weight.kg="weight.kg",
             age.group="  <=5  years", standing.var="standing")
datzi[,9:14]
```

```
##      heightc   bmi    hfaz    wfaz     bfaz    wfhz
## 1    104       10.1   3.38    −1.72    −2.6    −1.91
```

9.2.8. Calculate Prevalence from z-Scores Manually

Now that we have Z-scores for anthropometric measures, we can do nutritional status classification following WHO references and calculate their prevalence for our study data. Below are some examples of these calculations for the two age groups.

For Age $\leq$ 5 Years Old

According to the WHO, for children $\leq$ 5 years old, weight-for-height (WFH) is used for obesity and wasting classification. The example below classifies WFH Z-scores into categories "Obesity", "Overweight", "Normal WFH", "Wasting", calculates the prevalence and 95% CI of these categories, and displays the results as a table (Table 29).

```
w5l <-c("Obesity","Overweight","Normal WFH","Wasting")
datz5 <-datz5%>%
  mutate(wfh.catw= cut(
    wfhz,
    breaks = c(-Inf,-2, 2, 3, Inf), # Define break points
    labels = c("Wasting", "Normal WFH", "Overweight","Obesity"), # Assign labels
  right = TRUE))
datz5$wfh.catw <-factor(datz5$wfh.catw, levels=w5l) tbl <-
table(datz5$wfh.catw)
pci <-round(MultinomCI(tbl,
                       conf.level=0.95,
                       method="sisonglaz"),5)
tblb <- cbind(count=tbl, p.95ci=paste(pci[,1],
                                      " (",
                                      pci[,2],
                                      ", ",
                                      pci[,3],
                                      ")",
                                      sep=""))
kable(tblb,
      caption="Prevalence, 95\\%CI Obesity, Wasting calculated manually")%>%
  kable_styling(latex_options = c("scale_down", "hold_position"))
```

Table 29. Prevalence, 95% CI obesity, wasting calculated manually.

	Count	Prevalence (95% CI)
Obesity	410	0.02314 (0.0193, 0.02705)
Overweight	905	0.05107 (0.04723, 0.05498)
Normal WFH	16,201	0.91423 (0.91039, 0.91814)
Wasting	205	0.01157 (0.00773, 0.01548)

Source: Table by author.

The output shows the prevalence and 95% CI of each nutritional status category.

Similarly, the example below classifies height-for-age (HFA) z-cores into categories "Extreme tallness", "Normal HFA", and "Stunting", calculates the prevalence and 95% CI of these categories, and displays results as a table (Table 30).

```
stl <-c("Extreme tallness", "Normal HFA", "Stunting")
datz5 <-datz5%>% mutate(height.cats= cut(
    hfaz,
    breaks = c(-Inf,-2, 3, Inf), # Define break points
    labels = c("Stunting", "Normal HFA", "Extreme tallness"), # labels
    right = TRUE
  ))
```

```
datz5$height.cats <-factor(datz5$height.cats, levels=stl)
tbl  <- table(datz5$height.cats)
pci <-round(MultinomCI(tbl,
                       conf.level=0.95,
                       method="sisonglaz"),5)
tblb <- cbind(count=tbl, p.95ci=paste(pci[,1],
                                      " (",
                                      pci[,2],
                                      ", ",
                                      pci[,3],
                                      ")",
                                      sep=""))
kable(tblb,
      caption="Prevalence, 95\\%CI stunting under 5 calculated manually")%>%
  kable_styling(latex_options = c("scale_down", "hold_position"))
```

Table 30. Prevalence, 95% CI stunting under five calculated manually.

	Count	Prevalence (95% CI)
Extreme tallness	48	0.00267 (0.00267, 0.00277)
Normal HFA	17,367	0.96446 (0.96446, 0.96457)
Stunting	592	0.03288 (0.03288, 0.03298)

Source: Table by author.

For Age > 5 Years Old

For children > 5 years old, BMI-for-age is used for obesity and thinness classification. The following example calculates the prevalence and 95% CI of "Obesity", "Overweight", "Normal BMI", "Thinness", and displays results as a table, similar to the above (Table 31).

```
t18L <-c("Obesity","Overweight","Normal BMI","Thinness")
datz18 <-datz18%>% mutate(bmi.catt= cut(
    bfaz,
    breaks = c(-Inf,-2, 1, 2, Inf), # Define break points
    labels = c("Thinness", "Normal BMI", "Overweight","Obesity"), # labels
    right = TRUE
  ))
datz18$bmi.catt <-factor(datz18$bmi.catt, levels=t18L)
tbl  <- table(datz18$bmi.catt)
pci <-round(MultinomCI(tbl,
                       conf.level=0.95,
                       method="sisonglaz"),5)
tblb <- cbind(count=tbl, p.95ci=paste(pci[,1],
                                      " (",
                                      pci[,2],
                                      ", ",
                                      pci[,3],
                                      ")",
                                      sep=""))
kable(tblb,
      caption="Prevalence, 95\\%CI Obesity, Thinness calculated manually")%>%
  kable_styling(latex_options = c("scale_down", "hold_position"))
```

Table 31. Prevalence, 95% CI obesity, thinness calculated manually.

	Count	Prevalence (95% CI)
Obesity	11,992	0.15253 (0.14896, 0.1561)
Overweight	17,925	0.22799 (0.22443, 0.23156)
Normal BMI	46,752	0.59464 (0.59107, 0.59821)
Thinness	1954	0.02485 (0.02129, 0.02843)

Source: Table by author.

Again, similarly, the example below calculates the prevalence and 95% CI of "Extreme tallness", "Normal HFA", "Stunting", and displays results as a table (Table 32).

```
stl <-c("Extreme tallness", "Normal HFA", "Stunting")
datz18 <-datz18%>% mutate(height.cats=
  cut(
    hfaz,
    breaks = c(-Inf,-2, 3, Inf), # Define break points
    labels = c("Stunting", "Normal HFA", "Extreme tallness"), # labels
    right = TRUE
  ))
datz18$height.cats <-factor(datz18$height.cats, levels=stl)
tbl <- table(datz18$height.cats)
pci <-round(MultinomCI(tbl,
                       conf.level=0.95,
                       method="sisonglaz"),5)
tblb <- cbind(count=tbl, p.95ci=paste(pci[,1], " (",
                                      pci[,2], ", ",
                                      pci[,3], ")",
                                      sep=""))
kable(tblb,
      caption="Prevalence, 95\\%CI of stunting calculated manually")%>%
  kable_styling(latex_options = c("scale_down", "hold_position"))
```

Table 32. Prevalence, 95% CI of stunting calculated manually.

	Count	Prevalence (95% CI)
Extreme tallness	241	0.00305 (0.00304, 0.00309)
Normal HFA	78,121	0.98974 (0.98973, 0.98977)
Stunting	569	0.00721 (0.0072, 0.00724)

Source: Table by author.

9.2.9. WHO LMS Method for Constructing Normalized Growth Curves

This section demonstrates the statistical construction of reference growth curves using the Lambda-Mu-Sigma (LMS) method. While previous sections focused on calculating individual Z-scores against existing WHO standards, this section addresses the inverse problem: how to generate a smoothed, normalized growth reference from a raw population dataset. This is a critical skill for researchers working with a specific population where global WHO standards may not perfectly capture the local biological "norm".

The examples provided demonstrate how to:

- Model the three parameters of the LMS method: Lambda for skewness, Mu for the median, and Sigma for the coefficient of variation.
- Use the gamlss package to fit these parameters as smooth functions of age.
- Visualize these complex mathematical models as intuitive percentile curves (Centile Charts).

The LMS (Lambda, Mu, and Sigma) method enables efficient calculation of percentiles and Z-scores and smoothing of growth curves [88] and may be done with the "*gamlss*" package [82]. The example below fits the LMS model (*lms05b.h*) for height for boys $\leq$ 5 years old: *lms()* from the "*gamlss*" package tests several distributions (*families=c("BCCGo","BCPEo",* and *"BCTo")*) and select the best family to find the best-fitting L, M, and S parameters for height as a smooth function of "*age.m*" (*trans.x = TRUE*: Box–Cox transformation for age). The selected model is the one with the lowest deviance.

```
dat05b.h  <-na.omit(dat05b[,  c("height","age.m")])
lms05b.h <-lms(height, age.m, families=c("BCCGo","BCPEo","BCTo"), data=dat05b.h, points=FALSE,
               legend=TRUE,
               k=3, calibration=FALSE, trans.x=TRUE)
```

```
##  *** Checking for transformation for x ***
##  *** power parameters              0.64 ***
##  *** Initial                       fit***
##  GAMLSS-RS iteration 1:        Global    Deviance    =  55,128
##  GAMLSS-RS iteration 2:        Global    Deviance    =  55,128
##  *** Fitting BCCGo ***
##  GAMLSS-RS iteration 1:        Global    Deviance    =  54,049
##  GAMLSS-RS iteration 2:        Global    Deviance    =  53,993
##  GAMLSS-RS iteration 3:        Global    Deviance    =  53,993
##  GAMLSS-RS iteration 4:        Global    Deviance    =  53,993
##  *** Fitting BCPEo ***
##  GAMLSS-RS iteration 1:        Global    Deviance    =  54,134
##  GAMLSS-RS iteration 2:        Global    Deviance    =  53,837
##  GAMLSS-RS iteration 3:        Global    Deviance    =  53,837
##  GAMLSS-RS iteration 4:        Global    Deviance    =  53,837
##  GAMLSS-RS iteration 5:        Global    Deviance    =  53,837
##  *** Fitting BCTo ***
##  GAMLSS-RS iteration 1:        Global    Deviance    =  53,751
##  GAMLSS-RS iteration 2:        Global    Deviance    =  53,710
##  GAMLSS-RS iteration 3:        Global    Deviance    =  53,709
##  GAMLSS-RS iteration 4:        Global    Deviance    =  53,709
##  GAMLSS-RS iteration 5:        Global    Deviance    =  53,709
##  GAMLSS-RS iteration 6:        Global    Deviance    =  53,709
##  % of cases below 0.4 centile is0.313
##  % of cases below 2 centile is 1.92
##  % of cases below 10 centile is 10.6
##  % of cases below 25 centile is 25.6
##  % of cases below 50 centile is 49.9
##  % of cases below 75 centile is 74.7
##  % of cases below 90 centile is 89.8
##  % of cases below 98 centile is 98.4
##  % of cases below 99.6 centile is 99.7
```

```
lms05b.h$family
```

```
## [1] "BCTo"
"Box-Cox-t-orig."
```

```
lms05b.h$power
```

```
## [1] 0.64
```

The output shows:

- Model Fit: The output *lms05b.h$family* returns "BCTo" (Box–Cox-t-original), indicating that the model successfully applied a power transformation to normalize the skewed distribution of height in this population.
- The Power Parameter (Lambda): The result *lms05b.h$power* (0.64) represents the degree of transformation required. A value significantly different from 1 confirms that the raw height data were not perfectly normally distributed across all ages and required the LMS Box–Cox adjustment to achieve normality.
- Smoothing Trajectories: Unlike raw mean calculations, which can be "noisy," the LMS method produces smoothed centiles that reflect the underlying biological growth velocity while filtering out random sampling variation.

Next, we construct growth curves for height-for-age for boys $\leq$ 5 years old: *centiles()* of the "*gamlss*" package takes our fitted LMS model above ("*lms05b.h*") and uses it to draw the smoothed percentile curves (0.4th, 2nd, 10th, 25th, 50th, 75th, 90th, 98th, 99.6th) across the range of ages (Figure 115).

```
centiles(lms05b.h, dat05b.h$age.m,
cent=c(0.4,2,10,25,50,75,90,98,99.6),
                ylab="Height (cm)", xlab="Age (month)",
                legend=FALSE,
                ylim=c(70,130),
                main="Centile curves height for age (boys, 18 to 60 months)",
                yaxt = "n", xaxt = "n",
                points=FALSE, save = FALSE)
  axis(1,     at = seq(16,         60, by = 2), las=2)
  axis(2,     at = seq(70,         130, by = 5), las=2)
#Add horizontal grid
abline(h = seq(70, 130, by = 1), lty = 2, col = "lightgrey")
#Add vertical grid
abline(v = seq(16, 60, by = 1), lty = 2, col = "lightgrey")
```

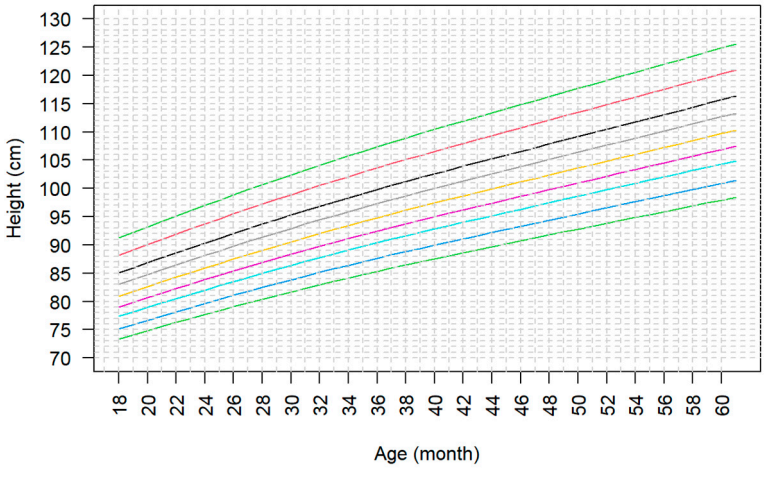


Figure 115. Centile curves for height for age. Source: Figure by author.

We can also create a fan chart using *centiles.fan()* of the "*gamlss*" package, which produces shaded percentile bands and provides a visually intuitive growth distribution (Figure 116).

```
centiles.fan(lms05b.h,dat05b.h$age.m,
                        cent=c(0.4,2,10,25,50,75,90,98,99.6),
                        ylab="Height (cm)", xlab="Age
                        (month)", ylim=c(70,130),
                        yaxt = "n", xaxt = "n",
                        main="Fan-chart height for age (boys, 18 to 60 months)",
                        points=FALSE)
axis(1, at = seq(16, 60, by = 2), las=2)
axis(2, at = seq(70, 130, by = 5), las=2)
#Add horizontal grid
abline(h = seq(70, 130, by = 1), lty = 2, col = "lightgrey")
#Add vertical grid
abline(v = seq(16, 60, by = 1), lty = 2, col = "lightgrey")
```

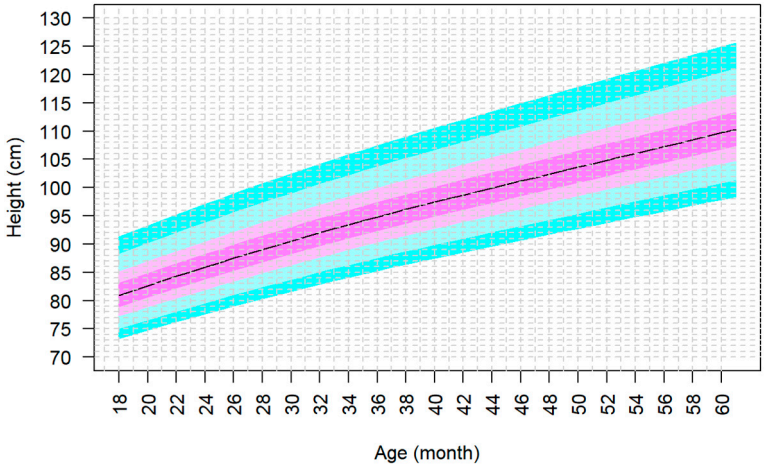


Figure 116. Fan-chart height for age. Source: Figure by author.

The outputs of the *centiles()* function (or *centiles.fan()*) are a plot featuring multiple smoothed lines.

- The 50th Percentile (Median or Mu): This represents the "average" growth trajectory.
- Outer Percentiles (0.4th, 98th): These represent the boundaries of a biologically plausible growth curve. If the curves are tightly packed, it indicates low variation in the population; if they spread significantly as age increases, it reflects increasing biological diversity in growth outcomes.
- The color bands are to distinguish between percentile ranges. The borders between color bands correspond to the above centile curves.

Above are example R codes for some basic analysis of child physical growth data. One may adapt the codes and elaborate the analysis for their own data.

9.3. Useful Resources

WHO 2007 child physical growth reference data may be downloaded from the WHO website for children aged 0 to 5 years old and 5 to 19 years old [76,77].

The original paper "Overweight & obesity epidemic, temporal trends and regional disparities in physical growth of Vietnamese children" published by Nhan Thi Ho et al. [75] contains a lot more details, description, explanation, and interpretation that may be helpful for readers.

9.4. Chapter Summary

- Newly created functions *iqr()* for median, quantile range plotting and *medci()* for median, 95% CI plotting with ggplot2;
- Function *MedianCI()* of the *DescTools* for median, 95% CI of anthropometric measures;
- Function *ci_median_diff()* of the *confintr* package for difference between medians and 95% CI of the difference;
- Average temporal trend of data with repeated measures may be evaluated with GEE model;
- Two sets of WHO 2007 reference data ($\leq$5 years old and >5 to 19 years old);
- Color code and ggplot2 for visualize observed data vs. (WHO) references;
- *"anthro"* and "*zscorer"* package to calculate Z-scores for $\leq$5 years old;
- Newly created function *zcal()* to calculate Z-scores for $\leq$5 years old and >5 to 19 years old for dataset or individual;
- Nutritional status classification (obesity, wasting, stunting, ...) based on Z-scores following WHO recommendation;
- Function *MultinomCI()* of the "*DescTools"* package for prevalence, 95% CI of nutritional status using Glaz & Sison's method;
- *Lms()* function of the *gamlss* package for construction of growth curves using WHO LMS (Lambda, Mu and Sigma) method.

10. Analysis of Clinical Trial Data with R

This chapter provides a practical guide to many steps for clinical trials using R. It includes example R codes for clinical trial planning, including sample size—power estimation for superiority and non-inferiority trials for the most common types of outcomes, as well as for randomization. With a detailed case study based on an example data set mimicking a vaccine trial, this chapter demonstrates how to use some specialized R packages to generate baseline summaries, evaluate efficacy outcomes, and summarize adverse events. Its hands-on examples are designed to help bio-medical researchers effectively work with clinical trial data to generate reporting-ready tables and figures.

10.1. Why R for Clinical Trial Data Analysis?

While traditional software like SAS has a long history in clinical trial analysis, R offers several distinct advantages, such as open-source and free, a vast package ecosystem, superior graphics, dynamic and reproducible reports, and growing active community. These advantages are driving its rapidly increasing adoption in clinical trial analysis, particularly in academia, clinical, and bio-medical research.

First, we load the required packages for clinical trial data analysis:

- "*pwr*" [24], "*powerSurvEpi*" [89], "*TrialSize*" [90], "*SampleSize4ClinicalTrials*" [91], "*gsDesign*" [92] for sample size—power estimation.
- "*blockrand*" [93], "*randomizeR*" [94] for randomization.

Other packages we will load later for our example clinical trial data analysis:

- "*arsenal*" [19] for baseline summary.
- "*survival*" [51] for analysis of time-to-event outcomes (e.g., for efficacy analysis).
- "*tidyverse*" [11] for data manipulation and summary (e.g., for adverse event summary).

Other supporting packages ("*rio*" [14], "*knitr*" [5], "*kableExtra*" [12]) are routinely used as in previous chapters.

```
#load multiple packages
packages <-c("rio","tidyverse","knitr", "kableExtra", "pwr","powerSurvEpi","TrialSize",
             "SampleSize4ClinicalTrials", "gsDesign","blockrand")
lapply(packages, library, character.only = TRUE)
```

10.2. Sample Size—Power Estimation for Clinical Trial

Sample size estimation is a fundamental component of clinical trial design, ensuring the study is appropriately powered to detect a clinically meaningful effect if one truly exists. Below are examples for sample size estimation for various clinical trial designs and outcomes [95].

10.2.1. Superiority Trials

Superiority trials are designed to show that the new treatment is better than the control. We go through examples for estimating sample size for the three most common types of outcomes.

Continuous Outcome

In our example below, we are designing a trial to test if a new drug reduces LDL cholesterol more than a placebo. We want to detect a mean difference of 10 mg/dL. Based on prior research, we expect the standard deviation in both groups to be 20 mg/dL. We desire 90% power with a two-sided alpha of 0.05. Package "*pwr*" [24] may be used. The effect size "*d*" is calculated as the mean difference divided by the standard deviation (10/20). The *pwr.t.test()* function takes this standardized effect size, the desired power (0.90), and significance level (0.05) to solve for sample size "*n*".

```
library(pwr)

# Parameters
mu_diff <- 10 # Expected difference in means
sd <- 20        # Expected standard deviation
sig_level    <- 0.05                    # Significance level (alpha)
power <- 0.90     # Desired power (1 - beta)

# Calculate Cohen's d (standardized effect size)
cohens_d <- mu_diff / sd
```

```
# Perform the sample size calculation
ss_continuous <- pwr.t.test(
    d = cohens_d,
    power = power,
    sig.level = sig_level,
    type = "two.sample",
    alternative = "two.sided"
)
ss_continuous
```

```
##
##     Two-sample t test power calculation
##
##     n = 85
##     d = 0.5
##     sig.level = 0.05
##     power = 0.9
##     alternative = two.sided
##
##     NOTE: n is number in *each* group
```

The output shows that 85 patients are needed per group (a total of 170 participants).

Note that one may use *power.t.test()* of the base R "*stats*" package, which allows one to input the raw difference in means (delta) and the standard deviation (sd) separately.

```r
# Note: The 'stats' package is loaded by default in R, so no library() call is needed.
# Parameters
mu_diff <- 10 # Expected difference in means
sd <- 20        # Expected standard deviation
sig_level <- 0.05  # Significance level (alpha)
power <-0.90     # Desired power (1 - beta)

# Perform the sample size calculation
ss_continuous <- power.t.test(
    delta = mu_diff,
    sd = sd,
    power = power,
    sig.level = sig_level,
    type = "two.sample",
    alternative = "two.sided"
)
ss_continuous
```

```
##     Two-sample t test power calculation
##
##     n = 85.03129
##     delta = 10
##     sd = 20
##     sig.level = 0.05
##     power = 0.9
##     alternative = two.sided
##
##     NOTE: n is number in *each* group
```

The output shows similar results to the above.

Binary Outcome

For our example below, a new cancer therapy is expected to have a 50% response rate, compared to a 35% response rate for the standard therapy. We want to determine the sample size needed to detect this difference with 80% power and a two-sided alpha of 0.05. We first calculate the effect size, Cohen's h, using the function *ES.h()*, then calculate sample size using the function *pwr.2p.test()*.

```
library(pwr)
# Parameters
p1 <-0.50              # Expected proportion in the new therapy group
p2 <- 0.35             # Expected proportion in the standard therapy group
sig_level <- 0.05
power <- 0.80

# Calculate effect size using Cohen's h
effect_size_h <- ES.h(p1 = p1, p2 = p2)

# Perform the sample size calculation
ss_binary <- pwr.2p.test(
    h = effect_size_h,
    power = power,
    sig.level = sig_level,
    alternative = "two.sided"
)
ss_binary
```

```
##     Difference of proportion power calculation for binomial distribution (arcsine
##     transformation)
##
##     h = 0.305
##     n = 169
##     sig.level = 0.05
##     power = 0.8
##     alternative = two.sided
##
##     NOTE: same sample sizes
```

The output shows that 169 patients are needed per group (a total of 352 participants).

Note that one may use *power.prop.test()* of the base R "*stats*" package, which does not require manually calculating Cohen's h (the effect size). Instead, it directly accepts the two proportions (p1 and p2).

```
# Parameters
p1 <- 0.50             # Expected proportion in the new therapy group
p2 <- 0.35             # Expected proportion in the standard therapy group
sig_level <- 0.05
power <- 0.80
# Perform the sample size calculation
ss_binary <- power.prop.test(
    p1 = p1,
    p2 = p2,
    power = power,
    sig.level = sig_level,
    alternative = "two.sided"
)
ss_binary
```

```
##      Two-sample comparison of proportions power calculation
##
##      n = 169.3114
##      p1 = 0.5
##      p2 = 0.35
##      sig.level = 0.05
##      power = 0.8
##      alternative = two.sided
##
##      NOTE: n is number in *each* group
```

The output shows similar results to the above.

Time-to-Event Outcome

Example: A clinical trial is planned to test a new drug for advanced pancreatic cancer. The ratio of participants in the experimental group (E) compared to the control group (C) is 1 ($k = 1$). The probability of death in the experimental group over the maximum time period of the study is 0.35 ($pE = 0.35$). The probability of death in the control group over the maximum time period of the study is 0.5 ($pC = 0.5$). The postulated hazard ratio is 0.7 ($RR = 0.7$). The desired power is 80% ($power = 0.8$) with a two-sided alpha of 0.05. The package "*powerSurvEpi*" [89] may be used.

```
library(powerSurvEpi)
ss_survival <-ssizeCT.default(power = 0.8,
                  k = 1,
                  pE = 0.35,
                  pC = 0.5,
                  RR = 0.7,
                  alpha = 0.05)
ss_survival
```

```
##    nE    nC
##   297   297
```

The output shows that the sample size needed for each group is 297 (a total of 594 participants).

10.2.2. Non-Inferiority Trial

A non-inferiority trial is designed to show that a new treatment is not worse than an active control.

Continuous Outcome

Example: A new, cheaper anti-hypertensive drug is being tested against the standard drug. The standard drug reduces systolic blood pressure (SBP) by 25 mmHg. The non-inferiority margin (Δ) is set at 5 mmHg. We assume the pooled standard deviation of SBP of the two groups is 15 mmHg, and the true mean difference in SBP between the two groups is 0. We want 90% power with a one-sided alpha of 0.025. The function *TwoSampleMean.NIS()* of the "*TrialSize*" package [90] may be used. This function takes our defined values for one-sided significance level (alpha), type 2 error (beta =1 − desired power), assumed true difference (margin), assumed SD of the outcome (sigma), the non-inferiority margin (delta), and ratio of n_new/n_standard (k) to estimate the required sample size n for our study.

```r
library(TrialSize)

# Define Parameters
alpha <- 0.025                # One-sided significance level
power <- 0.90                 # Desired power (so beta is 0.10)
true_difference <- 0          # Assumed true mu_new - mu_standard
sd_outcome <- 15              # Assumed SD of the outcome
ni_margin <- 5                # The non-inferiority margin (Δ)
allocation_ratio <- 1         # k: ratio of n_new / n_standard
# Perform the Sample Size Calculation
ss_ni_continuous <- TwoSampleMean.NIS(
    alpha = alpha,
    beta = 1 - power,
    sigma = sd_outcome,
    k = allocation_ratio,
    delta = ni_margin,
    margin = true_difference
)
ss_ni_continuous
```

```
## [1] 189
```

The output shows the sample size needed for each group (189 participants).

Binary Outcome

Example: A trial comparing a new single-dose antibiotic to a standard 7-day course. We assume the true success rate of the new antibiotic (p1) and the success rate of the standard treatment (p2) are 90%. The non-inferiority margin is 10%. We want 80% power with a one-sided alpha of 0.025. We use the function *TwoSampleProportion.NIS()* and input the parameters similarly to the above to calculate the required sample size.

```r
library(TrialSize)
# Define Parameters
alpha <- 0.025
power <- 0.80
p_standard <- 0.90            # Success rate for standard treatment
p_new <- 0.90                 # Assumed true success rate for new treatment
ni_margin <-0.10              # The non-inferiority margin (delta)

# Perform Calculation
ss_ni_binary <- TwoSampleProportion.NIS(
    p1 = p_new,
    p2 = p_standard,
    delta = p_new -p_standard,
    alpha = alpha,
    beta = 1 - power,
    k = 1,
    margin=ni_margin
)
ss_ni_binary
```

```
## [1] 141
```

The output shows the sample size needed for each group (141).

Another R package that may be used to compute sample size for various trial designs is "*SampleSize4ClinicalTrials*". We can use this package for the same example above.

```
library(SampleSize4ClinicalTrials)
ss_ni_binary2 <-ssc_propcomp(design=3L, #3L for non-inferiority trial
                        ratio=1, alpha=alpha,
                        power=power,
                        p1 = p_new,
                        p2 = p_standard,
                        delta=ni_margin)
ss_ni_binary2
```

```
##        Treatment     Control
## 1      142           142
```

The output shows the sample size needed for each group, similarly to the above.

Time to Event Outcome

Example: a trial comparing a new oral therapy with standard IV chemotherapy for cancer. The new drug is expected to have far fewer side effects, so even if it is slightly less effective, it would be a viable option. The primary outcome is Progression-Free Survival (PFS). The median PFS for the standard IV treatment is estimated to be 11 months. The clinical team has defined the non-inferiority margin (Δ) at a Hazard Ratio of 1.30. They are willing to accept the new drug if the hazard of progression is no more than 30% higher than the standard. For the sample size calculation, we will assume the true hazard ratio is 1.0. The trial will recruit patients over 24 months. After the last patient is enrolled, there will be an additional 18 months of follow-up. We desire 90% power with a one-sided alpha of 0.025. The *nSurv()* function of the "*gsDesign*" package [92] may be used.

```
library(gsDesign)
# Define Parameters
alpha <- 0.025              # One-sided significance level
power <- 0.90               # Desired power
beta <- 1 - power           # Type II error rate


# Survival and Non-Inferiority assumptions
median_standard <- 11             # Median PFS in months for standard arm
ni_margin_hr <- 1.30              # Non-inferiority margin (the null HR, Δ)
true_hr <- 1.0                    # Assumed TRUE hazard ratio

# Trial timing assumptions
accrual_duration <- 24            # Months to recruit patients (R)
follow_up_duration <- 18          # Months of follow-up after last patient in
total_duration <- accrual_duration + follow_up_duration # Total duration (T)

# Calculate Required Event Rate for the Function -
# nSurv requires the exponential event rate (lambda),
# not the median survival.
# We can calculate this assuming an exponential distribution.
# lambda = log(2) / median_survival
lambda_standard <- log(2) / median_standard
```

```r
# Perform the Sample Size Calculation
ss_ni_survival <- nSurv(
  lambdaC = lambda_standard,      # Monthly event rate in standard arm
  hr = true_hr,                   # *true* expected HR (usually 1 for NI)
  hr0 = ni_margin_hr,             # *null* hypothesis HR (NI margin)
  eta = 0,                        # Dropout rate (assuming 0 for simplicity)
  R = accrual_duration,           # Accrual duration
  T = total_duration,             # Total study duration
  minfup = follow_up_duration,
  alpha = alpha,                  # Already one-sided
  beta = beta, sided
  = 1
)
ss_ni_survival
```

```
##                                  Fixed design, two-arm trial with time-to-event
## outcome (Lachin and Foulkes, 1986).
## Solving for:                     Accrual rate
## Hazard ratio                     H1/H0 = 1/1.3
## Study duration:                  T = 42
## Accrual duration:                24
## Min. end-of-study follow-up:     minfup = 18
## Expected events (total, H1):     613
## Expected sample size (total):    735
## Accrual rates:
##                                  Stratum 1
## 0–24                             30.6
## Control event rates (H1):
##                                  Stratum 1
## 0-Inf                            0.063
## Censoring rates:
##                                  Stratum 1
## 0-Inf                            0
## Power:                           100*(1 – beta) = 90%
## Type I error (1-sided):          100*alpha = 2.5%
## Equal randomization:             ratio = 1
```

The output provides information for planning the trial:

- Total Events (d) is the most important number. The trial needs to continue until 613 events (disease progressions or deaths) have been observed. The final analysis can only happen when this target is met.
- Total Sample Size (n): To accumulate those 613 events within the planned 42-month total study duration, the trial will need to enroll 735 patients in total.
- Accrual Rate (gamma): The trial will need to enroll approximately 30.6 patients per month during 24 months to meet its goals.

10.3. Randomization

Randomization is the cornerstone of clinical trial design, serving as the primary mechanism to eliminate selection bias and ensure that study groups are comparable at baseline. The purpose of this section is to guide researchers through the transition from theoretical randomization to the practical generation of reproducible, auditable randomization sequences using R.

The packages "*blockrand*" [93] and "*randomizeR*" [94] are among the packages that may be used to generate a randomization list for clinical trials. Below are some examples using the "*blockrand*" package for common randomization scenarios.

10.3.1. Simple Randomization

This is the most straightforward method, akin to flipping a coin for each participant. Simple randomization can lead to imbalances in small sample sizes. For example, we do randomization for 100 participants into 2 equal-sized groups using simple randomization.

```
# Set a seed for reproducibility
set.seed(42)

# Define trial parameters
n_patients <- 100
treatment_arms <- c("New Drug", "Placebo")

# Generate the simple randomization list
randomization_list_simple <- data.frame(
    patient_id = 1:n_patients
) %>%
    mutate(
        assigned_arm = sample(x = treatment_arms,
                                                size = n_patients,
                                                replace = TRUE,
                                                prob = c(0.5, 0.5))
    )

# View the first 10 assignments
head(randomization_list_simple, 10)
```

```
##      patient_id    assigned_arm
##  1   1           New Drug
##  2   2           New Drug
##  3   3           Placebo
##  4   4           New Drug
##  5   5           New Drug
##  6   6           New Drug
##  7   7           New Drug
##  8   8           Placebo
##  9   9           New Drug
##  10  10          New Drug
```

```
# Check the final balance
table(randomization_list_simple$assigned_arm)
```

```
##
##      New Drug     Placebo
##      55           45
```

The output shows a small imbalance of sample size between arms (55 and 45, while the perfect balance should be 50 and 50).

10.3.2. Blocked Randomization (Permuted Block Randomization)

Blocked randomization is the most common method to prevent the imbalances seen in simple randomization. This technique helps maintain group balance over time, preventing long "runs" of a single treatment assignment. Using variable block sizes helps reduce the risk of predicting the last assignment in a block. The package *"blockrand"* may be used. We first need to define trial parameters, then use variable

block sizes to reduce predictability, and then input these values to generate the randomization list using the "*blockrand*" package [93].

```
install.packages("blockrand")
library(blockrand)
# Set a seed for reproducibility
set.seed(123)

# Define trial parameters
n_patients <- 120
treatment_arms <- c("Treatment A", "Control")
# Use variable block sizes to reduce predictability
block_sizes <- c(4, 6, 8)

# Generate the randomization list using the blockrand package
randomization_list_blocked <- blockrand(
  n = n_patients,
  num.levels = 2,
  levels = treatment_arms,
  block.sizes = c(2, 3, 4), # *blockrand* uses number of subjects per arm in block
  stratum = "Overall"              # use this for stratified randomization later
)

# View the first 12 assignments
head(randomization_list_blocked, 12)
```

```
##    id stratum    block.id  block.size  treatment
##    1 1  Overall  1         8           Treatment A
##    2 2  Overall  1         8           Treatment A
##    3 3  Overall  1         8           Control
##    4 4  Overall  1         8           Control
##    5 5  Overall  1         8           Control
##    6 6  Overall  1         8           Control
##    7 7  Overall  1         8           Treatment A
##    8 8  Overall  1         8           Treatment A
##    9 9  Overall  2         4           Control
##    10 10 Overall  2         4           Control
##    11 11 Overall  2         4           Treatment A
##    12 12 Overall  2         4           Treatment A
```

```
# Check the balance after 40 patients
table(randomization_list_blocked$treatment[1:40])
```

```
##
##     Control   Treatment A
##     20        20
```

```
# Check the final balance
table(randomization_list_blocked$treatment)
```

```
##
##    Control   Treatment A
##    60        60
```

The output shows a good balance between the two randomized groups.

10.3.3. Stratified Randomization

Stratified randomization creates a separate block randomization list for each combination of stratification factors, guarantees balance on key prognostic factors, and thus increases statistical power and credibility. It is essential for multi-center trials.

Example: A trial needs to ensure the treatment arms are balanced by Study Site (Site A, Site B) and Disease Stage (Early, Late). The package "*blockrand*" [93] may be used to generate a randomization list for each stratum, then combine the lists for all strata into a master randomization list. In the example codes below, we first define the strata, define trial parameters, use various block sizes, write a loop to generate a randomization list for each stratum and combine into the master randomization list, and then check for balance.

```
# Load the necessary library
library(blockrand)
# Set a seed for reproducibility
set.seed(101)
# Define the Strata
# Create a data frame where each row is a unique stratum (subgroup)
strata_df <- expand.grid(
    site = c("Site A", "Site B"),
    disease_stage = c("Early", "Late")
)

print(strata_df)
```

```
##      site      disease_stage
## 1    Site A    Early
## 2    Site B    Early
## 3    Site A    Late
## 4    Site B    Late
```

```
# Define Trial Parameters
n_per_stratum <- 20  # We will randomize 20 patients in EACH of the 4 strata
treatment_arms <- c("New Drug", "Control")
# We will use blocks of size 4 and 6 (2 and 3 per arm)
block_sizes_per_arm <- c(2, 3)
# Loop Through Strata and Build the Master List
# First, create an empty data frame to store the final, combined list
master_randomization_list <- data.frame()

# Now, loop through each row of the strata_df (i.e., each stratum)
for (i in 1:nrow(strata_df)) {
    # Get the current stratum's information
    current_site <- strata_df$site[i]
    current_stage <- strata_df$disease_stage[i]
    cat(paste("\n--- Generating list for Stratum:",
                    current_site, "-", current_stage, "---\n"))
    # Generate an independent randomization list for ONLY this stratum
    temp_rand_list <- blockrand(
        n = n_per_stratum,
        levels = treatment_arms,
        block.sizes = block_sizes_per_arm
    )
```

```
    # Add columns to "tag" this temporary list with the stratum info temp_rand_list$site <-
    current_site
    temp_rand_list$disease_stage <- current_stage
    # Append this temporary list to our master list
    master_randomization_list <- rbind(master_randomization_list,
                                       temp_rand_list)
}
```

```
##
## --- Generating list for Stratum: Site A - Early ---
##
## --- Generating list for Stratum: Site B - Early ---
##
## --- Generating list for Stratum: Site A - Late ---
##
## --- Generating list for Stratum: Site B - Late ---
```

```
# View and Verify the Final List
cat("\n Final Master Randomization List (First 10 rows) \n")
print(head(master_randomization_list, 10))
```

```
##
##  Final Master Randomization List (First 10 rows)
##    id block.id block.size treatment site disease_stage
## 1   1        1          4  New Drug Site A      Early
## 2   2        1          4   Control Site A      Early
## 3   3        1          4   Control Site A      Early
## 4   4        1          4  New Drug Site A      Early
## 5   5        2          4   Control Site A      Early
## 6   6        2          4  New Drug Site       A
## 7   7        2          4  New Drug Site       A
## 8   8        2          4   Control Site       A
## 9   9        3          4   Control Site       A
## 10 10        3          4  New Drug Site       A
```

```
cat("\n Final Master Randomization List (Last 10 rows) \n")
print(tail(master_randomization_list, 10))
```

```
##
##  Final Master Randomization List (Last 10 rows)
##          id block.id block.size treatment site disease_stage
##  75 11   3        6  New Drug Site B      Late
##  76 12   3        6   Control Site B      Late
##  77 13   3        6  New Drug Site B      Late
##  78 14   3        6   Control Site B      Late
##  79 15   3        6   Control Site B      Late
##  80 16   3        6  New Drug Site B      Late
##  81 17   4        4   Control Site B      Late
##  82 18   4        4   Control Site B      Late
##  83 19   4        4  New Drug Site B      Late
##  84 20   4        4  New Drug Site B      Late
```

```
cat("\n Final Verification of Balance \n")
```

```
##
##    Final Verification of Balance
```

```
# The final table is the most important check.
# It must show perfect balance.
balance_table <- table(master_randomization_list$treatment,
    master_randomization_list$site,
    master_randomization_list$disease_stage
)
balance_table
```

```
## , ,  = Early
##
##
##                  Site A  Site B
##       Control    12      10
##       New Drug   12      10
##
## , ,  Late
##
##
##                   Site A  Site B
##       Control     10      10
##       New Drug    10      10
```

The output shows a good balance between strata and groups.

10.4. Examples of Published Clinical Trial Data Analysis Using R

Below are some journal articles for clinical trials analyzed by the author Nhan Thi Ho et al. using R.

- Example Clinical Trial Paper 1: "Gatifloxacin versus ceftriaxone for uncomplicated enteric fever in Nepal" published in *Lancet Infectious Diseases* [96].
- Example Clinical Trial Paper 2: "A Trial of Itraconazole or Amphotericin B for HIV-Associated Talaromycosis" published in the N*ew England Journal of Medicine* [97].

10.5. Case Study: Example Analysis of a Vaccine Clinical Trial Data

The data for the examples below are generated to mimic the data from a multi-centered phase 1/2/3 COVID-19 vaccine trial in which the author Nhan Thi Ho participated as a core researcher and data scientist [98]. This generated dataset is used in the following examples to demonstrate the use of R for clinical trial data analysis and report (although the actual interim and final analyses for this trial for publication and authority approval were done in SAS). This case study is designed to provide example R codes to perform the common analysis of clinical trial data and present the results as publication-ready tables and figures. Interested readers should read the original paper for more details, description, explanation, and interpretation.

The example data contains the following datasets:

- *ran.ex*: randomization list and analysis sets;
- *base.ex*: baseline data;
- *ve.ex*: time to event outcome data;
- *sae.ex*: serious adverse event data;
- *ae.ex*: adverse event data.

10.5.1. Load Required Packages and Example Data

We load the packages "*arsenal*" [19], "*tidyverse*" [11] for baseline, outcome, adverse event summary, "*survival*" [51], "*survminer*" [52] for efficacy analysis of time-to-event outcomes, and load the example data from GitHub.

```
# load multiple packages
Packages <- c("arsenal", "tidyverse","survival", "survminer")
lapply(Packages, library, character.only = TRUE)
# Data path
url_base <- "https://raw.githubusercontent.com/nhanhocu/"
data_path <- "biodata-r/main/trialdata.rda"
data_url <- paste0(url_base, data_path)
# load datasets
Con <- url(data_url) print(load(con))
close(con)
# quick look at the datasets
str(ran.ex)
str(base.ex)
str(ve.ex)
str(sae.ex)
str(ae.ex)
```

10.5.2. Baseline Summary

Note that, for clinical trial data, we prepare a variable denoting analysis sets (e.g., intention to treat (ITT), per protocol, safety analysis sets) so that we can easily choose an analysis set for analysis and result display. This allows for a robust "sensitivity check"—evaluating if the study's conclusions remain consistent regardless of which subset of participants is analyzed, or the results are different with different analysis sets (which requires further evaluation for clinical trial issues such as the "Adherence Gap", Attrition Bias, or Internal Validity vs. Biological Signal. These are beyond the scope of this chapter.) The example below focuses on the ITT analysis set, which is the gold standard for randomized clinical trials.

The "Baseline Summary" table—universally recognized as "Table 1" in clinical trial publications—serves as the foundational evidence for the success of randomization. It provides an overview of the demographic and clinical profile of participants at the moment of enrollment between randomized arms, before any intervention effect could occur. The examples below are to illustrate the use of *tableby.control()* and *tableby()* of the package "*arsenal*" [19] for descriptive summary tables per treatment arm for ITT analysis set (*itt==1*). The codes are similar to the examples in previous chapters (Table 33).

```
# Set control option for data summary
d_controls <- tableby.control(
    test = TRUE,
    total = FALSE,
    numeric.test = "kwt",
    cat.test = "chisq",
    numeric.stats = c("meansd", "medianq1q3","meanCI","range","Nmiss2"),
    cat.stats = c("countpct", "Nmiss2"),
    stats.labels = list(
```

```
      meansd = "Mean (SD)",
      medianq1q3 = "Median (Q1, Q3)", range = "Min
      - Max",
      meanCI = "Mean (95%CI)",
      Nmiss2 = "Missing"
    )
)
dvar <- c("age","agecat","sex","riskcov")
# Set labels for variables to be summarized
mylabels  <-as.list(c("Age (year)","Age category (year)",
                      "Sex","Risk category"))
names(mylabels) <-dvar
# Create summary table
tabd  <- tableby(as.formula(paste("treatment",paste(dvar,collapse="+"),
                                  sep="~")),
                 data = subset(base.ex, itt=="1"),
                 control = d_controls)
kable(summary(tabd,labelTranslations = mylabels, text=TRUE), caption="Summary of
      baseline data")%>%
  kable_styling(latex_options = c("scale_down", "hold_position"))
```

Table 33. Summary of baseline data.

	Placebo (N = 7345)	**Vaccine (N = 7753)**	***p*-Value**
Age (year)			0.004
- Mean (SD)	46.539 (13.406)	45.964 (13.270)	
- Median (Q1, Q3)	48.000 (37.000, 57.000)	47.000 (36.000, 56.000)	
- Mean (95% CI)	46.539 (46.232, 46.846)	45.964 (45.668, 46.259)	
- Min−Max	17.000–89.000	17.000–85.000	
- Missing	0	0	
Age category (year)			0.022
- ≤60	6199 (84.4%)	6646 (85.7%)	
- >60	1146 (15.6%)	1107 (14.3%)	
- Missing	0	0	
Sex			0.684
- Mean (SD)	1.645 (11.650)	1.634 (11.340)	
- Median (Q1, Q3)	2.000 (1.000, 2.000)	2.000 (1.000, 2.000)	
- Mean (95% CI)	1.645 (1.378, 1.911)	1.634 (1.382, 1.887)	
- Min−Max	1.000–999.000	1.000–999.000	
- Missing	0	0	
Risk category			0.827
- At_risk	3705 (50.4%)	3897 (50.3%)	
- Healthy	3640 (49.6%)	3856 (49.7%)	
- Missing	0	0	

Source: Table by author.

The output table shows the baseline summary comparing the Vaccine and Placebo arms across a range of prognostic factors, including age, sex, and risk category. While the randomization process technically makes *p*-values in a baseline table redundant (since any differences are by definition due to chance), they remain a diagnostic tool in clinical trial reporting. A significantly small *p*-value (e.g., <0.05 for age) would flag a chance imbalance that might require adjustment in subsequent multivariable efficacy models.

10.5.3. Time-to-Event Outcome Summary

After performing baseline summary, the analysis shifts to the primary objective of the clinical trial: the efficacy of the intervention over time. Time-to-Event Outcome Summary quantifies the occurrence of the primary endpoint from the moment of randomization until the end of the predefined follow-up period.

Summary with Package "Arsenal"

We can do a quick summary of number of event and follow-up time by group for modified intention to treat (mITT) analysis set at Day 92 (*mitt_d92=="1"*). The function *tableby()* of the package *"arsenal"* [19] may also be used for overall summary and for subgroup stratification (Healthy and At risk) (Table 34).

Overall summary:

```
o_controls <- tableby.control(
    test = FALSE,
    total  =  FALSE,
    numeric.test  =  "kwt",
    cat.test  =  "chisq",
    numeric.stats  =  "arsenal_sum",
    cat.stats = c("countpct", "Nmiss2"),
    stats.labels = list(arsenal_sum ="Sum"))
ovar  <-  c("event","ttevent","event3692","ttevent3692")
mylabels <- as.list(c("Event (any time)","Follow-up time (day)",
                      "Event between Day 36-92", "Follow-up time between
                      36-92"))
names(mylabels) <-ovar
tabo <-  tableby(as.formula(paste("treatment",
                                  paste(ovar,collapse="+"),sep="~")),
                 data = subset(ve.ex, mitt_d92=="1"),
                 control  =  o_controls)
kable(summary(tabo,labelTranslations = mylabels, text=TRUE),
      caption="Event overall")%>%
    kable_styling(latex_options  =  c("scale_down",  "hold_position"))
```

Table 34. Event overall.

	Placebo (N = 6825)	**Vaccine (N = 6849)**
Event (any time) - Sum	735.000	539.000
Follow-up time (day) - Sum	618,244.000	625,475.000
Event between Day 36–92 - Sum	432.000	220.000
Follow-up time between 36 and 92 - Sum	601,174.000	607,577.000

Source: Table by author.

Summary table may be stratified by risk category (Healthy and At risk) by defining "*strata=riskcov*" within the *tableby*() function (Table 35):

```
Tabo <- tableby(as.formula(paste("treatment",
                                 paste(ovar,collapse="+"), sep="~")),
                data = subset(ve.ex, mitt_d92=="1"),
                strata=riskcov, #stratum for stratification
                control = o_controls)
kable(summary(tabo,labelTranslations = mylabels, text=TRUE),
      caption="Event by risk")%>%
  kable_styling(latex_options = c("scale_down", "hold_position"))
```

Table 35. Event by risk.

Riskcov		Placebo (N = 6825)	Vaccine (N = 6849)
Healthy	Event (any time)		
	- Sum	405.000	324.000
	Follow-up time (day)		
	- Sum	308,028.000	309,900.000
	Event between Day 36–92		
	- Sum	222.000	133.000
	Follow-up time between 36–92		
	- Sum	297,987.000	299,884.000
At_risk	Event (any time)		
	- Sum	330.000	215.000
	Follow-up time (day)		
	- Sum	310,216.000	315,575.000
	Event between Day 36–92		
	- Sum	210.000	87.000
	Follow-up time between 36–92		
	- Sum	303,187.000	307,693.000

Source: Table by author.

The output tables show the total (sum) number of events and the total follow-up times between groups in the overall data and in strata (healthy, at risk).

Manual Calculation for Later Table Combination

Usually, we want to present a compact outcome summary table combining multiple results. As such, we can do a manual summary for multiple components to make it easier to later combine multiple summaries into a table.

Summary of the number of events and follow-up time between arms for all data:

```
esum <- ve.ex %>%
  filter(mitt_d92=="1") %>%
  group_by(treatment) %>%
  summarise_at(.vars = vars(event3692, ttevent3692),
               .funs = sum) %>%
  pivot_wider(
    names_from = treatment,
    values_from = c(event3692, ttevent3692)
  )
esum
```

```
##     # A tibble: 1 × 4
##     event3692_Placebo   event3692_Vaccine   ttevent3692_Placebo
##     <dbl>               <dbl>               <dbl>
## 1   432                 220                 601,174
## # i 1 more variable: ttevent3692_Vaccine <dbl>
```

The summary may be stratified by each risk subgroup (healthy and at risk), and then the results of all groups and the results of the overall data are combined:

```
esums <-ve.ex %>%
   filter(mitt_d92=="1")  %>%
   group_by(treatment,riskcov) %>%
   summarise_at(.vars = vars(event3692, ttevent3692),
                        .funs = sum) %>%
   pivot_wider(
      names_from = treatment,
      values_from = c(event3692, ttevent3692)
     )
   esums
```

```
##      # A tibble: 2 × 5
##      riskcov    event3692_Placebo   event3692_Vaccine   ttevent3692_Placebo
##      <fct>      <dbl>               <dbl>               <dbl>
## 1    Healthy    222                 133                 297,987
## 2    At_risk    210                 87                  303,187
## # i 1 more variable: ttevent3692_Vaccine <dbl>
```

```
#Combined results
esuma <-bind_rows(esum, esums)
esuma <-esuma%>%
   select(riskcov, contains("Placebo"), contains("Vaccine"))
esuma
```

```
##      # A tibble: 3 × 5
##      riskcov    event3692_Placebo   ttevent3692_Placebo   event3692_Vaccine
##      <fct>      <dbl>               <dbl>                 <dbl>
## 1    <NA>       432                 601,174               220
## 2    Healthy    222                 297,987               133
## 3    At_risk    210                 303,187               87
## # i 1 more variable: ttevent3692_Vaccine <dbl>
```

10.5.4. Vaccine Efficacy Cox Model

We use the Cox proportional hazard model to estimate vaccine efficacy (vaccine efficacy = (1 − hazard ratio)*100%) and combine the results from the Cox models with the above event and follow-up time summary. We use the *coxph()* function from the "*survival*" package [51]. The model estimates the hazard ratio comparing the vaccine group with the placebo group.

Outcome Summary Table

In the examples below, we fit Cox models for all data and for each stratum, then combine the results of the Cox models (we select the analysis set mITT at Day 92). The mITT population typically includes participants who received at least one dose of the study product and were followed for outcome events according to the study protocol.

First, we fit the Cox model and summarize the results for all data:

```
# Cox model all
cfit  <-coxph(Surv(ttevent3692,event3692)~treatment,
                      data=subset(ve.ex, mitt_d92=="1"))
hrtab <-summary(cfit)$conf.int[,c(1,3:4)]
names(hrtab)  <-c("HR","lower.95","upper.95")
# HR all
hrtab
```

```
##    HR       lower.95    upper.95
##    0.498    0.424       0.586
```

```
# VE all
ve <-1-hrtab
ve <-c(ve[1],ve[3],ve[2])
names(ve) <-c("efficacy","lower.95","upper.95")
ve
```

```
##    efficacy    lower.95    upper.95
##    0.502       0.414       0.576
```

Then we do stratification by risk category (defined by the variable "riskcov" including two strata, healthy and at risk) by fitting the Cox model separately for risk strata and summarizing the results for each stratum. This is to better understand whether vaccine efficacy differs across participant subgroups. Stratified analyses are commonly conducted in vaccine trials to determine whether protective effects vary according to baseline health status or comorbidity risk.

At-risk participants (*riskcov=At_risk*):

```
datr <-subset(ve.ex,mitt_d92==1 & riskcov=="At_risk")
fitr <-coxph(Surv(ttevent3692,event3692)~treatment, data=datr)
tabr <-summary(fitr)$conf.int[,c(1,3:4)]
names(tabr) <-c("HR","lower.95","upper.95")
#  HR  riskcov=At_risk
tabr
```

```
##    HR       lower.95    upper.95
##    0.404    0.314       0.518
```

```
#  VE  riskcov=At_risk
ver <-1-tabr
ver <-c(ver[1],ver[3],ver[2])
names(ver) <-c("efficacy","lower.95","upper.95")
ver
```

```
##    efficacy    lower.95    upper.95
##    0.596       0.482       0.686
```

Healthy participants (*riskcov=Healthy*):

```
dath <-subset(ve.ex,mitt_d92==1 & riskcov=="Healthy")
fith <-coxph(Surv(ttevent3692,event3692)~treatment, data=dath)
tabh <-summary(fith)$conf.int[,c(1,3:4)]
names(tabh) <-c("HR","lower.95","upper.95")
#  HR  riskcov=Healthy
tabh
```

```
##    HR       lower.95    upper.95
##    0.589    0.475       0.730
```

```
#  VE  riskcov=Healthy
veh <-1-tabh
veh <-c(veh[1],veh[3],veh[2])
names(veh) <-c("efficacy","lower.95","upper.95")
veh
```

```
##   efficacy   lower.95   upper.95
##   0.411      0.270      0.525
```

Now, we combine the results from the above outcome (event and follow-up time) summary and from the Cox models for all data and each stratum into a nice table. These descriptive metrics provide important context for interpreting the efficacy estimates: the absolute burden of disease observed in each group (total events), the cumulative observation period across participants (total follow-up time), and vaccine efficacy and 95% CI (Table 36).

```
#Combine efficacy results
ve.all <- round(rbind(ve, veh, ver),2)
ve.s <-paste0(ve.all[,1]," (", ve.all[,2],",",ve.all[,3],")")

#Combine event summary with efficacy results:
esumas <-cbind(esuma,ve.s)
esumas$riskcov  <-c("All  participants","-  Healthy",
                    "- At risk")
kable(esumas,
      caption = "Summary of vaccine efficacy",
      col.names = c("Study population",
                    "Placebo: Total event",
                    "Placebo: Total follow-up (day)", "Vaccine: Total event",
                    "Vaccine: Total follow-up (day)", "Vaccine efficacy
                    (95%CI)"))%>%
  kable_styling(latex_options = c("scale_down", "hold_position"))
```

Table 36. Summary of vaccine efficacy.

Study Population	Placebo: Total Event	Placebo: Total Follow-Up (Day)	Vaccine: Total Event	Vaccine: Total Follow-Up (Day)	Vaccine Efficacy (95% CI)
All participants	432	601,174	220	607,577	0.5 (0.41,0.58)
- Healthy	222	297,987	133	299,884	0.41 (0.27,0.52)
- At risk	210	303,187	87	307,693	0.6 (0.48,0.69)

Source: Table by author.

The output table shows the total number of events, follow-up time for Vaccine and Placebo groups, and vaccine efficacy and 95% CI; e.g., vaccine efficacy in overall data is 0.5 (the hazard of events is reduced by 50% in the Vaccine group as compared to the Placebo group), and this is statistically significant (95% CI = (0.41, 0.58)). Vaccine efficacy in "Healthy" participants is 41%, and in "At risk" participants is 60%. This pattern suggests that the vaccine may provide greater benefit in populations with elevated baseline risk.

Cumulative Incidence Plot

In efficacy trials, we often plot the Cumulative Incidence ($1 - \text{Survival}$). This quantity represents the probability that a participant has experienced the event by time t. Similar to the examples in previous chapters, we create cumulative incidence plots for treatment groups for all data and stratify by risk groups using the function *surv_fit()* and *ggsurvplot()* (with *fun* = *"event"*) of the "*survminer*" package [52].

First, we create a *survminer::surv_fit* model object (note that we clarify *survminer::surv_fit* to tell R to use specifically the *surv_fit()* function of the "*survminer*" package) and plot the generated object to create the cumulative incidence plot for all data (Figure 117).

```
fit <-survminer::surv_fit(Surv(ttevent3692,event3692)~treatment,
                          data=subset(ve.ex, mitt_d92=="1"))
cuminca <-ggsurvplot(fit,
                     linetype="strata",
                     conf.int=FALSE,
                     pval=TRUE,
                     censor=FALSE,
                     pval.coord=c(43,0.15),
                     risk.table = TRUE,
                     risk.table.height = 0.25,
                     break.x.by =7,
                     xlim=c(36,92),
                     fun = "event", legend.title
                     ="",
                     tables.col = "strata",
                     legend.labs=c("Placebo","Vaccine"),
                     xlab="Follow-up day from Day 1",
                     ylab="Event cumulative incidence",
                     risk.table.y.text = FALSE)
cuminca$plot <-cuminca$plot+
   scale_x_continuous(breaks = c(0,36,43,50,57,64,71,78,85,92))
cuminca$table <-cuminca$table+
   scale_x_continuous(breaks = c(0,36,43,50,57,64,71,78,85,92))
cuminca
```

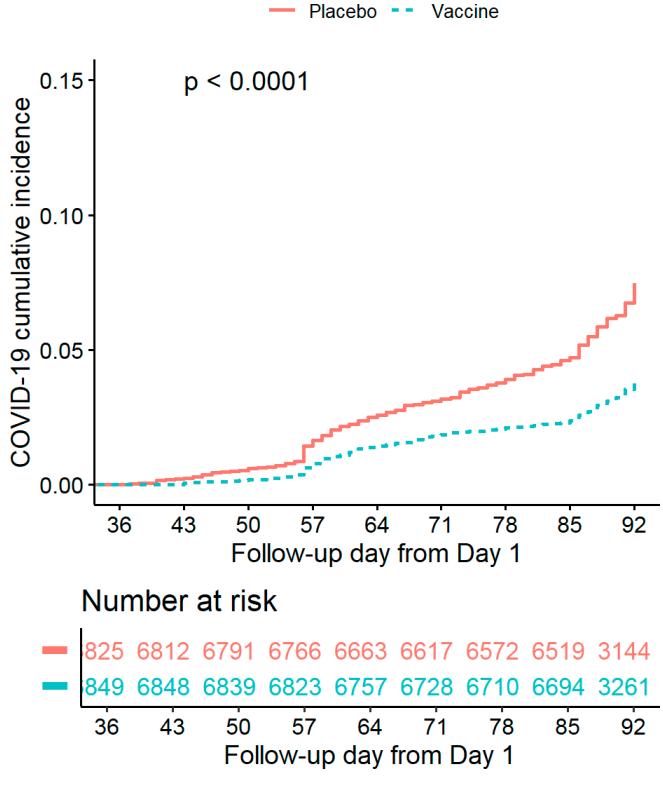


Figure 117. Cumulative incidence plot. Source: Figure by author.

The output plot shows a "widening gap" between the two curves, indicating an increasing protective effect of the vaccine over follow-up time. This visual pattern is consistent with the hazard ratio estimated from the Cox model.

Next, we create the plot stratified by Risk Category by adding "riskcov" to the model and plotting the generated *survminer::surv_fit* object. This plot allows investigators to visually assess whether the magnitude of vaccine protection differs between subgroups (Figure 118).

```
fit <-survminer::surv_fit(Surv(ttevent3692,event3692)~treatment + riskcov, data=subset(ve.ex,
                  mitt_d92=="1"))
cumincr <-ggsurvplot(fit, linetype="strata", conf.int=FALSE,
                pval=FALSE, censor=FALSE,
                risk.table = TRUE,
                risk.table.height = 0.3,
                break.x.by =7,
                xlim=c(36,92), fun =
                "event", legend.title
                ="",
                legend.labs=c("Placebo.Healthy", "Placebo.At_risk",
                              "Vaccine.Healthy","Vaccine.At_risk"),
                tables.col = "strata",
                xlab="Follow-up day from Day 1",
                ylab="Event cumulative incidence",
                risk.table.y.text = FALSE)
cumincr$plot <-cumincr$plot+
  scale_x_continuous(breaks = c(0,36,43,50,57,64,71,78,85,92))
cumincr$table <-cumincr$table+
  scale_x_continuous(breaks = c(0,36,43,50,57,64,71,78,85,92))
cumincr
```

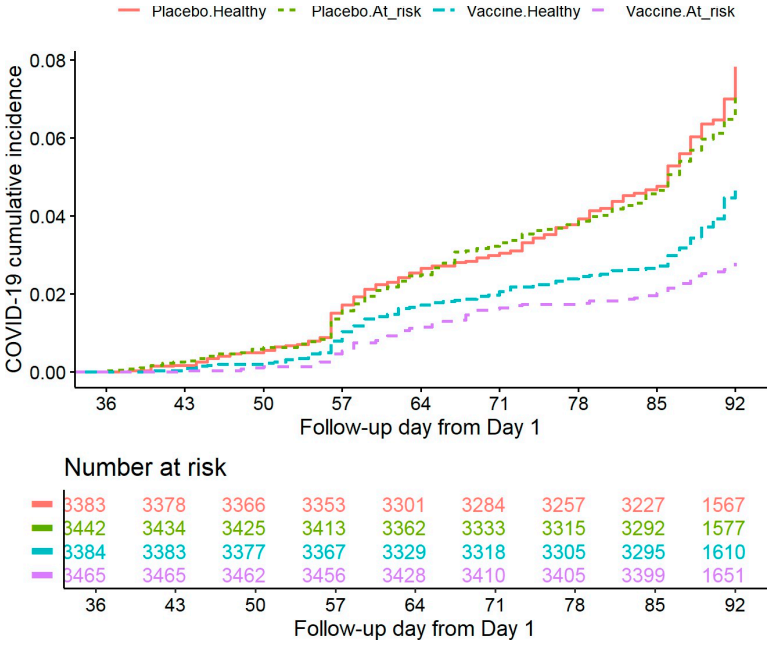


Figure 118. CIF stratified by risk category. Source: Figure by author.

The output figure shows that the gap between the Placebo vs. Vaccine group is wider in "At risk" participants as compared to "Healthy" participants. This

is consistent with the results in the vaccine efficacy summary table from the Cox model above.

The combination of Cox regression models, descriptive summaries of event counts and follow-up time, and cumulative incidence plots provides a comprehensive assessment of vaccine efficacy. These analyses and result presentations are commonly included in regulatory submissions and clinical study reports.

10.5.5. Safety Data Summary

Safety analyses represent a critical component of clinical trial reporting. While efficacy analyses evaluate whether an investigational product achieves its intended therapeutic effect, safety analyses assess whether the treatment introduces unacceptable risks to study participants. In regulatory submissions and clinical study reports, safety summaries are required to describe the frequency, severity, and clinical characteristics of adverse events across treatment groups.

The examples presented in this section illustrate how safety data can be summarized using R consistently with the tabular formats typically required in clinical study reports. These examples demonstrate a reproducible workflow for transforming raw adverse event datasets into reporting-ready tables that summarize key safety outcomes. Specifically, the examples show how to:

- Summarize Serious Adverse Events (SAEs) by severity, treatment relatedness, and death outcome;
- Summarize Adverse Events (AEs) by System Organ Class (SOC) and Preferred Term (PT);
- Calculate the number and percentage of affected participants;
- Generate tables suitable for inclusion in regulatory reports.

The R code uses functions from the "*tidyverse*" ecosystem, which provides efficient tools for data transformation, aggregation, and reporting. The "*tidyverse*" functions like *group_by()*, *summarise()*, *pivot_longer()*, and *pivot_wider()* are useful for these tasks [22].

Some parts of the example R codes below for safety analysis were adapted from the book "R for Clinical Study Reports and Submission" [99].

Summary of Serious Adverse Events (SAE)

First, we do a summary of Serious Adverse Events (SAE) by Severity, investigational product (IP) Relatedness, and Death on the safety analysis set. The Safety Analysis Set typically includes all participants who received at least one dose of the investigational product or placebo.

We calculate the total number of participants in each treatment group. These totals serve as the denominators for calculating event percentages.

```
pop <-ran.ex %>%
    filter(sas == "1") %>%
    dplyr::count(treatment,  name="tot")
pop
```

```
##       treatment   tot
## 1     Vaccine     7753
## 2     Placebo     7345
```

These counts (7753 in the Vaccine group and 7345 in the Placebo group) represent the population at risk and will be used to compute the percentage of participants experiencing each safety outcome.

Then we reformat SAE data and summarize the number and percentage of participants in the safety analysis set who meet each SAE criteria and for each category: IP relatedness ("drug"), seriousness ("ser"), and death ("die"); transform the dataset to a long dataset with *pivot_longer()*, write a simple function to summarize the number and percentage of participants who meet each AE criteria, summarize the number of participants with SAE, and then merge with the total number of participants to calculate the percentage. The output shows for each category the number of participants with SAE, the number of participants, and the percentage:

```
tidy_sae <- sae.ex %>% mutate(
    all = sas == "1",
    drug = aerel %in% "1", ser =
    aesev == "3",
    die = aeout == "1"
  ) %>%
  select(usubjid, treatment, all, drug, ser, die) %>%
  pivot_longer(cols = c(all, drug, ser, die))
#summarize the number and percentage of participants who meet each AE criteria.
fmt_num <- function(x, digits, width = digits + 4) { formatC(
    x,
    digits = digits, format
    = "f", width = width
  )
}
#summarize sae
sum.sae <-tidy_sae %>% filter(value ==
  TRUE) %>% group_by(treatment,
  name) %>%
  summarise(n = n_distinct(usubjid))
sum.sae.pop <-merge(sum.sae,pop,by = "treatment", all.x=TRUE)
ana <- sum.sae.pop %>%
  mutate(
    pct = fmt_num(n / tot * 100, digits = 1), n =
    fmt_num(n, digits = 0),
    pct = paste0("(", pct, ")")
      )
    ana
```

```
##   treatment   name   n    tot    pct
## 1Placebo     all    125  7345   (1.7)
## 2Placebo     die    6    7345   (0.1)
## 3Placebo     drug   7    7345   (0.1)
## 4Placebo     ser    31   7345   (0.4)
## 5Vaccine     all    130  7753   (1.7)
## 6Vaccine     die    9    7753   (0.1)
## 7Vaccine     drug   10   7753   (0.1)
## 8Vaccine     ser    44   7753   (0.6)
```

The output shows that the proportion of participants experiencing at least one SAE is similar between the two groups (Vaccine group: 130 participants (1.7%) vs. Placebo group: 125 participants (1.7%)). This finding suggests that the overall incidence of serious adverse events is comparable between groups.

Now we prepare a reporting-ready dataset for each AE group. After summarizing the SAE data, the results are reformatted into a structure suitable for reporting. This step involves converting the dataset back into wide format using *pivot_wider()* so that treatment groups appear as separate columns. The resulting table includes formatted strings combining counts and percentages (e.g., "125 (1.7)"). This formatting approach mirrors the presentation commonly used in clinical study reports.

Finally, the total number of participants in each treatment group is added to the top of the table to provide the denominator context for all percentages. The resulting output table is a concise summary of serious adverse events by treatment group.

```
# Prepare reporting-ready dataset for each AE group.
t_sae <- ana %>% pivot_wider(
      id_cols = "name",
      names_from = treatment,
      values_from = c(n, pct),
            values_fill = list(
            n = "     0",
         pct = "(      0.0)"
      )
   )%>%
   mutate(Placebo=paste0(n_Placebo, "  ", pct_Placebo), Vaccine=paste0(n_Vaccine, " ",
               pct_Vaccine))%>%
   select(name,Placebo, Vaccine)
t_sae
```

```
##      # A tibble: 4 × 3
##      name    Placebo       Vaccine
##      <chr>   <chr>         <chr>
## 1    all     "125 (1.7)"   "130 (1.7)"
## 2    die     "6 (0.1)"     "9 (0.1)"
## 3    drug    "7 (0.1)"     "10 (0.1)"
## 4    ser     "31 (0.4)"    "44 (0.6)"
```

Then we prepare a reporting-ready dataset for the number of participants by treatment group and combine it with the reporting-ready dataset for the AE group above (Table 37):

```
t_pop <- pop %>%
   mutate(name="sas")%>%
      pivot_wider(
      names_from = treatment, values_from =
      tot
   )%>%
   select(name, Placebo, Vaccine)
t_pop <-lapply(t_pop,as.character)
dp_sae <-bind_rows(t_pop,t_sae)%>%
   mutate(name = factor(
      name,
      c("sas","all", "drug", "ser", "die"),
      c("Participants in SAS",
         "- With one or more SAE", "- With IP-
         related SAE",  "- With grade 3 SAE",
         "- With SAE with death outcome"
      )
      ))
    dp_sae
```

```
##      # A tibble: 5 × 3
##      name                          Placebo      Vaccine
##      <fct>                         <chr>        <chr>
## 1    Participants in SAS           "7345"       "7753"
## 2    - With one or more SAE        "125 (1.7)"  "130 (1.7)"
## 3    - With SAE with death outcome "6 (0.1)"    "9 (0.1)"
## 4    - With IP-related SAE         "7 (0.1)"    "10 (0.1)"
## 5    - With grade 3 SAE            "31 (0.4)"   "44 (0.6)"
```

We now make a reporting-ready table for the dataset:

```
kable(dp_sae,
      caption = "Summary of serious adverse events",
      col.names = c("Study population",
                    "Placebo n(%)",
                    "Vaccine n(%)"))%>%
  kable_styling(latex_options = c("scale_down", "hold_position"))
```

Table 37. Summary of serious adverse events.

Study Population	Placebo n (%)	Vaccine n (%)
Participants in SAS	7345	7753
- With one or more SAE	125 (1.7)	130 (1.7)
- With SAE with death outcome	6 (0.1)	9 (0.1)
- With IP-related SAE	7 (0.1)	10 (0.1)
- With grade 3 SAE	31 (0.4)	44 (0.6)

Source: Table by author.

The output shows a concise publication-ready table for Serious Adverse Events (SAEs). One may apply similarly for summarizing Adverse Events (AEs).

Summary of AE by SOC and PT

Safety reporting often includes summarizing Adverse Events (AEs) by System Organ Class (SOC) and Preferred Term (PT) by treatment group. These classifications follow the MedDRA (Medical Dictionary for Regulatory Activities) hierarchy, which organizes adverse events according to physiological systems and specific clinical diagnoses.

The purpose of this analysis is to identify patterns in adverse events across different organ systems and to determine whether particular types of events occur more frequently in the Vaccine or Placebo group.

First, we reformat AE data and summarize the number and percentage of participants by SOC: the main logic is to count the unique subjects (*n_distinct(usubjid)*) who experienced an AE within each treatment and SOC group and then join this with the total subject count to calculate the percentage. Counting the number of unique subject identifiers within each treatment and event category ensures that each participant is counted only once within a given SOC (or PT) category, which aligns with regulatory guidance for safety reporting.

For each SOC category and treatment group, the example calculates the number of participants experiencing at least one AE within that SOC and the corresponding percentage relative to the total number of participants.

```
t1 <- ae.ex %>%
  group_by(treatment, soc) %>%
  summarise(n = fmt_num(n_distinct(usubjid), digits = 0)) %>%
  mutate(pt = soc, order = 0)%>%
  mutate(n=as.numeric(n))
t1.pop <-merge(t1, pop,by = "treatment", all.x=TRUE)
t1.np <- t1.pop %>%
  mutate(
    pct = fmt_num(n / tot * 100, digits = 1),
    n = fmt_num(n, digits = 0),
    pct = paste0("(", pct, ")")
  )
```

Next, the analysis then proceeds to a more detailed level by summarizing adverse events by Preferred Term (PT) within each SOC. These PT-level summaries allow researchers to identify specific clinical conditions that contribute to the broader SOC categories.

We reformat AE data and summarize the number and percentage of participants by SOC and PT:

```
t2 <- ae.ex %>%
  group_by(treatment, soc, pt) %>%
  summarise(n = fmt_num(n_distinct(usubjid), digits = 0)) %>%
  mutate(order = 1)%>%
  mutate(n=as.numeric(n))
t2.pop <-left_join(t2, pop, by="treatment")
t2.np <- t2.pop %>%
  mutate(
    pct = fmt_num(n / tot * 100, digits = 1),
    n = fmt_num(n, digits = 0),
    pct = paste0("(", pct, ")")
  )
head(t2.np)
```

```
###A tibble: 6 × 7
###Groups: treatment, soc [2]
## treatmentsoc                    pt          n     order tot   pct
## <fct>     <chr>                 <chr>       <chr><dbl><int><chr>
##1Vaccine  Blood and lymphaticsy~Anae~ "  ~   1     7753(0~
##2Vaccine  Blood and lymphaticsy~Lymp~"  ~   1     7753(0~
##3Vaccine  Blood and lymphaticsy~Lymp~"  ~   1     7753(0~
##4Vaccine  Cardiac disorders       Acut~ "  ~   1     7753(0~
##5Vaccine  Cardiac disorders       Angi~ "  ~   1     7753(0~
##6Vaccine  Cardiac disorders       Angi~ "  ~   1     7753(0~
```

Then, the last step combines SOC-level and PT-level summaries into a single hierarchical table. The *pivot_wider()* function is used to place the Vaccine and Placebo groups into separate columns, while the *mutate()* function constructs formatted values combining counts and percentages. Indentation is added for PT-level rows to visually distinguish them from their parent SOC categories. This formatting produces a structure commonly used in clinical safety tables (Table 38).

```
np_ae <-bind_rows(t1.np, t2.np) %>%
    pivot_wider(
        id_cols = c(soc, order, pt),
        names_from = treatment,
        values_from = c(n,pct),
        values_fill = fmt_num(0, digits = 0)
    ) %>%
    arrange(soc, order, pt) %>%
    mutate(Vaccine=paste0(n_Vaccine," ",pct_Vaccine),
                Placebo=paste0(n_Placebo," ",pct_Placebo)) %>%
    mutate(AE = case_when(
        order==1 ~ paste0("- ",pt),
        TRUE ~ pt)) %>%
    select(AE,Placebo,Vaccine)
pop.r <-c("Participants in SAS",
                pop$tot[pop$treatment %in% "Placebo"],
                pop$tot[pop$treatment %in% "Vaccine"])
names(pop.r) <-names(np_ae)
np_ae_pop <-bind_rows(pop.r, np_ae)
# Show first 30 rows
kable(head(np_ae_pop,30), format = "latex", longtabl = TRUE,
          col.names = c("Adverse events SOC and PT", "Placebo n (%)", "Vaccine n (%)"),
          caption = "Summary of adverse events by SOC and PT")%>%
          kable_styling(latex_options = c("scale_down", "hold_position"))
```

Table 38. Summary of adverse events by SOC and PT.

Adverse Events SOC and PT	Placebo n (%)	Vaccine n (%)
Participants in SAS	7345	7753
Blood and lymphatic system disorders	5 (0.1)	6 (0.1)
- Anemia	0 0	1 (0.0)
- Iron deficiency anemia	1 (0.0)	0 0
- Lymphadenitis	2 (0.0)	3 (0.0)
- Lymphadenopathy	2 (0.0)	2 (0.0)
Cardiac disorders	411 (5.6)	453 (5.8)
- Acute myocardial infarction	2 (0.0)	1 (0.0)
- Angina pectoris	2 (0.0)	1 (0.0)
- Angina unstable	1 (0.0)	1 (0.0)
- Arrhythmia	2 (0.0)	1 (0.0)
- Atrial fibrillation	1 (0.0)	0 0
- Atrioventricular block complete	0 0	1 (0.0)
- Bradycardia	4 (0.1)	6 (0.1)
- Chest discomfort	11 (0.1)	11 (0.1)
- Chest pain	8 (0.1)	14 (0.2)
- Dizziness	19 (0.3)	28 (0.4)
- Dyspnoea	9 (0.1)	4 (0.1)
- Heart valve incompetence	0 0	1 (0.0)
- Mitral valve incompetence	1 (0.0)	0 0
- Myocardial ischaemia	3 (0.0)	5 (0.1)
- Palpitations	0 0	1 (0.0)
- Sinus arrhythmia	0 0	1 (0.0)
- Syncope	2 (0.0)	1 (0.0)
- Tachycardia	352 (4.8)	382 (4.9)

Table 38. *Cont.*

Adverse Events SOC and PT	Placebo n (%)	Vaccine n (%)
Congenital, familial, and genetic disorders	1 (0.0)	0 0
- Atrial septal defect	1 (0.0)	0 0
Ear and labyrinth disorders	25 (0.3)	43 (0.6)
- Ear infection	1 (0.0)	2 (0.0)
- Ear pain	3 (0.0)	4 (0.1)

Source: Table by author.

Above are some example codes for the analysis and reporting of example clinical trial data. The examples in this section demonstrate how R can be used to construct publication-ready figures and tables efficiently and reproducibly for clinical trial reporting. One may apply what has been learned from previous chapters for further analysis and visualization.

10.6. Useful Resources

10.6.1. Books, Courses

- Fundamentals of Clinical Trials [100].
- Sample Size Calculation in Clinical Research [95].
- Courses: Design and Analysis of Clinical Trials (https://online.stat.psu.edu/stat509/node/164/, accessed on 1 January 2026).
- R for Clinical Study Reports and Submission [99].

10.6.2. Journal Articles

- Example Clinical Trial Paper 1 published in *Lancet Infectious Diseases* [96].
- Example Clinical Trial Paper 2 published in the *New England Journal of Medicine* [97].
- Example Clinical Trial Paper 3 published in *Lancet Global Health* [101].
- Example Clinical Trial Paper 4 published in *Nature Communications* [98].

10.7. Chapter Summary

- "*pwr*", "*powerSurvEpi*", "*TrialSize*", "*SampleSize4ClinicalTrials*", "*gsDesign*" are main packages for sample size—power estimation for various clinical trial designs and outcome types;
- "*blockrand*" and "*randomizeR*" packages may be used for various randomization approaches;
- Main analysis and report for clinical trial data:
    - Baseline summary (e.g., package "*arsenal*");
    - Outcome evaluation (various packages, approaches from previous chapters, e.g., "*survival*", "*survminer*" for time to event outcome);
    - Adverse event summary (e.g., package collection "*tidyverse*").

11. Analysis of Microbiome Data with "metamicrobiomeR"

The "*metamicrobiomeR*" package was developed by Nhan Thi Ho, the author of this book. During her research project at Columbia University Medical Center regarding the meta-analysis of the effects of exclusive breastfeeding on the infant gut microbiome across populations, many difficulties arose. First, there was a lack of statistical methods for the proper analysis of zero-inflated compositional microbiome relative abundance data. Second, available methods for differential analysis of microbiome data did not produce standardized effect estimates that might be pooled across studies for meta-analysis. To address these gaps, she implemented the Generalized Additive Model for Location, Scale, and Shape (GAMLSS) with the zero-inflated beta (BEZI) family for the analysis of microbiome relative abundance data with various options for data transformation or normalization to address compositional effects. Moreover, the differential effect estimates (log(odds ratio)) using this method may be pooled across studies for meta-analysis. The "*metamicrobiomeR*" package is the implementation of these approaches together with some other related functions for microbiome data analysis, meta-analysis, and visualization. The paper describing the "*metamicrobiomeR*" package may be found on BMC Bioinformatics [4]. The package is hosted in R cran [4], and an earlier version with a full package tutorial may be found on GitHub. The original research paper using the approaches implemented in the "*metamicrobiomeR*" package may be found in *Nature Communications* [102].

This chapter will guide you through some main steps for microbiome data analysis and meta-analysis using the "*metamicrobiomeR*" package. Interested readers should read the two papers mentioned above [4,102] and the package tutorial and manual [4] for further details, explanation, and result interpretation.

11.1. Statistical Approaches

The R package "*metamicrobiomeR*" applies Generalized Additive Models for Location, Scale, and Shape (GAMLSS) with a zero-inflated beta (BEZI) family (GAMLSS-BEZI) for the analysis of microbial taxonomy relative abundance data. GAMLSS is a general framework for fitting regression-type models in which the response variable can be any distribution [103]. With the BEZI family, this model allows direct and proper examination of microbiome relative abundance data, which resembles a zero-inflated beta distribution. In principle, this model is similar to the two-part mixed effect model proposed by Chen et al. [104] in that the presence or absence of the taxon in the samples is modeled with a logistic component, and the non-zero abundance of the taxon is modeled with a Beta component. Both logistic and beta components allow covariate adjustment and address longitudinal correlations with subject-specific random effects.

The GAMLSS-BEZI is based on the broadly applicable established GAMLSS framework that can be flexibly implemented and applied to different types of data and study designs (e.g., cross-sectional and longitudinal), as in the *taxa.compare()* functions. Importantly, the estimates (regression coefficients) from GAMLSS-BEZI are the log (odds ratio) of being in the case group (as compared to being in the control group) with changes in relative abundance of a specific bacterial taxon and thus are analogous across microbiome studies and can be directly combined using standard

meta-analysis approaches. As such, random effects or fixed effects meta-analysis models may be applied to pool the estimates and standard errors as implemented in the *meta.taxa()* functions of the "*metamicrobiomeR*" package. This approach allows examination of study-specific effects, heterogeneity between studies, and the overall pooled effects across studies. In the *taxa.compare()* function, all bacterial taxa or pathway data are first filtered to retain features with a mean relative abundance $\geq$ relative abundance threshold (e.g., $\geq$0.005%) and with prevalence $\geq$ prevalence threshold (e.g., present in $\geq$5% of the total number of samples). This pre-filtering step has been shown to improve the performance of various differential abundance detection strategies [105]. A filtered data matrix is then modeled by GAMLSS-BEZI and (μ) logit link and other default options using the R package "*gamlss*" version 5.0–5 [82].

For longitudinal data, subject-specific random effects can be added to the model. The default implementation is subject to random intercepts, as in practice, this is often sufficient to address the longitudinal correlations [106]. However, it is possible to extend the model to include random slopes depending on the specific research content.

In addition, different approaches to deal with compositional effects are also implemented, including Centered Log Ratio (CLR) transformation [107] with various zero-replacement options [108] and Geometric Mean of Pairwise Ratios (GMPR) normalization [109]. Multiple testing adjustment can be done using different methods (False Discovery Rate (FDR) control by default). For a more detailed description of statistical approaches and demonstration, please see the package paper at BMC Bioinformatics [4].

11.2. Example Data

For differential relative abundances analysis, the example dataset is published data from a cohort study of 50 healthy Bangladeshi infants, which included longitudinal gut microbiome data from 996 stool samples collected monthly from birth to 2 years of life [13]. This study included longitudinal monthly data regarding the infants' breastfeeding practices (exclusive, non-exclusive), duration of exclusive breastfeeding, infant age (months) at solid food introduction, and occurrence of diarrhea around the time of stool sample collection. A subset of samples collected around birth is used as an example cross-sectional dataset (50 samples), and data from all samples are used as a longitudinal dataset (996 samples). For meta-analysis, gut microbiome data from four published studies are used to demonstrate the application of random effects models for meta-analysis across microbiome studies. These four studies include:

- A cohort of healthy infants in Bangladesh [13] (the data of this study were also used in the examples demonstrating the use of GAMLSS-BEZI with *taxa.compare()* function);
- A cross-sectional study of Haitian infants negative for HIV who were exposed or unexposed to maternal HIV [110];
- A cohort of healthy infants in the USA (California and Florida [CA_FL]) [111];
- A small cohort of healthy infants in the USA (North Carolina [NC]) [112].

More details about the four studies included in the meta-analysis are described in the "*metamicrobiomeR*" package paper [4].

11.3. "metamicrobiomeR" Package Summary

The "*metamicrobiomeR*" package includes the functions in the table below (Table 39).

Table 39. Functions in the "metamicrobiomeR" package.

Functions	Description
taxa.filter	Filter relative abundances of bacterial taxa or pathways using prevalence and abundance thresholds
taxa.meansdn	Summarize mean, standard deviation of abundances, and number of subjects by groups for all bacterial taxa or pathways
taxa.mean.plot	Plot mean abundance by groups (from taxa.meansdn output)
taxa.compare	Compare relative abundances of bacterial taxa at all levels using GAMLSS or LM or LMAS
pathway.compare	Compare relative abundances of bacterial functional pathways at all levels using GAMLSS or LM or LMAS. Compare log(absolute abundances) of bacterial functional pathways using LM
taxcomtab.show	Display the results of relative abundance comparison (from taxa.compare or pathway.compare outputs)
meta.taxa	Perform meta-analysis of relative abundance estimates of bacterial taxa or pathways across studies using random/fixed effects
metatab.show	Display meta-analysis results of bacterial taxa or pathway relative abundances (from meta.taxa output)
meta.niceplot	Produce combined heatmap and forest plot for the meta-analysis results of bacterial taxa and pathway relative abundances
read.multi	Read multiple files in a path to R
alpha.compare	Calculate average alpha diversity indexes for a specific rarefaction depth, standardize, and compare indexes between groups
microbiomeage	Predict microbiome age using Random Forest model based on bacterial genera abundances

Source: Table by author.

11.4. Package Usage

This is a short, simplified version of the package tutorial. Only illustrations and examples for the analysis of differential relative abundance using GAMLSS-BEZI and meta-analysis across studies using random effects models are shown. Full illustration, examples of all implemented functions, workflows, and data are available at the

package's GitHub repository. A simplified version of the package vignette is also available in R cran [4].

First, we install and load the "*metamicrobiomeR*" package and other required packages (e.g., "*gdata*" [113]):

```
# may install package if not yet available by: install.packages("metamicrobiomeR")
library(metamicrobiomeR)
# Load other required packages
library(gdata)
```

11.4.1. Example Differential Analysis: Comparison Between Breastfeeding Statuses

Microbiome data are commonly normalized by converting raw counts into relative abundances, which represent the proportion of reads assigned to each taxonomic group within a sample.

The following example demonstrates comparing the relative abundance of bacterial taxa between infants (<6 months of age) who were exclusively breastfed versus those who were non-exclusively breastfed (the differential analysis). Breastfeeding is known to influence the development of the infant gut microbiome, and many studies have shown that breastfed infants often harbor microbial communities distinct from those of formula-fed infants. Plotting relative abundance by breastfeeding status and age, therefore, provides a visual assessment of how microbial populations evolve during early development.

In the example data, relative abundances of bacterial taxa at various taxonomic levels (from phylum to genus or species) are obtained via the *summarize_taxa.py* script in QIIME1 [114]. Bacterial functional pathway abundances (e.g., Kyoto Encyclopedia of Genes and Genomes (KEGG) pathway level 1 to 3) are obtained from metagenome prediction analysis using PICRUSt [115]. The relative abundance data should first be filtered with the *taxa.filter()* function. Then the function *taxa.meansdn()* is used to summarize the mean, standard deviation of abundances, and number of subjects by groups for all bacterial taxa of all levels or for pathways by comparison and grouping variables. Then the function *taxa.mean.plot()* is used to visualize data summary for the selected taxonomic level, as in the example codes and figure. The values to be displayed in the figure may be filtered by the *mean.filter* option, and taxonomic level names may be displayed as short or full long names via the option *show.taxname*.

Plot Relative Abundance

Visualizing relative abundance is an important exploratory step in microbiome analysis because it allows researchers to identify dominant microbial taxa, compare microbial composition between experimental groups, examine temporal changes in microbial communities, and detect potential associations between microbial taxa and clinical or environmental variables. These help identify patterns that may warrant further statistical investigation.

Now, we create a plot of Mean Relative Abundance by breastfeeding statuses and age at the phylum level (*tax.lev="l2"*), which corresponds to a high-level classification in the taxonomic hierarchy of microorganisms. Note that *tax.lev* is used to select the taxonomy level to be displayed, with values from "l2" for phylum to "l7" for species (Figure 119).

```
# Get and take a quick look at the data provided by
# the "metamicrobiomeR" package
data(taxtab6)
#str(taxtab6)
# data summary and plot
taxlist.rm <-taxa.filter(taxtab=taxtab6,
                         percent.filter = 0.05,
                         relabund.filter = 0.00005) taxa.meansdn.rm <-
taxa.meansdn(taxtab=taxtab6,
                                  sumvar="bf", groupvar="age.sample")
taxa.meansdn.rm <-taxa.meansdn.rm[taxa.meansdn.rm$bf!="No_BF",]
taxa.meansdn.rm$bf <-gdata::drop.levels(taxa.meansdn.rm$bf,
                                        reorder=FALSE)
#phylum
p.bf.l2 <-taxa.mean.plot(tabmean=taxa.meansdn.rm,
                         tax.lev="l2", comvar="bf",
                         groupvar="age.sample",
                         mean.filter=0.005,
                         show.taxname="short")
p.bf.l2$p
```

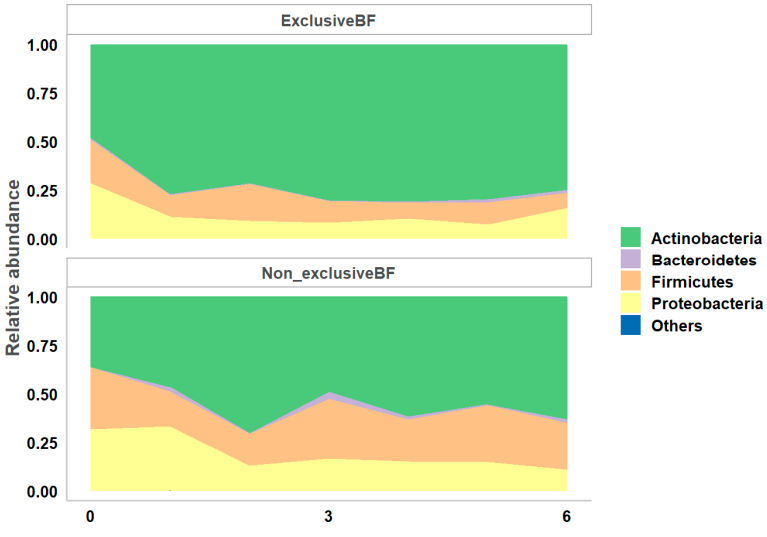


Figure 119. Taxa mean plot phylum. Source: Figure by author.

The output figure shows the mean relative abundance of major phyla across the study groups over time. We can see that the relative abundance of Actinobacteria is higher while the relative abundance of Firmicutes is lower in the exclusive breastfeeding group vs. the non-exclusive breastfeeding group.

Family-level aggregation provides a more granular view of the microbiome because each phylum contains many families with potentially distinct biological roles. By examining microbial composition at this level, researchers can identify more specific taxa that may be associated with breastfeeding status or developmental stage. To create a plot at the family level, we change to *tax.lev="l5"* (Figure 120):

```
p.bf.l5  <-taxa.mean.plot(tabmean=taxa.meansdn.rm,
                          tax.lev="l5", comvar="bf",
                          groupvar="age.sample",
                          mean.filter=0.005,
                          show.taxname="short")
p.bf.l5$p
```

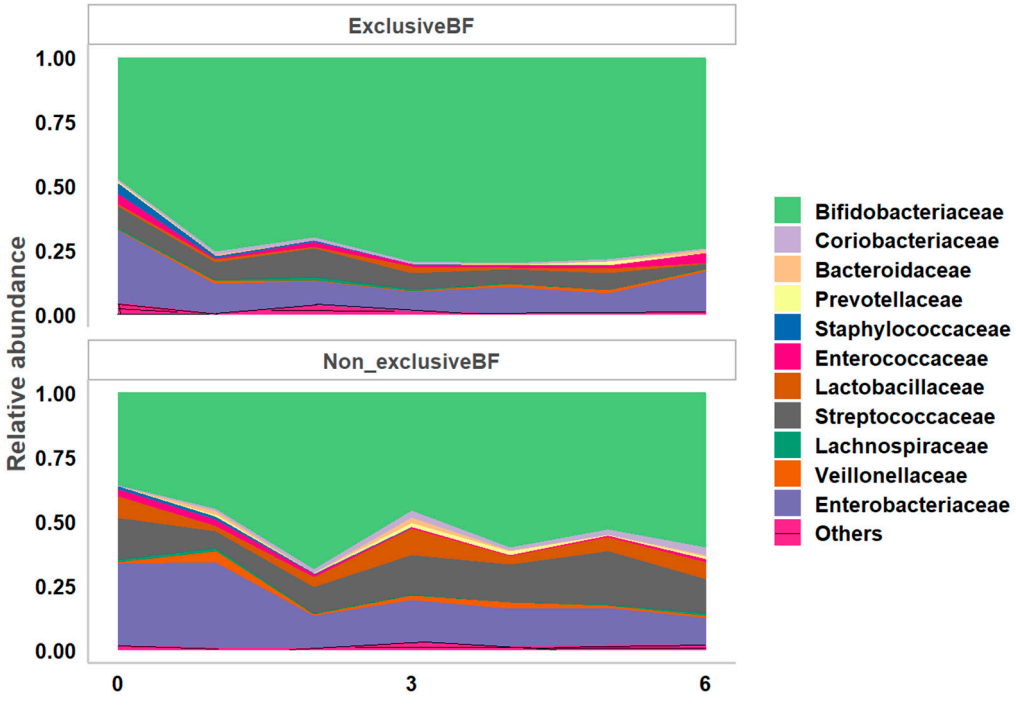


Figure 120. Taxa mean plot family. Source: Figure by author.

The output plot shows a more detailed representation of the microbial community than the phylum-level plot. Because the family level contains a larger number of taxa, the plot often includes a wider variety of microbial groups, each representing a smaller proportion of the total microbiome.

Comparison of Taxonomic Data Between Groups Using GAMLSS-BEZI

To address the characteristics of relative abundance data from microbiome sequencing, the GAMLSS-BEZI framework is particularly suitable because it can simultaneously model continuous relative abundance values between 0 and 1, excess zeros in the data, and covariate effects such as age or clinical variables. The examples in this section illustrate how differential abundance analysis can be conducted using the function *taxa.compare()*, which implements the GAMLSS-BEZI modeling approach for microbiome taxonomic data.

taxa.compare() and *pathway.compare()* are basically similar. There are just some adjustments to make these two functions readily usable with different data formats for pathway and taxonomy data. We need to specify the taxonomy dataset to be analyzed with *taxtab*, the comparison variable with *comvar*, and variables to be adjusted with *adjustvar*. Various statistical methods for differential analysis of relative abundance data may be selected by *propmed.rel* with default= "*gamlss*". Various data

transformations may be selected by *transform* with default= "*none*". Various methods for zero replacement may be selected by *zeroreplace.method* with default = "*none*". The data may be filtered before differential analysis by setting a filter threshold for prevalence with *percent.filter* and for relative abundance with *relabund.filter*. If the data is longitudinal, select the option *longitudinal* = "Yes". Various options for adjusting for multiple testing may be selected via the option *p.adjust.method* with default = "*fdr*".

Below is an example for comparison between breastfeeding statuses, adjusting for the age of infants at sample collection. Of note, the running time is not long in a regular laptop for both GAMLSS-BEZI analysis (~10s) and meta-analysis (~5s). However, to save running time, only taxonomies of one phylum are selected for the differential analysis example.

```
tab6 <-as.data.frame(taxtab6)
tl <-colnames(taxtab6)[grep("k__bacteria.p__fusobacteria",
                            colnames(taxtab6))]
taxacom.ex  <-taxa.compare(taxtab  =
                            tab6[,c("personid","x.sampleid", "bf","age.sample",
                                    tl)], propmed.rel="gamlss",
                           comvar="bf",
                           adjustvar="age.sample",
                           longitudinal="yes",
                           p.adjust.method="fdr")
```

The result output from the *taxa.compare()* function may be displayed using the *taxcomtab.show()* function. The taxa level to be displayed may be selected by *tax.lev* with options from "*l2*" (phylum) to "*l7*" (species) and the default is "*l2*". Options for multiple testing re-adjustment for only the level to be displayed or keeping the original multiple testing adjustment for all taxa of all levels may be selected with *readjust.p*. In the example below, we display the results for the comparison between non-exclusive vs. exclusive breastfeeding (BF) (*showvar="bfNon_exclusiveBF"*) at the phylum level (*tax.lev="l2"*).

```
tp  <-taxcomtab.show(taxcomtab=taxacom.ex,
                     tax.select="none",
                     showvar="bfNon_exclusiveBF",
                     tax.lev="l2",
                     readjust.p=TRUE,
                     p.adjust.method="fdr",
                     p.cutoff = 1)
tp
```

```
##   id                          Estimate.bfNon_exclusiveBFll
##1 k_bacteria.p_fusobacteria-0.06                          -0.77
##   ul                          Pr(>|t|).bfNon_exclusiveBF  pval.adjust.bfNon_exclusiveBF
## 10.65                         0.87                        0.87
```

The output shows the estimate (estimated Log Odds Ratio: "*Estimate.bfNon_exclusiveBF*"), 95% CI lower limit ("*ll*"), upper limit ("*ul*"), *p*-value ("*Pr(>|t|).bfNon_exclusiveBF*"), and multiple testing adjusted *p*-value ("*pval.adjust.bfNon_exclusiveBF*").

We can display the results of genus level by specifying *tax.lev="l6"*:

```
tp <-taxcomtab.show(taxcomtab=taxacom.ex,
                                tax.select="none",
                                showvar="bfNon_exclusiveBF",
                                tax.lev="l6",
                                readjust.p=TRUE,
                                p.adjust.method="fdr",
                                p.cutoff = 1)
head(tp)
```

```
##
## k_bacteria.p_fusobacteria.c_fusobacteriia.o_fusobacteriales.f_fusobacteriaceae.g_
    fusobacterium
##       Estimate.bfNon_exclusiveBF        ll         ul
##                            -0.12     -0.95       0.72
##        Pr(>|t|).bfNon_exclusiveBF     pval.adjust.bfNon_exclusiveBF
##                            0.785                             0.785
```

Comparison of Taxonomic Data Between Groups Using Linear Model with Arcsin Squareroot Transformation (LMAS)

This approach was formerly used with MaAsLin by The Huttenhower Lab [116]. This is also implemented in the *"metamicrobiomeR"* package for comparison and demonstration of the advantage of the GAMLSS-BEZI approaches for microbiome relative abundance data. We use the same *taxa.compare()* function and change the option *propmed.rel="lm"* and *transform="asin.sqrt"*. In the example below, we do a comparison of taxonomic data between breastfeeding statuses, adjusting for the age of infants at sample collection, and display the results at the phylum and family levels.

```
#phylum
ts <-taxcomtab.show(taxcomtab=taxacom.lmas,
                                tax.select="none",
                                showvar="bfNon_exclusiveBF",
                                tax.lev="l2",
                                readjust.p=TRUE,
                                p.adjust.method="fdr",
                                p.cutoff = 1,
                                digit=5, p.digit=5)
ts
taxacom.lmas <-taxa.compare(taxtab =
                                    tab6[,c("personid","x.sampleid","bf","age.sample",   tl)],
                               propmed.rel="lm",
                               transform="asin.sqrt", comvar="bf",
                               adjustvar="age.sample",
                               longitudinal="yes",
                               p.adjust.method="fdr")
```

```
##          id                                  Estimate.bfNon_exclusiveBF
## k_bacteria.p_fusobacteria                          0.00134
##                    ll                ul     Pr(>|t|).bfNon_exclusiveBF
##           -0.00134          0.00402              0.321
##          pval.adjust.bfNon_exclusiveBF
##                   0.321
```

```
#family
ts  <-taxcomtab.show(taxcomtab=taxacom.lmas,
                     tax.select="none",
                     showvar="bfNon_exclusiveBF", tax.lev="15",
                     readjust.p=TRUE,
                     p.adjust.method="fdr",
                     p.cutoff = 1,
                     digit=5,
                     p.digit=5)
head(ts)
```

```
##
## k_bacteria.p_fusobacteria.c_fusobacteriia.o_fusobacteriales. f_fusobacteriaceae
##         Estimate.bfNon_exclusiveBF         ll          ul
##                            0.00137   -0.00131    0.00405
##         Pr(>|t|).bfNon_exclusiveBF         pval.adjust.bfNon_exclusiveBF
##                               0.31                                  0.31
```

Comparison of Bacterial Functional (KEGG) Pathways Between Groups

KEGG pathways organize genes into hierarchical functional categories, allowing researchers to investigate microbial functional activity at multiple levels from level 1 (broad functional categories) to level 3 (detailed biological pathways). This section presents the examples to compare the relative abundance of predicted KEGG pathways between study groups and determine whether certain functional processes differ between groups. Functional pathway analyses provide insights into how microbial communities may differ in their metabolic potential. Integrating functional pathway analysis with taxonomic composition analysis provides a better understanding of microbiome dynamics.

The approach in *pathway.compare()* is similar to that in the *taxa.compare()* function, except for some adjustments for the data format obtained from metagenome prediction analysis using PICRUSt [115].

In the example below, we do a comparison of the relative abundance of bacterial functional (KEGG) pathways between male vs. female infants < 6 months, adjusting for breastfeeding statuses and age of infants at sample collection. We will do the analysis and display the results for pathway relative abundances for level 1 only (to save running time). We need to specify the pathway relative abundance data (*pathtab=kegg.12*), the covariate data (*mapfile=covar.rm*), the id variable to merge the two data files (*sampleid="sampleid"*), that we want to do the comparison for relative abundance (*pathsum="rel"*), and other options similar to those of the *taxa.compare()* function above.

```
# Get the data and take a quick look at the data
data(kegg.12)
#str(kegg.12)
data(covar.rm)
#str(covar.rm)
# Compare between groups
path1 <-pathway.compare(pathtab=kegg.12,
```

```
                                mapfile=covar.rm,
                                sampleid="sampleid",
                                pathsum="rel",
                                stat.med="gamlss",
                                comvar="gender", adjustvar=c("age.sample","bf"),
                                longitudinal="yes",
                                p.adjust.method="fdr",
                                percent.filter=0.05,
                                relabund.filter=0.00005)
```

We then use the same function, *taxcomtab.show()*, to display the results of KEGG pathway level 1 and level 2. Note that we specify *tax.lev="l2"* and change the dataset option *taxcomtab=path1$l1* or *taxcomtab=path1$l2* to display the results of level 1 or level 2 pathways.

```
#Level 1
tk <-taxcomtab.show(taxcomtab=path1$l1,
        sumvar="path",
        tax.lev="l2",
        tax.select="none",
        showvar="genderMale",
        p.adjust.method="fdr",
        p.cutoff=1)
 tk
```

```
##    id                                          Estimate.genderMale   ll
## 7 Organismal.Systems                           0.04                  0.02
## 5 Metabolism                                   0.01                  0.00
## 4 Human.Diseases                              -0.02                 -0.03
## 8 Unclassified                                -0.01                 -0.02
## 2 Environmental.Information.Processing        -0.01                 -0.02
## 3 Genetic.Information.Processing               0.01                  0.00
## 6 None                                        -0.06                 -0.16
## 1 Cellular.Processes                           0.00                 -0.03
##    ul        Pr(>|t|).genderMale   pval.adjust.genderMale
## 7 0.06       0.0000                0.0001
## 5 0.01       0.0155                0.0619
## 4 0.00       0.0273                0.0727
## 8 0.00       0.0521                0.1043
## 2 0.00       0.1705                0.2728
## 3 0.01       0.2454                0.3271
## 6 0.05       0.2877                0.3288
## 1 0.02       0.8100                0.8100
```

```
#Level 2
tk <-taxcomtab.show(taxcomtab=path1$l2,
                    sumvar="path",
                    tax.lev="l2",
                    tax.select="none",
                    showvar="genderMale",
                    p.adjust.method="fdr",
                    p.cutoff=1)
head(tk)
```

```
##      id
## 36   Cellular.Processes..Transport.and.Catabolism
## 9    Organismal.Systems..Endocrine.System
## 2    Metabolism..Biosynthesis.of.Other.Secondary.Metabolites
## 34   Genetic.Information.Processing..Transcription
## 1    Metabolism..Amino.Acid.Metabolism
## 20   Metabolism..Lipid.Metabolism
##      Estimate.genderMale  ll     ul     Pr(>|t|).genderMale
## 36   0.07                 0.04   0.10   0.0000
## 9    0.09                 0.05   0.12   0.0000
## 2    0.03                 0.02   0.04   0.0000
## 34   -0.02                -0.03  -0.01  0.0000
## 1    0.02                 0.01   0.03   0.0001
## 20   -0.01                -0.02  0.00   0.0004
##      pval.adjust.genderMale
## 36   0.0000
## 9    0.0000
## 2    0.0001
## 34   0.0001
## 1    0.0006
## 20   0.0026
```

Similar to the outputs of *taxa.compare()*, the outputs from *pathway.compare()* show the results for level 1 and level 2 KEGG pathways, including the estimates for the difference between genders (male vs. female), lower and upper 95% CI of the estimates, raw *p*-values, and *p*-values after adjusting for multiple testing.

11.4.2. Illustration of Meta-Analysis

Microbiome studies frequently involve relatively small sample sizes and may be conducted across different geographic locations, populations, and study designs. As a result, findings from individual studies may be inconsistent or may lack sufficient statistical power to detect modest differences in microbial relative abundance. Meta-analysis provides a rigorous statistical framework to combine evidence from multiple independent studies, thereby improving statistical power and producing more robust estimates of microbial differences between groups.

Meta-Analysis of Taxonomy or Pathway Relative Abundance Data

In the context of microbiome relative abundance analysis, meta-analysis allows investigators to integrate study-specific estimates of differential abundance obtained from statistical models such as GAMLSS-BEZI. By combining effect estimates across studies, researchers can determine whether a particular microbial taxon or predicted functional pathway shows consistent patterns of association across different populations.

This section demonstrates how study-specific differential abundance results can be integrated using meta-analysis methods and guides how the functions *meta.taxa()*, *metatab.show()*, and *meta.niceplot()* can be used to perform meta-analysis, summarize, and visualize pooled estimates across studies for taxonomic data. For predicted similarly.

Meta-analysis is illustrated with the example comparing relative abundances of gut bacterial taxa between male vs. female infants $\leq$6 months of age, adjusting for

feeding status and infant age at the time of stool sample collection. The meta-analysis for bacterial predicted functional pathways may be done similarly. GAMLSS-BEZI results from four studies (Bangladesh, Haiti, USA(CA_FL), USA(NC)) were used for meta-analysis with the *meta.taxa()* function. Summary measures may be selected by *summary.measure* with default= "RR" for estimates from GAMLSS with the BEZI family, and should be "RD" for estimates from the Linear/linear mixed effect model. The threshold percentage of the number of studies that a taxon is available to do meta-analysis may be selected by *percent.meta*. The result output from the *meta.taxa()* function may be displayed as a table using the *metatab.show()* function and then visualized as a combined heatmap and forest plot using the *meta.niceplot()* function. The example plot shows a heatmap of log(odds ratio) (log(OR)) of relative abundances of gut bacterial taxa at different taxonomic levels between male vs. female infants for each study and pooled estimates (meta-analysis) across all studies with 95% confidence intervals (95% CI) (forest plot). All log(OR) estimates of each bacterial taxa from each study were from Generalized Additive Models for Location, Scale, and Shape (GAMLSS) with beta zero-inflated family (BEZI) and were adjusted for feeding status and age of infants at sample collection. Pooled log(OR) estimates and 95% CI (forest plot) were from random effect meta-analysis models with inverse variance weighting and DerSimonian–Laird estimator for between-study variance based on the adjusted log(OR) estimates and corresponding standard errors of all included studies. Bacterial taxa with *p*-values for differential relative abundances < 0.05 were denoted with *, and those with *p*-values < 0.0001 were denoted with **. Pooled log(OR) estimates with pooled *p*-values < 0.05 are in red, and those with false discovery rate (FDR) adjusted pooled *p*-values < 0.1 are in triangle shape. Missing (unavailable) values are in white. (Abbreviation: USA: United States of America; CA: California; FL: Florida; NC: North Carolina).

First, we load saved GAMLSS-BEZI results of four studies for the comparison of bacterial taxa relative abundance between genders adjusted for breastfeeding and infant age at sample collection and perform a meta-analysis. We select only taxonomies of a phylum for the meta-analysis example (to save running time) and show the meta-analysis results in a table. We show the example results for phylum (*tax.lev="l2"*) and family level (*tax.lev="l5"*).

```
# Get and take a quick look at the dataset
data(tabsex4)
str(tabsex4)
```

```
## $ id
## 'data.frame'                    : 701 obs. of  23 variables: chr  "k_bacteria.p_actinobacteria" "k_
                                     bacteria. p_bacteroidetes" "k_bacteria.p_firmicutes" "k_bacteria.p_
                                     fusobacteria" ...
##  $ Estimate.genderMale           : num 0.2269 −0.1549 −0.056 −0.0648 −0.0887 ...
##  $ Std. Error.genderMale         : num 0.121 0.119 0.104 0.282 0.118 ...
##  $ t value.genderMale            : num 1.874 −1.305 −0.537 −0.23 −0.751 ...
## $ Pr(>|t|).genderMale            : num 0.0619 0.1931 0.5919 0.8186 0.4532 ...
##
$ Estimate.bfNon_exclusiveBF       : num  −0.3688 0.2557 0.2362 −0.0605 0.3699 ...
## $ Std.
Error.bfNon_exclusiveBF :           num 0.139 0.136 0.116 0.364 0.134 ...
##  $ t value.bfNon_exclusiveBF     : num −2.647 1.887 2.035 −0.166 2.764 ...
## $ Pr(>|t|).bfNon_exclusiveBF     : num 0.00857 0.0602 0.04276 0.86812 0.00606 ...
```

```
##  $ Estimate.bfNo_BF                : num −1.329 0.135 0.356 −0.403 1.153 ...
##  $ Std. Error.bfNo_BF              : num  0.412 0.499 0.392 1.083 0.407 ...
##  $ t value.bfNo_BF                 : num −3.223 0.27 0.908 −0.372 2.834 ...
## $ Pr(>|t|).bfNo_BF                 : num 0.00141 0.7873 0.36478 0.7103 0.0049 ...
##  $ Estimate.age.sample             : num 0.1251 0.0242 0.0115 0.0285 −0.1348 ...
##  $ Std. Error.age.sample           : num 0.0377 0.0367 0.0319 0.0857 0.038 ...
##  $ t value.age.sample              : num 3.318 0.66 0.359 0.333 −3.549 ...
##  $ Pr(>|t|).age.sample             : num 0.001022 0.510068 0.719525 0.73934 0.000448 ...
## $ pval.adjust.genderMale           : num 0.48 0.899 0.961 0.961 0.961 ...
##
$ pval.adjust.bfNon_exclusiveBF      :num 0.059 0.2333 0.1767 0.9976 0.0448 ...
##  $ pval.adjust.bfNo_BF             : num 0.0219 1 1 1 0.0468 ...
## $ pval.adjust.sample               : num 0.00786 0.82595 0.85375 0.85375 0.00505 ...
## $ study                            : chr "Subramanian et al. 2014 (Bangladesh)"
"Subramanian et al. 2014 (Bangladesh)" "Subramanian et al. 2014 (Bangladesh)" "Subramanian et al.
2014 (Bangladesh)" ...
: chr  "Bangladesh" "Bangladesh" "Bangladesh" "Bangladesh" ...
```

```
## $ pop       #select only taxonomies of a phylum for meta-analysis example (to save running time)
tlm <-tabsex4$id[grep("k   bacteria.p   actinobacteria",tabsex4$id)]
metab.sex <-meta.taxa(taxcomdat=tabsex4[tabsex4$id %in% tlm,],
                      summary.measure="RR",
                      pool.var="id",
                      studylab="study",
                      backtransform=FALSE,
                      percent.meta=0.5,
                      p.adjust.method="fdr")
#show results by table  #phylum
level
metatab.show(metatab=metab.sex$random,
             com.pooled.tab=tabsex4[tabsex4$id %in% tlm,], tax.lev="l2",
             showvar="genderMale",
             p.cutoff.type="p",
             p.cutoff=1,
             display="table")
```

```
##        id                              estimate      ll       ul      p
## 1 k    bacteria.p_actinobacteria       0.48          −0.27    1.23    0.208
##        p.adjust
## 1      0.462
```

```
#family  level
metatab.show(metatab=metab.sex$random,
             com.pooled.tab=tabsex4[tabsex4$id %in% tlm,],
             tax.lev="l5",
             showvar="genderMale",
             p.cutoff.type="p",
             p.cutoff=1,
             display="table")
```

```
##
## 11     k_bacteria.p_actinobacteria.c_coriobacteriia.o_coriobacteriales. f_coriobacteriaceae
## 10     k_bacteria.p_actinobacteria.c_actinobacteria.o_bifidobacteriales. f_bifidobacteriaceae
## 9      k_bacteria.p_actinobacteria.c_actinobacteria.o_actinomycetales. f_micrococcaceae
## 21     k_bacteria.p_actinobacteria.c_actinobacteria.o_bifidobacteriales. f_
       bifidobacteriaceae.other
## 8      k_bacteria.p_actinobacteria.c_actinobacteria.o_actinomycetales. f_corynebacteriaceae
## 7      k_bacteria.p_actinobacteria.c_actinobacteria.o_actinomycetales. f_actinomycetaceae
```

```
##              estimate      ll          ul        p              p.adjust
##      11        −0.26     −0.44       −0.08     0.0049         0.0344
##      10        0.52      −0.16       1.20      0.1345         0.4454
```

```
##      9      −0.09    −0.24          0.06    0.2198          0.4615
##     21      −0.12    −0.57          0.33    0.5925          0.8581
##      8      −0.02    −0.20          0.15    0.8026          0.8871
##      7       0.02    −0.34          0.38    0.9209          0.9209
```

The outputs show the meta-analysis pooled estimates (for the difference between groups), lower and upper limits of 95% CI, *p*-values, and *p*-values after being adjusted for multiple testing (FDR by default) for each taxon. For example, the taxon "11 k bacteria.p actinobacteria.c coriobacteriia.o coriobacteriales. f coriobacteriaceae" shows a significant difference between groups (male is lower vs. female) with a *p*-value = 0.0344 after adjusting for multiple testing.

Now, we show meta-analysis results as a combined heatmap and forest plot. We use the function *metatab.show()* to get a result table, then use *meta.niceplot()* to plot the result table.

```
metadat <-metatab.show(metatab=metab.sex$random,
                       com.pooled.tab=tabsex4, tax.lev="l5",
                       showvar="genderMale",
                       p.cutoff.type="p",
                       p.cutoff=1,
                       display="data")
meta.niceplot(metadat=metadat,sumtype="taxa",
              level="main", p="p",
              p.adjust="p.adjust",
              phyla.col="rainbow",
              p.sig.heat="yes",
              heat.forest.width.ratio =c(2.5, 2), leg.key.size=0.6,
              leg.text.size=8, heat.text.x.size=8,
              heat.text.x.angle=45,
              forest.axis.text.y=3,
              forest.axis.text.x=8,
              point.ratio = c(3,2),
              line.ratio = c(1.5,1))
```

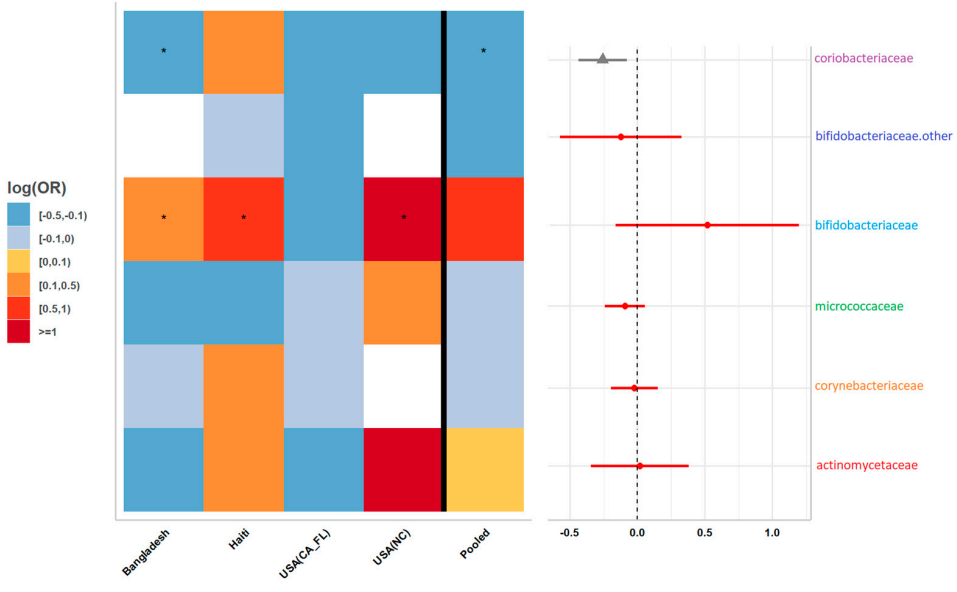


Figure 121. Meta-analysis nice plot. Source: Figure by author.

The output plot shows visually which taxa in which studies have positive or negative differences, the magnitude of the difference, and the significance, as well as the meta-analysis pooled estimates, 95% CI. Consistent with the output above, the taxon "actinobacteria.c coriobacteriia.o coriobacteriales. f coriobacteriaceae" is significantly different between groups (male is lower vs. female) (Figure 121).

Analysis and Meta-Analysis of Other Data

In addition to taxonomic relative abundance and predicted functional pathway data, microbiome studies often analyze other metrics, such as community diversity metrics. Meta-analysis may also be applied to other metrics of microbiome data, such as alpha diversity indexes. Similarly, meta-analysis helps synthesize evidence across studies and obtain more stable estimates of group differences in alpha diversity.

This section illustrates the application of meta-analysis for alpha diversity: how alpha diversity indices can be calculated and standardized within each study, analyzed using regression models that adjust for relevant covariates, and combined across studies using standard meta-analysis techniques. The alpha diversity indices commonly analyzed include Chao1, Observed Species, PD Whole Tree, and Shannon Index. Because these indices are measured on different numerical scales, they are standardized within each study so that they have a mean = 0 and a standard deviation = 1. Standardization ensures that diversity measures from different studies are comparable and can be combined in a meta-analysis. After standardization, alpha diversity indices are compared between study groups using regression models that adjust for relevant covariates. For cross-sectional datasets, the comparison is performed using a linear regression model (*lm()*). For longitudinal datasets, the analysis uses a linear mixed-effects model (*lmer()*). These models estimate the association between alpha diversity and the variable of interest while adjusting for confounding variables.

In the example below, for each study, the *alpha.compare()* function imports the outputs from the *alpha_rarefaction.py* QIIME1 script (*datlist=alphadat*), merges with the covariate file (*mapfile=covar.rm*) via sample id (*mapfile=covar.rm*), and calculates mean alpha diversity for different indices for each sample based on a user-defined rarefaction depth (*depth=3*). Mean alpha diversity indexes are standardized to have a mean of 0 and standard deviation of 1 to make these measures comparable across studies (*standardize=TRUE*). Standardized alpha diversity indexes are compared between groups, adjusting for covariates using a linear mixed effect model for longitudinal data (*comvar="gender", adjustvar=c("age.sample","bf"), longitudinal="yes"*). Meta-analysis across studies is then done and the results are displayed as a standard meta-analysis forest plot. The example below calculates mean alpha diversity indexes for a selected rarefaction depth, standardizes and compares standardized alpha diversity indexes between groups (male vs. female infants $\leq$ 6 months of age (*age.limit=6*)), and adjusts for covariates (feeding status and infant age at sample collection) using the Bangladesh data [13].

```
data(alphadat)
data(covar.rm)
covar.rm$sampleid <-tolower(covar.rm$sampleid)
alphacom <-alpha.compare(datlist=alphadat,
```

```
                                depth=3,
                                mapfile=covar.rm,
                                mapsampleid="sampleid",
                                comvar="gender",
                                adjustvar=c("age.sample","bf"),
                                longitudinal="yes",
                                age.limit=6,
                                standardize=TRUE)
alphacom$alphasum[,1:5]
```

```
##      id                  Estimate.genderMale  Std. Error.genderMale
## 1    chao1               -0.0654              0.150
## 2    observed_species    -0.1428              0.136
## 3    pd_whole_tree       -0.1325              0.145
## 4    shannon             -0.4401              0.199
##      t value.genderMale  Pr(>|t|).genderMale
## 1    -0.435              0.6635
## 2    -1.048              0.2947
## 3    -0.912              0.3619
## 4    -2.208              0.0272
```

The output shows the estimates ("Estimate.genderMale") representing the standardized difference in alpha diversity between male and female infants, standard error, t value, and *p*-value for the comparison. A negative estimate indicates that the diversity index tends to be lower in male infants compared with female infants, while a positive estimate would indicate the opposite. The "Shannon" index shows a statistically significant difference (*p*-value = 0.02722).

Now, to demonstrate the meta-analysis of alpha diversity indexes, we load and save result outputs of the *alpha.compare()* function of the four studies. The dataset in the examples below contains regression results for each alpha diversity index from the four studies mentioned above (Bangladesh, Haiti, USA (California and Florida), and USA (North Carolina)). Each row represents a study-specific estimate for a given diversity index, along with the corresponding standard error and *p*-value. Then we perform meta-analysis using the "*meta*" package [73] for a selected alpha diversity index (e.g., Shannon). The meta-analysis is conducted using the R package "meta", similarly to the meta-analysis we performed above for relative abundance. Note that, for alpha diversity, the estimates are from linear or linear mixed models, so we need to specify risk difference (sm="RD").

```
# load saved results of 4 studies
data(asum4)
asum4[,c(colnames(asum4)[1:5],"pop")]
```

```
##    id                  Estimate.genderMale  Std. Error.genderMale
## 1  chao1               0.06527              0.0722
## 2  observed_species    0.06529              0.0621
## 3  pd_whole_tree       0.03132              0.0502
## 4  shannon             -0.00128             0.0821
## 5  chao1               -0.08862             0.3528
## 6  observed_species    -0.09147             0.3453
## 7  pd_whole_tree       0.04257              0.2952
## 8  shannon             -0.00911             0.2936
## 9  chao1               -0.04503             0.1337
## 10 observed_species    -0.06538             0.1145
## 11 pd_whole_tree       -0.15902             0.1076
```

```
## 12shannon              −0.07053              0.1434
## 13chao1                0.49658               0.5925
## 14observed_species     0.23367               0.5294
## 15pd_whole_tree        −0.01076              0.7331
## 16shannon              0.17831               0.5553
##     t value.genderMalePr(>|t|).genderMale  pop
## 1  0.9042              0.366                 Bangladesh
## 2  1.0510              0.293                 Bangladesh
## 3  0.6233              0.533                 Bangladesh
## 4  −0.0155             0.988                 Bangladesh
## 5  −0.2512             0.803                 Haiti
## 6  −0.2649             0.792                 Haiti
## 7  0.1442              0.886                 Haiti
## 8  −0.0310             0.975                 Haiti
## 9  −0.3368             0.736                 USA(CA_FL)
## 10−0.5710              0.568                 USA(CA_FL)
## 11−1.4780              0.139                 USA(CA_FL)
## 12−0.4920              0.623                 USA(CA_FL)
## 130.8381               0.402                 USA(UNC)
## 140.4414               0.659                 USA(UNC)
## 15−0.0147              0.988                 USA(UNC)
## 160.3211               0.748                 USA(UNC)
```

```
# meta-analysis for Shannon index
shannon.sex <- meta::metagen(Estimate.genderMale,
                             `Std. Error.genderMale`, studlab=pop, data=subset(asum4,
                             id=="shannon"), sm="RD",
                             backtransf=FALSE)
# Save current margin settings
old_mar <- par("mar")
# Increase margins to avoid cropping
par(mar = c(5, 6, 4, 2))
meta::forest(shannon.sex,
             smlab="Standardized \n diversity difference", sortvar=subset(asum4,
             id=="shannon")$pop, lwd=2,
             xlim=c(-1, 1.5),
             fontsize = 8)
```

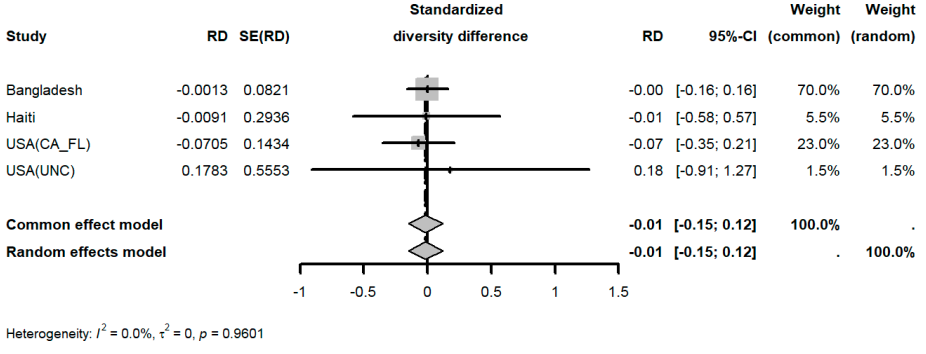


Figure 122. Meta-analysis forest plot. Source: Figure by author.

The output shows that all four studies and the meta-analysis are not statistically significant (95% CI includes zero) (Figure 122). Also, the heterogeneity between studies is not significant (p = 0.9601).

```
# Reset margins to original settings
par(mar = old_mar)
shannon.sex
```

```
## Number of studies: k = 4
##
##                          RD          95%-CI            z  p-value
## Common effect model  -0.0149 [-0.1495; 0.1198]  -0.22  0.8288
## Random effects model -0.0149 [-0.1495; 0.1198]  -0.22  0.8288
##
## Quantifying heterogeneity (with 95%-CIs):
## tau^2 = 0 [0.0000; 0.0134]; tau = 0 [0.0000; 0.1157]
## I^2 = 0.0% [0.0%; 84.7%]; H = 1.00 [1.00; 2.56]
##
## Test of heterogeneity:
##  Q     d.f.   p-value
##  0.30  3      0.9601
##
## Details of meta-analysis methods:
## Inverse variance method
## Restricted maximum-likelihood estimator for tau^2
## Q-Profile method for confidence interval of tau^2 and tau
## Calculation of I^2 based on Q
```

The outputs show the meta-analysis pooled estimates (RD), 95% CI, *p*-values, heterogeneity test results.

```
cbind(study=shannon.sex$studlab, pval=shannon.sex$pval)
```

```
##      study                pval
## [1,] "Bangladesh"         "0.987599902164408"
## [2,] "Haiti"              "0.9752390484969"
## [3,] "USA(CA_FL)"         "0.622739246734807"
## [4,] "USA(UNC)"           "0.748126030769876"
```

The output shows the *p*-values for the comparison for each of the studies included in the meta-analysis.

Above are some examples illustrating the use of the R package "*metamicrobiomeR*" for the analysis and meta-analysis of microbiome data. This package only provides some limited functionalities for microbiome data analysis. Readers may explore other R packages for microbiome data for more comprehensive and elaborate analysis.

11.5. Useful Resources

Readers should read these resources for further details, description, explanation, and interpretation:

- The paper describing the "*metamicrobiomeR*" package on BMC Bioinformatics [4].
- The "*metamicrobiomeR*" package hosted in R cran [4].
- The original paper on Nature Communications that applied the approaches implemented in the "*metamicrobiomeR*" package [102].
- An earlier version of the "*metamicrobiomeR*" package with a full package tutorial on Github (https://github.com/nhanhocu/metamicrobiomeR, accessed on 1 January 2026).

11.6. Chapter Summary

The "*metamicrobiomeR*" package has the following main functions:

- *taxa.compare()* and *pathway.compare()* functions utilize Generalized Additive Models for Location, Scale, and Shape with a zero-inflated beta family (GAMLSS-BEZI) for differential analysis of zero-inflated, compositional microbiome relative abundance data and produce standardized estimates (log odds ratios) suitable for meta-analysis
- Various options for data transformation are available;
- *taxcomtab.show()* to display results *taxa.compare()* and *pathway.compare();*
- *meta.taxa()* function is used for meta-analysis of the effect estimates from multiple studies using random-effects or fixed-effects models;
- *metatab.show()* function to display results from the meta-analysis as tables;
- *meta.niceplot()* function to visualize meta-analysis results as a combined heatmap and forest plot;
- *alpha.compare()* function to calculate, standardize, and compare alpha diversity indices between groups, with its outputs also being suitable for meta-analysis;
- *taxa.meansdn()* to summarize abundance data by group, *taxa.mean.plot()* to visualize these means.

References

1. R Core Team. *R: A Language and Environment for Statistical Computing*; R Foundation for Statistical Computing: Vienna, Austria, 2025.
2. Posit Team. *RStudio: Integrated Development Environment for R*; Posit Team: Boston, MA, USA, 2025; Available online: http://www.posit.co/ (accessed on 1 January 2025).
3. Xie, Y.; Allaire, J.J.; Grolemund, G. *R Markdown: The Definitive Guide*; Chapman and Hall/CRC: Boca Raton, FL, USA, 2018; Available online: https://bookdown.org/yihui/rmarkdown (accessed on 1 January 2025).
4. Ho, N.T.; Li, F.; Wang, S.; Kuhn, L. metamicrobiomeR: An R package for analysis of microbiome relative abundance data using zero-inflated beta GAMLSS and meta-analysis across studies using random effects models. *BMC Bioinform.* **2019**, *20*, 188. [CrossRef] [PubMed]
5. Xie, Y. *Dynamic Documents with R and Knitr*, 2nd ed.; Chapman and Hall/CRC: Boca Raton, FL, USA, 2015; Available online: https://yihui.org/knitr/ (accessed on 1 January 2025).
6. Allaire, J.; Xie, Y.; Dervieux, C.; McPherson, J.; Luraschi, J.; Ushey, K.; Atkins, A.; Wickham, H.; Cheng, J.; Chang, W.; et al. *rmarkdown: Dynamic Documents for R*; R Package v 2.29; 2024. Available online: https://cran.r-project.org/web/packages/rmarkdown (accessed on 2 July 2025).
7. Galili, T. *installr: Using R to Install Stuff on Windows OS (Such As: R, "Rtools", "RStudio", "Git", and More!)*; R Package v 0.23.4; 2022. Available online: https://cran.r-project.org/web/packages/installr/index.html (accessed on 20 June 2026).
8. Wickham, H.; Miller, E.; Smith, D. *haven: Import and Export "SPSS", "Stata" and "SAS" Files*; R Package v 2.5.5; 2025. Available online: https://cran.r-project.org/web/packages/haven/index.html (accessed on 20 June 2026).
9. Wickham, H.; Hester, J.; Bryan, J. *readr: Read Rectangular Text Data*; R Package v 2.1.5; 2024. Available online: https://cran.r-project.org/web/packages/readr/index.html (accessed on 20 June 2026).
10. Wickham, H.; Garbett, S.; Stephens, J.; Simonov, K.; Xie, Y.; Dong, Z.; Horner, J.; Beasley, W.; O'Connor, B.; Quinn, M.; et al. *yaml: Methods to Convert R Data to YAML and Back*; R Package v 2.3.10; 2024. Available online: https://cran.r-project.org/web/packages/yaml/index.html (accessed on 20 June 2026).
11. Wickham, H. *tidyverse: Easily Install and Load the Tidyverse*; 2023. Available online: https://cran.r-project.org/web/packages/tidyverse/index.html (accessed on 20 June 2025).
12. Zhu, H. *kableExtra: Construct Complex Table with kable and Pipe Syntax*; R Package v 1.4.0; 2024. Available online: https://haozhu233.github.io/kableExtra/ (accessed on 20 June 2025).
13. Subramanian, S.; Huq, S.; Yatsunenko, T.; Haque, R.; Mahfuz, M.; Alam, M.A.; Benezra, A.; DeStefano, J.; Meier, M.F.; Muegge, B.D.; et al. Persistent gut microbiota immaturity in malnourished Bangladeshi children. *Nature* **2014**, *510*, 417–421. [CrossRef] [PubMed]
14. Becker, J.; Chan Chong Schoch, D.; Leeper, T.J. rio: A Swiss-Army Knife for Data I/O. 2024. Available online: https://gesistsa.github.io/rio/ (accessed on 10 January 2025).
15. Wickham, H. *httr: Tools for Working with URLs and HTTP*; R Package v 1.4.7; 2023. Available online: https://cran.r-project.org/web/packages/httr/index.html (accessed on 10 June 2025).

16. Ooms, J. The jsonlite Package: A Practical and Consistent Mapping Between JSON Data and R Objects. *arXiv* **2014**, arXiv:14032805.
17. Wickham, H.; François, R.; Henry, L.; Müller, K.; Vaughan, D. *dplyr: A Grammar of Data Manipulation*; R Package v 1.1.4; 2023. Available online: https://cran.r-project.org/web/packages/dplyr/index.html (accessed on 10 June 2025).
18. Wickham, H. *ggplot2: Elegant Graphics for Data Analysis*; Springer: New York, NY, USA, 2016; Available online: https://ggplot2.tidyverse.org (accessed on 10 February 2025).
19. Heinzen, E.; Sinnwell, J.; Atkinson, E.; Gunderson, T.; Dougherty, G. *Arsenal: An Arsenal of R Functions for Large-Scale Statistical Summaries*. R Package v 3.6.3; 2021. Available online: https://github.com/mayoverse/arsenalhttps://github.com/mayoverse/arsenal (accessed on 10 February 2025).
20. Kassambara, A. *ggpubr: ggplot2 Based Publication Ready Plots*; R Package v 0.6.1. 2025. Available online: https://rpkgs.datanovia.com/ggpubr/ (accessed on 10 February 2025).
21. Aragón, T.J. Population Health Data Science with R. Available online: https://bookdown.org/taragonmd/phds/ (accessed on 10 February 2025).
22. Rodrigues, B. Modern R with the Tidyverse. Available online: https://modern-rstats.eu/ (accessed on 10 February 2025).
23. Fox, J.; Weisberg, S.; Price, B. *car: Companion to Applied Regression*, 3rd ed.; Sage: Thousand Oaks, CA, USA, 2019; Available online: https://r-forge.r-project.org/projects/car/ (accessed on 10 February 2025).
24. Champely, S. *pwr: Basic Functions for Power Analysis*; R Package v 1.3-0; 2020. Available online: https://cran.r-project.org/web/packages/pwr/index.html (accessed on 10 June 2025).
25. Amala, R.; Kumarapandiyan, G. *samplesizeestimator: Calculate Sample Size for Various Scenarios*; R Package v 1.0.0; 2023. Available online: https://cran.r-project.org/web/packages/samplesizeestimator/index.html (accessed on 10 June 2025).
26. Haynes, A.G.; Lenz, A.; Stalder, O.; Limacher, A. 'presize': An R-package for precision-based sample size calculation in clinical research. *J. Open Source Softw.* **2021**, *6*, 3118. [CrossRef]
27. Mathur, S.K. *Statistical Bioinformatics with R*, 1st ed.; Elsevier: Amsterdam, The Netherlands, 2009.
28. Zar, J.H. *Biostatistical Analysis*, 5th ed.; Prentice Hall: Upper Saddle River, NJ, USA, 2010.
29. Almeida, A.; Loy, A.; Hofmann, H. ggplot2 Compatible Quantile-Quantile Plots in R. *R J.* **2018**, *10*, 248–261. [CrossRef]
30. Calhoun, P. Exact: Unconditional Exact Test. 2024. Available online: https://CRAN.R-project.org/package=Exact (accessed on 10 February 2025). [CrossRef]
31. Gabriel, E.E.; Nason, M.; Fay, M.P.; Follmann, D.A. A boundary-optimized rejection region test for the two-sample binomial problem. *Stat. Med.* **2018**, *37*, 1047–1058. [CrossRef]
32. Yoshida, K.; Bartel, A. *tableone: Create "*Table 1*" to Describe Baseline Characteristics with or without Propensity Score Weights. CRAN: Contributed Packages*; R Package v 0.13.2; 2014. Available online: https://cran.r-project.org/web/packages/tableone/index.html (accessed on 10 June 2025).

33. Sjoberg, D.D.; Whiting, K.; Curry, M.; Lavery, J.A.; Larmarange, J. Reproducible Summary Tables with the gtsummary Package. *R J.* **2021**, *13*, 570–580. [CrossRef]
34. Lemeshow, S.; Hosmer, D.W.; Klar, J.; Lwanga, S.K. *Adequacy of Sample Size in Health Studies*; World Health Organization: Geneva, Switzerland, 1990.
35. Lenz, A.; Haynes, A.G.; Limacher, A. *presize: Precision Based Sample Size Calculation*; 2023. Available online: https://joss.theoj.org/papers/10.21105/joss.03118 (accessed on 10 June 2025).
36. Kohl, M. MKpower: Power Analysis and Sample Size Calculation. 2025. Available online: https://github.com/stamats/MKpower (accessed on 10 February 2025).
37. Kuhn, M.; Vaughan, D.; Hvitfeldt, E. *yardstick: Tidy Characterizations of Model Performance*; R Package v 1.3.2; 2025. Available online: https://cran.r-project.org/web/packages/yardstick/index.html (accessed on 10 June 2025).
38. Weisberg, S. *Applied Linear Regression*, 4th ed.; Wiley: Hoboken, NJ, USA, 2014.
39. Zeileis, A.; Köll, S.; Graham, N. Various Versatile Variances: An Object-Oriented Implementation of Clustered Covariances in R. *J. Stat. Softw.* **2020**, *95*, 1–36. [CrossRef]
40. Pinheiro, J.; Bates, D.; R Core Team. *nlme: Linear and Nonlinear Mixed Effects Models*; R Package v 3.1-168; 2025. Available online: https://cran.r-project.org/web/packages/nlme/index.html (accessed on 10 June 2025).
41. Bates, D.; Mächler, M.; Bolker, B.; Walker, S. Fitting Linear Mixed-Effects Models Using lme4. *J. Stat. Softw.* **2015**, *67*, 1–48. [CrossRef]
42. Lüdecke, D.; Ben-Shachar, M.S.; Patil, I.; Waggoner, P.; Makowski, D. performance: An R Package for Assessment, Comparison and Testing of Statistical Models. *J. Open Source Softw.* **2021**, *6*, 3139. [CrossRef]
43. Lüdecke, D.; Patil, I.; Ben-Shachar, M.S.; Wiernik, B.M.; Waggoner, P.; Makowski, D. see: An R Package for Visualizing Statistical Models. *J. Open Source Softw.* **2021**, *6*, 3393. [CrossRef]
44. Friedman, J.; Hastie, T.; Tibshirani, R.; Narasimhan, B.; Tay, K.; Simon, N.; Qian, J.; Yang, J. *glmnet: Lasso and Elastic-Net Regularized Generalized Linear Models*; 2025. Available online: https://www.jstatsoft.org/article/view/v033i01 (accessed on 10 June 2025).
45. Kuhn, M. Building Predictive Models in R Using the caret Package. *J. Stat. Softw.* **2008**, *28*, 1–26. [CrossRef]
46. James, G.; Witten, D.; Hastie, T.; Tibshirani, R. *Introduction to Statistical Learning: With Applications in R*, 2nd ed.; Spinger: New York, NY, USA, 2021.
47. Jackman, S. *pscl: Political Science Computational Laboratory*; R Package v 1.5.9; 2024. Available online: https://cran.r-project.org/web/packages/pscl/index.html (accessed on 10 June 2025).
48. Sing, T.; Sander, O.; Beerenwinkel, N.; Lengauer, T. ROCR: Visualizing classifier performance in R. *Bioinformatics* **2005**, *21*, 7881. [CrossRef]
49. Hilbe, J.M. *Logistic Regression Models*, 1st ed.; Chapman & Hall: Upper Saddle River, NJ, USA, 2009.
50. Therneau, T.M.; Grambsch, P.M. *Modeling Survival Data: Extending the Cox Model*; Springer: New York, NY, USA, 2000.
51. Therneau, T.M. Survival: Survival Analysis. R Package v 3.8-3. 2024. Available online: https://CRAN.R-project.org/package=survival (accessed on 10 February 2025).

52. Kassambara, A.; Kosinski, M.; Biecek, P. *survminer: Drawing Survival Curves Using ggplot2*; R Package v 0.5.0; 2024. Available online: https://cran.r-project.org/web/packages/survminer/index.html (accessed on 10 June 2025).
53. Grolemund, G.; Wickham, H. Dates and Times Made Easy with lubridate. *J. Stat. Softw.* **2011**, *40*, 1–25. [CrossRef]
54. Gray, B. cmprsk: Subdistribution Analysis of Competing Risks. 2024. Available online: https://www.R-project.org (accessed on 10 February 2025).
55. Moore, D.F. *Applied Survival Analysis Using R*; Springer: Cham, Switzerland, 2016.
56. Fine, J.P.; Gray, R.J. A Proportional Hazards Model for the Subdistribution of a Competing Risk. *J. Am. Stat. Assoc.* **1999**, *94*, 496–509. [CrossRef]
57. Crowley, J.; Hu, M. Covariance Analysis of Heart Transplant Survival Data. *J. Am. Stat. Assoc.* **1977**, *72*, 27. [CrossRef]
58. Halekoh, U.; Højsgaard, S.; Yan, J. The R Package geepack for Generalized Estimating Equations. *J. Stat. Softw.* **2006**, *15*, 1–11. [CrossRef]
59. Wood, S. *mgcv: Mixed GAM Computation Vehicle with Automatic Smoothness Estimation*; R Package v 1.9-3; 2025. Available online: https://cran.r-project.org/web/packages/mgcv/index.html (accessed on 10 June 2025).
60. Carey, V.J. *gee: Generalized Estimation Equation Solver*; R Package v 4.13-29; 2024. Available online: https://cran.r-project.org/web/packages/gee/index.html (accessed on 10 June 2025).
61. Lüdecke, D. *sjPlot: Data Visualization for Statistics in Social Science*; R Package v2.9.0; 2025. Available online: https://CRAN.R-project.org/package=sjPlot (accessed on 10 February 2025).
62. Lüdecke, D. sjlabelled: Labelled Data Utility Functions. R Package v 1.2.0. 2022. Available online: https://strengejacke.github.io/sjlabelled/ (accessed on 10 February 2025). [CrossRef]
63. Lüdecke, D. *sjmisc: Data and Variable Transformation Functions*; R Package v 2.8.11; 2025. Available online: https://joss.theoj.org/papers/10.21105/joss.00754 (accessed on 10 February 2025).
64. Ripley, B.; Venables, B. MASS: Support Functions and Datasets for Venables and Ripley's MASS. R Package v 7.3-65. 2025. Available online: https://www.stats.ox.ac.uk/pub/MASS4/ (accessed on 10 February 2025).
65. Singer, J.D.; Willett, J.B. *Applied Longitudinal Data Analysis*; Oxford University Press: New York, NY, USA, 2003. [CrossRef]
66. Galecki, A.; Burzykowski, T. *Linear Mixed-Effects Models Using R*; Springer: New York, NY, USA, 2013. [CrossRef]
67. Hubbard, A.E.; Ahern, J.; Fletscher, N.L.; Van der Laan, M.; Lippman, S.A.; Jewell, N.; Bruckner, T.; Satariano, W.A. To GEE or Not to GEE. *Epidemiology* **2010**, *21*, 467–474. [CrossRef] [PubMed]
68. Ho, N.T.; Abe, S.K.; Rahman, S.; Islam, R.; Saito, E.; Gupta, P.C.; Pednekar, M.S.; Sawada, N.; Tsugane, S.; Tamakoshi, A.; et al. Diabetes is associated with increased liver cancer incidence and mortality in adults: A report from Asia Cohort Consortium. *Int. J. Cancer* **2024**, *155*, 854–870. [CrossRef] [PubMed]

69. Auguie, B. *gridExtra: Miscellaneous Functions for "Grid" Graphics*; R Package v 2.3; 2017. Available online: https://cran.r-project.org/web/packages/gridExtra/index.html (accessed on 10 February 2026).
70. Harrell, F.E., Jr. *Hmisc: Harrell Miscellaneous*; R Package v 5.2-3; 2025. Available online: https://cran.r-project.org/web/packages/Hmisc/index.html (accessed on 10 February 2026).
71. Revelle, W. psych: Procedures for Psychological, Psychometric, and Personality Research. R Package v 2.5.6. 2025. Available online: https://CRAN.R-project.org/package=psych (accessed on 10 February 2025).
72. Warnes, G.R.; Bolker, B.; Lumley, T.; Johnson, R.C.; Jain, N.; Schwartz, M.; Rogers, J. *gmodels: Various R Programming Tools for Model Fitting*; R Package v 2.19.1; 2024. Available online: https://cran.r-project.org/web/packages/gmodels/index.html (accessed on 10 February 2026).
73. Schwarzer, G. meta: General Package for Meta-Analysis. 2025. Available online: https://github.com/guido-s/meta/ (accessed on 10 February 2025).
74. Pedersen, T.L. patchwork: The Composer of Plots. 2025. Available online: https://patchwork.data-imaginist.com (accessed on 10 February 2025).
75. Ho, N.T.; Bangsberg, D.; Hermiston, M.L.; Do, C.T.; Phung, L.N.; Van Nguyen, Q.; Dao, A.Q.; Tran, C.T.L.; Le, H.T.M.; Phung, L.T.; et al. Overweight & obesity epidemic, temporal trends and regional disparities in physical growth of Vietnamese children. *Sci. Rep.* **2026**, *16*, 7515. [CrossRef] [PubMed]
76. De Onis, M.; Onyango, A.W.; Borghi, E.; Siyam, A.; Nishida, C.; Siekmann, J. Development of a WHO growth reference for school-aged children and adolescents. *Bull. World Health Organ.* **2007**, *85*, 660–667. [CrossRef] [PubMed]
77. De Onis, M. WHO Child Growth Standards based on length/height, weight and age. *Acta Paediatr. Int. J. Paediatr.* **2006**, *95*, 76–85. [CrossRef]
78. Schumacher, D. *anthro: Computation of the WHO Child Growth Standards*; R Package v 1.0.1; 2023. Available online: https://cran.r-project.org/web/packages/anthro/index.html (accessed on 10 February 2026).
79. Myatt, M.; Guevarra, E. *zscorer: Child Anthropometry z-Score Calculator*; R Package v 0.3.1; 2019. Available online: https://cran.r-project.org/web/packages/zscorer/index.html (accessed on 10 February 2026).
80. Signorell, A. *DescTools: Tools for Descriptive Statistics*; R Package v 0.99.60; 2025. Available online: https://cran.r-project.org/web/packages/DescTools/index.html (accessed on 10 February 2026).
81. Mayer, M. *confintr: Confidence Intervals*; R Package v 1.0.2; 2023. Available online: https://cran.r-project.org/web/packages/confintr/index.html (accessed on 10 February 2026).
82. Stasinopoulos, M.; Rigby, R. gamlss: Generalized Additive Models for Location Scale and Shape. R Package v 5.4-22. 2024. Available online: https://www.gamlss.com/ (accessed on 10 April 2025).
83. Rudis, B. *hrbrthemes: Additional Themes, Theme Components and Utilities for ggplot2*; R Package v 0.8.7; 2024. Available online: https://cran.r-project.org/web/packages/hrbrthemes/index.html (accessed on 10 February 2026).
84. Garnier, S. *viridis: Colorblind-Friendly Color Maps for R*; R Package v 0.6.5; 2024. Available online: https://zenodo.org/records/7890878 (accessed on 10 February 2026).

85. Lenth, R.V. *emmeans: Estimated Marginal Means, aka Least-Squares Means*; R Package v 1.11.2; 2025. Available online: https://cran.r-project.org/web/packages/emmeans/index.html (accessed on 10 February 2026).
86. Aphalo, P.J. *ggpmisc: Miscellaneous Extensions to ggplot2*; R Package v 0.6.2; 2025. Available online: https://cran.r-project.org/web/packages/ggpmisc/index.html (accessed on 10 February 2026).
87. Wickham, H. *stringr: Simple, Consistent Wrappers for Common String Operations*; R Package v 1.5.1; 2023. Available online: https://cran.r-project.org/web/packages/stringr/index.html (accessed on 10 February 2026).
88. Cole, T.J. The LMS method for constructing normalized growth standards. *Eur. J. Clin. Nutr.* **1990**, *44*, 45–60. [PubMed]
89. Qiu, W.; Chavarro, J.; Lazarus, R.; Rosner, B.; Ma, J. *powerSurvEpi: Power and Sample Size Calculation for Survival Analysis of Epidemiological Studies*; R package v 0.1.5; 2025. Available online: https://cran.r-project.org/web/packages/powerSurvEpi/index.html (accessed on 10 February 2026).
90. Zhang, E.; Wu, V.Q.; Chow, S.-C.; Zhang, H.G. *TrialSize: R Functions for Chapter 3,4,6,7,9,10,11,12,14,15 of Sample Size Calculation in Clinical Research*; R Package v 1.4.1; 2024. Available online: https://cran.r-project.org/web/packages/TrialSize/index.html (accessed on 10 February 2026).
91. Qi, H.; Zhu, F. *SampleSize4ClinicalTrials: Sample Size Calculation for the Comparison of Means or Proportions in Phase III Clinical Trials*; R Package v 0.2.3; 2021. Available online: https://cran.r-project.org/web/packages/SampleSize4ClinicalTrials/index.html (accessed on 10 February 2026).
92. Anderson, K. *gsDesign: Group Sequential Design*; R Package v 3.6.9; 2025. Available online: https://cran.r-project.org/web/packages/gsDesign/index.html (accessed on 10 February 2026).
93. Snow, G. *blockrand: Randomization for Block Random Clinical Trials*; R Package v 1.5; 2020. Available online: https://cran.r-project.org/web/packages/blockrand/index.html (accessed on 10 February 2026).
94. Uschner, D.; Schindler, D.; Hilgers, R.D.; Heussen, N. randomizeR: An R Package for the Assessment and Implementation of Randomization in Clinical Trials. *J. Stat. Softw.* **2018**, *85*, 1–22. [CrossRef]
95. Chow, S.C.; Shao, J.; Wang, H.; Lokhnygina, Y. *Sample Size Calculations in Clinical Research*, 3rd ed.; Taylor & Francis: Boca Raton, FL, USA, 2017. [CrossRef]
96. Arjyal, A.; Basnyat, B.; Nhan, H.T.; Koirala, S.; Giri, A.; Joshi, N.; Shakya, M.; Pathak, K.R.; Mahat, S.P.; Prajapati, S.P.; et al. Gatifloxacin versus ceftriaxone for uncomplicated enteric fever in Nepal: An open-label, two-centre, randomised controlled trial. *Lancet Infect. Dis.* **2016**, *16*, 535–545. [CrossRef] [PubMed]
97. Le, T.; Van Kinh, N.; Cuc, N.T.; Tung, N.L.; Lam, N.T.; Thuy, P.T.; Cuong, D.D.; Phuc, P.T.; Vinh, V.H.; Hanh, D.T.; et al. A Trial of Itraconazole or Amphotericin B for HIV-Associated Talaromycosis. *N. Engl. J. Med.* **2017**, *376*, 2329–2340. [CrossRef] [PubMed]

98. Hồ, N.T.; Hughes, S.G.; Ta, V.T.; Phan, L.T.; Đỗ, Q.; Nguyễn, T.V.; Phạm, A.T.V.; Đặng, M.T.N.; Nguyễn, L.V.; Trịnh, Q.V.; et al. Safety, immunogenicity and efficacy of the self-amplifying mRNA ARCT-154 COVID-19 vaccine: Pooled phase 1, 2, 3a and 3b randomized, controlled trials. *Nat. Commun.* **2024**, *15*, 4081. [CrossRef] [PubMed]
99. Zhang, Y.; Xiao, N.; Anderson, K.; Zhu, Y. R for Clinical Study Reports and Submission. 2025. Available online: https://r4csr.org/ (accessed on 10 April 2025).
100. Friedman, L.M.; Furberg, C.D.; DeMets, D.L.; Reboussin, D.M.; Granger, C.B. *Fundamentals of Clinical Trials*, 5th ed.; Springer Nature: Cham, Switzerland, 2015.
101. Daniels, B.; Coutsoudis, A.; Moodley-Govender, E.; Mulol, H.; Spooner, E.; Kiepiela, P.; Reddy, S.; Zako, L.; Ho, N.T.; Kuhn, L.; et al. Effect of co-trimoxazole prophylaxis on morbidity and mortality of HIV-exposed, HIV-uninfected infants in South Africa: A randomised controlled, non-inferiority trial. *Lancet Glob. Health* **2019**, *7*, e1717–e1727. [CrossRef] [PubMed]
102. Ho, N.T.; Li, F.; Lee-Sarwar, K.A.; Tun, H.M.; Brown, B.; Pannaraj, P.S.; Bender, J.M.; Azad, M.B.; Thompson, A.L.; Weiss, S.T.; et al. Meta-analysis of effects of exclusive breastfeeding on infant gut microbiota across populations. *Nat. Commun.* **2018**, *9*, 4169. [CrossRef] [PubMed]
103. Rigby, R.A.; Stasinopoulos, D.M. Generalized additive models for location, scale and shape. *J. R. Stat. Soc.* **2005**, *54*, 507–554. [CrossRef]
104. Chen, E.Z.; Li, H. A two-part mixed-effects model for analyzing longitudinal microbiome compositional data. *Bioinformatics* **2016**, *32*, 2611–2617. [CrossRef] [PubMed]
105. McMurdie, P.J.; Holmes, S. Waste Not, Want Not: Why Rarefying Microbiome Data Is Inadmissible. *PLoS Comput. Biol.* **2014**, *10*, e1003531. [CrossRef] [PubMed]
106. Min, Y.; Agresti, A. Random effect models for repeated measures of zero-inflated count data. *Stat. Model.* **2005**, *5*, 1–19. [CrossRef]
107. Aitchison, J.; Barceló-Vidal, C.; Martín-Fernández, J.A.; Pawlowsky-Glahn, V. Logratio Analysis and Compositional Distance. *Math. Geol.* **2000**, *32*, 271–275. [CrossRef]
108. Palarea-Albaladejo, J.; Martin-Fernandez, J.A. zCompositions—R package for multivariate imputation of left-censored data under a compositional approach. *Chemom. Intell. Lab. Syst.* **2015**, *143*, 85–96. [CrossRef]
109. Chen, L.; Reeve, J.; Zhang, L.; Huang, S.; Wang, X.; Chen, J. GMPR: A robust normalization method for zero-inflated count data with application to microbiome sequencing data. *PeerJ* **2018**, *6*, e4600. [CrossRef] [PubMed]
110. Bender, J.M.; Li, F.; Martelly, S.; Byrt, E.; Rouzier, V.; Leo, M.; Tobin, N.; Pannaraj, P.S.; Adisetiyo, H.; Rollie, A.; et al. Maternal HIV infection influences the microbiome of HIV-uninfected infants. *Sci. Transl. Med.* **2016**, *8*, 349ra100. [CrossRef] [PubMed]
111. Pannaraj, P.S.; Li, F.; Cerini, C.; Bender, J.M.; Yang, S.; Rollie, A.; Adisetiyo, H.; Zabih, S.; Lincez, P.J.; Bittinger, K.; et al. Association Between Breast Milk Bacterial Communities and Establishment and Development of the Infant Gut Microbiome. *JAMA Pediatr.* **2017**, *171*, 647–654. [CrossRef] [PubMed]
112. Thompson, A.L.; Monteagudo-Mera, A.; Cadenas, M.B.; Lampl, M.L.; Azcarate-Peril, M.A. Milk- and solid-feeding practices and daycare attendance are associated with differences in bacterial diversity, predominant communities, and metabolic and immune function of the infant gut microbiome. *Front. Cell. Infect. Microbiol.* **2015**, *5*, 3. [CrossRef] [PubMed]

113. Warnes, G.R.; Gorjanc, G.; Magnusson, A.; Andronic, L.; Rogers, J.; MacQueen, D.; Korosec, A. *gdata: Various R Programming Tools for Data Manipulation*; R Package v 3.0.1; 2005. Available online: https://cran.r-project.org/web/packages/gdata/index.html (accessed on 10 February 2026).
114. Caporaso, J.G.; Kuczynski, J.; Stombaugh, J.; Bittinger, K.; Bushman, F.D.; Costello, E.K.; Fierer, N.; Gonzalez Peña, A.; Goodrich, J.K.; Gordon, J.I.; et al. QIIME allows analysis of high-throughput community sequencing data. *Nat. Methods* **2010**, *7*, 335–336. [CrossRef] [PubMed]
115. Langille, M.G.I.; Zaneveld, J.; Caporaso, J.G.; McDonald, D.; Knights, D.; Reyes, J.A.; Clemente, J.C.; Burkepile, D.E.; Vega Thurber, R.L.; Knight, R.; et al. Predictive functional profiling of microbial communities using 16S rRNA marker gene sequences. *Nat. Biotechnol.* **2013**, *31*, 814–821. [CrossRef] [PubMed]
116. Morgan, X.C.; Tickle, T.L.; Sokol, H.; Gevers, D.; Devaney, K.L.; Ward, D.V.; A Reyes, J.; A Shah, S.; LeLeiko, N.; Snapper, S.B.; et al. Dysfunction of the intestinal microbiome in inflammatory bowel disease and treatment. *Genome Biol.* **2012**, *13*, R79. [CrossRef] [PubMed]

Index

≤

≤ 5 years old, 239

>

> 5 years old, 239

9

95%CI, 52, 61, 219

A

accumulated risk, 150
Accuracy, 113
Additivity, 141, 142
Adjusted R-squared, 91
adjustvar, 292
adverse events, 258, 286
AIC, 98, 112, 119, 148
AIC(), 119, 147
Akaike Information Criterion, 90, 112, 119, 147
alpha diversity, 289
alpha.compare(), 301, 302
anova(), 146, 147, 174
anthro, 221, 239
anthro_prevalence(), 239
anthro_zscores(), 239
anthropometric, 223
Area Under the Curve, 117
array, 7
arsenal, 30, 46, 55, 99, 219, 221, 222, 258, 273, 286
as.Date(), 128
assumption, 50, 142
AUC, 117
Auto-regressive, 163

B

backward, 91, 147
barplot, 18
Base R, 12
baseline hazard function, 136
baseline summaries, 258
Bayesian Information Criterion, 90, 119, 147
Between-Subject Variance, 172
bias-corrected and accelerated, 226
BIC, 98, 119, 147
BIC(), 119, 147
binary, 99
Binary Outcome, 188, 260
binomial, 49, 99, 112, 188, 261
binomial distribution, 49
Blocked Randomization, 266
blockrand, 258, 265
bootstrap confidence interval, 226
Box–Cox transformation, 254
boxplot, 24

C

c(), 11
car, 46
case study, 258
case_when(), 237
categorical, 99
Categorical predictor, 73
categorical variables, 54
Censoring, 129
Centered Log Ratio, 288
centiles(), 256
centiles.fan(), 256
Central Limit Theorem, 46
child growth, 221

child growth reference data, 221
chisq.test(), 54
chi-square test, 54
ci_median_diff(), 221
CIF, 150
classification measures, 114
Classification Metrics, 113
cmprsk, 127, 153, 190
coef(), 65
coefficients, 73, 98, 101, 112, 138, 228
color code, 233
competing risk, 151
Competing Risk Analysis, 126
comvar, 292
Concordance Index, 143
confidence interval, 223
confint(), 65, 65
confintr, 221, 226
confusion matrix, 113
Continuous Outcome, 262
Continuous variables, 50
Cook's distance, 73
Correlation, 228
correlation model, 170
correlation structure, 228
corstr, 228
Count Outcome, 188
Covariance, 170
covariates, 138
Cox model, 142
Cox proportional hazards model, 126, 135
cox.zph(), 129, 131
coxph(), 129, 136
cuminc(), 152
Cumulative Hazard Function, 150
Cumulative Incidence Function, 150
cumulative incidence plots, 277
cumulative probability, 150
cut(), 242
cv.glmnet(), 149

D

data frame, 11
Data Manipulation, 15
data object, 9
Data visualization, 18
data.frame(), 11
dataset, 241
dates, 128
dbinom(), 49
density, 19
Dependence, 163
dependency, 70
DescTools, 221
Deviance, 112
Deviance Residuals, 112, 146
dfbeta, 73
diagnostic measures, 73
diagnostic plot, 69
difference, 224
differential, 292
difftime(), 128
Discrimination power, 117
distribution, 50
dnorm, 48
dplyr, 13, 14, 18

E

Each Study, 190
effect size, 58, 261
efficacy, 258
emmeans, 218
Entering Data, 11
Equal Variance Assumption, 69
Error rate, 113
errors, 70
ES.h(), 60, 60
event, 129
Exchangeable, 168
Exporting Data, 14
Extract effects, 174
Extreme tallness, 253

F

facet_grid(), 43
False Negative, 113
False Positive, 113
family, 112, 254, 255, 292
figures, 258
fill, 19
filter, 291
filter(), 15
Fine–Gray Model, 153
fixed effects, 157, 188
forest plot, 212, 289
forest(), 212
forward, 91, 147
function, 10, 147
functional pathway, 290

G

gamlss, 221, 257, 292
GAMLSS, 292
GAMLSS-BEZI, 292
gaussian, 188
Gaussian, 188
gdata, 291
gee, 157
GEE, 157
GEE model, 221
geeglm(), 167, 187, 228
geepack, 157, 221, 228
Generalized Additive Models for Location, Scale, and Shape, 221
Generalized Estimating Equations, 157, 187
Generalized Linear Mixed-Effects Models, 157, 188
Generalized Linear Models, 71, 163, 309
genus, 293
geom = "table", 231
geom_bar(), 18
geom_boxplot(), 24
geom_density(), 19
geom_dotplot(), 24
geom_histogram(), 18
geom_point(), 22
Geometric Mean of Pairwise Ratios, 288
ggarrange(), 44, 232
ggcompetingrisks(), 152
ggplot(), 78
ggplot2, 15, 18, 158, 221, 223
ggpmisc, 311
ggpubr, 44, 222
ggsurvplot(), 132, 277
GitHub API, 13
Glaz & Sison's method, 238
glm(), 71, 101, 102, 111
glmer(), 188
GLMM, 157
glmnet, 127, 149
gmodels, 190
Goodness-of-Fit, 144
gridExtra, 190, 222
group_by(), 25, 26, 280
growth curves, 221, 257
growth trajectory, 236
gsDesign, 258
gtsummary, 190

H

haven, 5
Hazard Function, 127, 136
hazard rate, 136
hazard ratio, 135, 137, 141
heatmap, 289
help for package, 5
heterogeneity, 220
Heterogeneity Test, 201
heteroscedasticity, 69
histogram, 18
Hmisc, 190
Homogeneity, 52
Homoscedasticity, 68, 69
HR, 138
hrbrthemes, 222
httr, 13

I

id, 221
if, 150
import(), 13
import_list(), 13
Importing Data, 12
incidence rate, 195
Independence, 50, 70, 228
independent, 68
individual, 241
Individual Data, 236
influence.measures(), 73
Influential observations, 72
Install, 4
install.packages(), 4
installr, 5
instantaneous rate, 138
interaction, 90, 142
intercept, 67
interpretation, 89, 112
interquartile, 223
interval(), 128
iqr(), 223

J

jsonlite, 13

K

kableExtra, 5, 46, 99, 127, 157, 221, 258, 307
Kaplan–Meier, 129, 135
Kaplan–Meier plot, 135
Kaplan–Meier Curve, 135
KEGG, 290
knitr, 5, 46, 99, 127, 157, 221, 258, 306

L

Lambda, 257
Lambda-Mu-Sigma, 221
lapply(), 4
LASSO, 149
Likelihood ratio test, 142, 148, 202
linear assumption, 68
Linear mixed-effects models, 188
Linear Regression line, 100
Linear relationship, 68
Linearity, 142, 143
link, 288
list, 9
lm(), 73
lme4, 157, 187
lmer(), 188
lmerTest, 172
LMS, 221, 257
lms(), 257
Load, 4
log (odds ratio), 287
log(odds), 103, 104
Logistic function, 99
Logistic Regression, 99
Logistic Regression curve, 100
log-likelihood, 107
log-rank test, 135
longitudinal, 293
longitudinal correlation, 170
Longitudinal data, 158
loop, 205, 208, 225
lstrends(), 229
lubridate, 128, 190

M

MaAsLin, 294
mae(), 95
Managing Biomedical Data, 11
Mantel–Haenszel test, 133
mapfile, 295
Marginal Effects, 179
Martingale Residuals, 145
MASS, 157, 222
matrix, 9
Maximum Likelihood, 104
Maximum Likelihood Estimation, 107
McFadden's Pseudo R-Squared, 120
McFadden's R-squared, 106

mean, 59
Mean Absolute Error, 95
mean model, 163
Mean Squared Error, 95
mean.filter, 290
medci(), 224, 235
median, 221, 234
Median survival time, 133
MedianCI(), 221
medians, 223
meta, 190, 212
meta.niceplot(), 297, 298
meta.taxa(), 288, 297, 298
meta-analysis, 189, 289, 293
metagen(), 209
metamicrobiomeR, 8, 291
metatab.show(), 297, 298
mgcv, 157
microbiomeage(), 284
Misclassification, 113
Model Accuracy, 143
Model calibration, 96
model complexity, 90
Model Diagnostics, 181
Model Evaluation, 113
model fit, 67, 90
model fit statistics, 119
Model selection, 92, 148
model validation, 93, 121, 126
Mu, 257
MultinomCI(), 249
Multiple linear regression, 89
Multiple testing, 293
multivariable, 99, 141
mutate(), 17

N

n_distinct(), 279
Negative, 114
Nelson–Aalen Estimator, 150
new function, 201, 223, 248
non-inferiority, 262
non-inferiority trial, 262
non-informative censoring, 135
non-linear term, 68
non-linear transformations, 68
non-parametric tests, 50
normal, 48
Normal BMI, 253
normal distribution, 48
Normal HFA, 253
Normal WFH, 251
normality, 71
normally distributed, 68
nSurv(), 264
nuisance, 187
Null deviance, 112
nutritional status classification, 221, 233, 237

O

obesity, 242, 253
offset, 188
operation, 9
outcome, 68, 98, 260
overweight, 242, 257

P

p.adjust.method, 293
Parameter, 61
parametric tests, 50
patchwork, 190, 221
pathsum, 295
pathtab, 295
Pathway, 290, 292
pathway.compare(), 292, 297, 305
pbinom(), 49
Penalized Regression, 149
percent.filter, 293
percent.meta, 298
Performance Metrics, 96
performance(), 114
Peto–Wilcoxon test, 134
phylum, 292
pipe operator, 14
pivot_longer(), 281
pivot_wider(), 284
plot, 231, 292

plot(), 68, 114
plot_model(), 180, 232
pnorm, 46
poisson, 188
Poisson, 188
pooled estimates, 297
population-average, 157
Positive, 113
Positive predictive value, 113
power, 55, 259, 260
powerSurvEpi, 258, 262
pR2(), 107, 120
prec_prop(), 62
Precision, 61, 113
predict(), 65, 78, 114
predicted values, 67
Prediction, 113
prediction(), 114
predictor, 68
Preferred Term, 284
presize, 46, 58, 62
prevalence, 221, 239, 253
probability, 49, 100
propmed.rel, 292
proportional hazards, 139, 141
Proportional Hazards Assumption, 126
pscl, 99, 108, 120
pspline(), 142
psych, 190
publication-ready table, 55
p-value, 137
pwr, 46, 58, 259, 260
pwr.2p.test(), 60, 260
pwr.t.test(), 59

Q

QIC(), 188, 228
qnorm, 51
Q-Q plot, 51, 70
qqline(), 51
qqnorm(), 51
quantiles, 223
Quasi Information Criterion, 167, 228

R

R, 9
R Markdown, 2
R Packages, 4
R^2, 107
random effects, 157, 218
random intercept, 182, 184
random slope, 176
randomization, 258
randomizeR, 258
read.multi(), 284
readjust.p, 293
readr, 5
Receiver Operating Characteristics, 118
relabund.filter, 291
relative abundance, 292
rename(), 16
rename_at(), 15
repeated measurements, 157
reporting-ready, 280
Residual deviance, 112
residuals, 68, 146, 186
residuals(), 65, 144
rio, 13, 157, 221
rmarkdown, 5, 46, 99, 190, 221
rmse(), 95
rnorm, 47
ROC, 116
ROCR, 99
Root Mean Squared Error, 95
rsq(), 96
R-squared, 96, 108
RStudio, 4

S

sample size, 58, 260
sample_frac(), 121
SampleSize4ClinicalTrials, 258, 264
samplesizeestimator, 46, 58, 61
scatter plot, 22

Score test, 134, 146
SD, 234
select(), 14
Sensitivity, 113
sensitivity analysis, 182
Serious Adverse Events, 283
show.taxname, 292
Sigma, 257
Sigmoid function, 101
significance level, 58, 260
Simple linear regression, 66
Simple Randomization, 266
sjlabelled, 157, 188
sjmisc, 157, 188
sjPlot, 157, 188, 222, 232
slope, 67
smooth function, 254
smoothed line, 23
smoothed percentile curves, 254
spaghetti plot, 158
Specificity, 114
standard deviation, 235
standard error, 66, 137, 212
Standardized, 301
standing, 250
start-stop format, 154
stat_smooth, 23
stat_summary(), 224
step(), 98, 145
stepwise, 90, 147
str(), 8
strata, 268
strata(), 139
Stratified Cox model, 140
Stratified Randomization, 267
stringr, 222
stunting, 242, 251
Subdistribution Hazard Model, 153
Subgroup Analysis, 201
subhe(), 201
subject-specific, 157
summarise(), 25, 280
Summary Statistics, 26
summary table, 30, 272
summary(), 66, 98
summary.measure, 298
superiority, 258
Superiority trials, 258
supine length, 246
Surv(), 264
surv_fit(), 277
survdiff(), 156
survfit(), 133
survival, 128, 131, 258, 262, 265
Survival data, 126
Survival Function, 129, 131
survival outcome, 135
survival probability, 131
survival time, 130
survminer, 127, 132, 220, 271
Survreg(), 129
System Organ Class, 283

T

t.test(), 52
tableby(), 30, 55, 220, 271, 273
tableby.control(), 31, 56, 271
tables, 258
tax.lev, 291
taxa.compare(), 292, 293
taxa.filter(), 290
taxa.mean.plot(), 290, 305
taxa.meansdn(), 290, 305
taxcomtab.show(), 293, 296
Taxonomic, 292
Taxonomy, 292
taxtab, 292
Temporal trend, 228
termplot(), 142
test, 121
Testing Set, 93
thinness, 239, 253
tidyverse, 5, 14, 99, 127, 157, 222, 237, 258, 271, 280
Time dependent covariates, 129, 154
time-to-event, 128

Time-to-Event Outcome, 262
training, 121
Training Set, 95
transform, 67, 293
transformation, 70, 150, 292
trend, 68
trend slopes, 230
TrialSize, 258, 262
true negatives, 114
true positive, 113
T-test, 50
TwoSampleMean.NIS(), 262
TwoSampleProportion.NIS(), 263
type, 227

U

Unequal variance, 70
Univariable Cox Model, 201
Unstructured, 164
Update, 4
updating packages, 5

V

vaccine trial, 270
validation, 149
variance, 50, 187
vector, 9
viridis, 222
Visualization, 222

W

Wald test, 146
wasting, 242, 257
wgsData, 244
WHO, 221
WHO 2007 Growth References, 233
WHO median, 234
WHO reference data, 237
wilcox.test(), 53
Wilcoxon's rank sum test, 53
Within-Subject Variance, 172
World Health Organization, 221
wrap_plots(), 208
writing a loop, 205

Y

yaml, 5

Z

zcal(), 249
zero-inflated beta, 287
zeroreplace.method, 293
zero-replacement, 288
zscorer, 222, 243
Z-scores, 221, 243
z-statistic, 136

MDPI AG
Grosspeteranlage 5
4052 Basel
Switzerland
Tel.: +41 61 683 77 34

MDPI Books Editorial Office
E-mail: books@mdpi.com
www.mdpi.com/books

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.

MDPI Books

Data analysis is at the heart of modern biomedical research, and R is the tool that makes it possible. This hands-on, practical guide introduces researchers, clinicians, and students in the health sciences to R programming and applied biostatistics from the ground up. No prior programming experience is required.

Structured in three progressive parts—foundational R skills, core statistical methods (linear and logistic regression, survival analysis, longitudinal data analysis), and advanced specialized topics (large consortium data, child growth, clinical trials, microbiome data analysis)—every chapter features real biomedical datasets, worked R code, and practice exercises.

Whether you are transitioning from SPSS, Stata, or SAS, or are new to programming entirely, this book gives you the practical skills to take full control of your research data—from cleaning and visualization to complex modeling and reporting.

Journal
Collaboration

machine learning & knowledge extraction

Publishing Open Access
Books & Series

mdpi.com/books

978-3-7258-5337-3